IS101

Introduction to Information Systems



Instructor and Course Coordinator

Prof. (Assoc.) Hany El-Ghaish

Faculty of Computers and Informatics

Tanta University

Academic Year 2026–2027

رومانی صبحی رزق جرجس مطر

Introduction

Information systems connect technology, data, people, and procedures to support organizational activities and decision-making. Understanding these connections helps students explain how digital services operate, evaluate the information they produce, and recognize the responsibilities involved in using them. Programming complements this understanding by providing practical tools for processing data, implementing rules, and developing software.

This book introduces these two areas through a connected study of Information Systems and Python Programming. Prepared for the IS101 course, it combines conceptual explanations with practical examples, explanatory figures, and review questions. It is intended for students beginning their study of computing and assumes no previous programming experience.

The information systems part consists of seven chapters:

1. **Introduction to Information Systems** explains the components, functions, and types of information systems and their role in organizations.

2. **Information Technology Fundamentals** introduces hardware, software, computer architecture, networks, and the infrastructure supporting digital services.

3. **Data and Information Management** examines how data is collected, organized, cleaned, integrated, protected, and transformed into useful information.

4. **Business Processes and Systems** explains how organizations create value through processes and how enterprise systems support and coordinate their work.

5. **Securing Information Systems** introduces security goals, threats, controls, policies, privacy, and risk management.

6. **Emerging Technologies and the Future of Information Systems** explores cloud computing, analytics, the Internet of Things, artificial intelligence, and other technologies, considering both their capabilities and limitations.

7. **Conclusion** brings the preceding concepts together and encourages students to apply them when evaluating a complete information system.

رومانی صبحی رزق جرجس مطر

Python Programming and Learning Approach

The Python programming part develops practical skills progressively, beginning with setting up a programming environment and writing a first program. Students then study variables, data types, input and output, operators, conditional statements, and loops. These foundations lead to functions, modules, strings, and collections such as lists, tuples, sets, and dictionaries. File handling and exception handling introduce ways to retain data and respond to errors.

The programming sequence culminates in Object-Oriented Programming (OOP). Students learn how classes and objects organize related data and behavior, then explore attributes, methods, constructors, encapsulation, inheritance, and polymorphism. Examples connect these concepts to familiar situations, such as representing students, courses, products, or accounts.

The two parts reinforce each other. Information systems explain why an organization needs accurate records, defined processes, and appropriate controls; programming demonstrates how selected operations can be implemented. For example, a simple Python application can validate input, calculate totals, search records, or represent entities through classes. Such exercises help students connect an organizational requirement to the logic of a working program.

Students should read the explanations, examine the figures, and attempt the chapter questions before reviewing difficult concepts with their instructor. In the programming part, they should run each example, predict its output, modify its inputs, and investigate errors. This approach develops understanding through observation and practice.

2025/2026 2025/2026 2025/2026

By the end of the book, students should be able to explain the foundations of information systems, assess their contribution to organizational work, and write Python programs using both procedural and object-oriented approaches. These abilities provide a foundation for further study in databases, software development, cybersecurity, data analytics, and artificial intelligence.

Instructor and Course Coordinator

Prof. (Assoc.) Hany El-Ghaish
Faculty of Computers and Informatics
Tanta University
Academic Year 2026–2027

رومانی صبحی رزق جرجس مطر

Chapter 1

Introduction to Information Systems

IS101 | Academic year 2026–2027
Faculty of Computers and Informatics
Tanta University

Instructor and Course Coordinator
Prof. (Assoc.) Hany El-Ghaish

رومانی صبحی رزق جرجس مطر

Chapter introduction

Information systems help organizations turn records into useful information and coordinate the work that depends on it. This chapter introduces their components, functions, organizational roles, and historical development. It also explains how different system types support decisions and illustrates their use in business, healthcare, education, and everyday services.

1 1 What is an Information System

An Information System (IS) is a set of interrelated components that work together to collect, process, store, and distribute information to support decision-making, coordination, control, analysis, and visualization within an organization. [1]

2025/2026 2025/2026 2025/2026

In simpler terms, an information system takes raw data and transforms it into useful information that people can use to make better decisions and run an organization more effectively.

Information systems combine technology, people, and processes to create value. A computer can perform calculations, but an effective service also needs reliable records, clear rules, and people who understand how to use its results.

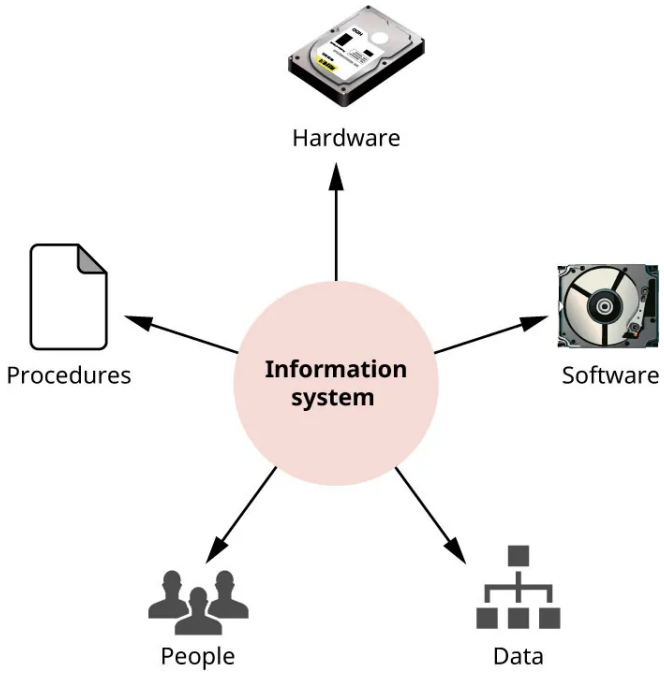


Figure 1.1. The input–process–output model. Kayau, Wikimedia Commons, CC BY-SA 3.0. Image content unchanged; converted from SVG to PNG. Source [10].

Example: a shop records quantities sold and unit prices. Processing these records produces a daily sales total that a manager can use when reviewing performance. The total becomes meaningful when its date, currency, and coverage are clear.

1 2 Components of an Information System

Understanding a system requires examining the resources that make it work. A common framework identifies five main components: hardware, software, data, people, and procedures. These components are connected; their effectiveness depends on how they work together. [1]



2025/2026

2025/2026

Figure 1.2. Components of an information system. Rice University, OpenStax, original Figure 1.2, CC BY 4.0. Image content unchanged; converted to PNG. Source [1].

Component	Description	Examples
Hardware	Physical devices used to input, process, store, and output data	Computers, servers, storage devices, printers, and network equipment
Software	Programs and instructions that tell hardware what to do	Operating systems, applications, and database systems
Data	Raw facts and figures processed into information	Customer names, sales figures, inventory levels, and sensor readings
People	Individuals who use, develop, manage, and maintain the system	End users, managers, IT specialists, and system analysts
Procedures	Rules, policies, and methods governing system use	Data entry rules, security policies, and backup procedures

These five components work together as a system. If a component is weak or missing, the whole service can become less effective. For example, an inventory application may produce an incorrect stock report if staff do not record returned goods. Correct software cannot compensate for missing transactions.

1 3 Functions of Information Systems

Information systems handle information through several related activities. The six functions below provide a practical overview: data collection, processing, storage, retrieval, distribution, and decision support. A particular system may emphasize some functions more than others.

1 Data Collection Gathering data from sources such as user input, sensors, external systems, and existing databases. A supermarket checkout collects item identifiers and quantities for each purchase.

2 Data Processing Converting raw data into meaningful information through calculations, sorting, classifying, and summarizing. For example, quantities and prices can be used to calculate revenue by product.

3 Data Storage Retaining raw data, processed records, and information in databases, data warehouses, or other storage systems for future use. Storage makes it possible to compare current results with earlier records.

4 Data Retrieval Allowing authorized users to access stored information when needed. A customer-service employee may retrieve an order using its identifier.

5 Data Distribution Delivering information to appropriate people through reports, dashboards, emails, or other channels. The format and timing should match the user's task.

6 Decision Support Providing analytical tools and insights that help managers and employees compare options and make informed decisions. A planning tool may compare different stock-order quantities.

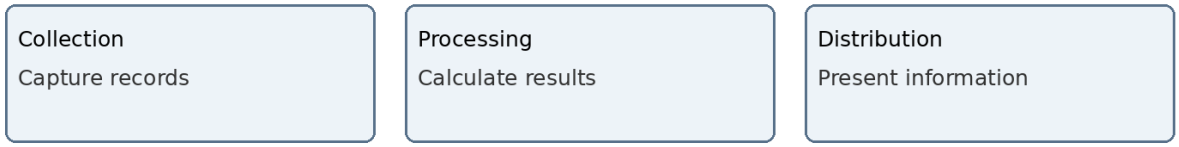


Figure 1.3. Selected activities in information handling. Storage and retrieval support these activities; decision support uses their results. Original teaching diagram.

A connected example

An online shop collects an order, calculates its value, stores the record, retrieves it for fulfillment, and sends a confirmation. Summarizing several weeks of orders can help staff assess demand. This example shows that the functions support a common activity rather than operating as unrelated tasks.

رومانی صبحی رزق جرجس مطر

1 4 Significance of Information Systems

Information systems matter because they can improve how work is performed and how information is used. These benefits depend on data quality, suitable design, and effective management. The presence of technology alone does not guarantee better organizational results.

Improved Decision Making Timely, accurate, and relevant information helps reduce uncertainty. Decision makers still need to examine assumptions and interpret the evidence.

Increased Efficiency Automation can reduce repeated manual work and some entry errors. An incorrectly configured rule can also repeat a mistake, so results require appropriate checks.

Enhanced Communication and Collaboration Shared information helps people inside and outside an organization coordinate activities. Teams can work from an agreed version of a document or record.

Competitive Advantage Effective use of information can help an organization respond faster to changing demand and customer needs. Advantage depends on how the service supports its wider business strategy.

Data Driven Insights Analytics can reveal patterns in customer behavior, operational performance, and market trends. These findings need sufficient context and evidence.

Global Reach Internet services and digital platforms allow organizations to reach users across geographic boundaries, subject to their operational capabilities and applicable requirements.

Security and Compliance Access controls, audit records, and other safeguards can support data protection and compliance. Compliance also requires appropriate policies, procedures, and oversight.

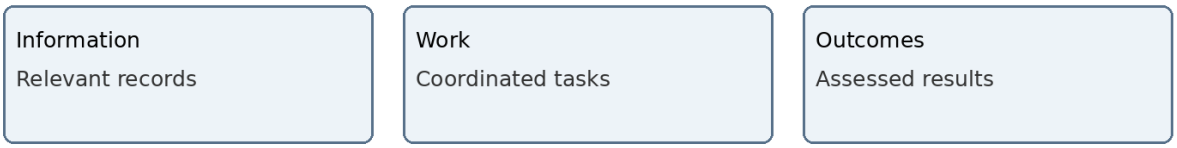


Figure 1.4. Connecting information resources with organizational outcomes. Original teaching diagram.

Example: a shared stock record can help sales and purchasing staff coordinate replenishment. The benefit should be assessed through outcomes such as fewer stock discrepancies, rather than the number of screens or reports introduced.

رومانی صبحی رزق جرجس مطر

1 5 Role of Information Systems in Organizations

Organizations use information at different levels. Operational activities concern routine work; tactical activities concern short-term planning and control; strategic activities concern longer-term direction. The distinction explains why users need different levels of detail and different reporting periods.

Level	Role	Examples
Operational	Process day-to-day transactions	Order processing, payroll, and inventory tracking
Tactical or middle management	Support short-term planning and control	Sales reports, budget monitoring, and resource allocation
Strategic or top management	Support long-term planning and major decisions	Market analysis, competitor intelligence, and strategic forecasting

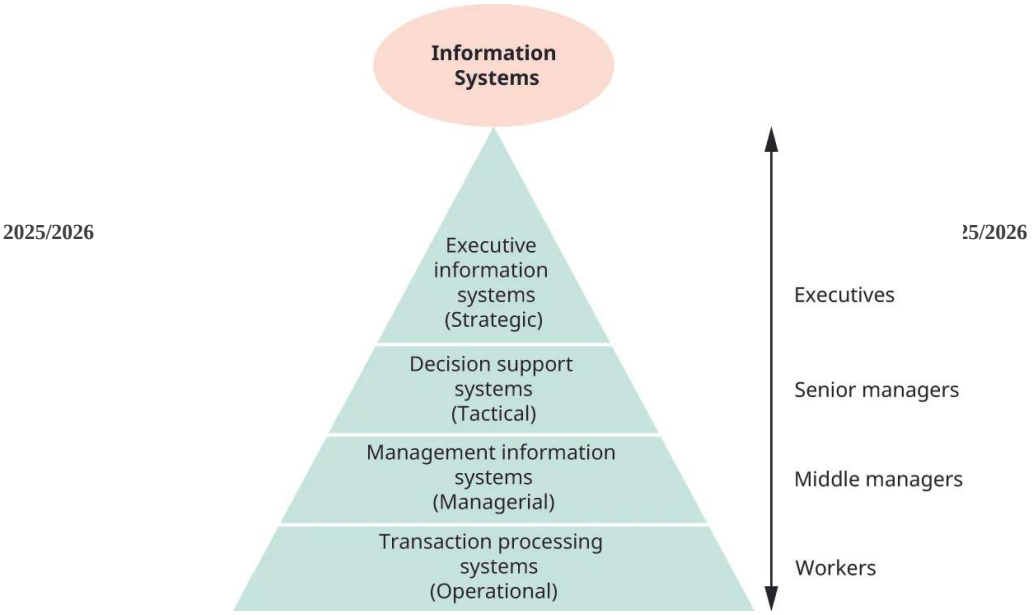


Figure 1.5. A simplified organizational view of information system categories. Rice University, OpenStax, original Figure 1.3, CC BY 4.0. Image content unchanged; converted to PNG. Source [1].

Key contributions

Across these levels, information systems support data and information management, decision support, business process automation, internal and external communication, performance monitoring, and strategic alignment. They can also support competitive positioning, data security, and regulatory compliance. The responsibilities and users of these capabilities may overlap.

Example: a branch records a sale at the operational level. A regional manager reviews monthly branch totals at the tactical level. Senior management examines longer-term trends when considering expansion. Related records support three different questions.

1 6 Historical Perspective and Evolution

The evolution of information systems reflects changes in computing, communications, and organizational needs. The periods below are approximate and overlapping. New technologies did not immediately replace earlier approaches; manual records, mainframes, and modern cloud services can coexist. [2–5]

Period	Key developments	Characteristics
Before the 1940s	Manual and mechanical systems	Paper records, card catalogs, and mechanical calculators
1940s–1950s	Early electronic computers	ENIAC publicly unveiled in 1946; scientific and military applications
1950s–1960s	Mainframe era	Centralized computing, batch processing, and development of COBOL from 1959
1960s–1970s	Minicomputers and early networking	Smaller systems, time sharing, and expanding computer networks
Late 1970s–1980s	Personal computing	Spreadsheets, word processors, and local networks; IBM PC introduced in 1981
1980s–1990s	Client server computing	Networked PCs, servers, relational databases, and graphical interfaces
1990s onward	Web and online services	Web invented in 1989; wider adoption in the 1990s supported e-commerce and online services
2000s onward	Cloud, mobile, IoT, big data, and AI adoption	Growing use of networked services, mobile devices, connected sensors, and advanced analytics

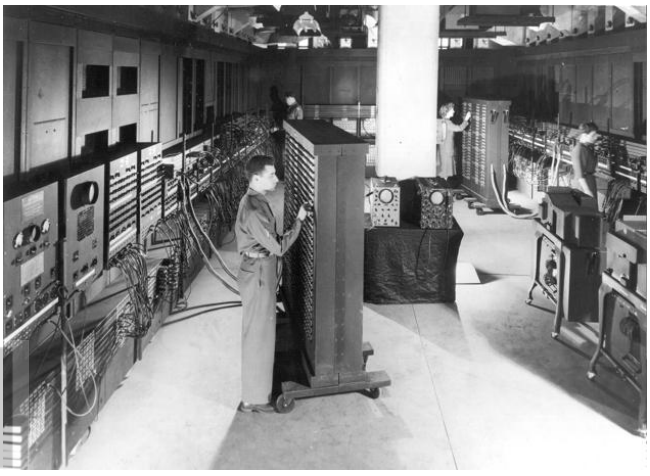


Figure 1.6. ENIAC at the Moore School in 1946. U. S. Army Photo; unidentified U.S. Army photographer. Public domain in the United States. Reproduced unchanged from Wikimedia Commons. Source [11].

Each stage expanded what organizations could do with information. The dates describe broad adoption patterns, not the invention dates of every technology. Artificial intelligence and mobile communication, for example, have histories that precede their widespread contemporary use. The Internet and the World Wide Web are also distinct: the Web is one service built on Internet infrastructure.

1 7 Decision Making and Strategic Planning

Information systems support decisions by making relevant records accessible and providing tools for interpretation. Their role is to inform judgment, compare alternatives, and monitor outcomes. A large volume of data is useful only when it relates to the decision and can be interpreted reliably.

Decision making support

- Provide access to internal and external data.
- Offer analysis and visualization through charts, dashboards, and reports.
- Support operational, tactical, and strategic decisions.
- Enable faster responses to changing conditions.
- Help challenge unsupported assumptions with evidence.

Data-informed analysis can reduce some forms of personal bias, but data and models can also contain bias. Decision makers should examine data coverage, assumptions, and possible consequences before accepting an analytical recommendation.

Strategic planning support

- Analyze past performance and current trends.
- Examine the external environment, including competitors, markets, and relevant regulations.
- Perform scenario analysis and what-if modeling.
- Compare resource allocation options.
- Monitor strategic initiatives using key performance indicators (KPIs).
- Identify opportunities for innovation and competitive advantage.

2025/2026 2025/2026

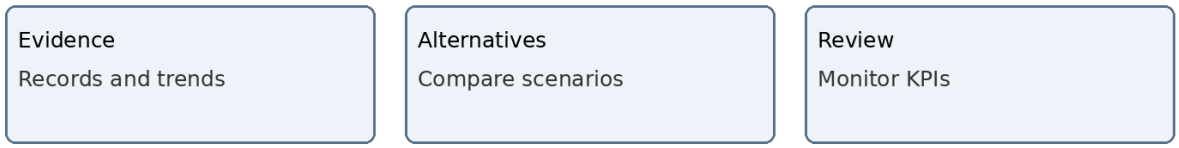


Figure 1.7. Evidence alternatives and review in planning. Original teaching diagram.

Example: a retailer considering a new branch compares demand estimates, operating costs, and possible locations. A what-if analysis examines how the result changes if demand is lower than expected. KPIs can later compare actual performance with the plan. A forecast remains an estimate rather than a guaranteed future outcome.

رومانی صبحی رزق جرجس مطر

1 8 Types of Information Systems

The classical categories below describe different kinds of support. They are useful for understanding the relationship between routine transactions, recurring reports, analytical decisions, and executive oversight. A modern platform may provide several of these capabilities within one application. [6]

Transaction Processing Systems TPS Handle daily routine transactions, such as accepted orders, payments, payroll events, and stock movements. Reliable transaction records provide a foundation for later reporting.

Management Information Systems MIS Provide regular reports and summaries for management, such as monthly sales by branch or budget expenditure by department.

Decision Support Systems DSS Help users analyze complex or non-routine decisions. Interactive models can compare alternatives and explore how results change under different assumptions.

Executive Information Systems EIS Provide high-level summaries and indicators for top executives. They help users review strategic performance and identify issues requiring further investigation.

Category	Illustrative question
TPS	Was this customer payment recorded?
MIS	What were total payments this month?
DSS	How might revenue change under a different pricing plan?
EIS	Are overall results aligned with strategic targets?

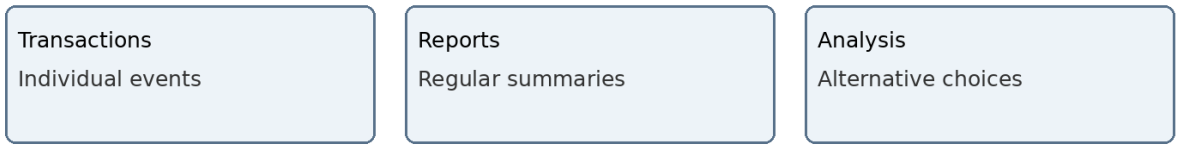


Figure 1.8. Different information needs supported by organizational systems. Original teaching diagram; these capabilities can coexist.

Modern organizations also use specialized systems such as enterprise resource planning, customer relationship management, electronic health records, and geographic information systems. These domain-oriented labels describe what work is supported and do not map exclusively to one classical category.

رومانی صبحی رزق جرجس مطر

1 9 Real World Examples of Information Systems

Information systems operate across industries and affect both organizations and individuals. The examples below connect a system category with its purpose and familiar products or services. Product names illustrate categories; capabilities vary by product and configuration.

System type	Purpose	Common examples
ERP enterprise resource planning	Integrate major business functions and related records	SAP ERP products, Oracle ERP products, Microsoft Dynamics 365
CRM customer relationship management	Manage customer interactions and sales activities	Salesforce, HubSpot
EHR electronic health records	Store and manage patient medical information	Epic, Oracle Health including Cerner Millennium
CMS content management systems	Manage websites and digital content	WordPress, Drupal
GIS geographic information systems	Manage, analyze, and display geographic data	ArcGIS, QGIS
Online banking systems	Allow customers to manage financial accounts	Bank mobile applications and websites

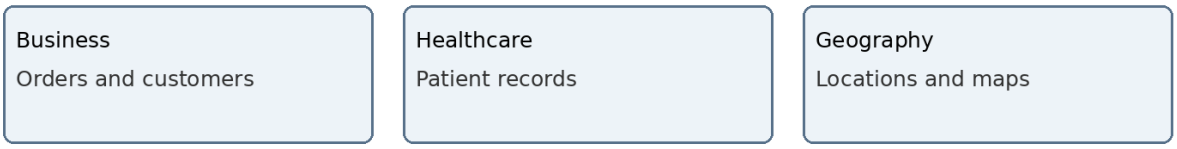


Figure 1.9. Examples of information system application domains. Original teaching diagram.

An ERP service can connect purchasing, inventory, finance, and other organizational activities. A CRM service focuses on customer-related interactions. An EHR supports patient information, while a CMS supports digital content. These systems serve different purposes even when they share technologies such as databases and web interfaces.

GIS supports spatial analysis as well as map display. ArcGIS and QGIS are representative GIS platforms. Google Maps is a familiar mapping and location service, but it should not be treated as equivalent to a full GIS platform. [7]

Cerner products are now associated with Oracle Health. The example above retains Cerner Millennium as a recognizable product name while reflecting that relationship. [8]

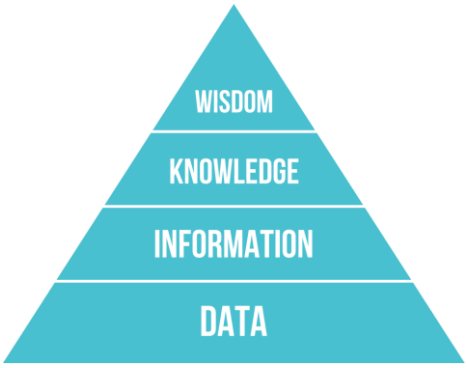
رومانی صیحی رزق جرس مطر

1 10 Chapter Summary

Information systems help organizations collect, process, store, retrieve, and distribute information. They combine hardware, software, data, people, and procedures. Their value depends on the quality of those resources and the way they support organizational work.

They support decisions at operational, tactical, and strategic levels, and can improve efficiency, communication, and coordination. Effective use of information can also contribute to competitive advantage. Security and compliance require suitable safeguards and organizational oversight.

Understanding components, functions, historical development, and strategic importance provides a foundation for more advanced study. Later chapters explore technological foundations, data management, business processes, security, and emerging technologies.



2025/2026 2025/2026 2025/2026

Figure 1.10. Data information knowledge and wisdom as a conceptual hierarchy. Longlivetheux, Wikimedia Commons, CC BY-SA 4.0. Image content unchanged; converted to PNG. Source [12].

Key terms

Term	Meaning
Information System IS	Interrelated resources handling information for a purpose
Hardware	Physical computing and communications devices
Software	Programs and instructions executed by computers
Data	Recorded facts and figures
Procedures	Rules and methods governing system use
Transaction Processing System TPS	System for recording routine transactions
Management Information System MIS	System providing recurring management reports
Decision Support System DSS	System supporting analysis of decision alternatives
Executive Information System EIS	System presenting strategic summaries for executives
Enterprise Resource Planning ERP	Integrated support for major business functions
Customer Relationship Management CRM	Support for customer interactions and sales

References and Image Credits

The following references support further study and identify the sources of the reproduced images. Original teaching diagrams are identified in their captions. The reproduced images retain their original licences; converting file format has not changed their content. Sources were checked on 9 October 2026.

[1] OpenStax Foundations of Information Systems section 1.1. Source of the components figure and organizational hierarchy figure.
https://openstax.org/books/foundations-information-systems/pages/1-1-introduction-to-information-systems

[2] Computer History Museum Timeline of Computer History.
https://www.computerhistory.org/timeline/

[3] Computer History Museum Committee Forms to Develop New Language 28 May 1959.
https://www.computerhistory.org/tdih/may/28/

[4] IBM The IBM PC.
https://www.ibm.com/history/personal-computer

[5] CERN The birth of the Web.
https://home.cern/science/computing/the-birth-of-the-web/

[6] OpenStax Introduction to Business section 13.3 Management Information Systems.
https://openstax.org/books/introduction-business/pages/13-3-management-information-systems

[7] Esri What is GIS.
https://www.esri.com/en-us/what-is-gis/overview

[8] Oracle Oracle Acquires Cerner to Form Oracle Health.
https://www.oracle.com/health/acquisition-of-cerner-the-future-of-healthcare/

2025/2026 2025/2026 2025/2026

[9] Rice University OpenStax Figures 1.2 and 1.3 reused here as Figures 1.2 and 1.5. CC BY 4.0.
https://creativecommons.org/licenses/by/4.0/

[10] Kayau IPO Diagram.svg Wikimedia Commons. CC BY-SA 3.0.
https://commons.wikimedia.org/wiki/File:IPO_Diagram.svg

[11] Unidentified U.S. Army photographer Classic shot of the ENIAC Wikimedia Commons. U. S. Army Photo. Public domain in the United States.
https://commons.wikimedia.org/wiki/File:Classic_shot_of_the_ENIAC.jpg

[12] Longlivetheux DIKW Pyramid.svg Wikimedia Commons. CC BY-SA 4.0.
https://commons.wikimedia.org/wiki/File:DIKW_Pyramid.svg

Creative Commons licence details

https://creativecommons.org/licenses/by-sa/3.0/

https://creativecommons.org/licenses/by-sa/4.0/

Chapter 1 Practice Questions

Choose one best answer for each question. Write its letter in the answer space provided.

Student name _____ Date _____

1. Which statement best describes an information system?

A. A collection of physical computing devices only

B. Interrelated components that collect, process, store, and distribute information

C. A program used exclusively to create documents

D. A communication network without people or procedures

Your answer: _____

2. A shop calculates daily sales totals from individual sales records. What does this illustrate?

A. Replacing all human judgment with technology

B. Removing the need to record transactions

C. Using hardware without procedures

D. Transforming raw data into useful information

Your answer: _____

3. Which item is a hardware component of an information system?

2025/2026 2025/2026 2025/2026

A. A server

B. A payroll application

C. A customer record

D. A backup policy

Your answer: _____

4. A written policy specifies who may enter data and when backups must occur. Which component does it represent?

A. Hardware

B. Data

C. Procedures

D. Software

Your answer: _____

رومانی صبحی رزق جرجس مطر

Chapter 1 Practice Questions Continued

5. An inventory report is incorrect because staff did not record returned goods. What is the best explanation?

A. Accurate software always guarantees accurate reports

B. System effectiveness depends on people, procedures, and data as well as technology

C. Inventory records are unrelated to information systems

D. Replacing the printer will necessarily correct the report

Your answer: _____

6. A checkout scanner captures product identifiers and quantities. Which function is being performed?

A. Data distribution

B. Decision support

C. Data retrieval

D. Data collection

Your answer: _____

7. A customer-service employee opens an existing order using its order number. Which function is this?

A. Data retrieval

B. Data collection

2025/2026 2025/2026 2025/2026

C. Data processing

D. Decision support

Your answer: _____

8. A system sends a monthly performance report to department managers. Which function is most directly illustrated?

A. Data storage

B. Data collection

C. Data distribution

D. Data retrieval

Your answer: _____

رومانی صبحی رزق جرجس مطر

Chapter 1 Practice Questions Continued

9. Which statement about the benefits of information systems is most accurate?

A. Automation prevents every organizational error

B. Benefits depend on data quality, suitable design, and effective management

C. Installing software automatically creates competitive advantage

D. Security software alone guarantees regulatory compliance

Your answer: _____

10. Which activity is primarily operational?

A. Selecting a five-year expansion strategy

B. Assessing entry into an international market

C. Setting the organization's long-term direction

D. Processing employees' monthly payroll transactions

Your answer: _____

11. A middle manager reviews monthly expenditure and reallocates next month's staffing budget. Which organizational level does this mainly represent?

A. Tactical

B. Strategic

C. Operational transaction entry

D. Executive long-term direction

Your answer: _____

12. Which development is most closely associated with the mainframe era described in this chapter?

A. Smartphone-based services

B. Public web shopping platforms

C. Centralized computing and batch processing

D. Widespread personal computer spreadsheets

Your answer: _____

Chapter 1 Practice Questions Continued

13. Which statement correctly distinguishes the World Wide Web from the Internet?

A. The two terms refer to exactly the same technology

B. The Web is a service built on Internet infrastructure

C. The Internet is a service that operates only within the Web

D. The Web replaced all earlier networking technologies

Your answer: _____

14. A retailer changes a demand estimate to see how it affects a proposed branch's profitability. Which technique is being used?

A. Transaction recording

B. Routine data entry

C. Access control

D. What-if analysis

Your answer: _____

15. What is the main purpose of key performance indicators in strategic planning?

A. To monitor progress toward strategic objectives

B. To guarantee that forecasts will be correct

2025/2026 2025/2026 2025/2026

C. To eliminate the need for managerial judgment

D. To replace all internal and external data

Your answer: _____

16. Which system type primarily records routine orders, payments, and stock movements?

A. Executive Information System (EIS)

B. Decision Support System (DSS)

C. Transaction Processing System (TPS)

D. Management Information System (MIS)

Your answer: _____

رومانی صبحی رزق جرجس مطر

Chapter 1 Practice Questions Continued

17. Which system type is most directly associated with regular management reports such as monthly sales by branch?

A. Transaction Processing System (TPS)

B. Management Information System (MIS)

C. Executive Information System (EIS)

D. Decision Support System (DSS)

Your answer: _____

18. Which pairing correctly matches a system type with its main purpose?

A. TPS — comparing complex non-routine alternatives

B. MIS — recording each individual checkout payment

C. EIS — entering individual inventory transactions

D. DSS — analyzing alternatives for complex decisions

Your answer: _____

19. An organization wants to connect purchasing, inventory, and finance through an integrated platform. Which category best fits this need?

A. Enterprise Resource Planning (ERP)

B. Customer Relationship Management (CRM)

C. Content Management System (CMS)

D. Electronic Health Record (EHR)

Your answer: _____

20. Which pairing correctly matches a specialized information system with its purpose?

A. CRM — managing patient medical records

B. CMS — analyzing geographic relationships

C. GIS — analyzing and displaying geographic data

D. EHR — managing website content

Your answer: _____

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

Chapter 2

Information Technology Fundamentals

IS101 | Academic year 2026–2027
Faculty of Computers and Informatics
Tanta University

Instructor and Course Coordinator
Prof. (Assoc.) Hany El-Ghaish

رومانی صبحی رزق جرجس مطر

2 1 Introduction

Information systems depend on technology that can capture data, execute instructions, retain records, and communicate results. Hardware, software, networks, and databases provide these capabilities. People and procedures determine how the resources are selected, operated, and used responsibly. This chapter explains the technological foundation introduced in Chapter 1 and connects individual components to familiar organizational services.

By the end of this chapter, you should be able to identify the elements of a computer-based information system, distinguish RAM from persistent storage, explain the role of an operating system, compare local and wide area networks, and trace a simple instruction through a computer. You should also recognize why processor architecture alone does not determine overall performance.

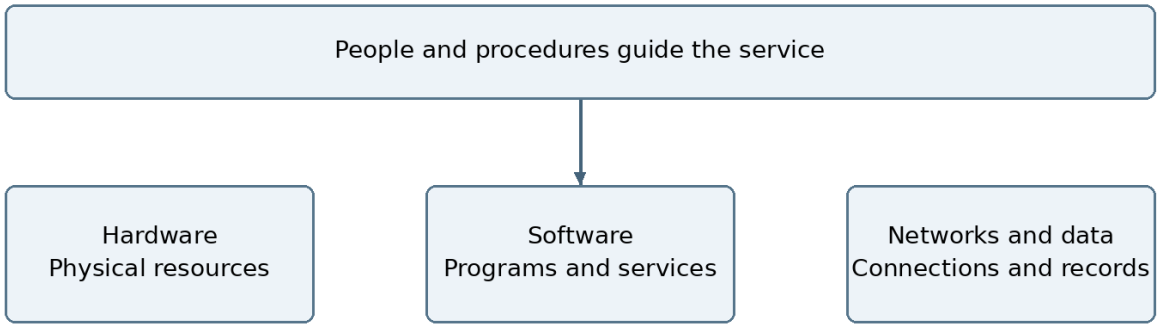


Figure 2.1. Technology supports an organizational service within human and procedural responsibilities. Original teaching diagram.

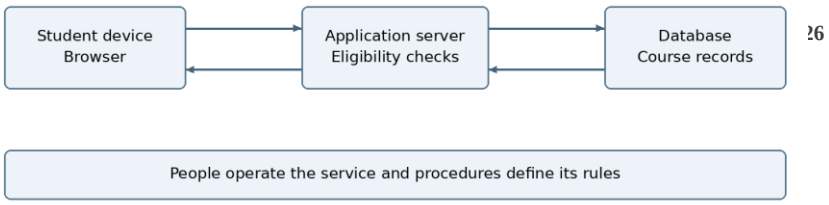
Throughout the chapter, a fictional university registration service illustrates the relationships. A student uses a device and browser, sends a request through a network, and receives a result from an application using stored student and course records.

2 2 Computer Based Information Systems

A computer-based information system (CBIS) is an organized combination of hardware, software, data and databases, telecommunications, people, and procedures used to turn data into useful information. The computer performs some operations, while the broader system includes the people and rules needed to make those operations meaningful. The component framework extends the information-system concepts studied in Chapter 1. [1]

Element	Registration service example
Hardware	Student laptop and application server
Software	Browser registration application and operating systems
Data and databases	Student identities course records and registrations
Telecommunications	Campus Wi-Fi wired links and Internet connection
People	Students advisers developers and administrators
Procedures	Eligibility rules approval steps and correction process

2025/2026



26

Figure 2.2. A simplified registration request and response across a CBIS. Original teaching diagram.

A request travels from the student's browser to the application. The application retrieves relevant records, checks eligibility, records an approved registration, and sends a response. The database retains the records; it is not the same thing as the application interface. Payroll, order entry, inventory control, online banking, and customer relationship management use comparable combinations of resources.

Embedded and pervasive computing

An embedded computer performs a function within a larger product, such as a washing-machine controller, a vehicle control unit, or a medical monitoring device. It can work without Internet access. Pervasive or ubiquitous computing describes computing spread through everyday environments. The terms are related but not interchangeable: a connected embedded sensor can contribute to pervasive computing, while a standalone controller remains an embedded system.

Technology infrastructure

Technology infrastructure consists of shared resources that support several services. A university may share servers, networks, database platforms, technical staff, and operating procedures across registration, payroll, and library systems. An outage in a shared resource can therefore affect more than one application.

2 3 Hardware

Hardware consists of the physical equipment used for computation, storage, input, output, and communication. A device's role depends on what it does in the system. A touchscreen, for example, displays results and accepts input. Hardware operates under software control and must suit the workload it supports. [2]

Component	Role	Examples
CPU	Executes instructions and performs operations	Intel Core AMD Ryzen Apple M-series
RAM	Holds active programs and data temporarily	8 GB 16 GB or 32 GB RAM
Storage	Retains programs and saved data without power	HDD and SSD
Input devices	Capture data and commands	Keyboard mouse scanner microphone
Output devices	Present results	Monitor printer speakers projector
Motherboard	Connects major components through circuits and connectors	CPU socket memory slots expansion interfaces
Expansion and peripheral hardware	Adds capabilities or connects external equipment	Graphics card network adapter external drive

2025/2026 2025/2026 2025/2026

The motherboard provides electrical and communication connections; the power supply supplies electrical power, which circuitry regulates and distributes. A graphics or network card is typically an expansion component. Peripheral is a broader term often used for devices such as printers and external drives, and its usage varies.

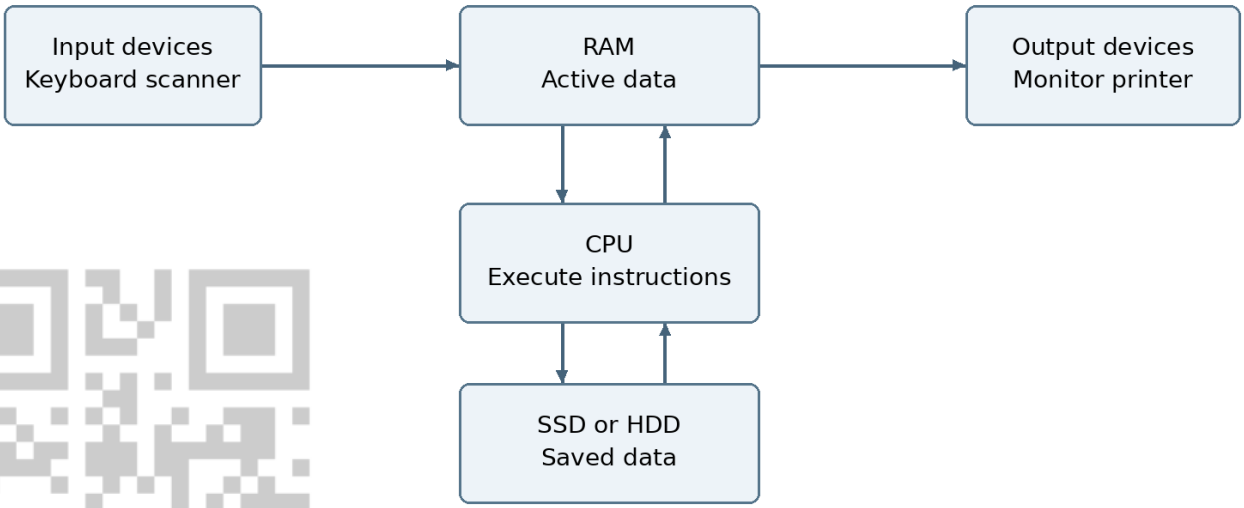


Figure 2.3. Simplified hardware relationships. Arrows show selected data transfers rather than every physical connection. Original teaching diagram.

2 3 Hardware Continued

Hardware performance depends on the combined system. A processor may wait for data from storage; an application may run slowly when its active data exceeds available RAM. More of one resource does not automatically remove a bottleneck elsewhere.

RAM and persistent storage

RAM is working memory. Its contents are normally lost when power is removed, so it is described as volatile. An HDD or SSD retains saved data without power and is described as non-volatile. Non-volatile does not mean indestructible or permanent: files can be deleted and devices can fail. Saving a document writes a persistent copy; editing an unsaved document may leave changes only in working memory.

Property	RAM	HDD or SSD
Main role	Active working area	Persistent files and programs
Power removed	Contents normally lost	Saved contents retained
Typical access speed	Faster than secondary storage	Slower than RAM
Example use	Working on an open spreadsheet	Keeping its saved workbook



Figure 2.4. Loading and saving connect a persistent file with a working copy in RAM. Original teaching diagram.

The upper arrow represents loading a file; the lower arrow represents saving changes. Neither arrow means the file disappears from its original location. A backup is a separate recovery copy, not simply the act of saving the current file.

A practical hardware example

When a student opens a spreadsheet, program instructions and needed data are loaded into RAM. The CPU executes calculations, and the display shows results. Saving writes the workbook to storage. A network adapter can then transfer it to a university submission service. The CPU, RAM, storage, and adapter perform different parts of one task.

رومانی صبحی رزق جرجس مطر

2 4 Software

Software consists of programs and related data that direct computer operations. A useful introductory classification distinguishes system software, application software, and programming tools. These categories can overlap. The operating system belongs to system software; it is not a separate category outside it. [3]

Category	Purpose	Examples
System software	Operate hardware and provide common services	Operating systems device drivers utilities
Application software	Perform tasks for users	Word processors spreadsheets browsers graphics editors
Programming tools	Create translate test and debug programs	Compilers interpreters debuggers IDEs

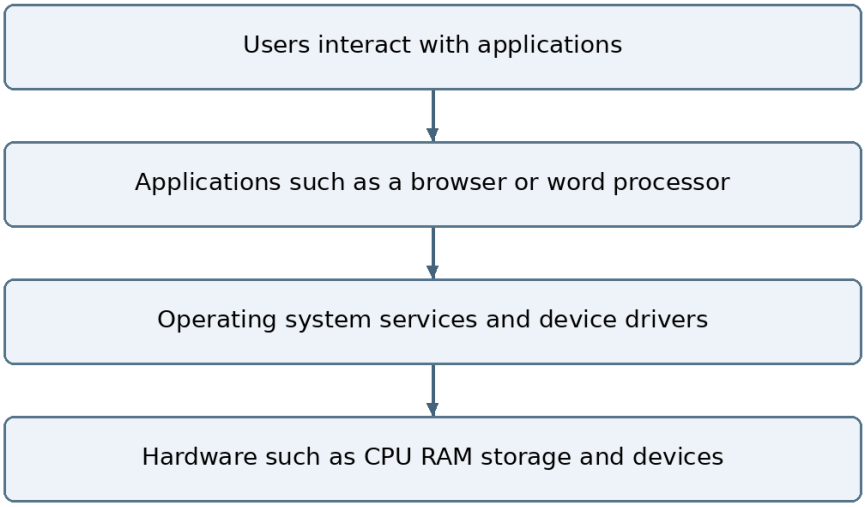


Figure 2.5. Applications use operating-system services to access hardware. Drivers help the OS communicate with devices. Original teaching diagram.

Application software

Applications support activities such as writing, calculation, communication, presentation, and database access. Microsoft Word and Google Docs support document preparation; Excel and Google Sheets support spreadsheet work; Chrome, Firefox, and Edge browse web content. Web applications may execute partly on the user’s device and partly on a remote server.

Utilities drivers and programming tools

A device driver helps the operating system control a particular device. Utilities assist with maintenance or protection, such as backup or malware scanning. A compiler translates source code into another representation, often machine code. An interpreter executes program instructions through an execution environment. An IDE combines facilities such as editing, debugging, and project management; real language implementations may combine compilation and interpretation.

رومانی صبحی رزق جرجس مطر

2 4 Operating Systems and Applications

An operating system coordinates shared resources and supplies services used by applications. Examples include Windows, macOS, Linux, Android, iOS, and Unix systems. Choosing an OS affects software compatibility, device support, administration, and user interaction; no single OS is best for every task.

OS function	What it manages	Example
Process management	Running programs and CPU scheduling	Sharing CPU time between a browser and editor
Memory management	Allocation and protection of memory	Giving programs their own address spaces
File management	Files directories and access permissions	Creating saving and opening a document
Device management	Input and output through drivers	Sending a job to a printer
User interface	Ways to interact with services	Desktop windows or command-line shell

Applications normally request protected services through APIs and system calls. For example, saving a file involves asking the operating system to access storage. Not every application action requires a system call: many calculations run directly in user mode. Multitasking allows multiple tasks to make progress, with scheduling deciding which task uses a processor core. Multiple cores can also execute tasks in parallel.

2025/2026 2025/2026 2025/2026

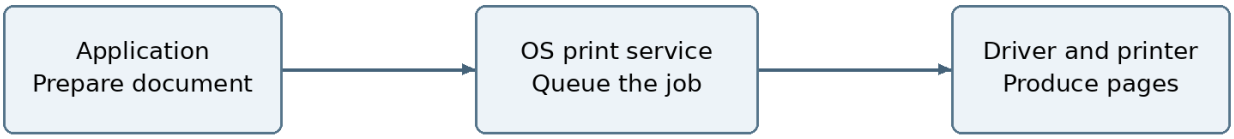


Figure 2.6. A simplified printing request. Each stage has a distinct responsibility. Original teaching diagram.

Memory protection and process isolation help prevent one program from corrupting another. They reduce interference but do not guarantee that software will never fail. A program can still contain errors, and shared resources can become overloaded.

Example of multitasking

A student edits notes while a browser downloads a file. The operating system schedules computation, manages each program's memory, and coordinates network and disk activity. A temporary delay in one application does not necessarily mean that the whole computer has stopped working.

رومانی صبحی رزق جرجس مطر

2 5 Networks

A network connects devices so they can exchange data and share resources. A node is a participating device, and communication protocols define how messages are structured and exchanged. Networking supports shared applications, remote access, communication, backups, and access to Internet services. [4]

Concept	Meaning	Example
Node	Device participating in a network	Laptop server printer or router
Protocol	Agreed rules for communication	IP addressing and TCP transport
Topology	Physical or logical arrangement of connections	Star bus ring or mesh
LAN	Network within a limited local area	Office or campus network
WAN	Network connecting locations across a wider area	Links between university campuses
Internet	Global interconnected network of networks	Infrastructure used to reach remote services

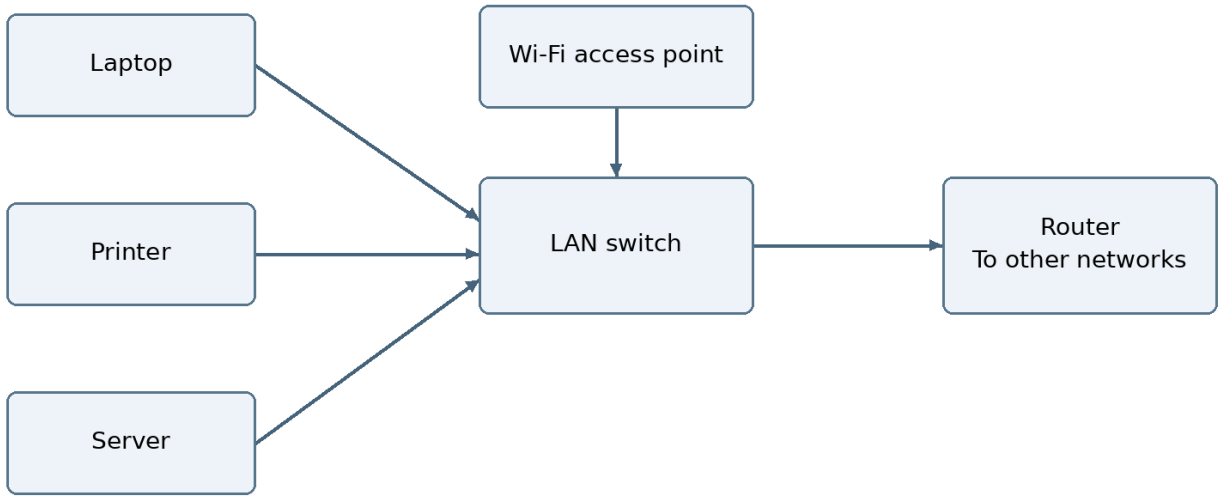


Figure 2.7. A local network can reach other networks through a router. A Wi-Fi access point connects wireless devices. Original teaching diagram.

A switch typically forwards frames between devices within a LAN. A router forwards packets between IP networks. A wireless access point gives Wi-Fi devices access to a local network. A home device may combine routing, switching, and wireless access in one enclosure. A LAN can work without Internet access, for example when sharing a local printer.

رومانی صبحی رزق جرجس مطر

2 5 Network Scope and Topology

LAN and WAN describe network scope, while topology describes how connections are arranged. A large campus LAN may support thousands of users; local does not mean that it must contain only a few devices. A WAN connects more widely separated locations and may use leased links or provider services. The Internet is a global example of interconnected networks. [4]

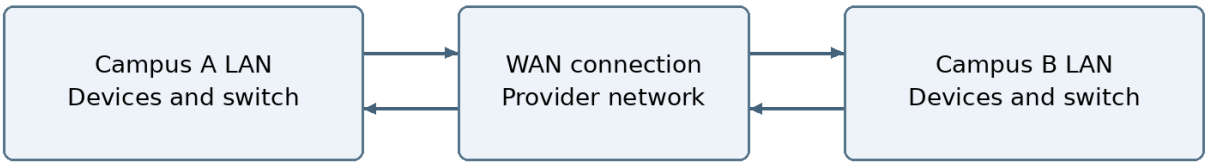


Figure 2.8. Two local networks connected across a wide area. The WAN is the intersite connection, not an individual office device. Original teaching diagram.

Common topologies

A star connects devices to a central connection point. A bus uses a shared communication path; a ring links devices around a closed loop; a mesh provides multiple interconnections. These are simplified patterns: real networks can combine them, and logical message flow may differ physical wiring.

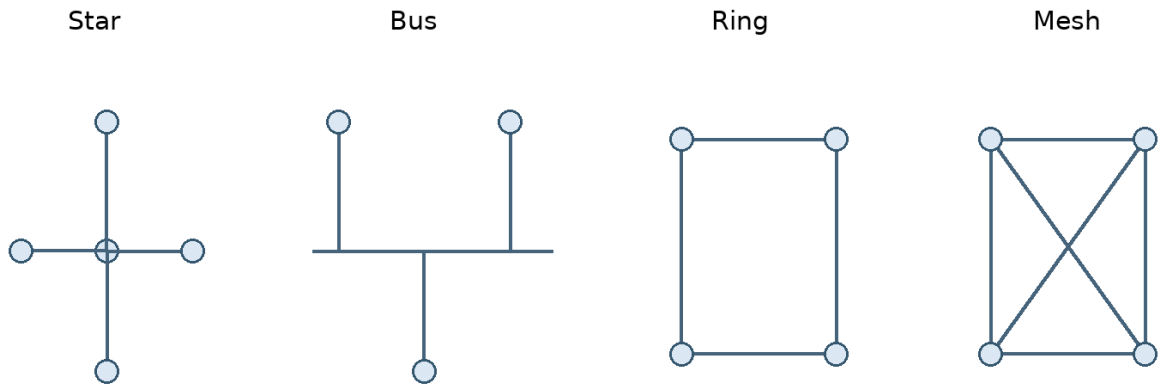


Figure 2.9. Simplified topology patterns. Circles represent nodes and lines represent links. Original teaching diagram.

TCP/IP names a family of protocols rather than one protocol. IP supports addressing and packet delivery between networks; TCP provides a reliable ordered byte stream for applications that use it. Other applications use different transport protocols. A protocol is a communication rule set, not the cable or wireless signal itself.

رومانی صبحی رزق جرجس مطر

2 6 Computer Architecture and Components

Computer architecture concerns the design of a computer and the interface its hardware provides to software. Computer organization concerns how components implement that design. At an introductory level, both help explain how the CPU, memory, input/output devices, and interconnections cooperate. The diagrams in this section are simplified models rather than complete engineering schematics. [5]

Inside the CPU

The arithmetic logic unit (ALU) performs arithmetic and logical operations, such as addition and comparisons. The control unit coordinates instruction execution and control signals. Registers hold small amounts of immediately needed information inside the CPU, such as operands or an instruction address. Modern processors contain multiple execution units, but these three concepts remain useful starting points.

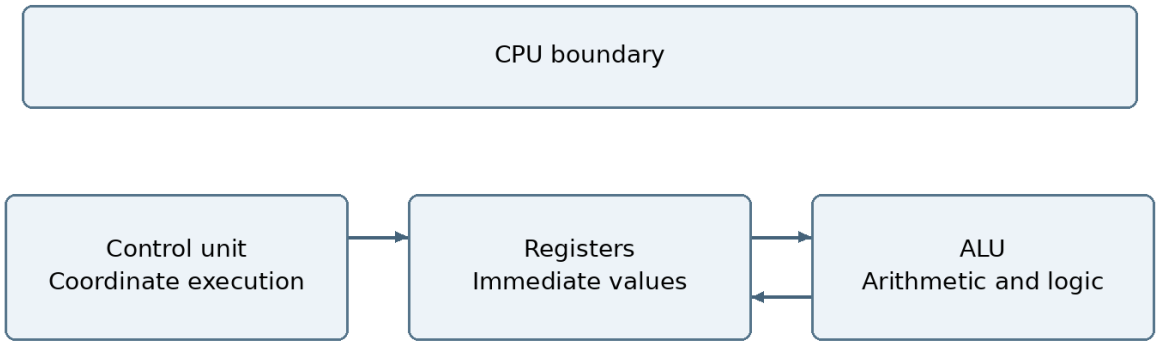


Figure 2.10. Conceptual CPU components exchange operands, results, and control information. Original teaching diagram.

Buses and input output

Interconnections transfer data and control information between components. Traditional descriptions distinguish data buses, address buses, and control signals. Modern computers also use point-to-point links and specialized interfaces, so a single shared bus is a teaching simplification. Input/output controllers coordinate exchanges with devices; the CPU does not manually move every byte of every transfer.

A keyboard event can be handled by device hardware, a driver, and the operating system before an application receives it. A storage controller can transfer blocks of data into RAM. These mechanisms explain why real data flow is more detailed than the basic input–process–output model.

رومانی صبحی رزق جرجس مطر

2 6 Memory Hierarchy and Instruction Execution

The memory hierarchy balances access speed, capacity, and cost. Registers are small and very fast; caches retain frequently used instructions or data close to the CPU; RAM holds larger working sets; secondary storage retains much larger amounts persistently. Typical speed and capacity relationships are trends, not fixed sizes for every device.

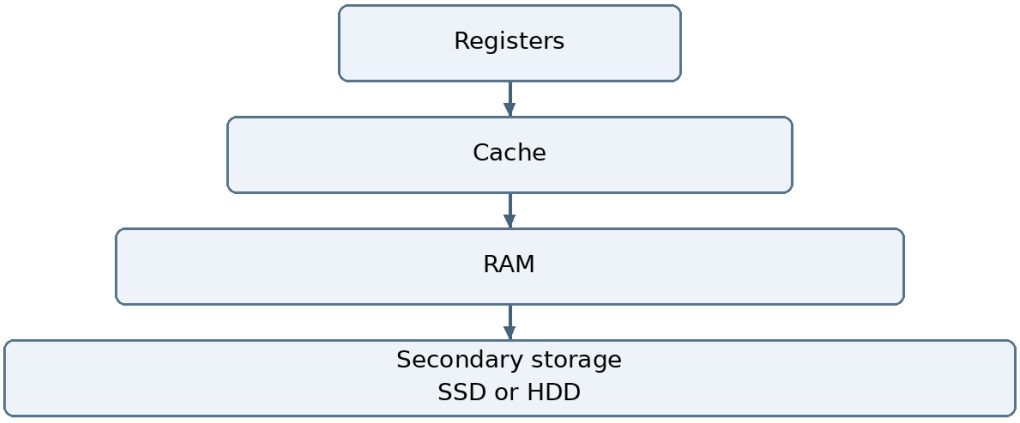


Figure 2.11. Typical memory hierarchy. Moving downward generally increases capacity and access time. Original teaching diagram.

Registers, cache, and ordinary RAM are normally volatile. Secondary storage is normally non-volatile. Cache reduces the need to wait for slower memory when recently or frequently used data is needed again. A faster CPU does not remove every delay caused by missing data or slow storage.

Fetch decode execute

The CPU fetches an instruction from memory, decodes its operation, and executes it. Execution may perform arithmetic, access memory, or change the next instruction address. Results may go to a register or memory, depending on the instruction. This conceptual cycle repeats while a program runs; real processors may overlap stages and execute several operations concurrently.

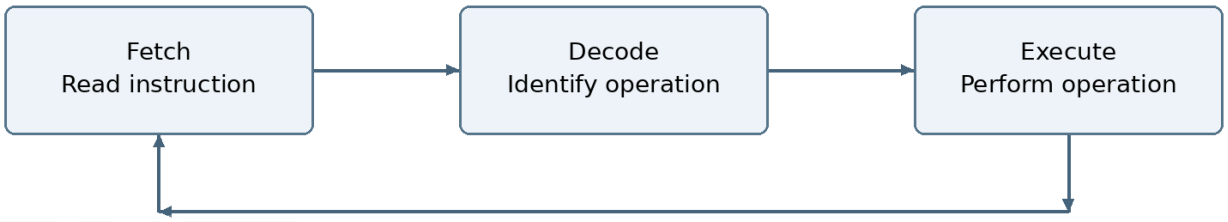


Figure 2.12. A conceptual instruction cycle. After execution, processing continues with the next required instruction. Original teaching diagram.

Worked example: to calculate 5 + 3, a program supplies instructions and operands. A simplified execution loads operands into registers, uses the ALU to produce 8, and retains the result. An application can later format and display it through OS services. The ALU computes the sum; the monitor presents its representation.

2 6 Machine Language and Instruction Sets

Machine instructions are encoded as binary bit patterns interpreted according to a processor's instruction set architecture (ISA). The ISA specifies operations, registers, data representations, and other behavior visible to software. An executable built for one ISA does not automatically run natively on another; translation or emulation may be needed. [6]

Programmers usually write high-level source code. Translation and execution tools convert or implement its operations using the target system's instructions. A source statement such as total = price * quantity can require several machine instructions. The binary encoding of a particular instruction depends on its ISA; an arbitrary sequence of zeros and ones has no universal instruction meaning.

Aspect	RISC tendency	CISC tendency
Design emphasis	Regular relatively simple instruction operations	Rich instruction set including more complex operations
Memory operations	Typically explicit load and store instructions	Some instructions operate directly on memory operands
Instruction encoding	Often regular but varies across ISAs	Often variable length in common x86 designs
Representative families	Arm and RISC-V	x86
Performance conclusion	Depends on implementation and workload	Depends on implementation and workload

RISC means Reduced Instruction Set Computer; CISC means Complex Instruction Set Computer. These labels describe broad design traditions, not a rule that one executes every program faster. Clock rate, core design, cache, compiler, power constraints, and the workload affect performance. Modern processors may combine ideas from both traditions, and instruction counts alone are insufficient for comparing speed.

A useful performance question

If two laptops take different times to process the same spreadsheet, investigate the complete task: processor capabilities, available RAM, storage activity, software, and background work. Knowing only that one processor is RISC and the other is CISC does not establish which computer will finish first.

2 7 Summary of Technology Infrastructure

Technology infrastructure brings the components together as shared organizational resources. Hardware supplies physical capacity; software directs operations; networks connect services; databases organize records; people operate and improve the systems; procedures define approved ways of working. These elements must be coordinated, since a failure in one can limit the usefulness of the whole service.

Resource	Contribution	University example
Hardware	Physical processing and storage	Servers student devices backup equipment
Software	Execution and resource management	OS registration application database software
Networks	Communication and access	Campus wired and Wi-Fi connections
Databases	Organized persistent records	Student course and registration tables
People	Use development and support	Students advisers developers administrators
Procedures	Consistent rules and operations	Access approvals updates and recovery steps

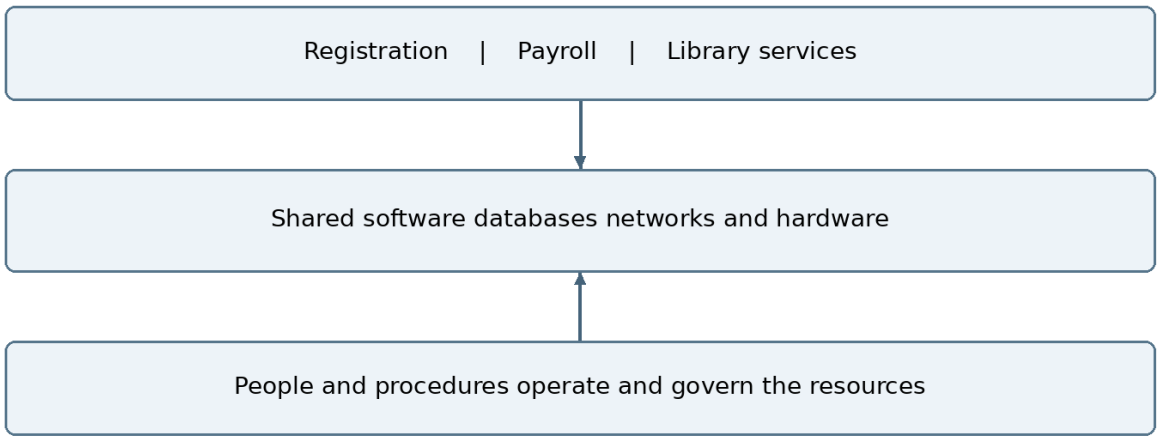


Figure 2.13. Shared infrastructure supports multiple services. People and procedures govern all layers. Original teaching diagram.

In the registration example, the network carries a request, the application checks rules, and the database retains the approved record. Administrators maintain the platforms, while approved procedures govern corrections and access. A database is an organized collection of data; a database management system (DBMS) is software used to manage it. Later study examines database design and transactions in greater depth.

2 8 Chapter Summary

A CBIS combines technological and organizational resources to support an information need. Hardware includes processing, working memory, persistent storage, and input/output equipment. RAM supports active work, while saved files reside on non-volatile storage. The operating system is system software that manages shared resources and provides services to applications.

Networks enable resource sharing and communication. LAN and WAN describe geographic scope, topology describes arrangement, and protocols describe communication rules. Computer architecture explains the interaction of CPU components, memory, and input/output. Registers, caches, RAM, and secondary storage form a typical hierarchy. Fetch, decode, and execute describe instruction processing at a conceptual level.

RISC and CISC identify instruction-set design traditions rather than guaranteed performance rankings. Understanding the combined infrastructure provides a basis for studying data management, business applications, security, and emerging technologies.

Key Terms

Term	Meaning
CBIS	Computer-based information system
Technology infrastructure	Shared resources supporting organizational systems
Hardware	Physical computing and communication equipment
Software	Programs and related data directing operations
Operating system	System software managing resources and services
Application software	Programs supporting user tasks
Network	Connected devices exchanging data
LAN and WAN	Local area network and wide area network
CPU	Central processing unit executing instructions
RAM	Volatile working memory
Computer architecture	Design and software-visible behavior of a computer
ISA	Instruction set architecture
RISC and CISC	Broad instruction-set design traditions

رومانی صیحی رزق جرجس مطر

References and Figure Credits

These sources support further reading and the technical explanations. All thirteen figures are original teaching diagrams prepared for this chapter. They are simplified conceptual illustrations rather than vendor schematics. Sources checked on 9 October 2026.

[1] OpenStax. Foundations of Information Systems section 1 1.

https://openstax.org/books/foundations-information-systems/pages/1-1-introduction-to-information-systems

[2] IBM. What is Computer Hardware.

https://www.ibm.com/think/topics/hardware

[3] IBM. What is an Operating System.

https://www.ibm.com/think/topics/operating-systems

[4] Cisco. LAN WAN and networking concepts.

https://www.cisco.com/site/us/en/learn/topics/networking/what-is-a-lan-local-area-network.html

[4a] Cisco. What is a WAN.

https://www.cisco.com/site/us/en/learn/topics/networking/what-is-a-wan-wide-area-network.html

[4b] Cisco. What is a Switch vs a Router.

https://www.cisco.com/site/us/en/learn/topics/small-business/network-switch-vs-router.html

[5] OpenStax. Introduction to Computer Science section 5 1 Computer Systems Organization.

https://openstax.org/books/introduction-computer-science/pages/5-1-computer-systems-organization

[5a] OpenStax. Introduction to Computer Science section 6 2 Fundamental OS Concepts.

https://openstax.org/books/introduction-computer-science/pages/6-2-fundamental-os-concepts

[6] Arm. What is Instruction Set Architecture.

https://www.arm.com/glossary/isa

رومانی صبحی رزق جرجس مطر

Chapter 2 Practice Questions

Choose one best answer for each question. Write its letter in the space provided.

Student name _____ Date _____

1. Which description best defines a CBIS?

A. A computer with no users or operating rules

B. A coordinated combination of technology, people, and procedures that processes data

C. A storage device used only for backups

D. A network cable connecting two printers

Your answer: _____

2. Which example best illustrates shared technology infrastructure?

A. A student's single unsaved note

B. The text of one customer email

C. One mark on an examination paper

D. A network and server platform supporting registration and payroll

Your answer: _____

3. A washing machine uses a controller to manage its wash cycle without Internet access. This is an example of what?

2025/2026 2025/2026 2025/2026

A. Embedded computing

B. A wide area network

C. A web browser

D. A compiler

Your answer: _____

4. Which hardware component normally loses its working contents when power is removed?

A. SSD

B. HDD

C. RAM

D. Saved backup drive

Your answer: _____

رومانی صبحی رزق جرجس مطر

Chapter 2 Practice Questions Continued

5. Which action most directly preserves changes to a document beyond loss of power?

A. Increasing screen brightness

B. Saving the document to non-volatile storage

C. Leaving the document open in RAM

D. Adding another mouse

Your answer: _____

6. Which device can serve as both an input and an output device?

A. Microphone

B. Printer

C. Keyboard

D. Touchscreen

Your answer: _____

7. Which CPU component performs arithmetic and logical operations?

A. ALU

B. Network adapter

C. File system

2025/2026 2025/2026 2025/2026

D. Printer controller

Your answer: _____

8. Which statement correctly describes the operating system?

A. It is a physical component on the motherboard

B. It is only a word-processing application

C. It is system software that manages resources and provides services

D. It replaces the need for application software

Your answer: _____

رومانی صبحی رزق جرجس مطر

Chapter 2 Practice Questions Continued

9. Which program is primarily application software?

A. A device driver

B. A spreadsheet editor

C. The operating-system scheduler

D. A memory-management component

Your answer: _____

10. What is a device driver primarily used for?

A. Writing an organization's approval policy

B. Permanently storing every active calculation

C. Replacing all application programs

D. Helping the OS communicate with a particular device

Your answer: _____

11. Which tool typically translates source code into another representation such as machine code?

A. Compiler

B. Monitor

C. Router

2025/2026 2025/2026 2025/2026

D. RAM module

Your answer: _____

12. A student edits a file while a browser downloads material. Which OS capability most directly supports progress in both tasks?

A. Changing network topology

B. Replacing persistent storage with a keyboard

C. Multitasking and scheduling

D. Removing all device drivers

Your answer: _____

رومانی صبحی رزق جرجس مطر

Chapter 2 Practice Questions Continued

13. Which scenario best represents a LAN?

A. A provider link connecting offices in several countries

B. An office network connecting computers and a shared printer

C. Every network on the global Internet

D. A processor's internal registers

Your answer: _____

14. Which device typically forwards packets between different IP networks?

A. Keyboard

B. Display

C. Local storage drive

D. Router

Your answer: _____

15. What does a communication protocol specify?

A. Rules for exchanging data

B. The color of a network cable

C. The physical size of a processor

2025/2026 2025/2026 2025/2026

D. The number of keys on a keyboard

Your answer: _____

16. Which topology connects devices to a central connection point?

A. Ring

B. Bus

C. Star

D. Fully connected mesh

Your answer: _____

رومانی صبحی رزق جرجس مطر

Chapter 2 Practice Questions Continued

17. Which ordering follows the typical hierarchy from fastest and smallest toward slower and larger storage?

A. SSD RAM cache registers

B. Registers cache RAM secondary storage

C. RAM secondary storage registers cache

D. Cache registers secondary storage RAM

Your answer: _____

18. In the conceptual instruction cycle, which stage identifies the operation specified by an instruction?

A. Save

B. Distribute

C. Backup

D. Decode

Your answer: _____

19. Which statement about RISC and CISC is most accurate?

A. They describe design traditions; performance depends on implementation and workload

B. RISC always executes every program faster

C. CISC cannot perform logical operations

2025/2026 2025/2026 2025/2026

D. They describe LAN and WAN geographic scope

Your answer: _____

20. Which statement correctly distinguishes a database from a DBMS?

A. A database is hardware and a DBMS is a network cable

B. They are two names for the CPU's registers

C. A database is an organized data collection; a DBMS is software that manages it

D. A DBMS eliminates all procedures and users

Your answer: _____

رومانی صبحی رزق جرجس مطر

Chapter 3

Data and Information Management

IS101 | Academic year 2026–2027
Faculty of Computers and Informatics
Tanta University

Instructor and Course Coordinator
Prof. (Assoc.) Hany El-Ghaish

رومانی صیحی رزق جرجس مطر

3 1 Introduction

Organizations use data to record activities, coordinate services, and evaluate results. A university needs reliable enrollment records; a retailer needs dependable sales and stock records; a hospital needs accurate patient information. The technology described in Chapter 2 provides a foundation, but its value depends on the quality, meaning, availability, and appropriate use of the records it holds.

This chapter explains how data is collected, stored, cleaned, integrated, protected, and governed. It then examines how analysis, transformation, visualization, business intelligence, and knowledge sharing turn records into useful evidence. A fictional campus bookshop provides a recurring example so that students can follow the same data through several activities.

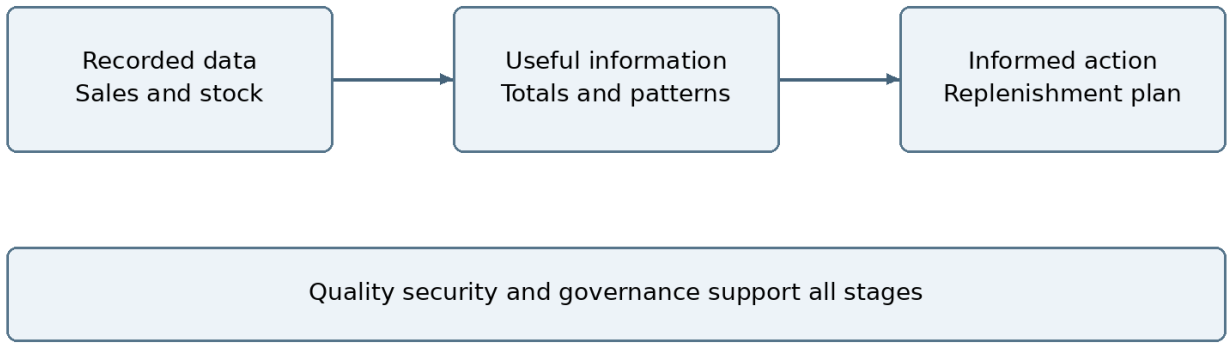


Figure 3.1. Records become useful through preparation, interpretation, and responsible application. Original teaching diagram.

Learning objectives

After studying this chapter, you should be able to distinguish data from information, explain the main management activities, diagnose common quality problems, compare storage approaches, and interpret a simple report. You should also be able to describe governance responsibilities, distinguish security from privacy, and explain the limits of conclusions drawn from a small dataset.

3 2 Data versus Information

Data consists of recorded observations, measurements, or facts. It may already be structured, such as rows in a transaction table, or may appear in less structured forms such as images and text. Information is data interpreted and organized in a context that answers a question. The distinction depends partly on the purpose and the reader; data does not have to be disorganized to qualify as data.

Concept	Meaning	Bookshop example
Data	Recorded facts available for interpretation	A transaction with customer Ahmed amount 1500 and a date
Information	Data with relevant context and meaning	Ahmed purchased goods worth 1500 EGP on 12 May 2026

The number 1500 alone could represent a price, a quantity, or an identifier. Naming the measure, currency, event, and date makes the record interpretable. A date such as 12/05/2026 can be ambiguous across conventions. The wording 12 May 2026, or an agreed year-month-day format such as 2026-05-12, reduces that ambiguity.



Figure 3.2. Labels and context explain what a record means. Original teaching diagram.

A worked example

Suppose three sales on one day have values of 200 EGP, 300 EGP, and 500 EGP. Their total is 1000 EGP and the average transaction value is 1000 ÷ 3 = 333.33 EGP, rounded to two decimal places. To use these results, specify the shop, date, included transactions, and treatment of returns. A correct sum of incomplete records can still misrepresent the day's sales.

Context matters more than appearance

A polished table becomes useful when its fields and scope are understood. Processing may involve calculation, but interpretation can also arise by labeling or organizing existing records. There is no universal rule that every fact must pass through an arithmetic operation before it becomes information.

رومانی صبحی رزق جرجس مطر

3 3 Data Management and the Data Lifecycle

Data management is the coordinated work needed to keep data usable throughout its lifecycle. It includes acquisition, storage, maintenance, integration, protection, and eventual archiving or deletion. Management decisions should reflect the purpose of the data, who needs it, how long it is needed, and which responsibilities apply. [1]



Figure 3.3. A simplified lifecycle. Quality checks and governance apply throughout, and errors can trigger correction at earlier stages. Original teaching diagram.

1 Data collection

Sources include user forms, applications, transactional systems, sensors and Internet of Things devices, external databases, and web or social-media sources. Collection begins by defining which observations are needed. A sales record may require a transaction identifier, product identifier, quantity, unit price, and timestamp. Collecting unrelated personal details adds responsibilities without necessarily improving the service.

Designing a reliable capture process

A clear form defines required fields, accepted units, and valid ranges. Automated systems also need checks: a sensor can malfunction and an external feed can change its format. Record the source and collection time so that later users can assess reliability. Metadata means descriptive information about data, such as a field's meaning, unit, source, and update schedule.

Example: a stock sensor reports 45 without a unit. Does this mean 45 items, kilograms, or boxes? A metadata definition resolves the uncertainty. The capture process should also explain whether the value is measured, entered manually, or estimated.

2025/2026 2025/2026 2025/2026

3 3 Data Storage

Storage should match the type, scale, access pattern, and intended use of the data. A relational database organizes related records in tables. A warehouse typically integrates prepared data for reporting and analysis. A lake can retain varied data, including raw files, for later processing. These approaches can coexist rather than compete as universal alternatives. [2]

Storage approach	Typical purpose	Example
Relational database	Maintain structured operational records and relationships	Sales orders and products
Data warehouse	Support integrated historical reporting and analysis	Monthly sales across branches
Data lake	Retain varied data for different processing needs	Logs images documents and raw feeds
Cloud storage	Provide storage through hosted infrastructure or services	Hosted object storage or managed database
File system	Organize files in directories	CSV exports images and documents

2025/2026 2025/2026 2025/2026

Cloud describes where or how a service is delivered, while database, warehouse, lake, and file system describe organizational or technical approaches. A cloud service can host a database or warehouse. A lake still needs access controls, cataloging, and quality practices; retaining a large collection does not make its contents easy to find or trustworthy.

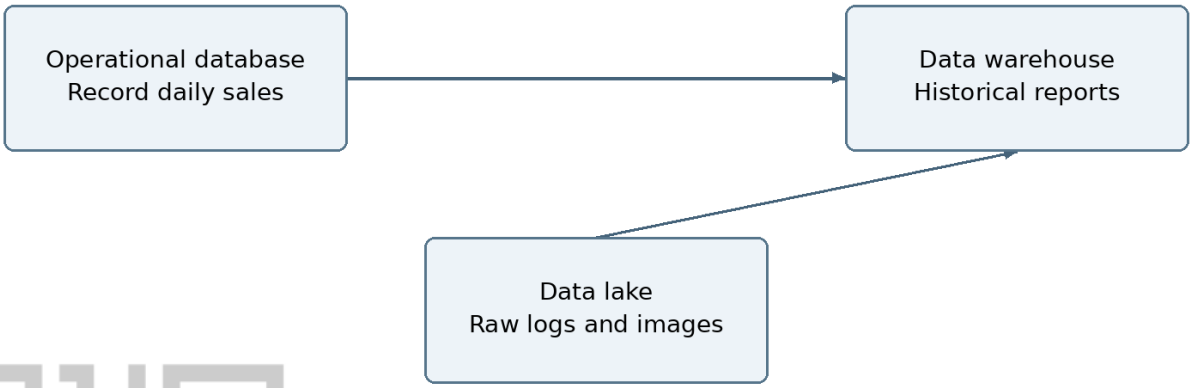


Figure 3.4. Operational and analytical storage support different tasks. The arrows show selected data movements. Original teaching diagram.

The bookshop records accepted sales in an operational database. Prepared summaries can feed a warehouse, while images or raw activity logs may be retained in a lake. The arrows indicate possible transfers after appropriate preparation, not an obligatory architecture for every organization.

3 3 Data Cleaning and Quality Assurance

Data cleaning investigates and resolves problems that affect intended use. Quality assurance includes preventing errors, applying checks, documenting corrections, and monitoring quality over time. Common dimensions include accuracy, completeness, consistency, validity, uniqueness, and timeliness. A valid format does not prove that a value is factually correct. [3]

Problem	Example	Appropriate response
Missing value	Quantity not recorded	Check the source or flag the record; do not invent a value
Duplicate record	Same sale imported twice	Check transaction identifiers before removing copies
Inconsistent format	Dates use different conventions	Apply an agreed format after interpreting the source
Incorrect value	Negative quantity in a normal sale	Investigate whether it is an error or a return
Outdated value	Old customer contact details	Verify and update through an approved process

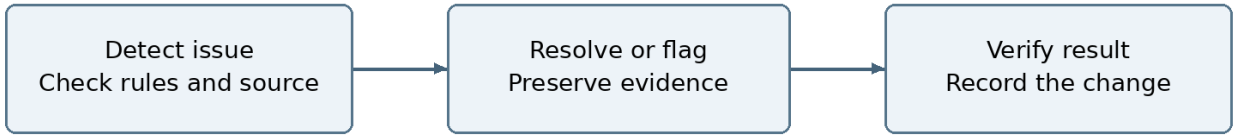


Figure 3.5. Quality checks should lead to documented decisions and verification. Original teaching diagram.

A duplicate changes a report

The bookshop has transactions T01 = 200 EGP, T02 = 300 EGP, and T03 = 500 EGP. If T03 is imported twice, the reported total becomes 1500 EGP instead of 1000 EGP. The error overstates the correct total by 500 EGP, or 50%. Checking the transaction identifier and source confirms that the extra row is a duplicate.

Two purchases of the same value by the same person are not necessarily duplicates. Likewise, a missing quantity is not equivalent to zero, and an unusual value is not automatically wrong. Keep original records or an audit trail so that a correction can be explained and reviewed.

رومانی صبحی رزق جرجس مطر

3 3 Data Integration

Data integration combines records from multiple sources into a usable view. It can reduce silos by connecting related information, but it requires agreement about identifiers, definitions, units, and update timing. Simply copying tables into one place does not resolve conflicting meanings or access responsibilities.

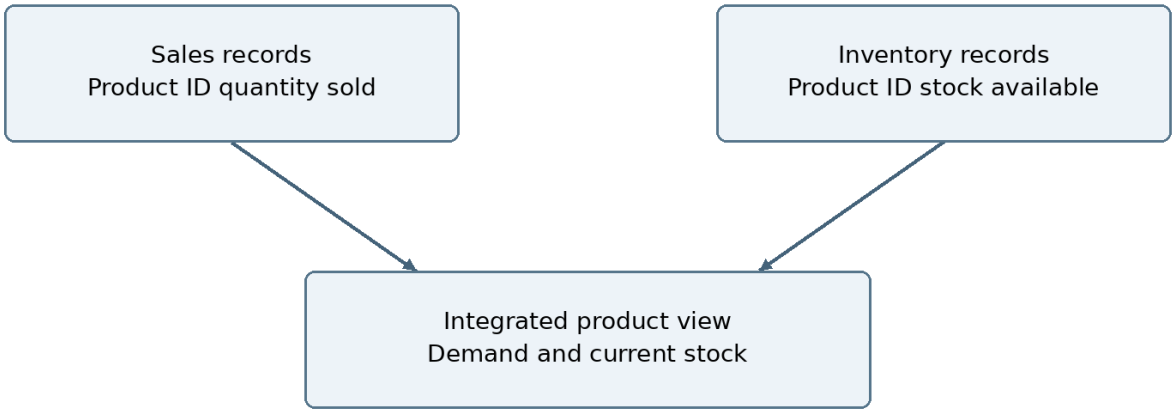


Figure 3.6. Shared product identifiers connect sales with stock records. A unified view depends on agreed definitions. Original teaching diagram.

Matching related records

A product identifier provides a more dependable match than an informal product name. One system may use "Intro IS Book" and another "Information Systems Introduction," even though they refer to the same item. A mapping can link source identifiers to an agreed master identifier. Such mappings need maintenance when products or systems change.

ETL and ELT

Extract, transform, load (ETL) means retrieving source data, preparing it, and then loading it into a target. Extract, load, transform (ELT) loads retrieved data before transformations occur in the target environment. The order differs; either approach still requires quality checks, security, and documentation. Transformations can include converting units, aligning field names, and resolving identifiers.

Example: sales uses quantities in individual books, while a supplier file uses cartons of 10 books. Integrating the records requires an explicit conversion. Three cartons correspond to 30 books; treating the number 3 as three individual books would distort the replenishment decision.

Lineage records where data came from and which transformations were applied. It helps answer why a dashboard total differs from a source report. Integration should also respect access rules: a shared analytical view need not expose every personal field from its sources.

2025/2026 2025/2026 2025/2026

3 3 Data Security and Governance

Data security protects confidentiality, integrity, and availability. Confidentiality concerns appropriate access; integrity concerns protection from improper alteration; availability concerns access when needed. Security controls work together with procedures and trained staff. Data governance defines responsibilities and standards for managing data across the organization. [4]

Control	Main contribution	Bookshop example
Authentication	Verify a claimed identity	Require a staff sign-in
Authorization	Limit permitted actions and records	Allow cashiers to record sales but restrict payroll access
Encryption	Protect readable content without an appropriate key	Encrypt sensitive files and communications
Backup and recovery	Restore data after loss or failure	Test recovery of sales records
Firewalls and monitoring	Restrict traffic and detect suspicious activity	Review unusual access attempts

Authentication asks who a user is; authorization asks what that user may do. Backups support recovery, but a backup that has never been tested may not restore successfully. Encryption protects content but does not correct inaccurate values or grant an appropriate right to use the data.

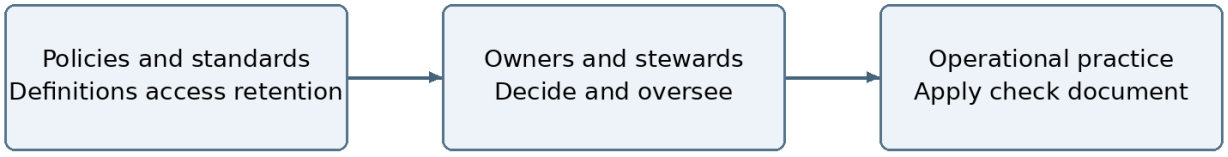


Figure 3.7. Governance connects standards, responsibility, and daily implementation. Original teaching diagram.

Responsibilities and accountability

A data owner is accountable for decisions about a data domain, such as approved access. A data steward helps maintain definitions, quality rules, and issue resolution. Technical custodians operate platforms and controls. Titles vary across organizations, so the responsibilities must be stated explicitly. Governance supports compliance, but a policy document alone does not establish compliance.

3 4 Information Management

Information management organizes the interpretation, presentation, distribution, and use of information. It overlaps with data management rather than forming a completely separate sequence. A reliable report depends on managed source records; user feedback about a misleading report may reveal a source-data problem.

1 Data analysis

Analysis examines patterns, trends, relationships, and anomalies in relation to a question. Descriptive analysis asks what happened, while diagnostic analysis investigates possible explanations. Predictive analysis estimates possible future outcomes. A pattern can guide further investigation, but an association does not establish that one factor caused another.

2 Data transformation

Transformation changes the representation or organization of data. Filtering selects records, aggregation combines them, and calculations derive measures. Scaling can place numeric variables on comparable ranges. The word normalization has different meanings: numeric scaling is different from relational database normalization, which organizes tables to reduce redundancy and update problems.

Technique	Operation	Bookshop example
Filtering	Select records satisfying a condition	Keep sales from May 2026
Aggregation	Combine records into summaries	Sum sales by day or product
Derived metric	Calculate a new measure	Revenue = quantity × unit price
Standardization	Align representations or units	Express all amounts in EGP

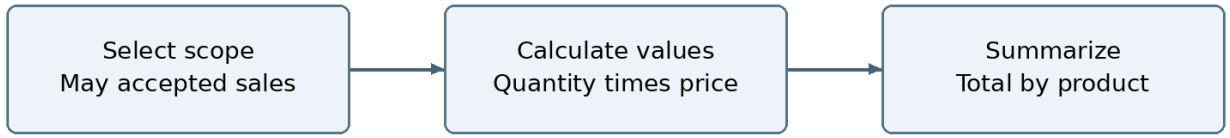


Figure 3.8. A report can filter, calculate, and aggregate records to answer a defined question. Original teaching diagram.

For three identical books sold at 100 EGP each, transaction revenue is 3 × 100 = 300 EGP. The sum of transaction revenue gives total revenue for the selected scope. This is not profit: profit requires relevant costs and other accounting definitions. Name measures carefully before using them in decisions.

رومانی صبحی رزق جرجس مطر

3 4 Visualization and Business Intelligence

Visualization presents information through charts, dashboards, and maps. Match the display to the question: bars compare categories, lines show ordered time trends, and maps show geographic patterns. Labels should identify the measure, unit, period, and source. A visually attractive display can still mislead if it uses an unsuitable scale or mixes incomparable values.

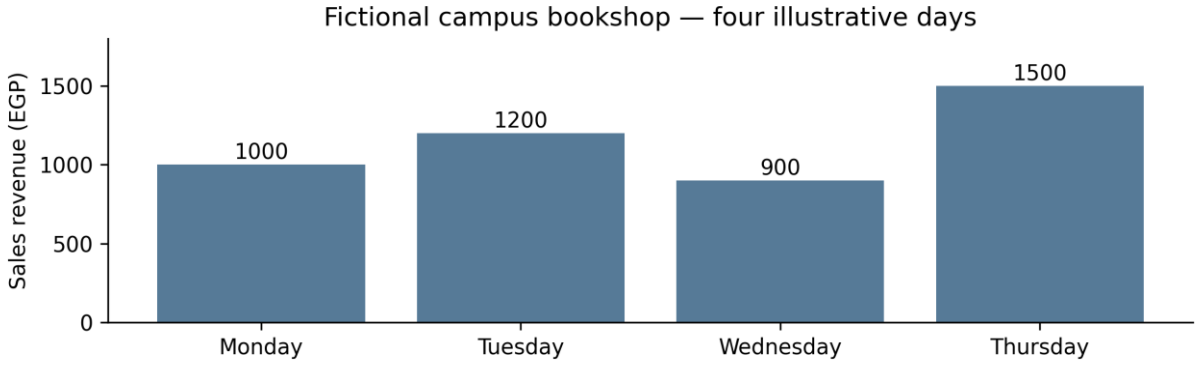


Figure 3.9. An original chart of fictional teaching data. The zero baseline supports comparison of bar lengths. Sales revenue does not establish profit.

Thursday has the largest displayed revenue, 1500 EGP. The four-day total is 4600 EGP. The chart does not establish why revenue changed, whether other days were similar, or whether profit increased. A short sample should not be treated as a complete picture of the business.

2025/2026 2025/2026 2025/2026

3 Data visualization and interpretation

Check both the visual and its underlying definitions. A dashboard should show its latest refresh time when currentness matters. Changing a filter changes the population being summarized; readers need to know whether they are seeing all branches or only one.

4 Business intelligence

Business intelligence (BI) brings together data access, analysis, reporting, and performance monitoring for organizational decisions. Capabilities can include interactive dashboards, ad-hoc reports, and analytical exploration. Microsoft Power BI is one example of a BI platform. Features differ by product and configuration, and a dashboard alone cannot guarantee an appropriate decision. [5]

A bookshop manager may filter sales by product and compare results with stock availability. The analysis is useful only if identifiers match, return rules are clear, and refresh timing is understood. Users should be able to trace a displayed total to its supporting records.

رومانی صبحی رزق جرجس مطر

3 4 Knowledge Management

Knowledge management captures, organizes, and shares useful experience and expertise. Documents can preserve explicit knowledge, such as an approved procedure. Tacit knowledge includes judgment developed through practice and may need discussion, mentoring, or demonstrations to be shared effectively.

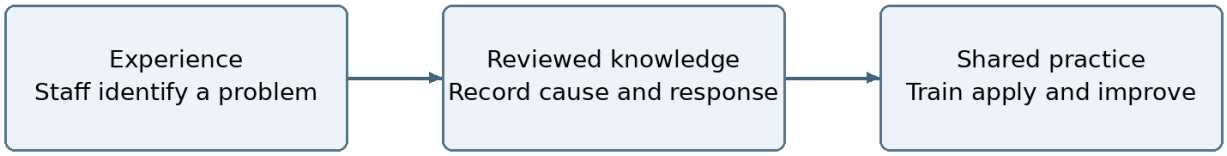


Figure 3.10. Experience becomes reusable when it is documented, reviewed, and applied with context. Original teaching diagram.

5 Capturing and sharing knowledge

Useful resources include lessons learned, best-practice guides, process documentation, expert explanations, and frequently asked questions. Each resource needs an owner, version, and review date where appropriate. A repository becomes less dependable if outdated guidance remains indistinguishable from current instructions.

Example: staff learn that a supplier uses a carton-based unit. A reviewed purchasing guide explains the conversion, names the supplier, and states when it applies. New employees can then avoid repeating the same error. The guidance should be revised if the supplier changes packaging.

Connecting management activities

The stock record is managed data. A report showing likely shortages is information. A verified explanation of recurring shortages supports knowledge. The manager still needs to consider cost, supplier lead time, uncertainty, and service priorities when selecting an action. Sharing an explanation does not remove the need to test whether it fits a new situation.

Student application

Imagine that two employees calculate different monthly totals. Identify the information needed to resolve the disagreement: reporting period, included records, return rules, currency, source, and refresh time. Explain how a shared metric definition and documented lineage could prevent the same disagreement in the next month.

رومانی صبحی رزق هرجس مطر

3 5 Importance of Data and Information Management

Effective management can improve decisions and daily operations by making relevant records dependable and easier to use. Benefits depend on implementation, user skills, and whether findings influence action. Collecting more data without clear purposes or responsibilities can increase cost and confusion.

Benefit	How it can arise	Example
Informed decisions	Relevant accurate timely evidence	Compare demand with available stock
Operational efficiency	Less searching and repeated correction	Use an agreed product identifier
Competitive advantage	Faster response to useful market evidence	Adjust stock to verified demand changes
Risk reduction	Quality controls and appropriate safeguards	Detect repeated imports and restrict access
Innovation	Investigate unmet needs and patterns	Pilot a service for recurring student requests
Customer experience	Reliable records and appropriate personalization	Resolve an order query using a current record

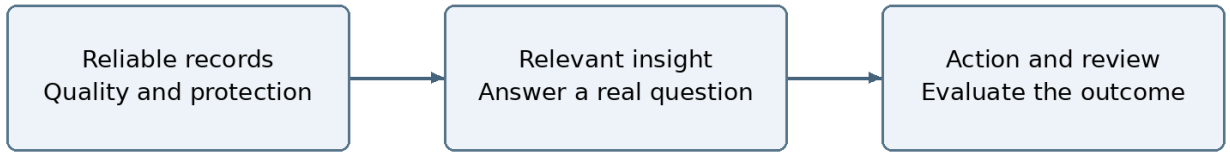


Figure 3.11. Organizational value depends on action and evaluation as well as reliable information. Original teaching diagram.

Suppose a manager uses verified sales patterns to reorder books earlier. Evaluate the improvement using relevant outcomes such as fewer stockouts, lower unnecessary stock, or shorter response times. The number of charts produced is a measure of activity, not proof that the service improved.

Personalization should use data for appropriate purposes with suitable controls. Better customer experience does not justify unnecessary collection or unrestricted access. Security, privacy, and usefulness must be considered together when designing a service.

رومانی صبحی رزق جرجس مطر

3 6 Data Information Knowledge and Wisdom

The data–information–knowledge–wisdom (DIKW) hierarchy is a conceptual model for discussing different forms of understanding. Data provides observations, information interprets them, knowledge connects evidence with understanding, and wisdom describes considered judgment about action. The model is useful for teaching, but it is not an automatic production pipeline. [6]

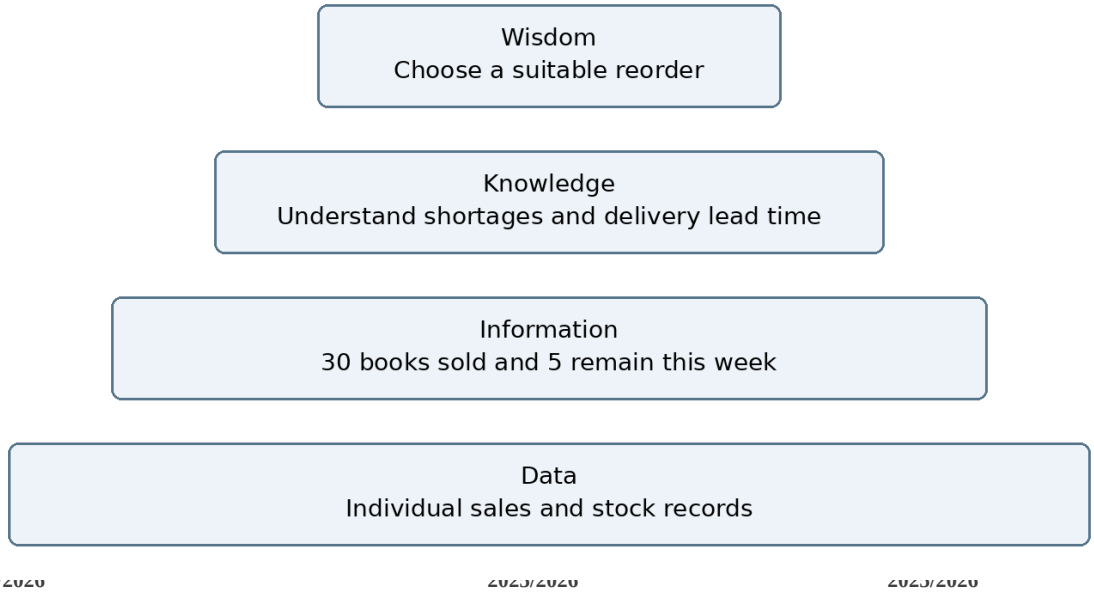


Figure 3.12. A DIKW teaching model with a single bookshop example. Each level requires interpretation rather than automatic promotion. Original teaching diagram.

The shop records individual sales. A weekly summary shows that 30 copies were sold and 5 remain. Staff combine this summary with verified demand patterns and delivery lead time to understand the risk of running out. The manager then weighs cash availability, uncertainty, and service needs before ordering more copies.

The weekly sales count alone does not determine the optimal order quantity. Knowledge may reveal additional constraints, and judgment may favor a smaller trial order or further investigation. Information systems support evidence and knowledge sharing, while responsibility for an appropriate decision remains with the relevant people.

Applying the model carefully

A record, report, or recommendation can be wrong at any level. An explanation that sounds plausible still needs supporting evidence. The hierarchy should help students identify what has been observed, what has been interpreted, and what assumptions connect the evidence to a proposed action.

3 7 Challenges in Data and Information Management

Growing datasets create technical and organizational challenges. Volume affects storage and processing; variety affects interpretation and integration; quality affects trust. A solution should address the observed problem rather than assume that purchasing more technology will resolve every difficulty.

Challenge	Risk or difficulty	Practical response
Growing volume	Slow processing or rising storage costs	Define retention and plan capacity
Data variety	Different formats meanings and tools	Catalog sources and specify transformations
Poor quality	Misleading totals and decisions	Validate investigate and monitor
Data silos	Incomplete views and inconsistent definitions	Agree identifiers and governed integration
Security and privacy	Misuse or exposure of sensitive records	Apply access controls and appropriate-use rules
Skills shortage	Weak analysis or unsupported maintenance	Train staff and clarify responsibilities
Compliance requirements	Practices conflict with applicable rules	Identify obligations and document implementation

2025/2026 2025/2026 2025/2026

Structured semi structured and unstructured data

Structured data follows an explicit schema, such as sales rows with defined columns. Semi-structured data has markers or keys, such as JSON messages, while allowing more flexible records. Unstructured data includes documents, audio, and images whose content does not fit a simple row-and-column schema. Unstructured does not mean that a file has no internal organization.

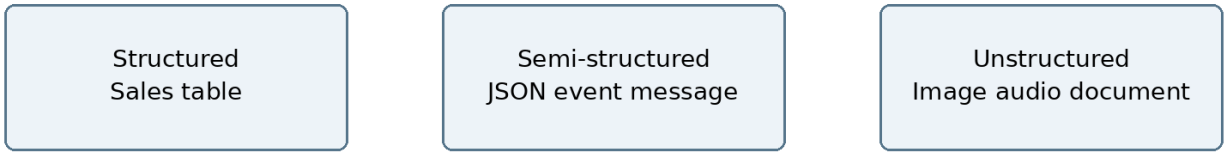


Figure 3.13. Different data forms require different preparation and interpretation methods. Original teaching diagram.

Example: a CSV sales table can be summed after its fields are interpreted. A JSON message needs its keys parsed and validated. A photographed receipt may need text extraction followed by checking. Converting the receipt to text does not guarantee correct amounts or an accurate transaction match.

3 7 Privacy and Responsible Data Use

Security concerns protecting data against threats, while privacy concerns appropriate handling of information about people. A securely stored file can still be used for an inappropriate purpose. Good practice therefore considers both technical safeguards and the purpose, extent, and duration of data use.

The European Commission describes GDPR principles including lawful and transparent processing, purpose limitation, data minimization, accuracy, storage limitation, security, and accountability. GDPR is one regulatory example; organizations must identify which rules apply to their circumstances rather than assume that one regulation covers every situation. [7]

A proportionate information need

A bookshop may need an order identifier and delivery contact for fulfillment. A stock-demand report may only need product quantities by period. Including named customer details in that report adds exposure without necessarily improving the analysis. Aggregate information can reduce exposure, although aggregation alone does not guarantee that individuals cannot be identified.

Balancing access and protection

Restrict sensitive records to authorized roles, document permitted uses, and apply retention decisions. Anonymization and pseudonymization are different: pseudonymized records can remain linkable to people using additional information. Removing a name alone does not reliably anonymize a dataset.

An integrated challenge

2025/2026 2025/2026 2025/2026

Imagine that the bookshop receives sales files from three branches. One uses cartons, another uses individual copies, and a third sends duplicate transactions. Before creating a consolidated dashboard, clarify units, investigate identifiers, define the reporting period, and determine authorized access. More rows or a faster charting tool would not resolve those inconsistencies by themselves.

Check your understanding: explain which problem concerns quality, which concerns integration, and which concerns privacy in this scenario. State what evidence you would inspect before changing a record.

3 8 Chapter Summary

Data records observations and facts, while information supplies context and meaning relevant to a question. Data can already be structured; the important distinction is how records are interpreted and used. Data management covers collection, storage, quality, integration, security, and governance throughout the lifecycle.

Information management connects analysis, transformation, visualization, business intelligence, and knowledge sharing. Accurate calculations require appropriate scope and definitions. Reliable records can support better decisions, efficiency, innovation, and service quality, but benefits need action and evaluation.

The DIKW hierarchy helps distinguish observations, interpretation, understanding, and judgment. Growing volume, varied formats, silos, skills gaps, and privacy requirements make continued improvement necessary. The principles studied here support later work on business processes, security, analytics, and emerging technologies.

Key Terms

Term	Meaning
Data	Recorded observations measurements or facts
Information	Data interpreted in a meaningful context
Data management	Coordinated management of data throughout its lifecycle
Data quality	Fitness of data for its intended use
Data integration	Combining sources into a usable view
Data governance	Policies standards responsibilities and oversight
Data security	Protection of confidentiality integrity and availability
Information management	Organizing interpreting distributing and using information
Business intelligence	Capabilities for organizational analysis and reporting
Data visualization	Visual presentation of information
Knowledge management	Capturing and sharing useful understanding and expertise
Data silo	Isolated collection that is difficult to combine or access
Metadata and lineage	Descriptions of data and records of its origin and transformations

رومانی صیحی رزق جرجس مطر

References and Figure Credits

The references support further study and the chapter's technical explanations. All thirteen figures are original teaching visuals. Figure 3.9 uses fictional illustrative data, not observed business results. Sources checked on 9 October 2026.

[1] IBM Developer. Data management.

https://developer.ibm.com/technologies/data-management/

[2] IBM. What Is a Data Warehouse.

https://www.ibm.com/think/topics/data-warehouse

[2a] IBM. What Is a Data Lake.

https://www.ibm.com/think/topics/data-lake

[3] IBM. What Is Data Quality Management.

https://www.ibm.com/think/topics/data-quality-management

[4] IBM Research. Data Governance and Compliance.

https://research.ibm.com/projects/data-governance-and-compliance

[5] Microsoft Learn. What is Power BI.

https://learn.microsoft.com/en-us/power-bi/power-bi-overview

[6] OpenStax. Foundations of Information Systems. Section 12 1.

https://openstax.org/books/foundations-information-systems/pages/12-1-ethics-sustainability-and-use-of-information-systems

[7] European Commission. Principles of personal data processing under the GDPR.

https://commission.europa.eu/law/law-topic/data-protection/information-business-and-organisations/principles-gdpr_en

2025/2026 2025/2026 2025/2026

Chapter 3 Practice Questions

Choose one best answer for each question. Write its letter in the answer space provided.

Student name _____ Date _____

1. Which statement best distinguishes data from information?

A. Data must always be unstructured

B. Information is any number without labels

C. Data can never appear in a table

D. Information is data interpreted in a relevant context

Your answer: _____

2. What additional detail makes the value 1500 interpretable in a sales report?

A. The database server's color

B. Its measure currency event and reporting context

C. The font used to display it

D. The monitor's screen size

Your answer: _____

3. Which activity belongs to data management?

2025/2026 2025/2026 2025/2026

A. Guaranteeing that every forecast is correct

B. Replacing all human decisions

C. Maintaining record quality and access throughout the lifecycle

D. Changing the organization's logo

Your answer: _____

4. Why is metadata useful?

A. It automatically corrects every missing value

B. It guarantees future business growth

C. It removes the need for access controls

D. It describes meanings units sources and other details about data

Your answer: _____

رومانی صبحی رزق جرجس مطر

Chapter 3 Practice Questions Continued

5. Which storage approach is typically designed for integrated historical reporting?

A. Keyboard buffer

B. Printer queue

C. Data warehouse

D. CPU register

Your answer: _____

6. Which statement about cloud storage is accurate?

A. Cloud storage is always a relational table

B. Cloud describes a delivery setting that can host different storage approaches

C. Cloud storage is the same as unstructured data

D. Cloud storage removes every security responsibility

Your answer: _____

7. A quantity is missing from a sales record. What is the most appropriate first response?

A. Delete every record for that customer

B. Check the source or flag the record for investigation

C. Replace it with zero without checking

2025/2026 2025/2026 2025/2026

D. Invent the most convenient value

Your answer: _____

8. A 500 EGP transaction is duplicated in a correct 1000 EGP total. By what percentage is the reported total overstated?

A. 100%

B. 150%

C. 50%

D. 5%

Your answer: _____

رومانی صبحی رزق جرجس مطر

Chapter 3 Practice Questions Continued

9. Why should product identifiers be agreed before integrating sales and stock data?

A. They help match related records consistently

B. They replace every access policy

C. They make all values factually correct

D. They eliminate the need for units

Your answer: _____

10. Which statement correctly describes ETL?

A. Evaluate transactions list users then delete records

B. Extract data transform it then load it into the target

C. Load data first and never transform it

D. Encrypt data transfer passwords then log out

Your answer: _____

11. A staff member signs in successfully but cannot open payroll records. Which control limits that access?

A. Authorization

B. Numeric scaling

C. Aggregation

D. Visualization

Your answer: _____

2025/2026 2025/2026 2025/2026

12. What is the main purpose of data governance?

A. Provide unlimited access to all staff

B. Define standards responsibilities and oversight for data

C. Replace all technical security controls

D. Guarantee that every database has no errors

Your answer: _____

رومانی صبحی رزق جرجس مطر

Chapter 3 Practice Questions Continued

13. Which operation combines individual transactions into daily totals?

A. Pseudonymization

B. Authentication

C. Aggregation

D. Encryption

Your answer: _____

14. Why is revenue different from profit?

A. The two measures always have the same value

B. Profit is simply the number of transactions

C. Revenue always includes every cost

D. Profit requires relevant costs and accounting definitions in addition to revenue

Your answer: _____

15. Which chart is generally suitable for comparing revenue across product categories?

A. An unlabeled image

B. A map without location data

C. Bar chart

2025/2026 2025/2026 2025/2026

D. A chart with no measure or units

Your answer: _____

16. Which capability is commonly associated with business intelligence?

A. Interactive reporting and performance monitoring

B. Allowing every user to access all personal data

C. Guaranteeing that a pattern proves causation

D. Removing the need for reliable source records

Your answer: _____

رومانی صبحی رزق جرجس مطر

Chapter 3 Practice Questions Continued

17. A reviewed guide records how a supplier's carton quantities should be converted. Which activity does this primarily illustrate?

A. Knowledge management

B. Replacing all storage devices

C. Network routing

D. Changing the processor instruction set

Your answer: _____

18. In the DIKW model, which example best represents wisdom?

A. Displaying a weekly sales total without interpretation

B. Storing one raw sales row

C. Recording a product identifier

D. Choosing a proportionate reorder after weighing evidence costs and uncertainty

Your answer: _____

19. A JSON event message uses named keys but permits flexible records. Which data form does it typically represent?

A. A physical hardware component

B. Only unstructured

C. A completely structureless bit stream

D. Semi-structured

Your answer: _____

20. Which statement best distinguishes privacy from security?

A. Securely protected personal data can still be used for an inappropriate purpose

B. Removing a name guarantees anonymization

C. Encryption always makes any use of personal data appropriate

D. Privacy concerns only processor speed

Your answer: _____

رومانی صبحی رزق جرجس مطر
2025/2026 2025/2026 2025/2026

Chapter 4

Business Processes and Systems

IS101 | Academic year 2026–2027
Faculty of Computers and Informatics
Tanta University

Instructor and Course Coordinator
Prof. (Assoc.) Hany El-Ghaish

رومانی صبحی رزق جرجس مطر

4 1 Introduction

Organizations create value through activities that deliver goods, services, or internal results. These activities form business processes. A university registers students, a shop fulfills orders, and a hospital schedules appointments. Each process depends on people, resources, rules, and information moving through a sequence of work.

Information systems help record events, coordinate responsibilities, automate suitable steps, and assess results. Their design should reflect the work that users actually perform, including exceptions and approvals. This chapter connects the technology and data-management principles studied earlier with organizational processes and enterprise systems.

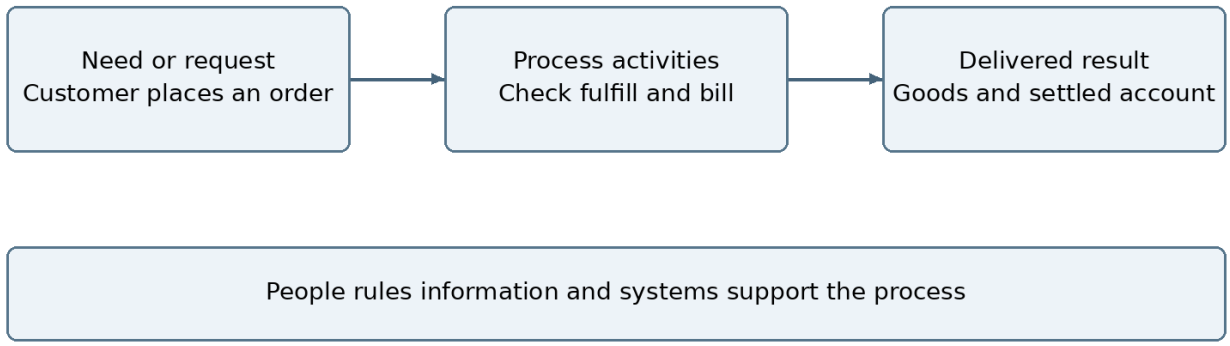


Figure 4.1. A process connects a need with a delivered result. Information systems support the work and its review. Original teaching diagram.

Learning objectives

By the end of this chapter, you should be able to define a business process, identify its components and classifications, and explain how information systems support it. You should also be able to compare ERP with CRM and other enterprise systems, read simple process models, calculate basic performance measures, and distinguish continuous improvement from reengineering.

4 2 What is a Business Process

A business process is a set of related activities that produces a defined result for an external customer or an internal user. A useful description states the purpose, trigger, scope, steps, responsible actors, and completion condition. The process can include alternative paths and exceptions rather than one fixed sequence for every case.

Characteristic	Question to ask	Order example
Purpose	What result is required?	Deliver the requested goods and settle payment
Activities	Which steps produce the result?	Validate check stock deliver invoice collect
Departmental scope	Who must cooperate?	Sales warehouse finance and customer service
Resources	What is consumed or used?	Staff time stock equipment and system capacity
Measurable output	How is completion recognized?	Order fulfilled and payment recorded

Order to cash

Order-to-cash covers the activities from receiving an order to fulfilling it and collecting payment. An order may need validation, a stock check, credit approval, delivery, invoicing, and payment reconciliation. The sequence varies with the business model: an online retailer may require prepayment, while an approved business customer may pay after receiving an invoice.

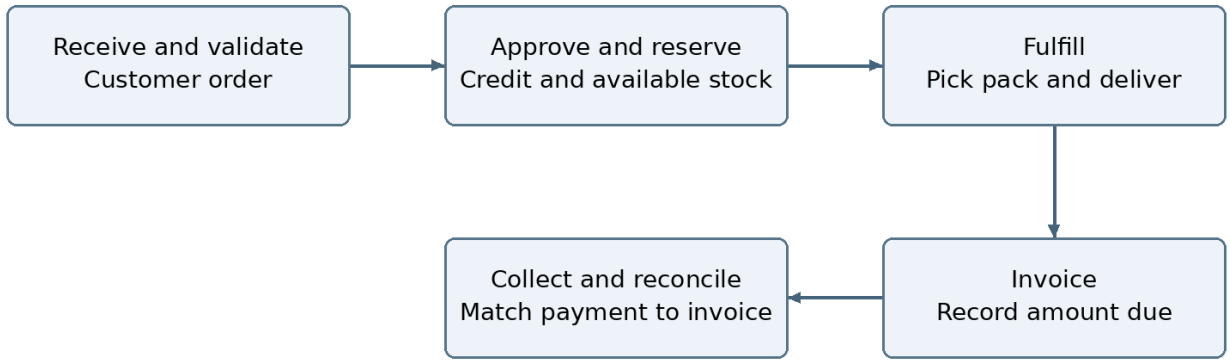


Figure 4.2. A simplified order-to-cash path for an approved customer buying on credit. Stock and credit exceptions require separate handling. Original teaching diagram.

A process is broader than a department or a screen in an application. Completing a sales-entry screen does not prove that the customer received the goods. The process boundary should identify which outcome is being evaluated.

4 2 Other Business Process Examples

Organizations carry out many processes with different purposes and timescales. Naming the start and finish prevents ambiguity about responsibility. A procurement process, for example, may begin with an approved need rather than with the supplier invoice that arrives near its end.

Process	Typical start	Typical result
Procure to pay	Need for goods or services	Required supply received and supplier paid
Hire to retire or exit	Need for an employee	Employment managed through eventual departure
Customer service	Question complaint or service request	Response or resolution communicated and recorded
Product development	Identified product opportunity	Validated design and release decision

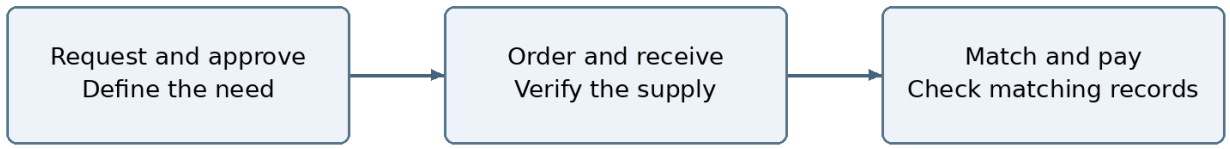


Figure 4.3. A simplified procure-to-pay sequence. Authorization and matching checks govern commitments and payment. Original teaching diagram.

Controls within a process

For a goods purchase, an organization may compare the purchase order, receipt record, and invoice before paying. The purchase order states what was authorized, the receipt records what arrived, and the invoice states what the supplier requests. Differences need investigation; automatically accepting every invoice would bypass the intended control.

Internal and external customers

The customer of a process can be inside the organization. An IT support process serves staff who need a working service. Its output might be restored access or an explained resolution. The same process concepts apply even when no physical product or external sale is involved.

Check your understanding: describe the trigger and completion condition of a university course-registration process. Identify at least two responsibilities involved in delivering the result.

رومانی صبحی رزق جرجس مطر

4 3 Components of a Business Process

A process model should explain what enters the work, what activities occur, who performs them, and what result is produced. Rules constrain valid actions, while measures help evaluate performance. These elements connect the organizational purpose with the records and software needed to support it.

Component	Meaning	Order example
Inputs	Requests data materials or other resources needed	Customer order product records available stock
Activities	Work that transforms or handles inputs	Validate order check credit pack goods
Outputs	Results delivered or recorded	Delivered goods invoice payment record
Actors	People roles or systems carrying out work	Sales employee warehouse staff ERP service
Rules and controls	Conditions and safeguards governing action	Credit limits approval thresholds stock checks
Performance measures	Defined indicators of process results	Cycle time cost per order on-time delivery

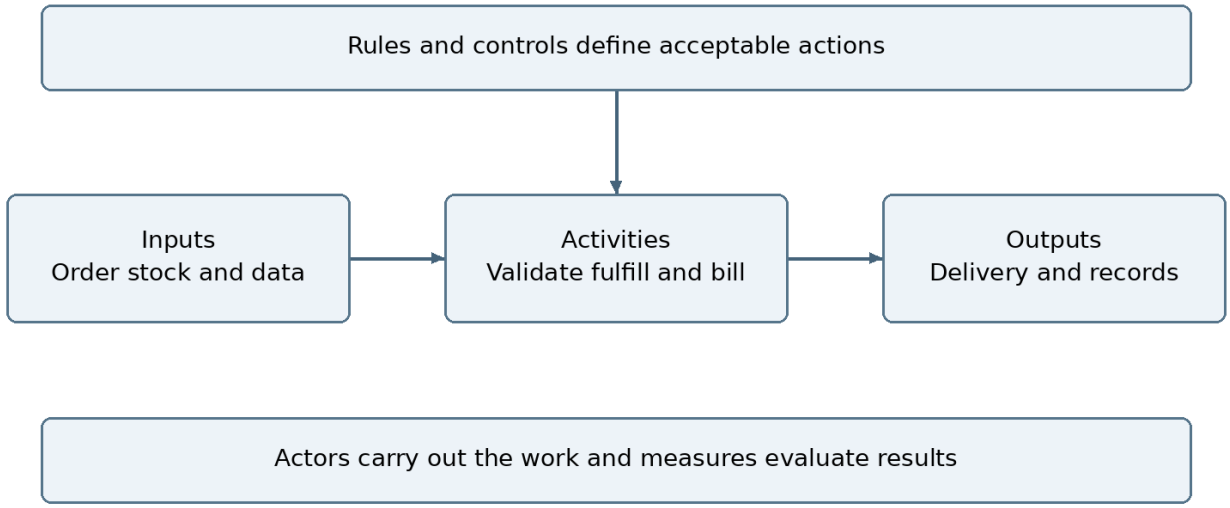


Figure 4.4. Activities use inputs to produce outputs under rules and controls. Actors perform the work and measures assess it. Original teaching diagram.

A customer order is an input; packing is an activity; delivery is an output. A salesperson is an actor rather than an output. The rule “orders above the approved credit limit require review” governs an activity. A measure such as average fulfillment time evaluates the resulting performance.

4 3 Measuring Process Performance

A useful metric defines the event being measured, its unit, its start and end points, and the reporting population. Without these definitions, two departments may report different values for the same process. Measures should combine speed, quality, cost, and service outcomes as appropriate.

Measure	Definition for this example	Interpretation
Cycle time	Elapsed time from accepted order to delivery	Includes active work and waiting
Processing time	Time spent actively performing tasks	Excludes separately identified waiting
Cost per order	Relevant process cost divided by completed orders	Requires an agreed cost scope
On-time delivery rate	On-time deliveries divided by eligible deliveries × 100	Requires an agreed promised-date rule
Error rate	Orders with defined errors divided by checked orders × 100	Depends on a consistent error definition

Worked calculations

Three orders take 4, 6, and 8 days from acceptance to delivery. Their average cycle time is (4 + 6 + 8) ÷ 3 = 6 days. If 60 of 75 eligible deliveries arrive on time, the on-time delivery rate is 60 ÷ 75 × 100 = 80%. These examples use fictional teaching data.

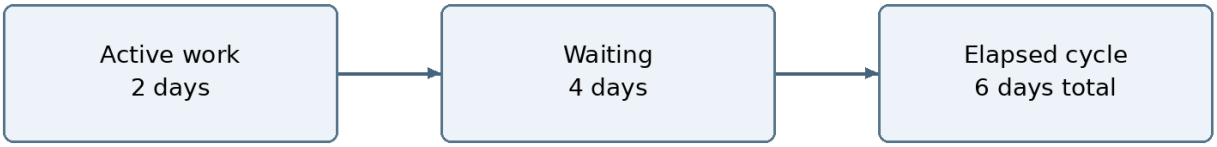


Figure 4.5. Illustrative elapsed cycle time contains both active work and waiting. The diagram is conceptual and not drawn to a time scale. Original teaching diagram.

Reducing an approval queue can improve cycle time without making packing faster. Averages can hide unusually delayed orders, so managers may also examine the distribution and reasons for exceptions. Faster completion should not be achieved by bypassing necessary checks or increasing errors.

A bottleneck is a constraint that limits process flow or capacity. Locate it using evidence such as queue lengths, workload, service capacity, and waiting time. The step with the most visible activity is not automatically the bottleneck.

4 4 Types of Business Processes

Processes can be classified by organizational role, degree of structure, and position in the value chain. These are different dimensions rather than mutually exclusive labels. A process can be operational and structured, while also crossing several departments.

Classification	Meaning	Example
Operational or core	Directly delivers the organization's main goods or services	Order fulfillment or student teaching
Supporting	Enables core activities	IT support HR administration
Management	Sets direction monitors and controls work	Budget planning and performance review

The classification depends on the organization's purpose. Payroll is generally a supporting process in a retailer but can be a core service in a payroll-service provider. Its routine execution can also be described as operational work. State the classification framework before assigning labels.

Degree of structure	Characteristics	Example
Structured	Clear repeatable steps and decision rules	Routine payroll calculation
Semi structured	Defined steps plus judgment in some decisions	Credit approval with unusual circumstances
Unstructured	Variable knowledge-intensive work	Exploratory new-product design

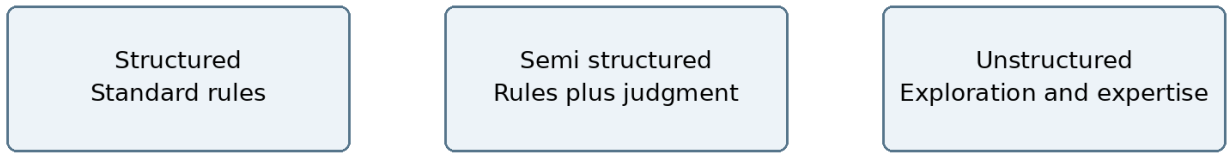


Figure 4.6. Process structure varies with how much of the work can be specified in advance. Original teaching diagram.

In a value-chain view, primary processes directly contribute to the customer offering, while support processes enable them. Primary does not mean that supporting work is unimportant. An unreliable IT service can interrupt an otherwise well-designed customer process.

4 5 Role of Information Systems in Business Processes

Information systems support the movement of work and information. They can automate well-defined tasks, connect related records, route responsibilities, and provide performance evidence. Their contribution depends on sound process rules, usable data, appropriate controls, and people who understand their responsibilities.

Role	Contribution	Example
Automation	Perform suitable repeatable tasks	Send an order confirmation after acceptance
Integration	Connect related departmental information	Share approved order and stock records
Coordination	Route work and clarify responsibility	Send an approval request to the assigned manager
Monitoring and control	Track status and identify exceptions	Alert staff when an order is overdue
Analysis and improvement	Identify patterns and test changes	Investigate delays by activity and branch

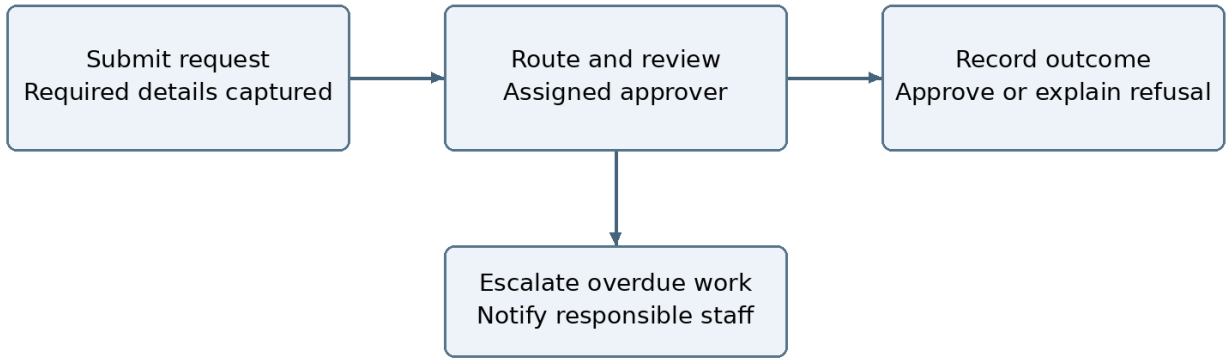


Figure 4.7. A workflow assigns work and follows its status. An exception requires an accountable response. Original teaching diagram.

An entered sales order may trigger a stock reservation and create related financial records, but the exact update depends on the order status and configured rules. An order does not automatically mean that goods were delivered or revenue was recognized. Integration must preserve these distinctions.

Monitoring may use live events or periodic updates. A dashboard's refresh schedule determines how current its view is. Automation can repeat an incorrect rule quickly, so inspect the process and data before deciding which steps to automate.

رومانی صیحی رزق جرجس مطر

4 6 Enterprise Systems

Enterprise systems provide coordinated support for important processes across organizational functions. Some also exchange information with customers, suppliers, and partners. Their purpose is to reduce fragmented records and support end-to-end work. A product's label does not prove that every relevant process is integrated.

4 6 1 Enterprise Resource Planning Systems

An enterprise resource planning (ERP) system is an integrated suite supporting functions such as finance, sales, inventory, procurement, production, and human resources. A common database or shared data model connects modules so that users work with consistent definitions and related records. This describes logical integration; real deployments can contain multiple databases and connected services. [1]

Typical module	Main focus	Related record
Finance and controlling	Accounting costs and financial reporting	Invoice payment and account entry
Sales and distribution	Orders delivery and billing	Customer order delivery record
Materials and inventory	Purchasing receipts and stock movements	Purchase order and stock balance
Production planning	Materials capacity and manufacturing schedules	Production order
Human resources	Employee and employment administration	Employee record
Customer functions where provided	Customer-related activities	Contact service case or sales opportunity

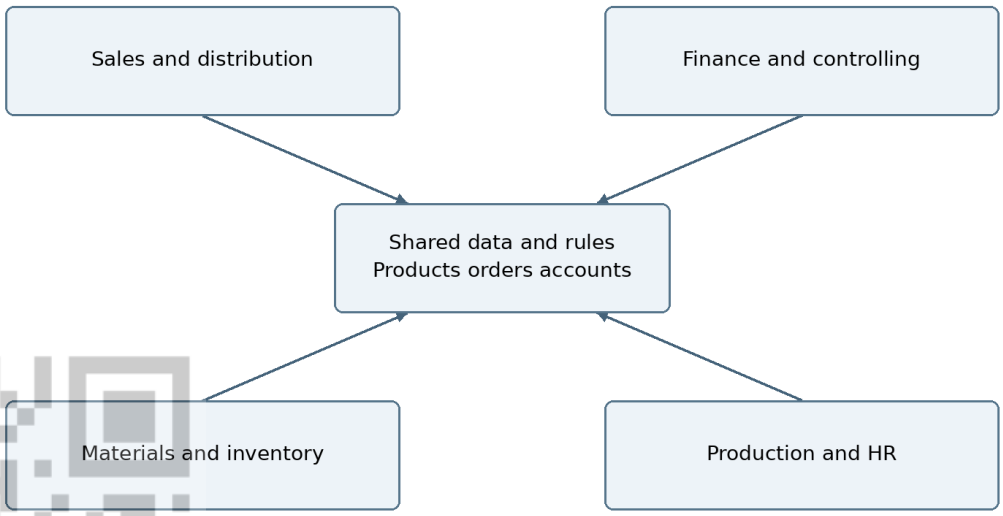


Figure 4.8. ERP modules share agreed data and coordinated operations. This is a conceptual integration model, not a physical database diagram. Original teaching diagram.

4 6 1 ERP Benefits and Implementation Challenges

ERP can reduce repeated entry, improve consistency, and make cross-departmental status easier to review. For example, an authorized stock movement can update the inventory record used by purchasing and fulfillment. However, a shared source can still contain incorrect data. The phrase single source of truth expresses a consistency goal rather than an assurance of factual accuracy.

Potential benefit	What supports it	Implementation challenge
Consistent records	Agreed master data and validation	Migrating duplicate or conflicting records
Better coordination	Shared statuses and clear responsibilities	Reconciling departmental practices
Faster reporting	Integrated data and suitable reporting tools	Defining measures and access rights
Standardized processes	Approved common procedures	Training users and managing change
Reduced repeated entry	Controlled interfaces and automation	Maintaining integrations and customizations

Cost complexity and change

Implementation can require licensing or service costs, process analysis, configuration, data migration, training, integration, and ongoing support. Customization can accommodate a necessary requirement but can also increase testing and upgrade effort. Review whether the requirement can be met through configuration or a process change before modifying core software.

2025/2026 2025/2026 2025/2026

A practical implementation example

If three branches use different product codes, the organization needs an agreed mapping before migrating inventory records. Staff then test receipts, sales, returns, and accounting effects. A successful login is not enough evidence that the business processes work. Pilot users should verify both normal transactions and meaningful exceptions.

Representative products include SAP ERP offerings, Oracle Fusion Cloud ERP, Microsoft Dynamics 365 Finance or Business Central, and Odoo. Product families contain different applications and deployment choices; examples identify categories rather than recommend a purchase. Microsoft describes Business Central as supporting functions including finance, sales, manufacturing, and shipping. [2]

رومانی صبحی رزق جرجس مطر

4 6 2 Customer Relationship Management Systems

Customer relationship management (CRM) systems organize information and work involving current and potential customers. They can connect contacts, sales opportunities, campaigns, and service cases. CRM supports relationship management, while the quality of the relationship still depends on how people use the information and respond to customers. [3]

Function	Purpose	Example
Contact and account management	Maintain appropriate customer details	Record verified business contact information
Sales automation	Track opportunities and follow-up work	Assign the next action for a quotation
Campaign management	Coordinate marketing activity	Track responses to an authorized campaign
Customer service	Record and route service issues	Assign a delivery complaint to support staff
Customer analysis	Examine interactions and outcomes	Review response times and repeat purchases

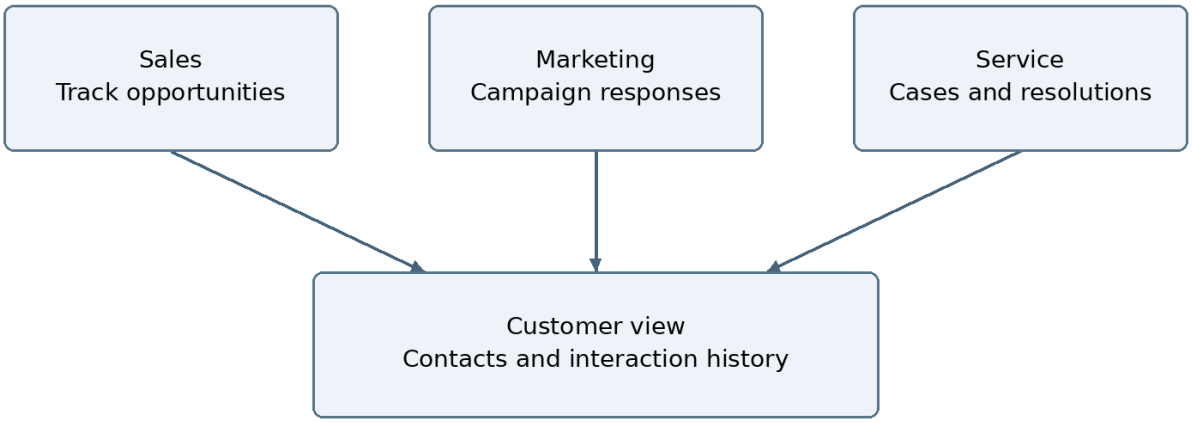


Figure 4.9. Customer-related records connect sales, marketing, and service work through a shared view. Original teaching diagram.

A support employee can review an open case and related order status before replying to a customer. The CRM may obtain the order status from an ERP interface. Appropriate access, current records, and clear follow-up responsibilities are necessary; collecting more contact details alone does not guarantee satisfaction or loyalty.

Representative examples include Salesforce, HubSpot, Microsoft Dynamics 365 Sales and Customer Service, and Zoho CRM. The capabilities available depend on the selected products and configuration. A sales pipeline shows opportunities at different stages; an opportunity is not the same as a completed sale.

رومانی صیحی رزق جرجس مطر

4 6 3 Other Important Enterprise Systems

Additional enterprise systems address particular organizational needs. Their functions can overlap with ERP or CRM, and vendors may offer them as connected suites. Understanding the purpose of each category is more useful than assuming that each organization needs a separate product for every label.

System	Primary purpose	Representative examples
Supply Chain Management SCM	Coordinate supply demand goods and related information	Oracle Fusion Cloud SCM and SAP supply-chain solutions
Human Capital Management HCM	Support recruitment employment development and exit	Workday and SAP SuccessFactors
Product Lifecycle Management PLM	Manage product definitions versions and changes	Siemens Teamcenter and PTC Windchill
Enterprise Content Management ECM	Manage controlled documents and digital content	OpenText and Microsoft SharePoint content capabilities

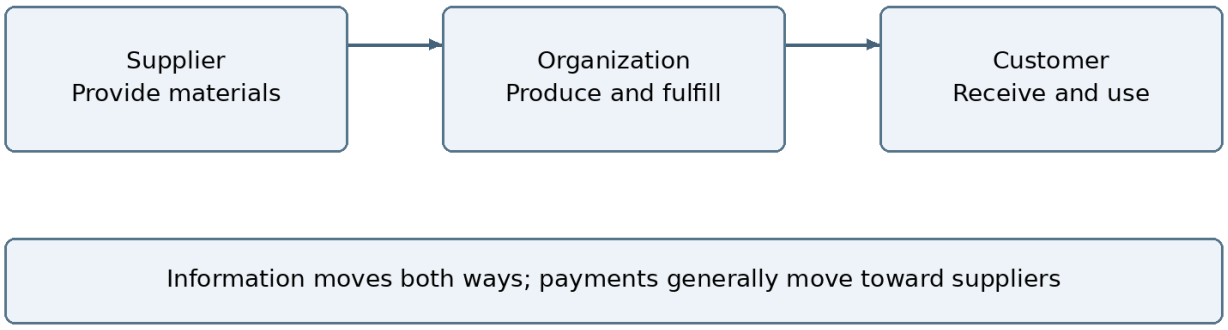


Figure 4.10. Supply-chain coordination connects organizations. Goods information and financial flows have different directions. Original teaching diagram.

The upper arrows show a simplified flow of goods. SCM also concerns planning, inventory, logistics, and coordination with trading partners. Oracle describes its SCM suite as a set of supply-chain applications. A diagram of goods movement alone does not capture all the information and financial relationships. [4]

ERP and CRM compared

Aspect	ERP emphasis	CRM emphasis
Main question	How are enterprise resources and transactions coordinated?	How are customer interactions and opportunities managed?
Typical records	Products stock invoices accounts and purchases	Contacts opportunities campaigns and cases
Connection	Provides operational order and financial status	Provides customer context and follow-up activity

4 7 Business Process Modeling

Process modeling represents how activities, decisions, and responsibilities fit together. An As-Is model describes the current process as observed. A To-Be model describes a proposed future process. Compare the two using evidence, requirements, and realistic constraints rather than assume that the proposed diagram is automatically better.

Technique	Main use	What it makes visible
Flowchart	Explain activities and decision branches	Order of work and alternatives
Swimlane diagram	Allocate work across roles or departments	Responsibility and handoffs
BPMN	Use a standardized process notation	Events tasks gateways and defined flows
Value stream mapping	Examine end-to-end value and flow	Work information and waiting across the stream

Business Process Model and Notation (BPMN) is a standard maintained by the Object Management Group. A basic vocabulary includes events, activities, gateways, and flows. Sequence flow orders work within a process; message flow describes communication between separate participants. These distinctions matter when a formal BPMN model is required. [5]

2

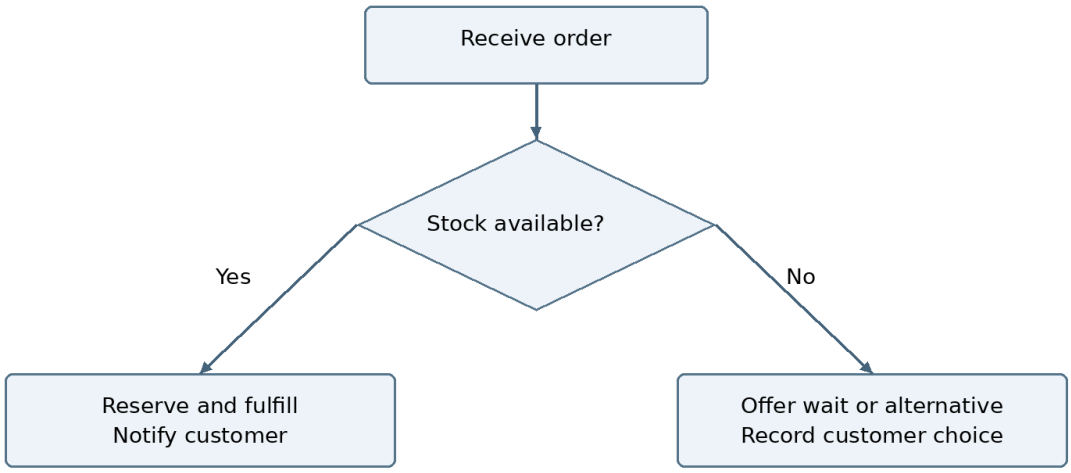


Figure 4.11. A simplified flowchart with a stock decision. Rounded boxes show activities and the diamond shows a decision. This is not a formal BPMN diagram. Original teaching diagram.

رومانی صبحی رزق جرجس مطر

4 7 Responsibilities and Process Handoffs

A swimlane diagram groups activities by the role that performs them. It can reveal delays caused by handoffs, unclear ownership, or repeated entry. A handoff transfers work or information between actors; each transfer needs a defined result and enough context for the receiving role.

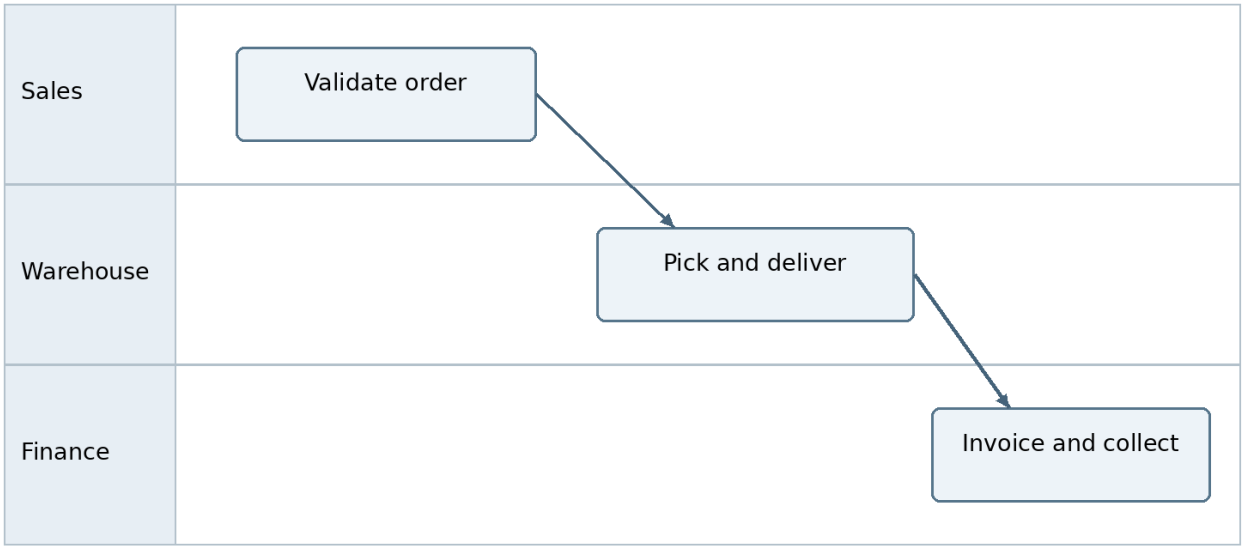


Figure 4.12. A simplified swimlane diagram for an approved credit order. The lanes identify responsibility and arrows identify handoffs. Original teaching diagram.

Sales validates the order and passes fulfillment details to the warehouse. Finance uses the relevant order and delivery records to invoice and collect payment. The diagram omits checks already discussed and is intended to show responsibility, not every possible exception.

How to build a useful model

Begin with a defined process boundary and observe real cases. Ask participants about normal work, exceptions, queues, and workarounds. Name activities using clear verbs, identify decision conditions, and confirm the model with the people who perform the work. Include records and control points that affect the result.

Using a model for system design

A model helps identify required data, user roles, interfaces, status changes, and audit records. It can also expose inconsistent interpretations of completion. A system designer should resolve those disagreements before implementing automated transitions.

Value stream mapping provides another view of material and information flow and time spent waiting. It helps examine the complete stream rather than optimize one activity at the expense of the overall result. [6]

4 8 Business Process Improvement and Reengineering

Process improvement changes how work is performed to achieve defined outcomes. Continuous improvement, often associated with Kaizen, makes ongoing incremental changes. Business process reengineering (BPR) fundamentally rethinks and redesigns a process. The approaches differ in scope, but both require evidence and assessment of consequences. [7, 8]

Aspect	Continuous improvement	Business process reengineering
Typical scope	Incremental changes within an existing process	Fundamental redesign of the overall process
Example	Reduce duplicate entry in one approval step	Replace fragmented approvals with a redesigned service
Implementation	Often narrower pilots and repeated evaluation	Often extensive coordination and change management
Risk	Can be more limited but depends on the change	Can be substantial because many responsibilities change
Evidence needed	Baseline outcome and effect of each change	Baseline future design and validated process outcomes

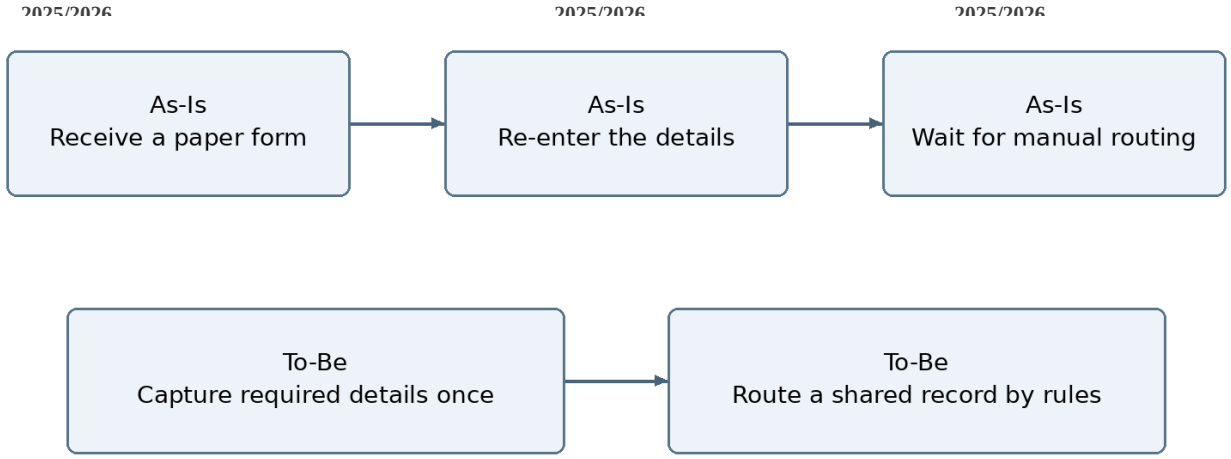


Figure 4.13. A conceptual As-Is and To-Be comparison. Shared information reduces re-entry only when the new design is validated. Original teaching diagram.

Information systems can enable shared records, rule-based routing, and performance tracking. However, automating the old sequence without questioning unnecessary work may preserve its inefficiencies. Redesign should also address people, policy, training, exception handling, and access responsibilities.

رومانی صبحی رزق جرجس مطر

4 8 Evaluating a Process Change

An improvement proposal needs a specific problem, a baseline, an intended outcome, and a way to assess the result. Evaluate the end-to-end process rather than count only how many activities became automated. A change that speeds one department but creates a longer queue in another may not improve the overall service.

A worked improvement example

Suppose a pilot reduces average fulfillment cycle time from 6 days to 4 days. The reduction is 2 days. The percentage reduction relative to the baseline is (6 − 4) ÷ 6 × 100 = 33.3%, rounded to one decimal place. This result uses fictional teaching data and applies to the measured pilot, not automatically to every future order.

Check the explanation and the safeguards

Compare equivalent order types and reporting periods. Examine delivery errors, cost, customer experience, and control compliance as well as elapsed time. A seasonal demand change or different order mix could explain part of the observed difference. A before-and-after comparison alone does not prove that the system caused every improvement.

Evaluation step	Question to answer
Define the issue	Which outcome is unsatisfactory and for whom?
Measure the baseline	How is performance defined and what cases are included?
Design the change	Which activities rules or handoffs will change?
Pilot and verify	Do normal work and exceptions still function correctly?
Review outcomes	Did time quality cost and service improve as intended?
Sustain the result	Who owns ongoing monitoring and corrective action?

A process owner is accountable for the overall process and its performance, even when activities belong to several departments. Without clear ownership, a dashboard can identify an issue while nobody has responsibility for resolving it.

Student application: choose a course-registration or equipment-purchase process. Identify one unnecessary re-entry or waiting point, propose a modest change, and state two measures that would test its effect. Explain which approval or control must be preserved.

رومانی صبحی رزق جرجس مطر

4 9 Chapter Summary

A business process connects related activities to a defined result for a customer or internal user. It has inputs, activities, outputs, actors, controls, and performance measures. Process classifications depend on organizational role, structure, and value-chain perspective. End-to-end work often crosses departmental boundaries.

Information systems support automation, integration, coordination, monitoring, and analysis. ERP integrates major organizational functions; CRM emphasizes customer interactions. SCM, HCM, PLM, and ECM address additional needs and can connect with other enterprise systems. Integration requires shared definitions, reliable data, appropriate rules, and responsibilities.

Models help explain current work and proposed changes. Flowcharts show sequences and decisions, swimlanes clarify responsibility, BPMN provides standardized notation, and value stream mapping examines overall flow. Continuous improvement and BPR offer different scales of change. Evaluate improvements using defined measures and meaningful safeguards.

Key Terms

Term	Meaning
Business process	Related activities producing a defined result
Operational or core process	Work directly delivering the organization’s main offering
Supporting process	Work enabling core activities
ERP	Enterprise resource planning integrating organizational functions
CRM	Customer relationship management supporting customer work
SCM	Supply chain management coordinating supply and demand
Business process modeling	Representation of activities decisions and responsibilities
BPR	Business process reengineering through fundamental redesign
Workflow	Routing and coordination of tasks and their status
Process integration	Connecting related activities and information
As-Is and To-Be	Current and proposed process models
Cycle time	Elapsed time between defined start and end events
Process owner	Role accountable for end-to-end process performance

References and Figure Credits

All thirteen figures are original teaching diagrams prepared for this chapter. The process examples and numeric calculations are fictional illustrations. Figures 4.11 and 4.12 are simplified models rather than formal BPMN diagrams. The following sources support further reading. Sources checked on 9 October 2026.

[1] SAP. What is ERP.

https://www.sap.com/africa/products/erp/what-is-erp.html

[2] Microsoft Learn. Welcome to Dynamics 365 Business Central.

https://learn.microsoft.com/en-us/dynamics365/business-central/welcome

[3] Salesforce. What is CRM.

https://www.salesforce.com/crm/what-is-crm/

[4] Oracle. Supply Chain Management.

https://www.oracle.com/scm/

[5] Object Management Group. Business Process Model and Notation.

https://www.omg.org/bpmn/

[5a] Object Management Group. BPMN Specification.

https://www.omg.org/spec/BPMN/

[6] Lean Enterprise Institute. Value Stream Mapping.

https://www.lean.org/lexicon-terms/value-stream-mapping/

[7] Lean Enterprise Institute. Continuous Improvement.

https://www.lean.org/lexicon-terms/continuous-improvement/

[8] IBM. What is Business Process Reengineering.

https://www.ibm.com/think/topics/business-process-reengineering

2025/2026 2025/2026 2025/2026

Chapter 4 Practice Questions

Choose one best answer for each question. Write its letter in the answer space provided.

Student name _____ Date _____

1. Which statement best defines a business process?

A. Related activities that produce a defined result for a customer or internal user

B. A computer's physical circuit board

C. Any collection of unrelated documents

D. A department name without activities or outcomes

Your answer: _____

2. Which event best represents the completion of an order-to-cash process for an approved credit sale?

A. A product code is created

B. A sales employee opens an application

C. The goods are fulfilled and payment is recorded and reconciled

D. The customer views an advertisement

Your answer: _____

3. Which sequence best represents procure-to-pay?

2025/2026 2025/2026 2025/2026

A. Receive a customer order deliver and collect

B. Design a campaign and capture customer responses

C. Recruit train and manage employee exit

D. Request approve order receive match and pay

Your answer: _____

4. In an order process, which item is an input?

A. The average delivery time

B. Packing the goods

C. The customer order

D. The warehouse employee role

Your answer: _____

رومانی صبحی رزق جرجس مطر

Chapter 4 Practice Questions Continued

5. Which process component is illustrated by a credit-limit approval rule?

A. Processing equipment alone

B. Customer satisfaction metrics

C. Rules and controls

D. Outputs

Your answer: _____

6. Three orders take 4, 6, and 8 days to fulfill. What is their average cycle time?

A. 6 days

B. 8 days

C. 4 days

D. 18 days

Your answer: _____

7. If 60 of 75 eligible deliveries are on time, what is the on-time delivery rate?

A. 60%

B. 125%

C. 75%

2025/2026 2025/2026 2025/2026

D. 80%

Your answer: _____

8. Which statement about classifying payroll is most accurate?

A. It has no relationship to supporting processes

B. Its classification depends on the organization’s purpose and the framework used

C. It can never be described as routine operational work

D. It must be a core process in every organization

Your answer: _____

رومانی صبحی رزق جرجس مطر

Chapter 4 Practice Questions Continued

9. Which example most clearly represents a semi-structured process?

A. An activity with no defined purpose or result

B. Copying a file without making a decision

C. Routine payroll calculation using fully specified rules

D. Credit approval with defined checks and judgment for unusual cases

Your answer: _____

10. A system routes a purchase request to the assigned approver. Which role does this most directly illustrate?

A. Replacement of all controls

B. Coordination

C. Product disposal

D. Physical manufacturing

Your answer: _____

11. Which statement about automating a process is most accurate?

A. Automation guarantees that every decision is correct

B. Automation always eliminates all waiting time

2025/2026 2025/2026 2025/2026

C. Automation removes the need for exception handling

D. Incorrect rules can be repeated quickly and need validation

Your answer: _____

12. What is the main purpose of ERP?

A. Integrate major organizational functions and related data

B. Replace every employee judgment

C. Describe the physical layout of a LAN

D. Manage only advertising images

Your answer: _____

رومانی صبحی رزق جرجس مطر

Chapter 4 Practice Questions Continued

13. Which issue is especially important when implementing ERP?

A. Resolving conflicting master data and preparing users for changed work

B. Changing screen colors as the only project objective

C. Avoiding all tests of returns and exceptions

D. Assuming that every imported record is correct

Your answer: _____

14. Which activity is most directly associated with CRM?

A. Designing a physical motherboard

B. Controlling CPU instruction decoding

C. Tracking customer opportunities and service cases

D. Managing only warehouse shelving materials

Your answer: _____

15. Which system category primarily supports coordination of supply demand and logistics?

A. PLM

B. ECM

C. SCM

2025/2026 2025/2026 2025/2026

D. HCM

Your answer: _____

16. Which system category primarily manages product definitions versions and design changes?

A. CRM

B. PLM

C. HCM

D. ECM

Your answer: _____

رومانی صبحی رزق جرجس مطر

Chapter 4 Practice Questions Continued

17. What is the purpose of an As-Is process model?

A. Specify only the color of software screens

B. Guarantee that a proposed future process will succeed

C. Eliminate the need to consult process participants

D. Describe how the current process actually works

Your answer: _____

18. Which modeling technique is especially useful for showing departmental responsibility and handoffs?

A. An unlabeled sales chart

B. Swimlane diagram

C. A memory hierarchy diagram

D. A list of processor instruction codes

Your answer: _____

19. Which statement best describes BPR?

A. Fundamental redesign of a process rather than only a small incremental change

B. Changing one report font without reviewing work

C. Preserving every existing activity as an absolute requirement

2025/2026 2025/2026 2025/2026

D. Guaranteeing improvement without measuring results

Your answer: _____

20. A pilot reduces average cycle time from 6 to 4 days. What is the reduction relative to the baseline, rounded to one decimal place?

A. 50.0%

B. 33.3%

C. 66.7%

D. 2.0%

Your answer: _____

رومانی صبحی رزق جرجس مطر

Chapter 5

Securing Information Systems

IS101 | Academic year 2026–2027
Faculty of Computers and Informatics
Tanta University

Instructor and Course Coordinator
Prof. (Assoc.) Hany El-Ghaish

رومانی صبحی رزق جرجس مطر

5 1 Introduction

Organizations depend on information to serve customers, coordinate work, and make decisions. A loss of student records, an unauthorized change to an invoice, or an unavailable registration service can disrupt those activities. Security protects both information and the services that use it. Its scope includes digital systems, paper records, physical facilities, and human decisions.

This chapter explains security objectives, threats, controls, policies, privacy, and risk management. Protection must fit the organization's needs: overly broad access exposes information, while unnecessarily restrictive access can prevent legitimate work. Effective security combines safeguards with usable procedures, monitoring, response, and recovery.

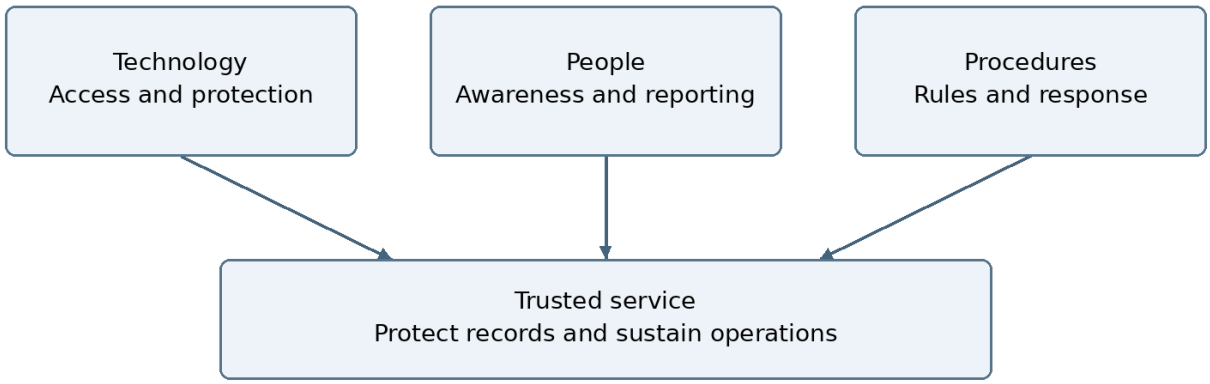


Figure 5.1. Security supports a dependable organizational service through complementary responsibilities. Original teaching diagram.

Learning objectives

After studying this chapter, you should be able to explain the CIA triad, distinguish threats from vulnerabilities and risk, and match controls to practical problems. You should also be able to distinguish authentication from authorization, explain MFA and encryption, interpret a basic risk register, and describe the relationship between security, privacy, policies, and recovery.

5 2 What is Information Security

Information security protects information and the systems that handle it against unauthorized reading, use, alteration, destruction, and disruption. The central objectives are confidentiality, integrity, and availability, often called the CIA triad. These objectives provide a useful way to describe the harm that a security event could cause. [1]

Objective	Meaning	University example
Confidentiality	Only authorized parties can access protected information	A student's private record is disclosed to an unrelated person
Integrity	Information is protected against improper alteration or destruction	A grade is changed without approval
Availability	Authorized users can access and use information when required	Registration becomes inaccessible during enrollment

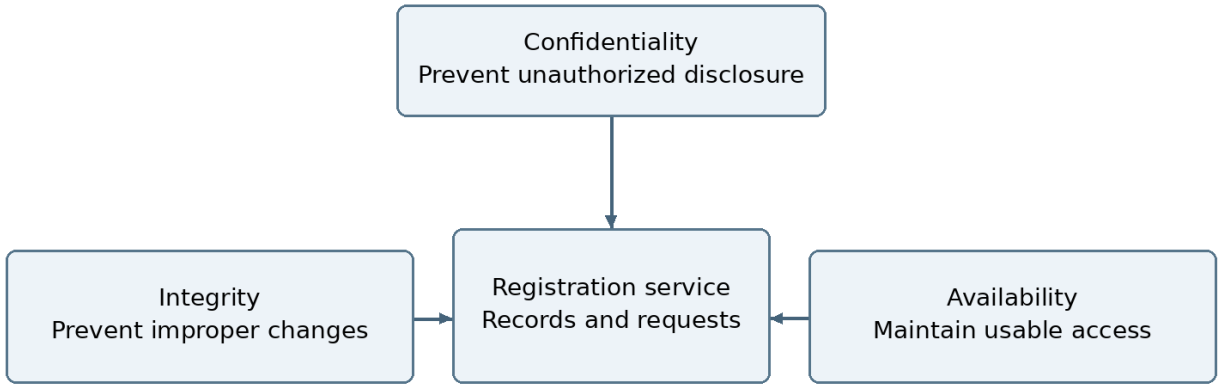


Figure 5.2. The CIA objectives apply together to one service. A single incident can harm more than one objective. Original teaching diagram.

Balancing the objectives

Encrypting a database can support confidentiality, but losing its encryption keys can damage availability. A backup can restore lost records, but an unprotected backup can expose the same records. Choose controls as a coordinated set rather than treating one objective as sufficient.

Authentication and non repudiation

Authentication establishes confidence in a claimed identity. Non-repudiation provides evidence of an action or message origin so that a later denial can be assessed; digital signatures and protected evidence can support it. Neither a password nor an ordinary log alone proves beyond all dispute that a particular person acted. Shared accounts, compromised keys, and unreliable evidence weaken attribution.

رومانی صبحی رزق جرجس مطر

5 3 Common Security Threats

A threat is a potential cause of harm. A vulnerability is a weakness that could allow the harm to occur. Risk concerns the likelihood and consequences of that harm in context. A stolen password is dangerous because of the access it can provide; the importance of the affected service determines the potential impact.



Figure 5.3. A risk scenario connects a source of harm, an exploitable weakness, and a business consequence. Original teaching diagram.

5 3 1 Human Threats

Threat	How it occurs	Illustration
Social engineering	Deception or pressure influences a person’s actions	A caller pretends to be IT support and asks for a login code
Insider threat	Authorized access is misused deliberately or unintentionally	A staff member exports records beyond an approved purpose
Human error	A mistake exposes or damages information	A confidential file is emailed to the wrong recipient

Phishing is a form of social engineering delivered through deceptive messages or websites. It can also involve technical mechanisms such as imitation login pages. The human and technical categories therefore overlap. Blaming users alone misses design problems such as confusing prompts, excessive permissions, and difficult reporting procedures.

A practical reporting example

A student receives an urgent message claiming that registration will be canceled unless a password is entered through a link. The student should open the university portal through a known route, verify the request through an official channel, and report the suspicious message. If credentials were entered, prompt reporting helps the support team contain the exposure.

رومانی صبحی رزق جرجس مطر

5 3 2 Technical Threats

Technical attacks exploit software behavior, communication paths, or system capacity. Their consequences depend on what the attacker can reach and what protections are in place. The examples below describe the concepts without requiring students to execute attacks.

Threat	Mechanism	Possible consequence
Malware	Malicious software runs on a device or service	Files stolen damaged or made inaccessible
Phishing	Deceptive messages or sites request unsafe actions	Credentials or payment details exposed
Denial of service (DoS) and distributed DoS (DDoS)	Resources are exhausted; DDoS uses distributed sources	Legitimate users cannot obtain service
SQL or code injection	Untrusted input is interpreted as a command	Data accessed or modified without authorization
Man in the middle	An adversary intercepts and may alter communication	Messages observed or substituted
Zero-day exploitation	A flaw is exploited before defenders have an effective patch available	Defenses based only on known patches may be insufficient

Recognizing malware categories

Category	Distinguishing feature
Virus	Attaches to a host file or program and spreads when it executes
Worm	Can propagate across systems without attaching to a host file
Trojan horse	Appears legitimate while carrying malicious behavior
Ransomware	Disrupts access and demands payment; may also steal data
Spyware	Collects information about activity without appropriate authorization

These labels describe different properties and can overlap. A Trojan can install ransomware, and stolen credentials can be used without malware. A firewall alone does not correct an application that builds database commands unsafely. OWASP recommends parameterized queries to keep query instructions separate from input values. [2]

5 3 3 Environmental Threats and Layered Defense

Security incidents do not always involve a malicious actor. Fire, water damage, power interruption, excessive heat, and equipment failure can prevent information systems from operating. Hardware failure is usually an operational or technical event rather than a natural disaster, but it belongs in the same continuity discussion.

Event	Potential effect	Relevant safeguard
Power interruption	Unexpected shutdown and disrupted transactions	Uninterruptible power supply (UPS) and safe shutdown
Fire or water damage	Facilities and equipment become unusable	Detection suitable suppression and alternate arrangements
Hardware failure	A server or storage device stops working	Monitoring redundancy and recoverable backups
Cooling failure	Equipment overheats and service is interrupted	Environmental monitoring and maintenance

Defense in depth

Defense in depth uses multiple complementary controls so that one failure does not immediately expose the whole service. In a registration system, identity checks, limited permissions, secure application design, network restrictions, logging, and backups protect different stages. Each layer must have a purpose and an owner.

2025/2026 2025/2026 2025/2026

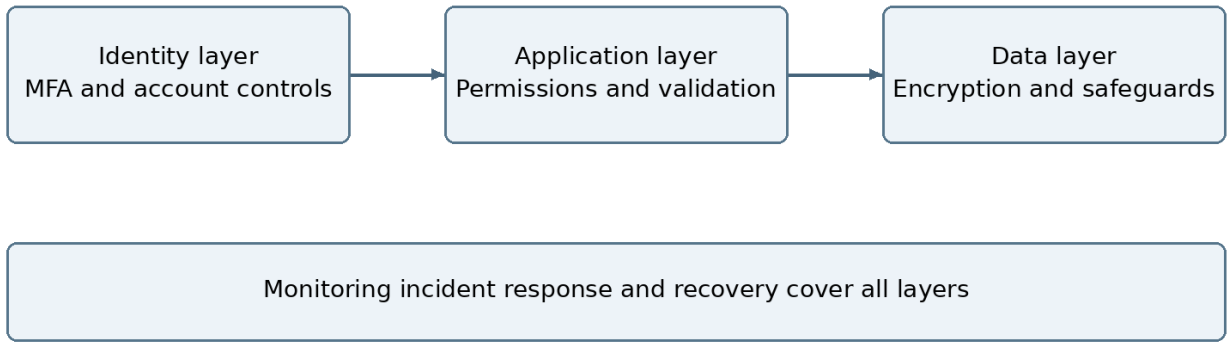


Figure 5.4. Complementary layers protect access and data; monitoring and recovery support the whole service. Original teaching diagram.

Redundancy improves resilience to some failures, but it is not a replacement for backup: accidental deletion or corruption may be replicated. Physical protection and alternate recovery arrangements address failures that affect an entire room or site.

5 4 Security Controls and Countermeasures

A security control is a safeguard that modifies risk. Controls are often grouped as technical, administrative, and physical. Another classification describes their role: preventive controls try to stop an event, detective controls reveal it, and corrective controls help limit damage or restore operation. One control can serve several roles.

5 4 1 Technical Controls

Control	Main purpose	Important limitation
Authentication	Verify a claimed identity	Successful login does not authorize every action
Access control	Permit only approved operations and resources	Permissions require review as roles change
Encryption	Protect data using cryptographic keys	Keys and decrypted endpoints still need protection
Firewall	Permit or restrict traffic according to rules	Allowed traffic can still contain harmful activity
Anti-malware	Detect or block malicious software behavior	Detection is imperfect and requires maintenance
IDS and IPS	Identify suspicious activity; IPS can block selected traffic	Alerts need investigation and tuning
Backup and recovery	Restore usable information and services	A backup must be protected and restoration tested

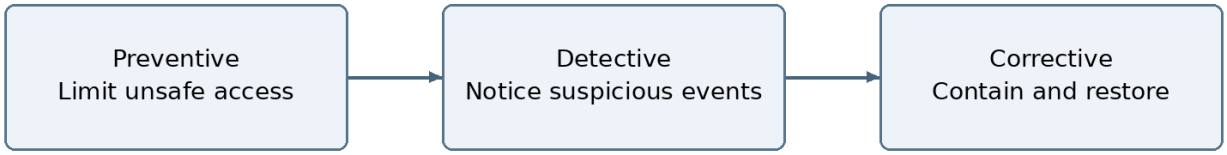


Figure 5.5. Controls contribute before during and after an incident. These roles complement rather than replace one another. Original teaching diagram.

An intrusion detection system (IDS) mainly raises alerts about suspicious activity. An intrusion prevention system (IPS) is positioned to take preventive action, such as dropping selected traffic. Blocking decisions can interrupt legitimate work, so rules need testing and monitoring. A web application firewall examines web requests, while secure coding addresses the underlying application behavior.

5 4 Authentication Authorization and Least Privilege

Authentication asks who is requesting access. Authorization asks which operations that identity may perform. A student and an instructor may both authenticate successfully while receiving different permissions. Least privilege means granting only the access needed for an approved task, for the appropriate duration.

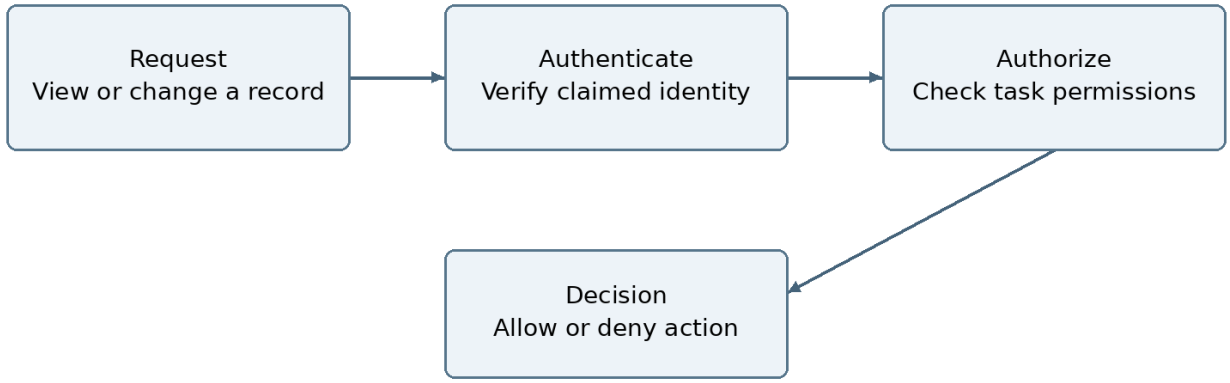


Figure 5.6. Identity verification and permission checks are separate decisions. An authenticated user can still be denied a particular operation. Original teaching diagram.

Multi factor authentication

Multi-factor authentication (MFA) requires distinct types of evidence, such as something known and something possessed. A password and a hardware security key are different factors; two passwords are not. Phishing-resistant methods, such as properly deployed FIDO-based authenticators, better resist imitation login sites than reusable codes. Code-based MFA still adds protection, but codes can be stolen through phishing. [3]

Role	Example permitted action	Example restricted action
Student	View own registration and submit a request	Change another student’s grade
Instructor	Enter approved results for assigned classes	Administer unrelated financial accounts
System administrator	Maintain systems under approved procedures	Browse personal records without a work purpose

Managing access throughout employment

Approve access when a person joins, revise it when responsibilities change, and remove it promptly on departure. Separate routine accounts from privileged administrative use where appropriate. Periodic reviews help detect permissions that remain after their original purpose has ended.

5 4 Encryption Backup and Recovery

Encryption transforms readable data into ciphertext using a cryptographic algorithm and key. Authorized decryption recovers the content. Data in transit moves between systems; data at rest is stored on devices or services. HTTPS (Hypertext Transfer Protocol Secure) uses TLS (Transport Layer Security) to protect a connection when certificates are validated correctly. It does not prove that the destination's business is trustworthy.

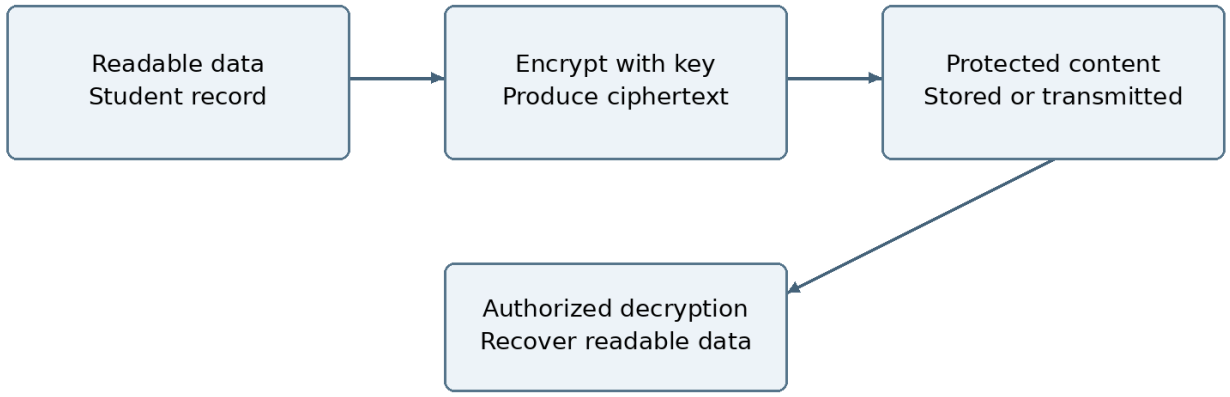


Figure 5.7. Encryption protects content during storage or transfer. Key management and endpoint security remain essential. Original teaching diagram.

2025/2026 2025/2026 2025/2026

Encryption does not prevent an authorized account from misusing data after decryption. Hashing produces a digest for purposes such as comparison or integrity checks and is not reversible encryption. Digital signatures support checking origin and integrity; they do not automatically hide message content.

Planning for restoration

Backups preserve recoverable copies. Recovery includes restoring data, applications, configurations, and access so that the service works again. Maintain copies isolated from ordinary production access, such as offline or appropriately protected immutable backups. CISA's ransomware guidance emphasizes safeguarded backups and recovery readiness. A successful backup job is not proof of a successful restore. [4]

Planning term	Meaning	Illustrative requirement
Recovery point objective RPO	Target maximum amount of data loss measured in time	Recover to a point no more than 4 hours before disruption
Recovery time objective RTO	Target elapsed time for restoring a service after disruption	Restore priority registration service within 8 hours

RPO and RTO are targets rather than guarantees. Choose a backup schedule and recovery design that can meet them, then test actual restoration. A four-hour RPO and eight-hour RTO describe different requirements: allowable lost history and allowable outage time.

5 4 Administrative Physical and Response Controls

Administrative controls define responsibilities and procedures. Physical controls protect facilities and equipment. These categories support technical safeguards: access software is weakened if staff share accounts, and strong network controls cannot compensate for an unlocked room containing unprotected records.

Category	Examples	Contribution
Administrative	Training access reviews incident plans and approved screening	Define expected conduct and accountable decisions
Physical	Locked rooms access cards cameras and equipment protection	Restrict physical access and reveal unauthorized entry
Environmental	Fire detection cooling and power safeguards	Reduce damage and sustain safe operation

Incident reporting and response

An event is an observable occurrence; an incident is an occurrence that compromises security or requires a security response under the organization’s criteria. Students and employees should know where to report suspected incidents. The response team assesses the report, contains the affected activity, investigates the cause, and coordinates recovery. Evidence handling and communication should follow the approved plan.

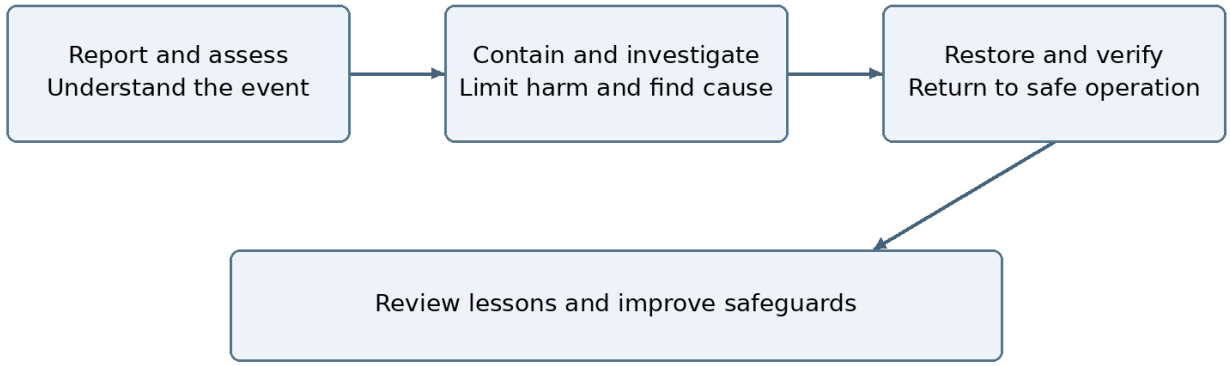


Figure 5.8. A simplified incident-handling example. Actions may repeat as new evidence appears; this is not a rigid universal sequence. Original teaching diagram.

For a suspected compromised account, authorized responders might restrict sessions, reset affected credentials, investigate activity, and confirm the account is safe before normal use resumes. Users should report promptly rather than independently delete evidence or attempt unapproved investigation. Clear communication helps staff act effectively under pressure.

5 5 Security Policies

A security policy states management's expectations for protecting information assets. It defines scope, responsibilities, rules, and accountability. A standard gives specific required criteria, while a procedure describes how to perform a task. A policy should be understandable, approved, communicated, and reviewed when systems or risks change.

Policy topic	Question it should answer
Acceptable use	Which activities are allowed on organizational systems?
Authentication	Which credentials and MFA methods are required?
Access management	Who approves reviews and removes permissions?
Data classification	Which handling rules apply to each sensitivity level?
Communication	How may information be emailed shared or published?
Incident reporting	Where and how quickly should concerns be reported?
Accountability	How are exceptions and violations addressed?

Classifying information

2025/2026 2025/2026 2025/2026

Illustrative label	Example	Handling approach
Public	Published course catalog	Approved publication and change control
Internal	Routine staff procedure	Share within authorized organizational use
Confidential	Personal student record	Limit access and protect storage and transfer
Restricted	Highly sensitive credentials or records	Apply especially strict access and handling

Labels and requirements vary by organization. Classification should follow the effect of disclosure or misuse, and handling rules should be concrete. Merely marking a document confidential does not enforce access restrictions.

A usable authentication policy

Require unique accounts, appropriate MFA, secure account recovery, and prompt reporting of suspected compromise. Encourage long unique passwords and password managers. NIST's current guidance rejects arbitrary routine password changes absent evidence of compromise and emphasizes length and checks against commonly used or compromised passwords. This is a technical guidance example, not a universal legal requirement. [5]

5 6 Privacy and Regulatory Compliance

Privacy concerns how personal information is collected, used, shared, and retained, and the effects on individuals. Security protects that information against misuse and exposure. A secure database can still create a privacy problem if it collects unnecessary information or uses it for an incompatible purpose.

Principle	Practical application
Purpose and transparency	Explain what information is needed and how it will be used
Data minimization	Collect information necessary for the stated purpose
Lawful processing	Identify a valid legal basis; obtain consent where required
Accuracy and rights	Maintain appropriate accuracy and support applicable individual rights
Storage limitation	Define retention periods and lawful deletion arrangements
Security and accountability	Protect information and demonstrate responsible handling

For example, registering for a routine class may require a student identifier and course selection, but not unrelated medical history. Access to necessary information should be limited to relevant roles. The European Commission explains these principles for GDPR processing; consent is one possible legal basis rather than a requirement for every processing activity. [6, 7] 2025/2026 2025/2026



Figure 5.9. Privacy decisions span the information lifecycle and require an identified purpose. Original teaching diagram.

Examples of regulatory frameworks

The EU General Data Protection Regulation (GDPR) governs personal-data processing within its scope, including some organizations outside the EU. California's Consumer Privacy Act (CCPA), amended by the California Privacy Rights Act (CPRA), gives covered consumers rights concerning personal information and creates duties for covered businesses. Examples include rights to know, correct, delete, and opt out of certain sale or sharing, subject to conditions and exceptions. Other jurisdictions have their own requirements. [8]

Organizations need to establish which laws apply to their activities and maintain evidence of compliance. Consequences can include enforcement, financial penalties, operational disruption, and loss of trust. Specific obligations depend on jurisdiction and circumstances; a generic checklist cannot establish compliance.

5 7 Risk Management in Information Security

Risk management helps an organization choose safeguards proportionate to its assets, threats, vulnerabilities, and objectives. The aim is to keep risk within approved limits, not to promise perfect protection. Risk assessment needs both technical knowledge and an understanding of business consequences.

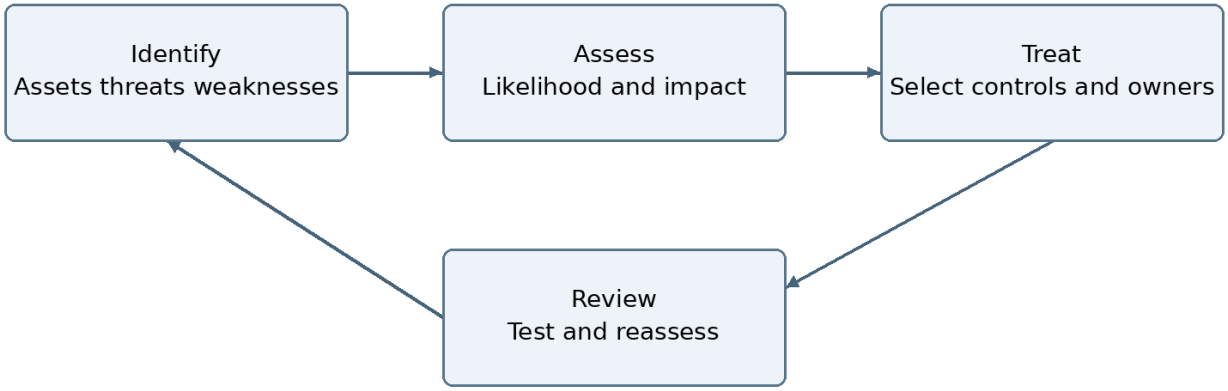


Figure 5.10. Risk management links assessment treatment and review. Review feeds the next assessment as conditions change. Original teaching diagram.

The basic process

Identify the information and services that need protection, including dependencies such as network links and identity providers. Describe realistic threat scenarios and relevant weaknesses. Assess likelihood and impact using an agreed method. Select safeguards, assign owners, and review whether the intended reduction in risk has occurred.

Response	Meaning	Example
Avoid	Stop or change the activity creating the risk	Discontinue an unnecessary collection of sensitive data
Reduce	Apply safeguards to lower likelihood or impact	Add MFA and restrict sensitive access
Transfer or share	Allocate selected consequences through a contract or insurance	Purchase insurance for specified covered losses
Accept	Approve remaining exposure within defined limits	Record a low-priority residual risk for review

Residual risk remains after treatment. Insurance does not prevent an incident or remove all organizational responsibility. Acceptance should have an accountable owner, a rationale, and a review date; leaving a weakness unattended is not the same as a documented acceptance decision.

5 7 Worked Risk Assessment and Governance

A simple teaching method rates likelihood and impact from 1 to 5, then multiplies them to prioritize scenarios. In the table, L means likelihood and I means impact. This ordinal score is a screening aid, not a probability, a financial loss estimate, or a universal formula. Two scenarios with the same score can require different responses.

Scenario	L	I	Score	Selected treatment
Staff account used to expose student records	4	5	20	MFA least privilege and access review
Short local power interruption affects registration	3	4	12	UPS safe shutdown and tested recovery
Wrong recipient receives a sensitive attachment	3	4	12	Sharing controls verification and reporting

For the account scenario, 4 × 5 = 20. After safeguards are implemented and tested, a reassessment might rate likelihood 2 and impact 5, producing 2 × 5 = 10. This fictional example does not claim that risk was precisely halved: the ratings express judgment on an ordinal scale. Severe consequences still deserve attention.

A practical risk register

Record the asset, scenario, existing controls, assessment, proposed action, owner, deadline, residual risk, and next review. For a registration service, the owner should consider peak enrollment periods, recovery dependencies, and the effect on students. Review evidence from exercises and actual incidents rather than rely only on a statement that a control exists.

Connecting risk to governance

NIST’s Cybersecurity Framework 2.0 organizes outcomes into Govern, Identify, Protect, Detect, Respond, and Recover. Govern addresses direction and accountability; the other functions cover understanding risk, protection, detection, incident handling, and restoration. These functions interact continuously rather than form a one-time checklist. [9]

Student application

A department keeps student records in a shared folder that all staff can edit. Identify the asset, one threat, one vulnerability, and possible CIA consequences. Propose a preventive control, a detective control, and a recovery measure. Explain which role should approve access and review the remaining risk.

5 8 Best Practices for Securing Information Systems

Good practice combines routine prevention with the ability to recognize problems and recover. Select actions according to the service's risks, then verify that they work. Users, managers, developers, and administrators each contribute; usable systems and clear responsibilities support safer behavior.

Practice	Purpose	Evidence of implementation
Strong authentication	Reduce misuse of accounts	MFA enrollment and protected recovery processes
Timely updates	Address known vulnerabilities	Patch inventory deployment records and exception review
Least privilege	Limit unnecessary access	Approved roles and completed access reviews
Protect sensitive data	Reduce exposure in storage and transit	Encryption configuration and key access controls
Continuous education	Improve recognition and reporting	Relevant exercises and an accessible reporting route
Monitoring	Reveal suspicious activity	Reviewed alerts protected logs and response records
Tested recovery	Restore service after disruption	Documented restore exercises and measured results
Regular assessment	Find weaknesses and verify controls	Authorized audit findings and tracked corrective actions

A day to day example

A staff member uses a unique account with MFA, accesses only assigned records, checks recipients before sharing, and reports suspicious messages. Administrators maintain updates, monitor meaningful events, and test restoration. Management provides resources, assigns ownership, and reviews whether safeguards fit operational needs.

Measuring security performance

Useful measures include coverage of MFA on relevant accounts, time to correct prioritized vulnerabilities, completion of access reviews, and recovery exercise results. A high number of blocked messages alone does not prove that a system is secure. Measures should relate to risks and reveal gaps that require action.

Check your understanding: explain why training cannot replace secure application design, why a firewall cannot replace least privilege, and why a backup should be tested. For each, describe the different problem addressed by the complementary control.

5 9 Chapter Summary

Information security protects confidentiality, integrity, and availability across information and services. Authentication verifies claimed identity, authorization determines permitted actions, and reliable evidence supports accountability. Threats can involve people, software, communication, facilities, or operational failures; categories can overlap.

Technical, administrative, and physical controls work together. MFA, least privilege, encryption, secure application design, monitoring, and tested recovery reduce different risks. Policies establish expectations, while privacy addresses appropriate collection and use as well as protection. Applicable legal duties require attention to context and jurisdiction.

Security remains an ongoing management responsibility. Identify assets and scenarios, assess likelihood and impact, select treatments, and review residual risk. Continue to test safeguards and learn from incidents as technology and organizational needs change.

Key Terms

Term	Meaning
Information security	Protection of information and the services handling it
CIA triad	Confidentiality integrity and availability objectives
Threat	Potential cause of a harmful event
Vulnerability	Weakness that can enable harm
Malware	Software with malicious behavior
Phishing	Deceptive messages or sites that encourage unsafe actions
Authentication	Verification of a claimed identity
Authorization	Determination of permitted actions
MFA	Authentication using distinct factor types
Encryption	Key-based transformation protecting readable content
Firewall	Traffic control according to defined rules
Least privilege	Only the access necessary for an approved task
Security policy	Approved expectations for information protection
Data privacy	Appropriate processing and protection of personal data
Risk management	Assessment treatment and review of risk
Residual risk	Risk remaining after treatment

رومانی صبحی رزق جرجس مطر

References and Figure Credits

The ten figures are original explanatory diagrams. University scenarios and risk scores are fictional teaching examples. The sources below support the technical guidance and privacy overview. Sources checked on 9 October 2026.

[1] NIST CSRC. Information security glossary.

https://csrc.nist.gov/glossary/term/information_security

[2] OWASP. SQL Injection Prevention Cheat Sheet.

https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

[3] CISA. Implementing Phishing Resistant MFA.

https://www.cisa.gov/sites/default/files/2023-01/fact-sheet-implementing-phishing-resistant-mfa-508c.pdf

[4] CISA. StopRansomware Guide.

https://www.cisa.gov/stopransomware/ransomware-guide

[5] NIST. SP 800 63B 4 Digital Identity Guidelines Authentication and Authenticator Management.

https://pages.nist.gov/800-63-4/sp800-63b.html

[6] European Commission. Principles of personal data processing under the GDPR.

https://commission.europa.eu/law/law-topic/data-protection/information-business-and-organisations/principles-gdpr_en

[7] European Commission. Legal grounds for processing data.

https://commission.europa.eu/law/law-topic/data-protection/information-business-and-organisations/legal-grounds-processing-data_en

[8] California Attorney General. California Consumer Privacy Act.

https://oag.ca.gov/privacy/ccpa

[9] NIST. Cybersecurity Framework.

https://www.nist.gov/cyberframework

2025/2026 2025/2026 2025/2026

Chapter 5 Practice Questions

Choose one best answer for each question. Write its letter in the answer space provided.

Student name _____ Date _____

1. Which event most directly violates confidentiality?

A. An unauthorized person reads a student's private record

B. An approved correction updates a record

C. A scheduled server shutdown delays access

D. An authorized student submits a request

Your answer: _____

2. A grade is changed without approval. Which objective is most directly affected?

A. Availability

B. Confidentiality alone

C. Integrity

D. Data minimization

Your answer: _____

3. Which event most directly threatens availability?

2025/2026 2025/2026 2025/2026

A. A student uses an approved password manager

B. An approved retention rule deletes expired records

C. A traffic flood prevents legitimate access to a portal

D. An authorized adviser reads a permitted record

Your answer: _____

4. Which statement distinguishes a threat from a vulnerability?

A. A vulnerability is always a malicious person

B. A threat can cause harm; a vulnerability is a weakness that can enable it

C. A threat is always a software defect

D. The two terms refer only to financial loss

Your answer: _____

رومانی صبحی رزق جرجس مطر

Chapter 5 Practice Questions Continued

5. How is phishing related to social engineering?

A. It is a form of deception that can use technical delivery mechanisms

B. It is only a natural disaster

C. It requires physical access to a server room

D. It always spreads without a person taking any action

Your answer: _____

6. Which malware category can propagate without attaching to a host file?

A. Worm

B. Password manager

C. Firewall

D. Virus

Your answer: _____

7. What distinguishes DDoS from an ordinary single-source DoS attack?

A. DDoS cannot affect websites

B. DDoS can only change paper records

C. DDoS always depends on a stolen encryption key

2025/2026 2025/2026 2025/2026

D. Distributed sources contribute to service disruption

Your answer: _____

8. Which application design measure directly helps prevent SQL injection?

A. Turning off audit logs

B. Storing passwords in readable form

C. Parameterized queries that separate instructions from input values

D. A shared administrator account

Your answer: _____

رومانی صبحی رزق جرجس مطر

Chapter 5 Practice Questions Continued

9. Which measure is a physical security control?

A. Restricting entry to a server room with controlled door access

B. Reviewing an employee's application role

C. Scanning a downloaded file for malware

D. Encrypting database content

Your answer: _____

10. Which question is answered by authorization?

A. Which algorithm compresses a file?

B. How much backup storage is installed?

C. Which identity is being claimed?

D. Which actions may this verified identity perform?

Your answer: _____

11. Which example uses distinct authentication factors?

A. Two answers to knowledge-based questions

B. A password and a hardware security key

C. A username and a display name

2025/2026 2025/2026 2025/2026

D. A password and a second password

Your answer: _____

12. What is the principle of least privilege?

A. Retain every former employee account indefinitely

B. Grant only the access needed for an approved task

C. Give every user administrator access

D. Allow all access after a successful login

Your answer: _____

رومانی صبحی رزق جرجس مطر

Chapter 5 Practice Questions Continued

13. Which statement about encryption is accurate?

A. Hashing and reversible encryption are identical

B. Authorized endpoints and decryption keys still require protection

C. HTTPS proves that every website operator is trustworthy

D. Encryption prevents every authorized user from misusing data

Your answer: _____

14. What is the primary distinction between IDS and IPS?

A. IDS encrypts files while IPS stores backups

B. IDS replaces all investigation while IPS replaces policies

C. IDS approves personnel access while IPS hires staff

D. IDS mainly detects and alerts while IPS can block selected traffic

Your answer: _____

15. A service has a four-hour RPO. What does this target describe?

A. The required password reset interval

B. A guaranteed four-hour total outage

C. The minimum age of every backup

2025/2026 2025/2026 2025/2026

D. The maximum targeted lost data history measured in time

Your answer: _____

16. Why should an organization test backup restoration?

A. Testing automatically fulfills every privacy obligation

B. Testing guarantees that no future incident can occur

C. Restoration tests eliminate the need to protect backups

D. A completed backup job does not prove that usable service can be restored

Your answer: _____

رومانی صبحی رزق جرجس مطر

Chapter 5 Practice Questions Continued

17. Which practice best illustrates data minimization?

A. Keep every record forever

B. Collect only personal information needed for the stated purpose

C. Give all staff access to all personal records

D. Collect all available personal information in case it becomes useful

Your answer: _____

18. Which statement about consent and privacy law is most accurate?

A. Consent is required in some circumstances; other lawful bases can apply

B. CCPA applies identically to every organization worldwide

C. Encryption automatically establishes a lawful processing basis

D. Every processing operation everywhere must use consent

Your answer: _____

19. Using the chapter's illustrative scoring method, likelihood 3 and impact 4 produce which score?

A. 1

B. 7

C. 12

2025/2026 2025/2026 2025/2026

D. 34

Your answer: _____

20. What is residual risk?

A. Only the cost of buying a firewall

B. Risk that is always eliminated by insurance

C. Risk that remains after controls or other treatment

D. A guarantee that all controls will fail

Your answer: _____

رومانی صبحی رزق جرجس مطر

Chapter 6

Emerging Technologies and the Future of Information Systems

IS101 | Academic year 2026–2027
Faculty of Computers and Informatics
Tanta University

Instructor and Course Coordinator
Prof. (Assoc.) Hany El-Ghaish

رومانی صبحی رزق جرجس مطر

6 1 Introduction

Information systems evolve as organizations adopt new ways to collect, connect, analyze, and use information. Cloud computing, connected devices, and artificial intelligence can expand a service's capabilities, but useful adoption depends on the problem being solved. A technology creates value when it improves an outcome under realistic cost, reliability, security, and human constraints.

This final chapter introduces important technologies and explains how they can work together. Some, including cloud services and many analytics tools, are already widely established. Others have emerging applications or remain research-intensive. Future developments should be treated as possibilities to evaluate rather than guaranteed benefits.

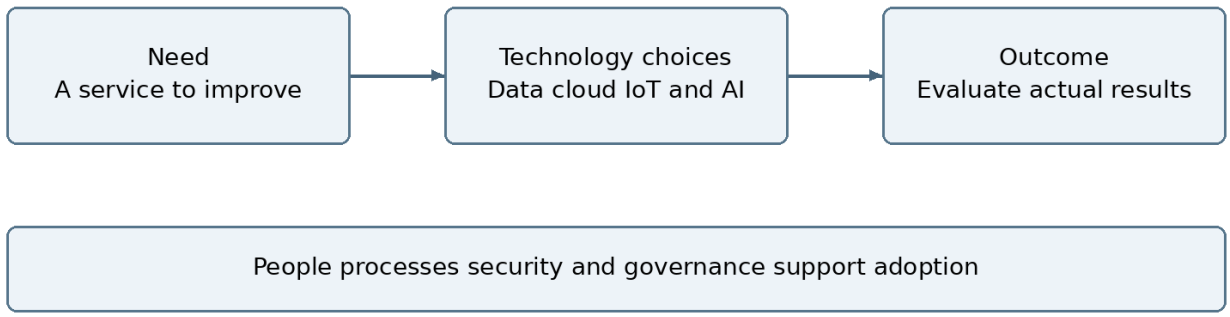


Figure 6.1. Technology choices connect organizational needs with measurable outcomes. Original teaching diagram.

Learning objectives

You should be able to compare cloud service and deployment models, explain Big Data characteristics, and trace an IoT data flow. You should also distinguish AI from ML, compare learning approaches, explain edge computing and digital twins, and assess an adoption proposal using benefits, costs, evidence, and risks.

6 2 Cloud Computing

Cloud computing provides convenient access over a network to configurable computing resources that can be provisioned and released with limited management effort. Services may include virtual machines, storage, databases, and applications. The public Internet is common, but private cloud access can use organizational networks. Pay-as-you-go pricing is common rather than a requirement for every cloud deployment. [1]

Essential characteristics

Characteristic	Meaning	Example
On-demand self-service	Users provision resources through an agreed service interface	Request a virtual machine without ordering a physical server
Broad network access	Services are reachable over networks using suitable clients	Access an application from a laptop or approved phone
Resource pooling	Provider resources support multiple consumers with allocation controls	Compute capacity is assigned across workloads
Rapid elasticity	Capacity can expand or contract with demand	Increase capacity during enrollment
Measured service	Resource use is monitored and accounted for	Track storage usage or processing time

These five characteristics are part of NIST’s widely used cloud definition. Buying a server and connecting it to the Internet does not, by itself, establish a cloud service. Elasticity concerns adapting capacity to demand; scalability concerns the ability to handle a larger workload. [1]

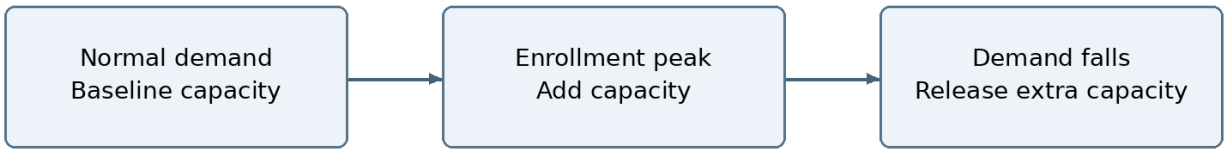


Figure 6.2. Elastic capacity follows changing demand when scaling is configured and supported. The illustration is conceptual rather than measured data. Original teaching diagram.

A university may temporarily add application capacity during enrollment. The database, network, and application design must also support the peak load. Automatic scaling cannot repair an inefficient query or an incorrect business rule, and unused provisioned capacity can still generate charges.

6 2 Cloud Service Models

Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS) describe which parts of a computing service are supplied and managed. They offer different levels of control and operational work. The exact allocation depends on the product and contract. [1, 2]

Model	Provider generally operates	Customer still handles	Examples
IaaS	Physical infrastructure and virtualization	Guest OS applications data access and configuration	Amazon EC2 and Azure Virtual Machines
PaaS	Infrastructure OS and managed runtime or platform	Application code data access and relevant configuration	Google App Engine and Azure App Service
SaaS	Application and underlying platform and infrastructure	Data use accounts permissions configuration and endpoint security	Microsoft 365 Salesforce and Google Workspace

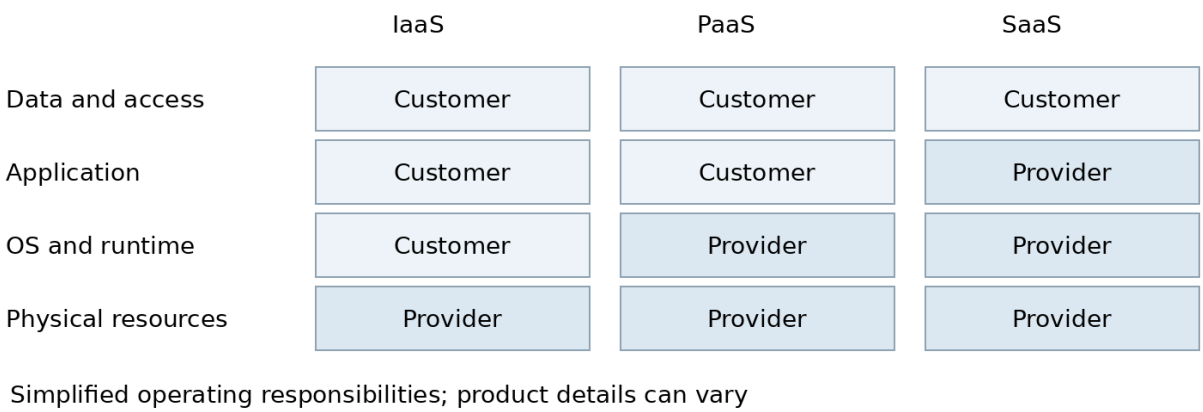


Figure 6.3. A simplified cloud responsibility matrix. Provider operation does not remove customer duties for data use identity access and configuration. Original teaching diagram.

SaaS does not mean that the customer has no security responsibilities. Microsoft's shared-responsibility guidance emphasizes continuing customer responsibilities for information, identities, and access. Customers also need to understand retention, export, recovery, and service availability arrangements. [2]

Choosing a model

An organization maintaining a custom server environment may prefer IaaS. A development team may use PaaS to deploy its application without administering the underlying operating system. A department needing a standard collaboration application may choose SaaS. Greater control usually brings more management responsibility.

6 2 Deployment Models Benefits and Challenges

Deployment models describe who a cloud serves and how cloud environments are related. A private cloud can be hosted on-site or externally; private does not automatically mean secure. A public cloud is offered for public use, with isolation and access controls for customers. Public infrastructure does not make each customer’s records public.

Model	Arrangement	Illustration
Public	Services offered to a broad set of customers	A provider hosts services for many organizations
Private	Exclusive use by one organization	A university operates a dedicated cloud environment
Hybrid	Distinct cloud environments connected for coordinated use	A private cloud exchanges workloads or data with a public cloud
Community	Exclusive use by a group with shared concerns	Several institutions share a cloud for a common mission

NIST includes community cloud as well as public, private, and hybrid deployment. Hybrid is more specific than simply using two unrelated online applications. Integration must address identity, data movement, networking, and operating procedures. [1]

2025/2026 2025/2026 2025/2026

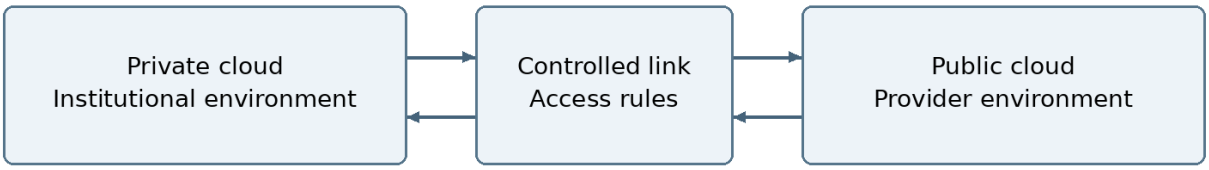


Figure 6.4. A hybrid arrangement links distinct cloud environments. The connection requires explicit data and access rules. Original teaching diagram.

Potential benefit	Associated planning concern
Lower upfront equipment spending	Total recurring charges can exceed expectations
Flexible capacity and faster provisioning	Scaling needs appropriate design and cost controls
Remote access	Connectivity and endpoint protection remain necessary
Managed maintenance	The provider updates only the layers covered by the service
Reduced local infrastructure work	Provider dependency portability and contracts need review

Plan for outages, export formats, transfer charges, identity protection, data location, and applicable privacy duties. Vendor lock-in means that switching provider may be difficult or costly because applications depend on proprietary interfaces, formats, or services. Evaluate exit options before committing to a design.

6 3 Big Data and Analytics

Big Data describes datasets whose scale, arrival rate, diversity, or variability creates a need for scalable methods of storage and analysis. There is no universal number of rows or bytes that makes data “big.” A workload that overwhelms one system may be manageable on another. NIST emphasizes scalable architecture in its definition. [3]

Characteristic	Meaning	Campus illustration
Volume	The amount of data to store or process	Many years of building-sensor readings
Velocity	The arrival rate and required response speed	Continuous alerts from equipment
Variety	Different data structures and formats	Tables JSON messages images and maintenance notes
Veracity	Reliability quality and uncertainty	A sensor produces inaccurate readings
Value	Useful outcomes from analysis	Identify avoidable energy waste

Volume, velocity, and variety are a common introductory framework. Additional lists may include veracity, value, or variability; there is no single mandatory list of Vs. Data quality and useful questions matter even when large-scale processing is technically possible.

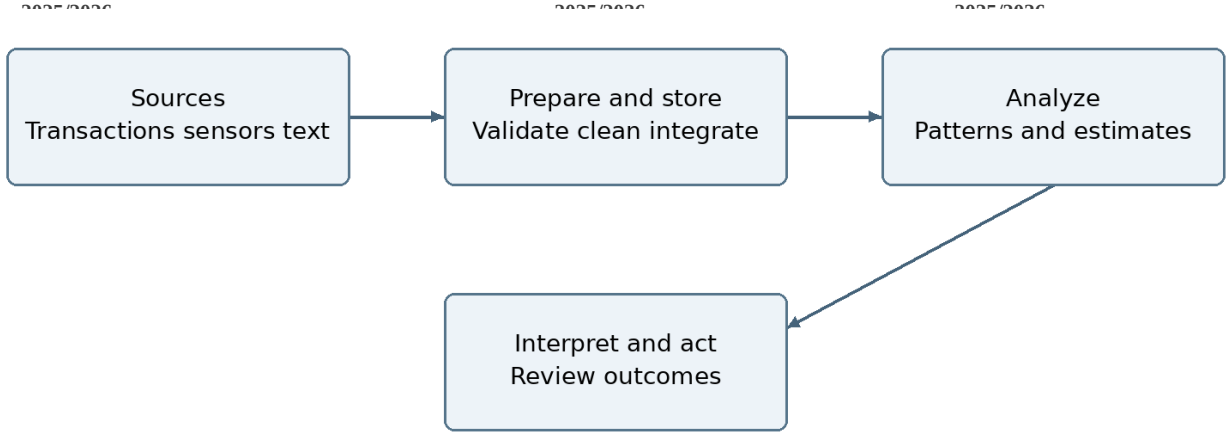


Figure 6.5. An analytics pipeline turns varied sources into usable evidence. Quality access controls and traceability apply throughout. Original teaching diagram.

Batch and streaming analysis

Batch processing analyzes a collected set at scheduled intervals, such as daily energy totals. Streaming processing handles events as they arrive, such as a temperature alert. Near-real-time results still have delays from sensing, communication, computation, and delivery. The required response time should follow the operational need.

Distributed processing divides work across computers, but coordination, data movement, and consistency introduce overhead. More machines do not automatically make every calculation faster. Choose a design according to the workload rather than the label Big Data.

6 3 Analytics Questions and Practical Value

Analytics should begin with a decision or question. A chart of past performance, a forecast, and a recommended action answer different questions. Data preparation, appropriate evaluation, and interpretation connect the technical output to an organizational result.

Analytics type	Question	Example
Descriptive	What happened?	Report electricity used by building last month
Diagnostic	What factors may explain the result?	Compare operating hours and usage patterns
Predictive	What is likely to happen?	Estimate next week's demand
Prescriptive	What action could improve the outcome?	Suggest schedules under comfort and cost constraints

A worked data-volume example

Suppose 100 sensors each record one reading per minute. They produce 100 × 60 × 24 = 144,000 readings per day. If each stored record averages 100 bytes, the raw daily total is 14,400,000 bytes, about 14.4 MB using decimal units. Thirty days produce about 432 MB before indexes, metadata, replicas, or backups. The example is fictional and does not establish a Big Data threshold.

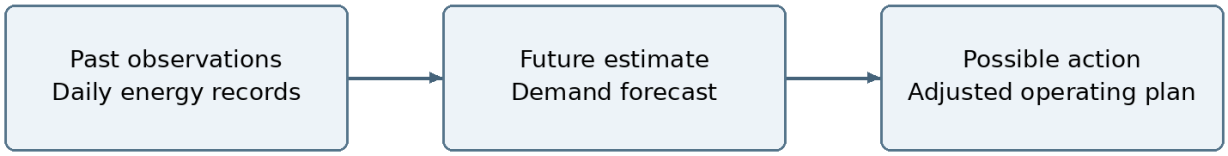


Figure 6.6. The same data can support different questions. Moving to a recommendation adds assumptions and constraints. Original teaching diagram.

Applications and limitations

Organizations use analytics for customer patterns, fraud alerts, maintenance planning, marketing, and clinical decision support. A detected pattern may help prioritize investigation, but correlation alone does not establish causation. A system that flags a transaction still needs an appropriate review process. Clinical applications require domain-specific evaluation and accountable professional oversight.

For the campus example, an increase in electricity use might reflect longer building hours rather than faulty equipment. Compare relevant conditions before deciding on maintenance. Useful analysis states uncertainty, documents assumptions, and avoids treating incomplete measurements as certain evidence.

رومانی صبحی رزق جرجس مطر

6 4 Internet of Things

The Internet of Things (IoT) connects physical devices with computing and communication so they can observe or affect their environment. Sensors measure properties such as temperature, movement, or pressure. Actuators change a physical state, such as opening a valve. IoT combines sensing, communication, processing, and sometimes actuation. [4]

Setting	Example device	Possible use
Home	Connected thermostat	Monitor temperature and adjust heating
Personal activity	Wearable motion sensor	Summarize activity patterns
Industry	Machine vibration sensor	Detect abnormal equipment behavior
Transport	Connected vehicle sensor	Report condition and operational events
City or campus	Environmental or occupancy sensor	Support lighting traffic or building management

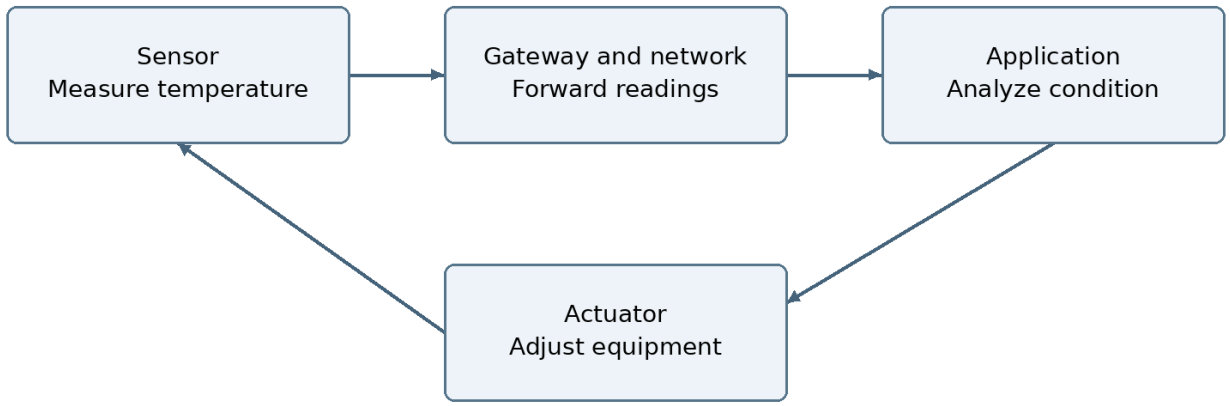


Figure 6.7. A simplified IoT feedback loop. Sensing provides data and approved control decisions may drive an actuator. Original teaching diagram.

A gateway can aggregate devices and translate protocols before forwarding data. Processing may happen on the device, at a nearby edge server, or in a remote service. Devices do not all require a direct public Internet connection. Some systems monitor conditions without automatically controlling equipment.

A campus monitoring example

A sensor reports a laboratory's temperature through a gateway. An application displays readings and alerts the facilities team when a configured limit is exceeded. The limit should reflect validated operational requirements, and the team should verify the condition before taking action where safety or uncertainty warrants review.

رومانی صبحی رزق جرجس مطر

6 4 IoT Reliability Security and Edge Computing

Connected devices need maintenance throughout their lifetime. Poorly calibrated sensors, missing timestamps, interrupted power, and network outages can all weaken the resulting information. Device identity, secure updates, access control, and support arrangements should be considered before deployment.

Concern	Why it matters	Planning response
Security and privacy	Devices can expose records or physical controls	Limit access protect communication and minimize personal data
Interoperability	Devices may use incompatible formats or protocols	Use defined interfaces and test integration
Power and reliability	Battery failure or disconnection creates missing observations	Monitor health and plan maintenance
Data quality	Poor calibration or timing gives misleading results	Validate readings timestamps and units
Lifecycle support	Unsupported devices may retain unresolved weaknesses	Maintain inventory update plans and replacement criteria

رومانی صیحی رزق جرجس مطر

2025/2026 2025/2026 2025/2026

Edge computing

Edge computing performs processing near the data source, such as on a device or local gateway. It can reduce network transfers and support faster local responses. Cloud services can still retain longer-term records and perform broader analysis. NIST’s fog-computing model discusses distributed processing between devices and centralized services. [5]

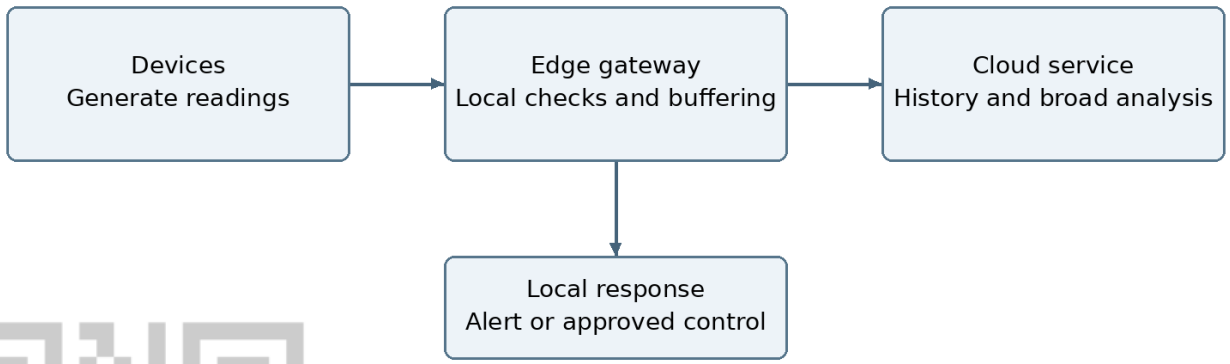


Figure 6.8. Local processing and cloud analysis serve different needs. The edge can forward summaries while retaining a defined local function. Original teaching diagram.

For a classroom, a local controller might continue its validated temperature rule during a network outage while buffering readings for later upload. Edge processing does not remove security duties or guarantee safe behavior: local software, stored data, power, and physical access still need protection. Define what should happen when readings or communication fail.

6 5 Artificial Intelligence and Machine Learning

Artificial intelligence (AI) refers to systems designed to perform tasks involving capabilities such as reasoning, perception, language processing, or decision support. Machine learning (ML) is an approach within AI that learns patterns from data. It does not require programmers to specify every prediction rule, but people still choose the task, data, learning method, and evaluation criteria. [6]

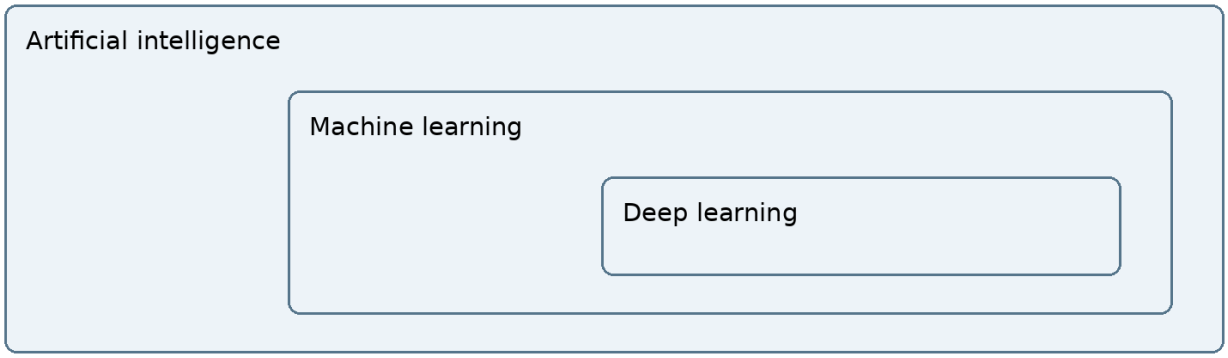


Figure 6.9. Machine learning is one approach within AI and deep learning is an approach within machine learning. The nesting does not imply that every AI application requires deep learning. Original teaching diagram.

Learning approach	Training signal	Example
Supervised	Examples include target labels or values	Predict spam versus legitimate messages
Unsupervised	Examples have no supplied target labels	Group similar patterns of energy usage
Reinforcement	An agent receives feedback or rewards from interaction	Learn a control policy in a safe simulation

Supervised learning includes classification, which predicts a category, and regression, which predicts a numerical value. Unsupervised groups do not automatically correspond to meaningful business categories. Reinforcement learning seeks a policy that improves expected reward over time; a poorly designed reward can encourage unwanted behavior. Other approaches, including self-supervised learning, also exist. [6]

Applications in information systems

AI can rank recommendations, detect unusual transactions, recognize images or speech, support forecasts, and assist users through conversational interfaces. These capabilities support organizational processes, but model output is not a guarantee of truth, fairness, or an appropriate action.

رومانی صیحی رزق جرجس مطر

6 5 Training Evaluation and Responsible Use

Training estimates model behavior from examples. Evaluation checks performance on appropriate data that did not guide fitting. Inference applies a trained model to a new input. A model does not necessarily keep learning after deployment; updating its behavior requires an explicitly designed process.

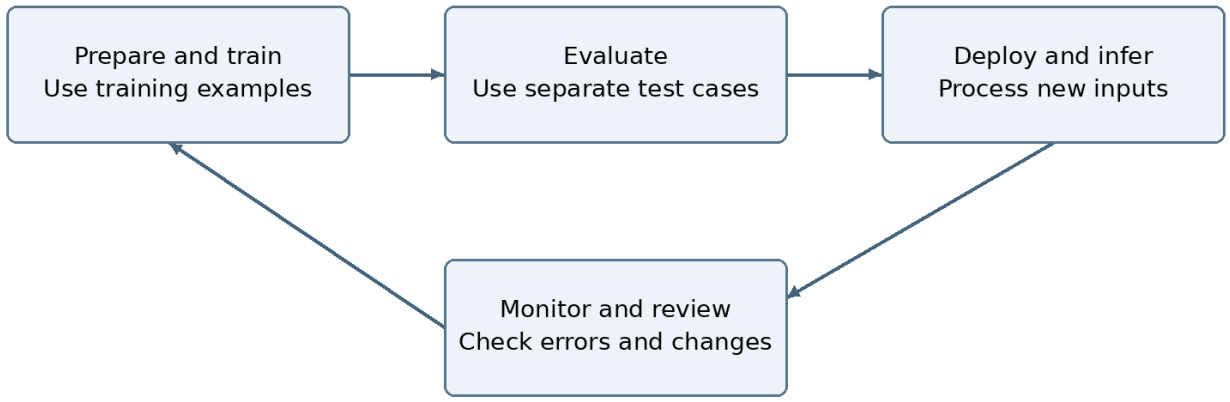


Figure 6.10. A simplified ML lifecycle separates development evaluation and operational use. Monitoring can trigger a controlled revision. Original teaching diagram.

A practical prediction example

A maintenance team labels past equipment records as normal or requiring attention. A classifier learns from suitable training examples and is evaluated on separate records. If closely related observations from the same event appear in both sets, the test may overstate generalization. For time-dependent use, evaluation should reflect the intended future prediction setting.

Consider a fictional test with 1,000 cases and only 10 true faults. Predicting “normal” for every case achieves 990 ÷ 1,000 × 100 = 99% accuracy while missing every fault. Evaluate measures related to the purpose, including fault recall, false alerts, and the consequences of errors. High accuracy alone can be misleading.

Generative AI and human review

Generative AI produces content such as text, images, or code. It can help draft a response or summarize material, but it can also produce convincing false content, often called hallucination or confabulation. NIST’s generative-AI profile discusses this and other risks, including privacy and harmful bias. Verify important outputs and keep sensitive information within approved tools and purposes. [7]

An adviser might review a draft answer to a routine student question, confirm it against current university rules, and correct it before use. AI should not silently determine a student’s entitlement or replace an accountable decision process. Monitor performance across relevant groups, provide escalation, and review changes in data or operating conditions.

2025/2026 2025/2026 2025/2026

6 6 Other Important Emerging Technologies

Technologies differ in maturity, purpose, and implementation needs. Some complement existing systems; others address specialized problems. Assess each by what it enables, what assumptions it requires, and which risks remain. Adoption should follow an appropriate use case rather than a fashionable label.

Technology	Core idea	Possible use	Key limitation
Blockchain	Shared ledger with linked records and agreed validation	Coordinate records across parties	Does not establish truth of input data
Edge computing	Process near devices or local users	Local monitoring and reduced transfers	Distributed maintenance remains necessary
5G networks	Fifth-generation mobile communication	Connectivity for users and devices	Actual latency and coverage vary
Digital twins	Data-linked model of a physical asset or system	Monitoring and scenario exploration	Needs suitable data and a validated model
Augmented reality AR	Overlay digital content on a physical view	Guided maintenance or contextual data	Alignment and usability need testing
Virtual reality VR	Present an immersive simulated environment	Training or design exploration	Comfort accessibility and equipment matter
Quantum computing	Use quantum states and algorithms	Research on selected computational tasks	Not a general speedup for every problem

Blockchain explained

Blockchain organizes validated records into cryptographically linked blocks across participating systems. A consensus mechanism governs acceptance. NIST describes ledgers as tamper-evident and tamper-resistant rather than absolutely immune to modification. A public network and a permissioned consortium have different participation and disclosure rules. [8]

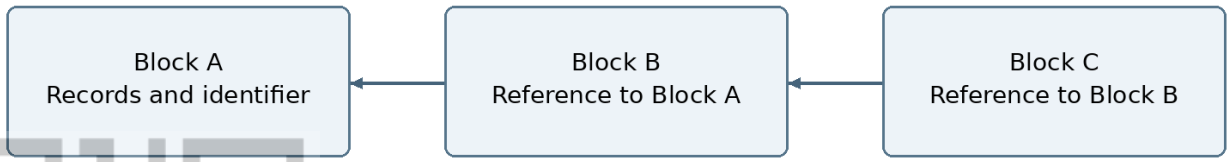


Figure 6.11. Each new block contains a reference linking it to the preceding block. Validation and consensus are additional mechanisms beyond this simplified picture. Original teaching diagram.

Recording an incorrect shipment statement does not make it correct. Participants still need reliable input, key protection, and governance. An ordinary database may be more appropriate when one trusted organization controls the records. Smart contracts execute programmed rules, and errors in those rules can produce unwanted results.

6 6 Digital Twins Immersive Interfaces and Quantum Computing

A digital twin is a digital representation linked to a physical object or system through relevant data. It can help compare expected and observed behavior or explore changes. It is more than a static 3D image: the useful relationship is between the physical asset, its observations, and the model's purpose. [9]

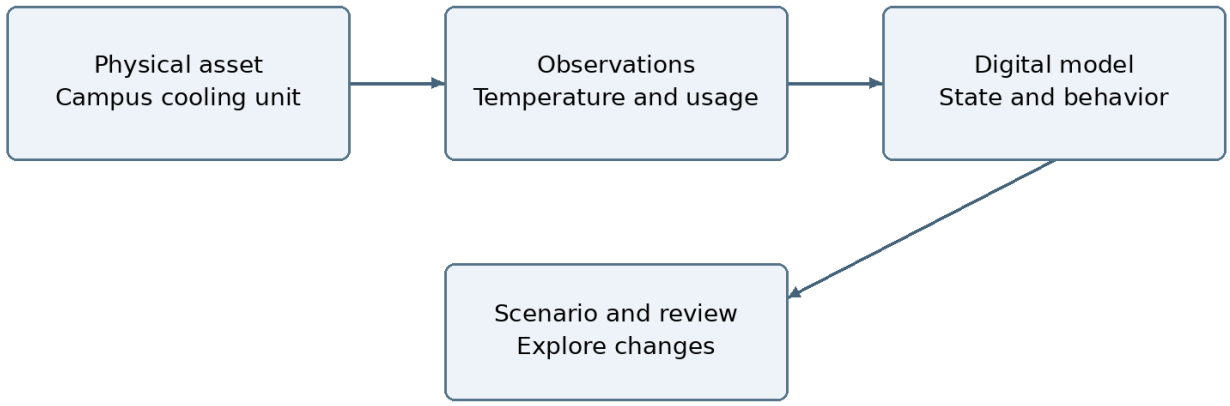


Figure 6.12. A digital twin combines an asset data and a model to support investigation. Proposed actions require appropriate validation and oversight. Original teaching diagram.

2025/2026 2025/2026 2025/2026

For example, a facilities team could compare actual energy use with a model of expected behavior. A disagreement might reflect a fault, inaccurate measurements, or a model that needs revision. Simulated improvements should be checked against real constraints before changing equipment operation.

AR VR and 5G

Augmented reality adds digital content to a view of the physical world; virtual reality presents an immersive virtual environment. An AR interface might display a maintenance instruction beside equipment, while VR could simulate a training setting. Neither removes the need for accessible design, accurate content, and safe interaction. Microsoft describes these experiences within a wider mixed-reality spectrum. [10]

5G denotes fifth-generation mobile networks. ITU describes enhanced broadband, large-scale device communication, and low-latency reliable communication as major usage scenarios. These capabilities depend on deployment, devices, radio conditions, and the end-to-end service. A 5G label does not guarantee a particular response time or safe autonomous operation. [11]

Quantum computing

Quantum computing uses qubits and quantum operations in specialized algorithms. It may offer advantages for selected problems, including aspects of simulating physical systems, but it is not faster for every workload. Hardware noise, error correction, scale, and algorithm design remain important constraints. It should not be described as solving all currently impossible problems. [12]

6 7 The Future of Information Systems

Future systems are likely to combine more distributed data collection, automation, and intelligent assistance. The pace and value of adoption will differ across organizations. The trends below are plausible directions supported by current capabilities, not promises that every organization or service will develop in the same way.

Direction	Possible development	Question for the organization
Automation and intelligence	AI supports more routine and analytical work	Which actions need review or escalation?
Connected environments	IoT edge and mobile networks link more devices	Can the service manage failures and device lifecycles?
Data as a strategic asset	Integrated records support better analysis	Are quality purpose and ownership clear?
Security and privacy	Safeguards become part of system design	How are access data use and recovery controlled?
Human and AI collaboration	Systems assist people with interpretation and action	Who remains accountable for the result?
Sustainability	Energy and equipment impact influence design	How is resource use measured against useful outcomes?

Sustainability and green computing

Green computing aims to reduce avoidable environmental impact across operation and equipment lifecycles. Appropriate capacity, efficient software, sensible data retention, maintenance, and responsible disposal can contribute. Cloud use is not automatically greener: workload design, utilization, energy sources, and data movement affect the result.

A campus project might measure electricity per successfully monitored building, equipment lifetime, and maintenance requirements. Reducing resource use should be considered alongside service quality. A design that saves energy but cannot provide necessary safety monitoring fails its main purpose.

Skills for future professionals

Students need foundations in programming, databases, networks, statistics, security, and business processes as well as familiarity with new tools. Communication, critical evaluation, and ethical judgment help translate technical output into appropriate decisions. Continuous learning should build on these foundations rather than replace them with isolated product knowledge.

6 8 Opportunities and Challenges for Organizations

Emerging technologies can support new services, more efficient operations, improved customer experiences, and faster experimentation. Benefits depend on integration, adoption by users, and measurable outcomes. An impressive demonstration does not establish that a system will work reliably in production.

Opportunity	Associated challenge	Evaluation approach
New digital services	Uncertain demand and return	Pilot a defined user need and measure uptake
Operational efficiency	Integration and workflow change	Compare time cost quality and failure rates
Better experiences	Privacy bias and accessibility	Review relevant groups and support routes
Faster innovation	Skills and maintenance demands	Assign ownership and support resources
Data-driven decisions	Unreliable or incomplete evidence	Validate data assumptions and error consequences

An adoption process

Define the problem and baseline, identify stakeholders, and choose a small feasible scope. Assess technical fit, data readiness, privacy, security, and operational responsibilities. Run a pilot with success criteria and a rollback or fallback plan. Expand only when evidence supports the benefit and the organization can sustain operation.

2025/2026 2025/2026 2025/2026

An integrated smart campus case

A university wants to reduce avoidable energy use while preserving teaching conditions. IoT sensors record temperature and equipment activity. An edge gateway validates and buffers readings. Cloud storage retains appropriate history, and analytics compares buildings under similar schedules. A predictive model estimates demand; facilities staff review recommendations before changing settings.

Measure energy relative to operating hours and relevant conditions, not only a raw before-and-after total. Track comfort complaints, missing readings, false alerts, and recovery from disconnection. Protect devices and access, avoid unnecessary personal monitoring, and specify maintenance ownership. The pilot can succeed without blockchain, VR, or quantum computing if those technologies do not address its needs.

Student application

Propose a small campus service using two technologies from this chapter. State its purpose, data sources, processing location, human responsibilities, security controls, and two success measures. Explain one important failure mode and whether an established simpler solution could meet the same need.

رومانی صبحی رزق جرجس مطر

6 9 Chapter Summary

Cloud computing supplies configurable resources through service and deployment models. It changes the division of operational work while preserving customer responsibilities for appropriate data use and access. Big Data methods address demanding workloads, and analytics turns observations into evidence for decisions.

IoT connects physical environments with digital systems. Edge computing supports local processing, while remote services can provide shared history and broader analysis. AI includes machine learning methods that learn patterns from data; their value depends on evaluation, monitoring, and accountable use. Generative output requires verification when errors matter.

Blockchain, digital twins, 5G, immersive interfaces, and quantum computing address different needs. Their benefits are conditional, and a newer technology is not automatically a better solution. Future information systems will depend on strong data practices, integration, security, human judgment, and measurable organizational value.

Key Terms

Term	Meaning
Cloud computing	On-demand access to configurable shared computing resources
IaaS PaaS SaaS	Infrastructure platform and software service models
Elasticity	Adjusting resource capacity to demand
Big Data	Data workloads requiring scalable methods
Volume velocity variety	Amount arrival rate and diversity of data
IoT	Connected physical devices with sensing computing or actuation
Edge computing	Processing near a source or local user
AI	Systems performing tasks involving intelligent capabilities
ML	Learning patterns from data for a defined task
Inference	Applying a trained model to a new input
Generative AI	Models that produce content such as text or images
Blockchain	Distributed ledger with linked records and agreed validation
Digital twin	Data-linked digital representation of an asset or system
AR and VR	Augmented physical views and immersive virtual environments
Quantum computing	Computing with qubits and quantum operations

رومانی صبحی رزق جرجس مطر

The Six Chapter Learning Sequence

The six chapters provide a foundation for understanding information systems as combinations of technology, data, people, and processes. Use the sequence below to connect each chapter's concepts with a complete organizational service.

No	Title	Central question
1	Introduction to Information Systems	What makes information systems useful?
2	Information Technology Fundamentals	Which technologies support the service?
3	Data and Information Management	How is data made reliable and useful?
4	Business Processes and Systems	How does the organization create value?
5	Securing Information Systems	How are records and operations protected?
6	Emerging Technologies and the Future of Information Systems	Which new capabilities are worth adopting?

Bringing the chapters together

For any proposed system, identify its purpose, users, and decisions. Describe the hardware, software, and communication needed; define the data and its quality requirements; and map the process it supports. Assign security, privacy, and recovery responsibilities. Evaluate whether an emerging technology improves the resulting service.

A successful solution is judged by its dependable contribution to organizational goals, not simply by the number of technologies included. The same reasoning applies to registration, banking, healthcare, retail, manufacturing, and public services.

Final reflection

Choose a familiar information system and explain how all six chapters apply to it. Identify one practical improvement, the evidence needed to justify it, and a limitation that must be managed. Use clear examples and distinguish observed facts from expectations.

رومانی صبحی رزق جرجس مطر
2025/2026 2025/2026 2025/2026

References and Figure Credits

All twelve figures are original explanatory diagrams. Campus scenarios, data-volume calculations, and classification results are fictional teaching examples. Technology capabilities and product examples depend on configuration and may change. Sources checked on 9 October 2026.

[1] NIST. SP 800 145 The NIST Definition of Cloud Computing.

https://csrc.nist.gov/pubs/sp/800/145/final

[2] Microsoft Learn. Shared responsibility in the cloud.

https://learn.microsoft.com/en-us/azure/security/fundamentals/shared-responsibility

[3] NIST CSRC. Big Data glossary.

https://csrc.nist.gov/glossary/term/big_data

[4] NIST. SP 800 183 Networks of Things.

https://csrc.nist.gov/pubs/sp/800/183/final

[5] NIST. SP 500 325 Fog Computing Conceptual Model.

https://csrc.nist.gov/pubs/sp/500/325/final

[6] Amazon Web Services. What is Machine Learning.

https://aws.amazon.com/what-is/machine-learning/

[7] NIST. AI 600 1 Generative Artificial Intelligence Profile.

https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf

[8] NIST. IR 8202 Blockchain Technology Overview.

https://csrc.nist.gov/pubs/ir/8202/final

[9] IBM. What is a Digital Twin.

https://www.ibm.com/think/topics/digital-twin

[10] Microsoft Learn. What is mixed reality.

https://learn.microsoft.com/en-us/windows/mixed-reality/discover/mixed-reality

[11] International Telecommunication Union. IMT 2020 and 5G.

https://www.itu.int/en/itu-r/study-groups/rsg5/rwp5d/imt-2020/pages/default.aspx

[12] IBM. What is Quantum Computing.

https://www.ibm.com/think/topics/quantum-computing

2025/2026 2025/2026 2025/2026

Chapter 6 Practice Questions

Choose one best answer for each question. Write its letter in the answer space provided.

Student name _____ Date _____

1. Which characteristic allows a customer to provision cloud resources through an agreed interface without ordering physical equipment?

A. Manual hardware procurement
B. On-demand self-service
C. Unrestricted public data access
D. Permanent fixed capacity

Your answer: _____

2. Which responsibility typically remains with the customer using IaaS?

A. Maintaining the provider's building cooling
B. Replacing the provider's physical servers
C. Managing the guest operating system and applications
D. Operating the provider's virtualization hardware

Your answer: _____

3. Which statement about SaaS is most accurate?

2025/2026 2025/2026

A. Every customer must administer the application's physical server
B. The customer has no responsibility for permissions
C. SaaS requires all customer records to be public
D. The provider operates the application while the customer still manages appropriate data use and access

Your answer: _____

4. Which arrangement best illustrates hybrid cloud?

A. A public website containing only a static image
B. Two unrelated email accounts with no integration
C. One personal computer with no network access
D. Distinct private and public cloud environments connected for coordinated use

Your answer: _____

Chapter 6 Practice Questions Continued

5. What does velocity describe in the Big Data framework?

A. The rate of data arrival and required processing response

B. The honesty of every data provider

C. Only the total number of stored bytes

D. The number of file formats

Your answer: _____

6. Which example most directly concerns veracity?

A. A system receives one million events each second

B. A warehouse stores several petabytes of records

C. A sensor's readings are inaccurate and need validation

D. A dataset contains ten different file formats

Your answer: _____

7. If 100 sensors each record one reading per minute, how many readings are produced in one day?

A. 2,400

B. 144,000

C. 6,000

D. 14,400

2025/2026 2025/2026 2025/2026

Your answer: _____

8. Which analytics task is predictive?

A. Listing yesterday's transactions

B. Reporting last month's measured usage

C. Estimating next week's electricity demand

D. Displaying the names of current buildings

Your answer: _____

رومانی صبحی رزق جرجس مطر

Chapter 6 Practice Questions Continued

9. What distinguishes an actuator from a sensor?

A. An actuator always stores an entire cloud database

B. A sensor must always be disconnected from networks

C. The two terms refer to identical functions

D. An actuator changes a physical state while a sensor measures a property

Your answer: _____

10. Which statement about IoT processing is accurate?

A. It can occur on a device at an edge gateway or in a remote service

B. It must always occur in a public cloud

C. Every sensor must have a direct public Internet address

D. IoT eliminates the need for device maintenance

Your answer: _____

11. What is a main reason to use edge computing?

A. Make every application a blockchain system

B. Support local processing and reduce transfers to a remote service

C. Guarantee that a device never loses power

D. Eliminate all access-control requirements

Your answer: _____

12. Which relationship between AI ML and deep learning is correct?

A. Deep learning is an approach within ML and ML is an approach within AI

B. ML excludes learning from data

C. AI is always a subset of deep learning

D. All AI systems must use deep learning

Your answer: _____

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

Chapter 6 Practice Questions Continued

13. Which task most clearly uses supervised learning?

A. Training on messages labeled spam or legitimate

B. Saving raw readings without modeling

C. Learning a policy from interaction rewards

D. Grouping usage patterns without supplied target labels

Your answer: _____

14. What is the main training feedback in reinforcement learning?

A. A requirement that every example have a category label

B. An assumption that every action is equally good

C. Only a list of file names

D. Rewards associated with interaction and outcomes

Your answer: _____

15. A model predicts normal for all 1,000 cases when 10 cases are true faults. Which statement is correct?

A. It has 99% accuracy but detects none of the faults

B. Accuracy proves that it is suitable for maintenance

C. It detects all faults because accuracy is high

2025/2026 2025/2026 2025/2026

D. It has 1% accuracy and perfect fault recall

Your answer: _____

16. How should important generative-AI output be handled?

A. Replace every applicable policy with the generated text

B. Accept it because fluent wording proves correctness

C. Verify it against reliable evidence and use an accountable review process

D. Assume that it cannot expose personal information

Your answer: _____

رومانی صبحی رزق جرجس مطر

Chapter 6 Practice Questions Continued

17. Which limitation of blockchain is important?

A. It guarantees that every participant is honest

B. It removes every need for governance

C. It automatically corrects every programming error

D. It does not establish that input data accurately describe real events

Your answer: _____

18. Which feature most clearly distinguishes a useful digital twin from a static illustration?

A. The file is stored in a presentation

B. A relevant data link connects the model with its physical counterpart

C. The model has no connection with observed conditions

D. The image uses more colors

Your answer: _____

19. Which statement about quantum computing is appropriate?

A. It can solve every impossible computational problem

B. It may benefit selected problems but is not faster for every workload

C. It guarantees an immediate advantage for every business database

D. It removes all need for classical computing

Your answer: _____

20. What is the strongest basis for expanding an emerging-technology pilot?

A. The number of fashionable technologies in the proposal

B. The absence of any user feedback

C. Evidence of defined benefits together with sustainable operation and managed risks

D. A demonstration with no success criteria or fallback plan

Your answer: _____

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

Chapter 7

Conclusion

IS101 | Academic year 2026–2027
Faculty of Computers and Informatics
Tanta University

Instructor and Course Coordinator
Prof. (Assoc.) Hany El-Ghaish

رومانی صبحی رزق جرجس مطر

7 1 Introduction

This book has introduced information systems through six connected areas: their purpose, technological foundations, data management, business processes, security, and emerging technologies. Together, these areas explain how organizations use information to deliver services and make decisions. This conclusion brings the ideas together and helps you apply them to a complete organizational situation.

An effective information system depends on more than functioning equipment. It needs reliable data, appropriate procedures, people who understand their responsibilities, and controls that protect the service. Its value should be assessed through the outcomes it supports, such as accurate transactions, shorter waiting times, dependable access, and better informed decisions.

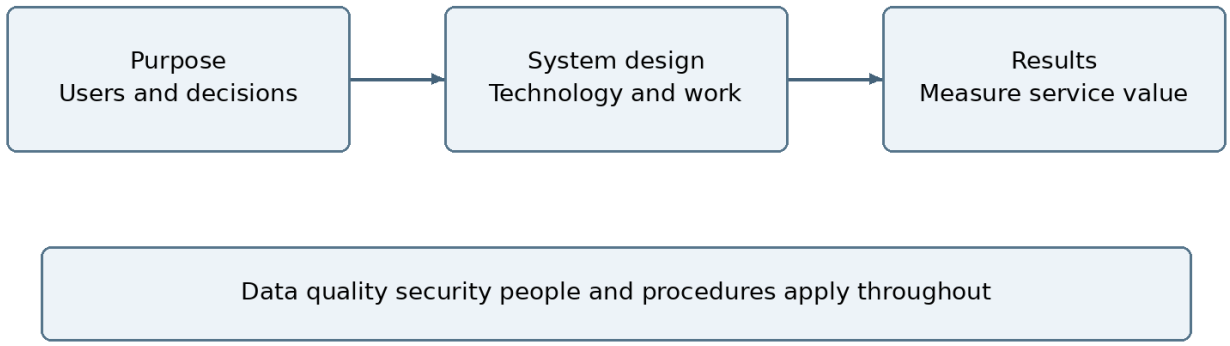


Figure 7.1. A complete information system connects purpose resources and results. Original teaching diagram.

Learning objectives

After this chapter, you should be able to summarize the six core chapters, connect their concepts to one service, distinguish a technical output from an organizational outcome, and evaluate a proposed improvement. You should also explain why security, data quality, human judgment, and continuous learning remain important as technologies change.

7 2 Summary of Key Concepts

Each chapter examined a different part of the same organizational system. Reading them together shows why technology choices cannot be separated from information needs, work practices, and responsibility. The following summaries identify the main lesson and its practical implication.

Chapter 1 Introduction to Information Systems

An information system combines hardware, software, data, people, and procedures to collect, process, store, and distribute information. It supports operational work and decisions at different management levels. The important starting question is what information a person needs to perform a task or make a decision. Buying software alone does not establish a well coordinated system.

Chapter 2 Information Technology Fundamentals

Hardware supplies processing, working memory, persistent storage, and input and output capabilities. Software directs these resources, while networks enable communication. The operating system manages resources and provides services to applications. Infrastructure must be selected for the workload: a faster processor does not automatically correct a slow network, insufficient memory, or an inefficient application.

Chapter 3 Data and Information Management

Data consists of recorded facts; information gives those facts meaningful context. Collection, storage, cleaning, integration, security, and governance help maintain trustworthy records. Analysis and visualization make records useful for decisions. A clear report based on inaccurate data can still produce a poor decision, so quality and interpretation must be considered together.

2025/2026 2025/2026 2025/2026

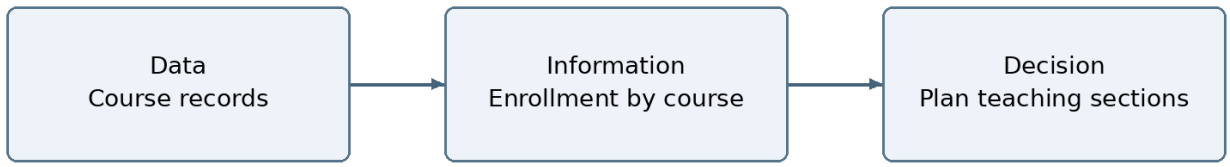


Figure 7.2. Recorded facts become useful evidence through context and interpretation. Original teaching diagram.

For example, individual registration records become information when summarized by course and capacity. A department can use that information to investigate demand. Planning additional sections also requires room availability, staffing, budgets, and academic judgment.

رومانی صبحی رزق جرجس مطر

7 2 Key Concepts Continued

The remaining chapters connected information resources to organizational work, protection, and technological change. They showed that a service must be useful, dependable, and maintainable, rather than simply technically possible.

Chapter 4 Business Processes and Systems

A business process is a related set of activities that produces a service or product. Information systems can automate steps, integrate records across departments, coordinate work, and monitor performance. ERP supports integrated organizational processes, while CRM focuses on customer relationships. Process modeling helps identify delays and handoffs before deciding what should be automated.

Chapter 5 Securing Information Systems

Security protects confidentiality, integrity, and availability. Technical, administrative, and physical controls work together to reduce risk. Policies establish responsibilities, and risk management helps prioritize protection. Security is an ongoing responsibility: organizations must monitor conditions, prepare responses, and test recovery rather than assuming that a single control guarantees protection.

NIST's Cybersecurity Framework 2.0 organizes cybersecurity outcomes around Govern, Identify, Protect, Detect, Respond, and Recover. This reinforces the connection between management responsibility and operational protection. These functions are related activities, not a one-time sequence completed when software is installed. [1]

2025/2026 2025/2026 2025/2026

Chapter 6 Emerging Technologies and the Future of Information Systems

Cloud computing, analytics, IoT, and AI can expand system capabilities. Cloud services offer configurable resources, IoT connects physical observations to digital services, and analytics and AI can support interpretation and prediction. Benefits depend on design, data, evaluation, cost, and oversight. Providers and customers retain different responsibilities, and emerging capabilities should be evaluated through evidence rather than assumed benefits. [2]

Core area	Question to ask about a proposed system
Purpose	Which users tasks and decisions will it support?
Infrastructure	Can the resources handle the required workload?
Data	Are records accurate consistent and appropriately governed?
Processes	Will the workflow produce the intended service?
Security	How will risks access and recovery be managed?
Emerging technologies	What evidence justifies adopting a new capability?

These questions provide a practical review framework. An improvement in one area may expose a weakness elsewhere: faster collection can create more data-quality work, and wider access can require stronger identity and permission controls.

7 3 The Bigger Picture

Information systems influence how organizations design services, coordinate work, interact with customers and partners, and make decisions. Their strategic importance comes from their contribution to organizational goals. A useful evaluation connects system behavior to a result that matters to the organization and its users.

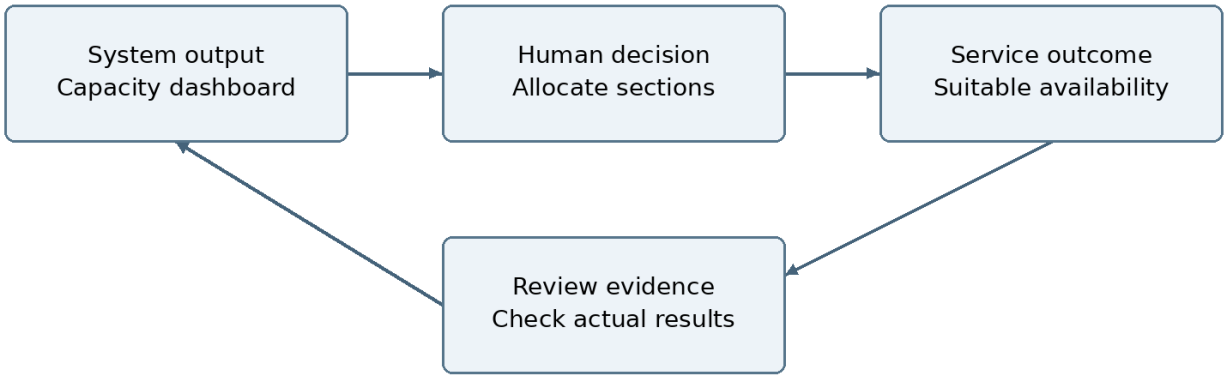


Figure 7.3. System outputs support decisions whose outcomes must be reviewed. Original teaching diagram.

Outputs and outcomes

An output is something the system produces, such as a report, confirmation, or alert. An outcome is the effect of its use, such as fewer registration errors or improved service availability. A dashboard may be accurate and timely yet have little effect if staff cannot act on its findings. Evaluation should therefore examine the process and its results as well as the technology.

2025/2026 2025/2026 2025/2026

Organizational concern	Possible system contribution	Evidence to examine
Decision quality	Provide relevant timely information	Accuracy and usefulness for the decision
Process performance	Reduce unnecessary manual steps	Cycle time errors and exception handling
User experience	Make status and instructions clear	Completion rates complaints and feedback
Data responsibility	Apply ownership and access rules	Quality checks permissions and retention
Service continuity	Prepare monitoring and recovery	Outages response times and restore tests

These measures require a baseline and context. A shorter completion time is not a success if important checks are skipped. Better performance should preserve the controls and service quality that users depend on.

رومانی صبحی رزق جریس مطر

7 3 An Integrated University Example

Consider a fictional university registration service. Students submit course requests, the application checks eligibility and capacity, approved registrations are saved, and staff review enrollment reports. This familiar example shows how the six core chapters describe different responsibilities within one service.

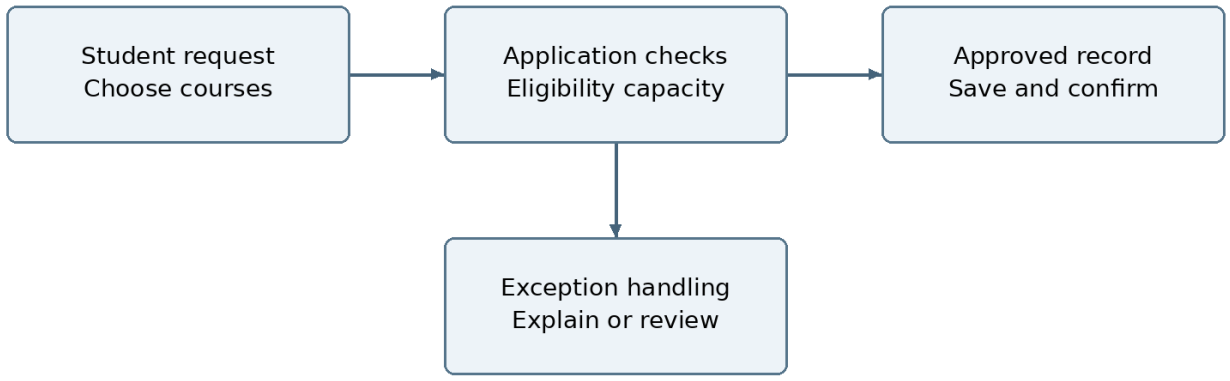


Figure 7.4. A simplified registration workflow includes rule checks and an exception path. Original teaching diagram.

Chapter	Application to the registration service
1	Identify students advisers administrators and their information needs
2	Provide devices networks servers software and storage
3	Maintain correct identities prerequisites capacities and registrations
4	Coordinate requests validation approval and exception handling
5	Restrict access protect records monitor events and test recovery
6	Evaluate elastic cloud capacity or forecasting when justified

A worked evaluation example

Suppose a fictional pilot reduces average registration time from 10 minutes to 6 minutes. The reduction is 10 − 6 = 4 minutes, or 4 ÷ 10 × 100 = 40%. This suggests faster completion for the measured sample. It does not, by itself, prove fewer errors, lower cost, or better access for every student. Compare similar conditions and review unsuccessful attempts and exceptions as well.

A demand forecast could help staff plan course sections, but it should be evaluated against relevant past observations and checked for changing conditions. Staff remain responsible for constraints such as teaching availability and approved academic rules. The example is illustrative; its numbers are not measured university results.

7 4 Final Thoughts

Studying information systems develops the ability to connect technical resources to organizational needs. The terms introduced in this book give you a vocabulary, but professional understanding also requires explaining relationships, checking evidence, and recognizing limitations. Those habits remain useful when individual products and technologies change.

Apply a disciplined improvement approach

Begin with a clearly defined service problem. Describe the existing process, the records involved, the users affected, and the measures of success. Compare feasible alternatives, including procedural improvements. Test a suitable change on a limited scale, assess results and risks, and assign responsibility for ongoing operation before expansion.

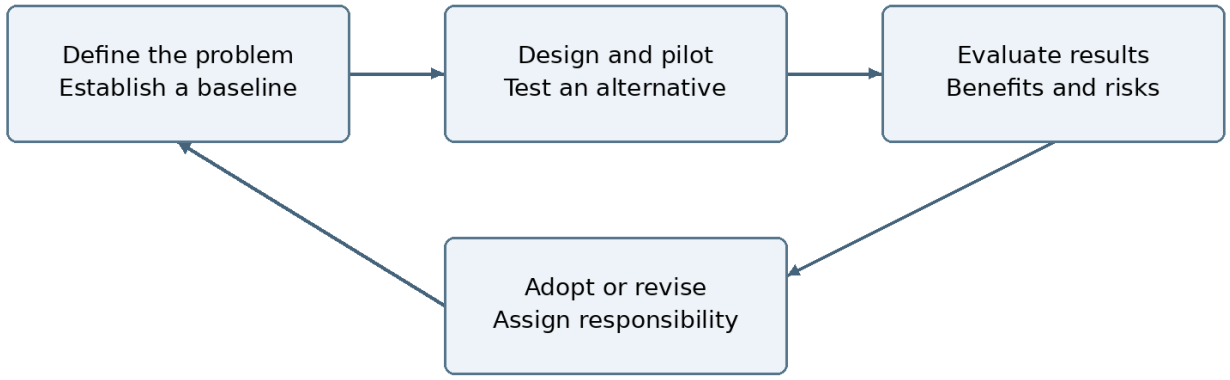


Figure 7.5. Improvement requires evaluation and a route back to revision. Original teaching diagram.

Build transferable skills

Practice reading a process diagram, interpreting a small dataset, explaining an access-control decision, and comparing technology alternatives. Communicate assumptions and uncertainty in plain language. Responsible use of AI includes verifying important outputs and keeping accountable human review where the consequences require it.

Reflection activity

Choose a library, clinic, retail, banking, or university service. Identify its users, input data, main activities, outputs, and decisions. Explain one data-quality risk and one security risk. Propose an improvement, name the evidence you would collect, and explain what could make the improvement unsuitable.

7 5 Closing Statement

This book has provided a structured introduction to information systems for IS101. We hope it has helped you build a clear understanding of how technology, data, people, and procedures support organizational work. Use this foundation to ask precise questions, evaluate evidence, and design improvements that serve real needs. Continued study and practical experience will deepen your ability to contribute responsibly to the organizations and communities in which you work.

Chapter Summary and Further Reading

The six core chapters form a connected foundation, and this seventh chapter provides the concluding synthesis. An information system supports a purpose through technology, reliable records, coordinated work, and human responsibility. Its contribution must be evaluated through meaningful outcomes, with security and privacy considered throughout its use.

Key Terms

Term	Meaning in this conclusion
Information system	Resources and activities that support information needs
Infrastructure	Shared technological and operational foundations
Data quality	Fitness of records for their intended use
Business process	Related activities producing a service or product
CIA triad	Confidentiality integrity and availability
Governance	Responsibilities policies and oversight guiding decisions
System output	A result produced by a system such as a report
Organizational outcome	The effect of using a system in practice
Baseline	Starting evidence against which change is compared
Pilot	Limited trial used to evaluate a proposed change

References and figure credits

The summaries synthesize Chapters 1–6 of this book. All five figures are original teaching diagrams. The registration service and numerical pilot are fictional examples. The following authoritative sources support further reading on protection and cloud services. Sources checked on 9 October 2026.

[1] NIST. The NIST Cybersecurity Framework 2.0.

https://doi.org/10.6028/NIST.CSWP.29

[2] NIST. SP 800 145 The NIST Definition of Cloud Computing.

https://csrc.nist.gov/pubs/sp/800/145/final

Chapter 7 Review Questions

Choose one best answer for each question. These questions review the conclusion and the six core chapters. Write the letter in the space provided.

Student name _____ Date _____

1. Which description best represents an information system?

A. A collection of devices without defined users

B. Only the computers owned by an organization

C. Only the applications installed on a server

D. Technology data people and procedures working toward information needs

Your answer: _____

2. What should guide the initial design of an information system?

A. The appearance of the dashboard alone

B. The largest available server

C. Users tasks and the decisions the system must support

D. The newest product regardless of purpose

Your answer: _____

3. Which component normally holds active programs and data temporarily?

A. A stored paper file

B. An archived backup

C. RAM

D. A printed report

Your answer: _____

4. What is a main responsibility of an operating system?

A. Manage resources and provide services to applications

B. Replace the need for application software

C. Define every organizational policy

D. Guarantee that all input data are correct

Your answer: _____

Chapter 7 Review Questions Continued

5. Which example is information rather than an isolated raw value?

A. An identifier without context

B. An unexplained date

C. The number 42 without a label

D. Enrollment totals organized by course and capacity

Your answer: _____

6. What is the strongest reason to clean data before analysis?

A. Cleaning guarantees that every forecast is correct

B. Errors and inconsistencies can distort the resulting findings

C. Cleaning removes all need for access control

D. Cleaning makes business rules unnecessary

Your answer: _____

7. What does a business process describe?

A. Related activities that produce a service or product

B. Only the price of software

C. A list of unrelated records

2025/2026 2025/2026 2025/2026

D. Only the physical arrangement of computers

Your answer: _____

8. Which system is primarily intended to integrate major organizational processes?

A. A device driver

B. A single image file

C. ERP

D. A standalone calculator

Your answer: _____

رومانی صبحی رزق جرجس مطر

Chapter 7 Review Questions Continued

9. Which system focuses on interactions with current and potential customers?

A. An operating-system scheduler

B. A physical network cable

C. RAM

D. CRM

Your answer: _____

10. Which security goal is threatened when grades are changed without permission?

A. Only availability

B. Integrity

C. Only processing speed

D. Only storage capacity

Your answer: _____

11. Which control follows least privilege?

A. Share one privileged account with all staff

B. Give every user administrator access

C. Remove all access reviews

2025/2026 2025/2026 2025/2026

D. Give each user only access required for authorized tasks

Your answer: _____

12. Why should recovery arrangements be tested?

A. To check that required data and services can actually be restored

B. To replace all preventive controls

C. To make security policies unnecessary

D. To eliminate every possible incident

Your answer: _____

رومانی صبحی رزق جرجس مطر

Chapter 7 Review Questions Continued

13. Which cloud statement is appropriate?

A. Responsibilities depend on the service model and agreement

B. SaaS removes all customer responsibility

C. A private cloud is automatically secure

D. Cloud services always eliminate recurring costs

Your answer: _____

14. Which capability is most directly associated with IoT?

A. Removing all network dependencies

B. Collecting physical observations through connected devices

C. Guaranteeing that every device uses AI

D. Correcting every inaccurate record automatically

Your answer: _____

15. How should an important AI-generated recommendation be handled?

A. Accept it because fluent language proves accuracy

B. Verify evidence and retain appropriate accountable review

C. Use it without examining the relevant constraints

D. Assume it cannot contain false statements

Your answer: _____

16. Which example is an organizational outcome?

A. An automatic confirmation message

B. Fewer verified registration errors after an evaluated change

C. A database query result

D. A dashboard file created by the system

Your answer: _____

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

Chapter 7 Review Questions Continued

17. What is the percentage reduction from 10 minutes to 6 minutes?

A. 4%

B. 100%

C. 60%

D. 40%

Your answer: _____

18. What does the time reduction alone establish in the fictional pilot?

A. All errors were eliminated

B. Every student received better service

C. The measured average completion time decreased

D. Total costs certainly fell

Your answer: _____

19. What is the most useful purpose of a baseline?

A. Remove all uncertainty from forecasting

B. Guarantee that the proposed solution will work

C. Provide starting evidence for comparison after a change

2025/2026 2025/2026 2025/2026

D. Replace the need to evaluate a pilot

Your answer: _____

20. Which approach best supports responsible improvement?

A. Define the problem pilot a change evaluate results and assign responsibility

B. Automate every activity without reviewing its purpose

C. Install the newest technology before identifying users

D. Measure only successful attempts and ignore exceptions

Your answer: _____

رومانی صبحی رزق جرجس مطر

AI Diploma Lecture 2: Introduction to Python Basics for AI

رومانی صبحی رزق جرجس مطر

1 Simple Definition of Artificial Intelligence

Artificial Intelligence (AI) is the simulation of human intelligence processes by machines, especially computer systems. These processes include learning (the acquisition of information and rules for using the information), reasoning (using rules to reach approximate or definite conclusions), and self-correction.

Exercise: Research and write your own simple definition of AI. Compare it with the one provided and discuss how AI is used in everyday applications like voice assistants or recommendation systems.

Try writing, running, and testing the code yourself to deepen your understanding.

2 Differences Between Traditional Programming and Machine Learning

The following table highlights the key differences between traditional programming and machine learning.

Aspect	**Traditional Programming**	**Machine Learning**
Approach	Programmer writes explicit rules and logic to solve a problem.	Algorithm learns patterns from data to make predictions or decisions without explicit rules.
Input	Data and predefined rules.	Data (features and labels) for training.
Output	Deterministic results based on rules.	Probabilistic models that improve with more data.
Handling Complexity	Struggles with complex, unpredictable scenarios.	Excels in handling large, complex datasets and uncertainty.
Examples	Calculating sums, sorting lists.	Image recognition, spam detection.

Exercise: Create a simple traditional program to add two numbers. Then, think about how machine learning could be used to predict sums based on patterns in data. Discuss the pros and cons of each approach.

Try writing, running, and testing the code yourself to deepen your understanding.

رومانی صبحی رزق جرجس مطر

3 Variable Naming Rules

This table covers the rules for naming variables in Python, how to use them, with valid and invalid examples.

Rule/Usage	Description	Valid Examples	Invalid Examples
Valid Characters	Letters (a-z, A-Z), digits (0-9), underscores (_). Must not start with digit.	`user_name`, `age2`	`2age`, `user-name`
Case Sensitivity	Names are case-sensitive.	`Age` and `age` are different.	N/A
Reserved Keywords	Cannot use Python keywords like `if`, `for`.	`total_sum`	`if`, `class`
Style Convention	Use `snake_case` for variables. Be descriptive.	`number_of_students`	`nOs` (poor style)
Usage	Assign values with `=`, e.g., `var = value`.	`score = 90`	`score =` (missing value)

Exercise: Declare variables for your name, age, and height following the rules. Try invalid names and note the errors when running the code.

Try writing, running, and testing the code yourself to deepen your understanding.

4 The Print Function

This table explains the print function in Python, its roles, and usage with examples.

Aspect	Description/Rules	Examples
Basic Role	Outputs text or values to the console.	`print("Hello, World!")` Output: Hello, World!
Printing Variables	Use inside print, separated by commas.	`name = "Alice"` `print("Name:", name)` Output: Name: Alice
Multiple Items	Print multiple items with sep (default space) and end (default newline).	`print(1, 2, 3, sep="-", end=".")` Output: 1-2-3.
Formatting	Use f-strings for formatted output.	`age = 25` `print(f"Age: {age}")` Output: Age: 25

Exercise: Write a program that prints your name and age using variables and f-strings. Experiment with different separators and ends.

Try writing, running, and testing the code yourself to deepen your understanding.

5 The Input Function

This table explains the `input()` function in Python, its rules, and usage for reading different data types with examples. Indentation is shown with spaces (4 spaces per level).

Aspect	Description/Rules	Examples
Basic Role	Reads user input as a string. Use a prompt for clarity.	`name = input("Enter name: ")` Input: Alice Output: name = "Alice"

Reading Integer	Convert string input to int using `int()`. Use try-except for error handling. Indent block 4 spaces.	`try:`
		`age = int(input("Enter age: "))`
		`print("Age:", age)`
		`except ValueError:`
		`print("Invalid input")`
		Input: 25
		Output: Age: 25

Reading Float	Convert string input to float using `float()`. Handle errors. Indent block 4 spaces.	`try:`
		`height = float(input("Enter height: "))`
		`print("Height:", height)`
		`except ValueError:`
		`print("Invalid input")`
		Input: 1.75
		Output: Height: 1.75

Reading Boolean	Convert string input to bool. Non-empty strings are `True`, empty is `False`. Use `eval()` carefully for explicit `True`/`False`. Indent block 4 spaces.	`try:`
		`is_student = bool(eval(input("Student? (True/False): "))`
		`print("Student:", is_student)`
		`except:`
		`print("Invalid input")`
		Input: True
		Output: Student: True

Reading String	Input is already a string; no conversion needed.	`city = input("Enter city: ")` Input: Paris Output: city = "Paris"

2025/2026 2025/2026 2025/2026

Exercise: Write a program that uses `input()` to collect name (str), age (int), height (float), and student status (bool). Print them back. Test with valid and invalid inputs, adding try-except to handle errors with proper indentation (4 spaces). Run and observe the results.

Try writing, running, and testing the code yourself to deepen your understanding.

6 Python Operators

This table lists all major Python operators, their usage, and simple examples with results.

رومانی صبحی رزق جرجس مطر

Type	Operator	Description	Example and Result
Arithmetic	+	Addition	5 + 3 = 8
	-	Subtraction	5 - 3 = 2
	*	Multiplication	5 * 3 = 15
	/	Division	5 / 2 = 2.5
	//	Floor Division	5 // 2 = 2
	%	Modulus	5 % 2 = 1
	**	Exponentiation	2 ** 3 = 8
Assignment	=	Assign value	x = 5 (x is 5)
	+=	Add and assign	x = 5; x += 3 (x=8)
	-=	Subtract and assign	x = 5; x -= 3 (x=2)
	*=	Multiply and assign	x = 5; x *= 3 (x=15)
	/=	Divide and assign	x = 5; x /= 2 (x=2.5)
	//=	Floor divide and assign	x = 5; x //= 2 (x=2)
	%=	Modulus and assign	x = 5; x %= 2 (x=1)
	**=	Exponent and assign	x = 2; x **= 3 (x=8)
Relational	==	Equal to	5 == 5 (True)
	!=	Not equal to	5 != 3 (True)
	>	Greater than	5 > 3 (True)
	<	Less than	5 < 3 (False)
	>=	Greater than or equal	5 >= 5 (True)
	<=	Less than or equal	5 <= 3 (False)
Logical	and	True if both true	True and False (False)
	or	True if either true	True or False (True)
	not	Inverts boolean	not True (False)
Bitwise	&	AND	5 & 3 = 1
2025/2026	|	OR 2025/2026	5 | 3 = 7 2025/2026
	^	XOR	5 ^3 = 6
	~	NOT	~5 = -6
	«	Left shift	5 « 1 = 10
	»	Right shift	5 » 1 = 2
Membership	in	Check membership	3 in [1,2,3] (True)
	not in	Check non-membership	4 not in [1,2,3] (True)
Identity	is	Check same object	x is y (if same)
	is not	Check different object	x is not y (if different)

Exercise: Write expressions using each operator type. Run them and verify results. Create a calculator program using arithmetic and assignment operators.

Try writing, running, and testing the code yourself to deepen your understanding.

7 Types of Programs: Sequential, Branching, and Looping

This table describes the types of program structures in Python, their definitions, importance, and simple examples, with indentation shown using spaces (1cm} for 4 spaces).

رومانى صبحى رزق جرجس مطر

Type	Definition	Importance	Simple Example

Sequential	Code executes line by line in order. No indentation needed beyond base level.	Basic structure; ensures predictable flow.	`print("Step 1")`
			`print("Step 2")`
			Output: Step 1
			Step 2

Branching	Uses conditionals (if, elif, else) to execute different code based on conditions. Indent block 4 spaces.	Allows decision-making; handles different scenarios.	`x = 10`
			`if x > 5:`
			`print("Greater")`
			Output: Greater

Looping	Repeats code blocks using for or while loops. Indent loop body 4 spaces.	Efficient for repetitive tasks; avoids code duplication.	`for i in range(3):`
			`print(i)`
			Output: 0
			1
			2

Exercise: Write a sequential program to print numbers 1-5. Add branching to print if even/odd with proper indentation (4 spaces). Use looping to iterate and check, ensuring correct indentation. Test and modify for different inputs.

Try writing, running, and testing the code yourself to deepen your understanding.

8 Indentation in Python

Indentation in Python refers to the use of whitespace (typically 4 spaces or one tab) at the beginning of a line to define the scope and hierarchy of code blocks, such as those in loops, conditionals, functions, or try-except statements. Unlike other programming languages that use braces {}, Python enforces indentation to structure code, making it both a syntax requirement and a readability feature. Incorrect indentation leads to `IndentationError` or unexpected behavior. The standard practice, per PEP 8, is to use 4 spaces per indentation level. For example, code inside an `if` statement or a `for` loop must be indented to indicate it belongs to that block. Below is an example with try-except, using spaces (`1cm}` for 4 spaces) to highlight indentation:

```
try:
    age = int(input("Enter age: "))
    print(f"Age: {age}")
except ValueError:
    print("Invalid input")
```

Input: 25
Output: Age: 25

Exercise: Write a program with a loop and a conditional statement, ensuring proper indentation (4 spaces). Intentionally introduce an indentation error, run the code, and observe the `IndentationError`. Fix it and verify the correct output in blue.

Try writing, running, and testing the code yourself to deepen your understanding.

رومانی صبحی رزق جرجس مطر

Python Lecture: Arithmetic, Relational, Logical Operators, Branching, and Assignments

رومانی صبحی رزق جرجس مطر

1 Arithmetic Operators in Python

Arithmetic operators perform mathematical operations on numbers (integers, floats, etc.). Here are the common ones:

- `+`: Addition (e.g., $5 + 3 = 8$)
- `-`: Subtraction (e.g., $5 - 3 = 2$)
- `*`: Multiplication (e.g., $5 \times 3 = 15$)
- `/`: Division (returns a float, e.g., $5/3 \approx 1.666$)
- `//`: Floor division (integer division, discards remainder, e.g., $5//3 = 1$)
- `%`: Modulus (remainder, e.g., $5\%3 = 2$)
- `**`: Exponentiation (power, e.g., $5 * *3 = 125$)

1.1 Operator Precedence

2025/2026 2025/2026 2025/2026

Python follows a precedence order (like PEMDAS in mathematics). Operators with higher precedence are evaluated first; same-precedence operators are evaluated left to right.

Precedence (highest to lowest):

1. `()`: Parentheses
2. `**`: Exponentiation
3. `*`, `/`, `//`, `%`: Multiplication, division, floor division, modulus
4. `+`, `-`: Addition, subtraction

Examples:

- `2 + 3 * 4`: Multiply first ($3 \times 4 = 12$), then add ($2 + 12 = 14$). Result: 14.
- `(2 + 3) * 4`: Parentheses first ($2 + 3 = 5$), then multiply ($5 \times 4 = 20$). Result: 20.
- `2 ** 3 + 1`: Exponent first ($2^3 = 8$), then add ($8 + 1 = 9$). Result: 9.
- `10 / 2 * 3`: Divide first ($10/2 = 5.0$), then multiply ($5.0 \times 3 = 15.0$). Result: 15.0.
- `10 % 3 // 2`: Modulus first ($10\%3 = 1$), then floor divide ($1//2 = 0$). Result: 0.

2 Sequential Programs Using Arithmetic Operators

Below are five simple sequential programs demonstrating arithmetic operators. Each program takes input, performs calculations, and displays output.

2.1 Program 1: Basic Arithmetic

Description: Takes two numbers and computes their sum, difference, product, and division.

```
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))
sum_result = num1 + num2
diff_result = num1 - num2
prod_result = num1 * num2
div_result = num1 / num2
print("Sum:", sum_result)
print("Difference:", diff_result)
print("Product:", prod_result)
print("Division:", div_result)
```

2.2 Program 2: Rectangle Area and Perimeter

Description: Calculates the area and perimeter of a rectangle using length and width inputs.

```
length = float(input("Enter length: "))
width = float(input("Enter width: "))
area = length * width
perimeter = 2 * (length + width)
print("Area:", area)
print("Perimeter:", perimeter)
```

2.3 Program 3: Simple Interest

Description: Computes simple interest using principal, rate, and time.

```
principal = float(input("Enter principal amount: "))
rate = float(input("Enter annual rate (as decimal): "))
time = float(input("Enter time in years: "))
interest = (principal * rate * time) / 100
print("Simple Interest:", interest)
```

2.4 Program 4: Powers and Roots

Description: Computes square, cube, and approximate square root of a number using exponentiation.

```
num = float(input("Enter a number: "))
square = num ** 2
cube = num ** 3
sqrt_approx = num ** 0.5
print("Square:", square)
print("Cube:", cube)
print("Square Root (approx):", sqrt_approx)
```

2.5 Program 5: Temperature Conversion

Description: Converts temperature from Celsius to Fahrenheit.

رومانی صبحی رزق جرجس مطر

```python
celsius = float(input("Enter temperature in Celsius: "))
fahrenheit = (celsius * 9 / 5) + 32
print("Temperature in Fahrenheit:", fahrenheit)
```

3 Relational and Logical Operators

3.1 Relational Operators

These compare two values and return `True` or `False`:

- `==`: Equal to (e.g., $5 == 5 \rightarrow$ `True`)
- `!=`: Not equal to (e.g., $5! = 3 \rightarrow$ `True`)
- `>`: Greater than (e.g., $5 > 3 \rightarrow$ `True`)
- `<`: Less than (e.g., $5 < 3 \rightarrow$ `False`)
- `>=`: Greater than or equal to (e.g., $5 >= 5 \rightarrow$ `True`)
- `<=`: Less than or equal to (e.g., $5 <= 3 \rightarrow$ `False`)

3.2 Logical Operators

These combine boolean expressions:

- `and`: True if both operands are True (e.g., $(5 > 3)$ `and` $(2 < 4) \rightarrow$ `True`)
- `or`: True if at least one operand is True (e.g., $(5 > 3)$ `or` $(2 > 4) \rightarrow$ `True`)
- `not`: Inverts the boolean value (e.g., `not` $(5 > 3) \rightarrow$ `False`)

Example: `x > 0 and x < 10` checks if `x` is between 1 and 9.

4 Branching Structures in Python

Branching allows programs to execute different code based on conditions, using indentation to define blocks. Conditions use relational and logical operators.

4.1 if Statement

Executes code if the condition is `True`; skips it otherwise.

Structure:

```python
if condition:
    # code if True
# code after if (always runs)
```

How it works: Python evaluates the condition. If `True`, executes the indented block; if `False`, skips to the next non-indented line.

4.2 if-else Statement

Executes one block if `True`, another if `False`.

Structure:

رومانی صبحی رزق جرجس مطر

```
if condition:
    # code if True
else:
    # code if False
```

How it works: Exactly one block runs based on the condition.

4.3 if-elif-else Statement

Checks multiple conditions sequentially, executing the first `True` block, or `else` if none are `True`.

Structure:

```
if condition1:
    # code if condition1 True
elif condition2:
    # code if condition1 False and condition2 True
elif condition3:
    # code if previous False and condition3 True
else:
    # code if all above False
```

How it works: Checks conditions in order, stops at the first `True`. `elif` can repeat; `else` is optional.

5 Branching Program Examples

5.1 if Statement Examples

5.1.1 Program 1: Positive Number Check

Description: Checks if a number is positive and prints a message if so.

```
num = float(input("Enter a number: "))
if num > 0:
    print("The number is positive.")
print("Program ended.")
```

5.1.2 Program 2: Voting Eligibility

Description: Checks if a person is eligible to vote (age ≥ 18).

```
age = int(input("Enter your age: "))
if age >= 18:
    print("You are eligible to vote.")
print("Thank you.")
```

5.2 if-else Statement Examples

5.2.1 Program 1: Even or Odd

Description: Determines if a number is even or odd.

```
num = int(input("Enter a number: "))
if num % 2 == 0:
    print("The number is even.")
```

```
else:
    print("The number is odd.")
```

5.2.2 *Program 2: Freezing Check*

Description: Checks if temperature is above freezing (0°C).

```
temp = float(input("Enter temperature in Celsius: "))
if temp > 0:
    print("Above freezing.")
else:
    print("At or below freezing.")
```

5.3 if-elif-else Statement Examples

5.3.1 *Program 1: Grading System*

Description: Assigns a grade based on a score: A (90+), B (80-89), C (70-79), else Fail.

```
score = float(input("Enter score (0-100): "))
if score >= 90:
    print("Grade: A")
elif score >= 80:
    print("Grade: B")
elif score >= 70:
    print("Grade: C")
else:
    print("Fail")
```


5.3.2 *Program 2: Number Classification*

Description: Classifies a number as positive, negative, or zero.

```
num = float(input("Enter a number: "))
if num > 0:
    print("Positive")
elif num < 0:
    print("Negative")
else:
    print("Zero")
```

6 Assignments

Below are five assignments to test your understanding of arithmetic operators, their behavior with data types, and their use in sequential and branching structures.

6.1 Assignment 1: Testing Arithmetic Operators

Task: Write a Python program that takes two numbers (integers or floats) as input and performs all arithmetic operations (+, -, *, /, //, %, **). Print the result of each operation with a clear label. Ensure the program handles division by zero by checking if the second number is zero before performing division or modulus.

```
# Assignment 1: Test arithmetic operators
num1 = float(input("Enter first number: "))
```

```
num2 = float(input("Enter second number: "))
print("Addition:", num1 + num2)
print("Subtraction:", num1 - num2)
print("Multiplication:", num1 * num2)
if num2 != 0:
    print("Division:", num1 / num2)
    print("Floor Division:", num1 // num2)
    print("Modulus:", num1 % num2)
else:
    print("Division: Cannot divide by zero")
    print("Floor Division: Cannot divide by zero")
    print("Modulus: Cannot divide by zero")
print("Exponentiation:", num1 ** num2)
```

رومانی صبحی رزق جرجس مطر

6.2 Assignment 2: True/False - Arithmetic Operators and Data Types

Task: For each statement below, determine if it is `True` or `False` based on whether arithmetic operators work with the given data types. Write a Python program that tests each statement by attempting the operation and catching any errors using `try-except`. Print whether each statement is `True` or `False` based on the result.

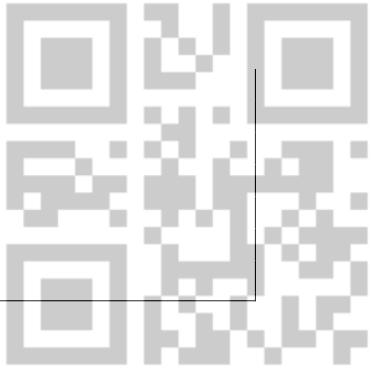
- Statement 1: Addition (+) works on strings (e.g., `"5" + "3"`).
- Statement 2: Multiplication (*) works on strings and integers (e.g., `"hello" * 3`).
- Statement 3: Division (/) works on strings (e.g., `"10" / "2"`).
- Statement 4: Addition (+) works on integers and floats (e.g., `5 + 3.2`).
- Statement 5: Exponentiation (**) works on floats (e.g., `2.5 ** 2`).


```
# Assignment 2: Test arithmetic operators with data types
print("Statement 1: Addition (+) works on strings")
try:
    result = "5" + "3"  # String concatenation
    print("Result:", result, "-> True (concatenates strings)")
except:
    print("Result: False")

print("\nStatement 2: Multiplication (*) works on strings and integers")
try:
    result = "hello" * 3  # String repetition
    print("Result:", result, "-> True (repeats string)")
except:
    print("Result: False")

print("\nStatement 3: Division (/) works on strings")
try:
    result = "10" / "2"  # Should raise TypeError
    print("Result:", result)
except:
    print("Result: False (division not supported for strings)")

print("\nStatement 4: Addition (+) works on integers and floats")
try:
    result = 5 + 3.2  # Integer + Float
    print("Result:", result, "-> True")
except:
    print("Result: False")
```



```
29
30 print("\nStatement 5: Exponentiation (**) works on floats")
31 try:
32     result = 2.5 ** 2  # Float exponentiation
33     print("Result:", result, "-> True")
34 except:
35     print("Result: False")
```

6.3 Assignment 3: Sequential Structure Program

Task: Write a sequential program that calculates the total cost of items purchased, including tax. Ask the user for the price of an item and the quantity purchased. Calculate the subtotal (price × quantity), apply a 5% tax, and display the subtotal, tax amount, and total cost.

```
1  # Assignment 3: Calculate total cost with tax
2  price = float(input("Enter item price: "))
3  quantity = int(input("Enter quantity: "))
4  subtotal = price * quantity
5  tax_rate = 0.05
6  tax = subtotal * tax_rate
7  total = subtotal + tax
8  print("Subtotal: $", subtotal)
9  print("Tax (5%): $", tax)
10 print("Total: $", total)
```

6.4 Assignment 4: Branching Structure Program

Task: Write a program using an `if-elif-else` structure to determine a discount based on the purchase amount. If the amount is over $100, apply a 10% discount; if over $50, apply a 5% discount; otherwise, no discount. Display the original amount, discount amount, and final amount after discount.

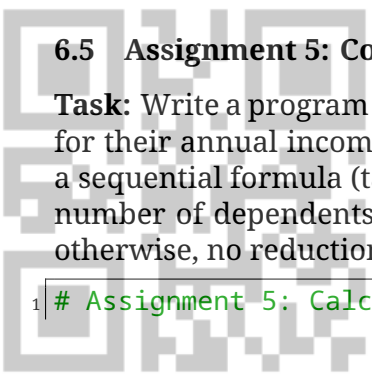
```
1  # Assignment 4: Apply discount based on purchase amount
2  amount = float(input("Enter purchase amount: $"))
3  if amount > 100:
4      discount = amount * 0.10
5  elif amount > 50:
6      discount = amount * 0.05
7  else:
8      discount = 0
9  final_amount = amount - discount
10 print("Original Amount: $", amount)
11 print("Discount: $", discount)
12 print("Final Amount: $", final_amount)
```

6.5 Assignment 5: Combined Sequential and Branching Program

Task: Write a program that combines sequential and branching structures. Ask the user for their annual income and the number of dependents. Calculate a tax amount using a sequential formula (tax = income × 0.15). Then, use branching to adjust the tax: if the number of dependents is 3 or more, reduce the tax by 20%; if 1 or 2, reduce by 10%; otherwise, no reduction. Display the income, initial tax, tax reduction, and final tax.

```
1  # Assignment 5: Calculate tax with dependent-based reduction
```

رومانی صبحی رزق جرجس مطر



```
income = float(input("Enter annual income: $"))
dependents = int(input("Enter number of dependents: "))
tax = income * 0.15  # Sequential calculation
if dependents >= 3:
    reduction = tax * 0.20
elif dependents >= 1:
    reduction = tax * 0.10
else:
    reduction = 0
final_tax = tax - reduction
print("Income: $", income)
print("Initial Tax (15%): $", tax)
print("Tax Reduction: $", reduction)
print("Final Tax: $", final_tax)
```

رومانی صبحی رزق جرجس مطر

Lecture Programs

Program 1:

```
# Arithmeatic Operators
'''
+ add      -  sub     * Multi     / div
// integer divsion         % mod        ** power
'''
# Sequantial Program

num1 = float(input("Enter first number1: "))
num2 = float(input("Enter Second number1: "))
summ = num1 + num2
subtract = num1 - num2
mult= num1 * num2
div = num1 / num2
intdiv = num1 // num2
power = num1 ** num2
moduls = num1 % num2
print(num1 , " + ", num2 , " = ", summ)
print(num1 , " - ", num2 , " = ", subtract)
print(num1 , " * ", num2 , " = ", mult)
print(num1 , " / ", num2 , " = ", div)
print(num1 , " // ", num2 , " = ", intdiv)
print(num1 , " ** ", num2 , " = ", power)
print(num1 , " % ", num2 , " = ", moduls)
```

Program 2:

```
# Assigment Operators

'''
=      +=          -=
*=      /=       //=     %=
'''

x = 10

print ("x = ", x)

x += 30    # x= x % 30
print ("x = ", x)
```

رومانی صبحی رزق جرجس مطر

Program 3:

```
# Sequential Program
# CylinderVolume = 3.14 * Radius ** 2 * Height

Radius = float(input("Enter Cylinder Radius: "))
Height= float(input("Enter Cylinder Height: "))

Cylinder_Volume = 3.14* Radius ** 2 * Height

print("Cylinder Volume = ", Cylinder_Volume)
```

Program 4:

```
# Sequential Program

Salary = float(input(" Enter your Salary: "))
tax= float(input("Enter Tax value: "))

NetSalary= Salary - (Salary * tax)

print("Salary = ", Salary)
print("tax = ", tax)
print("Net Salary= ", NetSalary)
```

Program 5:

```
# Sequential Program

Cel = float(input("Enter Celesice Temperature: "))

Fah = (Cel * 9 /5) + 32

print("Cel. Temperature =", Cel)

print("Faher. = ", Fah)
```

رومانی صبحی رزق جرجس مطر

رومانی صبحی رزق جرجس مطر

Program 6:

```
# Conditions -->  Relational Operator True/ False
'''
Condition
relation operators ( > , >= , < , <= , == , !=)
logical Operators ( and, or , not )

'''
x = int(input(" Enter x: "))
y= int (input("Enter y: "))
z= int (input("Enter z "))
condition1 = x > y   # True
condition2 = x > z   # true
condition3 = y == z  # true
and_condition =  condition1 and condition2 and condition3
or_condition =  condition1 or condition2 or condition3
not_condition = not condition1

print (" condition 1= ", condition1)
print (" condition 2= ", condition2)
print (" condition 3= ", condition3)
print (" And condition = ", and_condition)
print (" Or condition 1= ", or_condition)
print (" Not condition 1= ", not_condition)
```

Program 7: One Way Selection

```
# One Way



x = int(input("Enter X: "))

if x>3:
    print("X is Greater than 3")


print("Now we are out of iF")
```

رومانی صبحی رزق جرجس مطر

Program 8: One Way Selection

```
# Two way Selection  if condition else ...

x = int(input("Enter X: "))

if x>3:
    print("X is Greater than 3")
else:
    print("X is Less than or Equal 3")



print("Now we are out of iF")
```

Program 9: Two Way Selection

```
# Two way Selection  if condition else ...

x = int(input("Enter X: "))

if x>3:
    print("X is Greater than 3")
else:
    print("X is Less than or Equal 3")


print("Now we are out of iF")
```

Program 10:

```
# RectangleArea =  Width * Length

Width = float(input("Enter Rectangle Width : "))

Length = float(input("Enter Rectangle Length : "))

if Width > 0 and Length > 0:
        RectArea =Width * Length
        print ("Width = ", Width)
        print ("Length = ", Length)
        print ("Rectangle Area = ", RectArea)
else:
    print("Error: You must both Width and Length as Postive value")
```

رومانی صبحی رزق جرجس مطر

2025/2026

2025/2026

2025/2026

Python Lecture 3: Nested Branching, Looping, Break/Continue, and Examples

September 02, 2025

رومانی صبحی رزق جرجس مطر

1 Revision of Lecture 2 Programs

To reinforce concepts from Lecture 2, we revise three programs with slight modifications for clarity. These cover arithmetic operators, relational/logical operators, and basic branching.

1.1 Revised Program 1: Basic Arithmetic (from Lecture 2, Program 1)

Write a program that takes two numbers and computes their sum, difference, product, and division, with added error handling for division by zero.

.

```
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))
sum_result = num1 + num2
diff_result = num1 - num2
prod_result = num1 * num2
print("Sum:", sum_result)
print("Difference:", diff_result)
print("Product:", prod_result)
if num2 != 0:
    div_result = num1 / num2
    print("Division:", div_result)
else:
    print("Division: Cannot divide by zero")
```

2025/2026 2025/2026 2025/2026

Example Input: 10, 2
Output: Sum: 12.0
Difference: 8.0
Product: 20.0
Division: 5.0

1.2 Revised Program 2: Even or Odd (from Lecture 2, Program 5.2.1)

Write a program that determines if a number is even or odd, with added input validation.

.

```
try:
    num = int(input("Enter a number: "))
    if num % 2 == 0:
        print("The number is even.")
    else:
```

```
        print("The number is odd.")
except ValueError:
    print("Invalid input; please enter an integer.")
```

Example Input: 4
Output: The number is even.
Example Input: invalid
Output: Invalid input; please enter an integer.

1.3 Revised Program 3: Grading System (from Lecture 2, Program 5.3.1)

Write a program that assigns a grade based on score, with range check.

.

```
score = float(input("Enter score (0-100): "))
if score < 0 or score > 100:
    print("Invalid score; must be between 0 and 100.")
else:
    if score >= 90:
        print("Grade: A")
    elif score >= 80:
        print("Grade: B")
    elif score >= 70:
        print("Grade: C")
    else:
        print("Fail")
```

Example Input: 85
Output: Grade: B
Example Input: 101
Output: Invalid score; must be between 0 and 100.

2 Nested If Statements

Nested if statements involve placing an if-else structure inside another if or else block. This allows for more complex decision-making by checking multiple conditions hierarchically. Indentation is crucial; each nested level adds 4 spaces.

Structure:

```
if condition1:
    # code if condition1 True
    if condition2:
        # code if condition1 and condition2 True
    else:
        # code if condition1 True but condition2 False
else:
    # code if condition1 False
```

Write a program to check if a number is positive and even.

.

```
num = int(input("Enter a number: "))
if num > 0:
    if num % 2 == 0:
```

```
        print("Positive and even.")
    else:
        print("Positive but odd.")
else:
    print("Non-positive.")
```

Example Input: 4
Output: Positive and even.
Example Input: -1
Output: Non-positive.

3 Control Statements by Repetition (Looping)

Looping allows code to repeat until a condition is met. Python has two main loops: while and for.

3.1 While Loop

Executes as long as a condition is true. Useful for unknown iteration counts.

Structure:

```
while condition:
    # code to repeat
```

Example: Print numbers 1 to 5.

```
count = 1
while count <= 5:
    print(count)
    count += 1
```

2025/2026 2025/2026 2025/2026

Output: 1
2
3
4
5

3.2 For Loop

Iterates over a sequence (e.g., range, list). Useful for known iteration counts.

Structure:

```
for variable in sequence:
    # code to repeat
```

Example: Print numbers 1 to 5.

```
for i in range(1, 6):
    print(i)
```

Output: 1
2
3
4
5

رومانی صبحی رزق جرجس مطر

4 Break and Continue

- **break**: Exits the loop immediately. - **continue**: Skips the rest of the current iteration and goes to the next.

Example with break:

```
for i in range(1, 10):
    if i == 5:
        break
    print(i)
```

Output: 1
2
3
4

Example with continue:

```
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
```

Output: 1
2
4
5

5 Example Programs

Below are 10 programs demonstrating nested branching, loops, break, and continue step by step, each with expected output.

5.1 Program 1: Nested If for Age Group

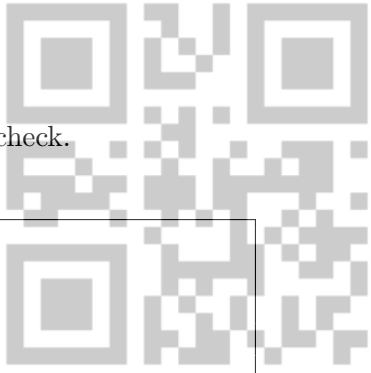
Write a program that classifies age into child, teen, or adult.

.

```
age = int(input("Enter age: "))
if age > 0:
    if age < 13:
        print("Child")
    elif age < 20:
        print("Teen")
    else:
        print("Adult")
else:
    print("Invalid age")
```

Example Input: 15
Output: Teen
Example Input: -1
Output: Invalid age

رومانی صبحی رزق جرجس مطر



5.2 Program 2: Nested If for Login Check

Write a program that simulates simple login with username and password check.

.

```
username = input("Enter username: ")
if username == "admin":
    password = input("Enter password: ")
    if password == "1234":
        print("Login successful")
    else:
        print("Invalid password")
else:
    print("Invalid username")
```

Example Input: admin, 1234
Output: Login successful
Example Input: user, 1234
Output: Invalid username

5.3 Program 3: While Loop for Sum

Write a program that sums numbers until user enters 0.

0.

```
total = 0
num = int(input("Enter number (0 to stop): "))
while num != 0:
    total += num
    num = int(input("Enter number (0 to stop): "))
print("Sum:", total)
```

2025/2026 2025/2026 2025/2026

Example Input: 5, 3, 0
Output: Sum: 8

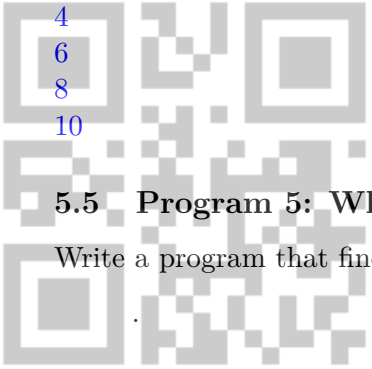
5.4 Program 4: For Loop with Range

Write a program that prints even numbers from 0 to 10.

0 10.

```
for i in range(0, 11, 2):
    print(i)
```

Output: 0
2
4
6
8
10



5.5 Program 5: While with Break

Write a program that finds first even number greater than input.

.

رومانی صبحی رزق جرجس مطر

```
num = int(input("Enter starting number: "))
while True:
    num += 1
    if num % 2 == 0:
        print("First even:", num)
        break
```

Example Input: 3
Output: First even: 4

5.6 Program 6: For with Continue

Write a program that prints numbers 1-10, skipping multiples of 3.

1-10 3.

```
for i in range(1, 11):
    if i % 3 == 0:
        continue
    print(i)
```

Output: 1
2
4
5
7
8
10

5.7 Program 7: Nested Loop (For in For)

Write a program that prints a 3x3 grid.

3x3.

```
for row in range(1, 4):
    for col in range(1, 4):
        print(row * col, end=" ")
    print()
```

Output: 1 2 3
2 4 6
3 6 9

5.8 Program 8: While with Nested If

Write a program that counts positive/negative numbers until 0.

/ 0.

```
positive = 0
negative = 0
num = int(input("Enter number (0 to stop): "))
while num != 0:
    if num > 0:
        positive += 1
    else:
```

```
        negative += 1
    num = int(input("Enter number (0 to stop): "))
print("Positive:", positive, "Negative:", negative)
```

Example Input: 5, -3, 0
Output: Positive: 1 Negative: 1

5.9 Program 9: For with Break in Nested Loop

Write a program that breaks out of inner loop when condition met.

.

```
for i in range(1, 4):
    for j in range(1, 4):
        print(i, j)
        if j == 2:
            break
```

Output: 1 1
1 2
2 1
2 2
3 1
3 2

5.10 Program 10: Infinite Loop with Break and Continue

Write a program that echoes input until "quit", skips "skip".

2025/2026 2025/2026 2025/2026

"quit" "skip".

```
while True:
    text = input("Enter text (quit to stop): ")
    if text == "quit":
        break
    if text == "skip":
        continue
    print("Echo:", text)
```

Example Input: hello, skip, world, quit
Output: Echo: hello
Echo: world

6 Assignments

Below are 5 assignments to test your understanding of nested branching, loops, break, and continue. Solutions are not provided; practice by writing and testing the code.

6.1 Assignment 1: Nested If for Eligibility

Write a program that checks if a person is eligible for a job: age >=18 and experience >2 years (nested inside age check). Print appropriate messages.

: >=18 >2 ( ). .

رومانی صبحی رزق جرجس مطر

6.2 Assignment 2: While Loop for Factorial

Write a program to calculate factorial of a number using while loop. Handle invalid input.

while. .

6.3 Assignment 3: For Loop with Continue

Write a program to print numbers 1-20, skipping those divisible by 5 using continue.

1-20 5 continue.

6.4 Assignment 4: Nested Loop Pattern

Write a program to print a triangle pattern using nested for loops (e.g., 1
22
333).

for (1
22
333).

6.5 Assignment 5: Game Loop with Break/Continue

Write a program to simulate a simple guessing game: guess a number 1-10, continue until correct or "quit" to break.

: 1-10 "quit" .

رومانی صبحی رزق جرجس مطر

Python Lecture 3: Revision Programs for Looping Concepts

September 09, 2025

رومانی صبحی رزق جرجس مطر

1 Revision Programs

To refresh the concepts from Lecture 3 on nested branching and looping, here are four programs. These programs incorporate for loops, while loops, if-elif-else structures, and input validation to reinforce the material.

1.1 Program 1: Multiplication Table

Write a program that creates a multiplication table for numbers 1 to 10 using nested for loops.

1 10 for .

```
for i in range(1, 11):
    for j in range(1, 11):
        print(f"{i} * {j} = {i * j}", end="\t")
    print()
```

Output: 1 * 1 = 1 1 * 2 = 2 ... 1 * 10 = 10
2 * 1 = 2 2 * 2 = 4 ... 2 * 10 = 20
...
10 * 1 = 10 10 * 2 = 20 ... 10 * 10 = 100

2025/2026 2025/2026 2025/2026

1.2 Program 2: Updated Grading System with Validation

Write a program that updates the grading code to allow the student to continue entering a valid score if the input is less than 0 or greater than 100, using a while loop for validation within a for loop for multiple students.

0 100 while for .

```
for i in range(0, 4):
    valid = False
    while not valid:
        try:
            score = float(input("Enter score between 0 and 100: "))
            if score < 0 or score > 100:
                print("Your score must be between 0 and 100")
            else:
                valid = True
        except ValueError:
            print("Invalid input; please enter a number.")
    if score >= 90:
        print("Grade A")
    elif score >= 80:
        print("Grade B")
```

```
    elif score >= 70:
        print("Grade C")
    else:
        print("Fail")
print("END")
```

Example Input (for one student): 110, 85
Your score must be between 0 and 100
Grade B
(Continues for 3 more students)
END

1.3 Program 3: Mixed Loop with While, For, and If-Elif-Else

Write a program that uses a while loop to repeatedly ask for a number until a valid positive integer is entered, then uses a for loop to iterate from 1 to that number, classifying each as "even", "div by 3", or "other" using if-elif-else.

while for 1 "" " 3" "" if-elif-else.

```
valid = False
while not valid:
    try:
        num = int(input("Enter a positive integer: "))
        if num > 0:
            valid = True
        else:
            print("Must be positive.")
    except ValueError:
        print("Invalid input; enter an integer.")
for i in range(1, num + 1):
    if i % 2 == 0:
        print(i, "even")
    elif i % 3 == 0:
        print(i, "div by 3")
    else:
        print(i, "other")
```

2025/2026 2025/2026 2025/2026

Example Input: -1, abc, 5
Must be positive.
Invalid input; enter an integer.
1 other
2 even
3 div by 3
4 even
5 other

1.4 Program 4: Three Nested For Loops

Write a program that uses three nested for loops to create a 3D-like pattern, printing coordinates (i, j, k) where i ranges from 1 to 2, j from 1 to 2, and k from 1 to 2, demonstrating how each level of nesting iterates fully before moving to the next.

for (i, j, k) i 1 2 j 1 2 k 1 2 .

```
for i in range(1, 3):
```

روماني صبحي رزق جرجس مطر

```
    for j in range(1, 3):
        for k in range(1, 3):
            print(f"({i}, {j}, {k})", end=" ")
        print()
```

Output: (1, 1, 1) (1, 1, 2)
(1, 2, 1) (1, 2, 2)
(2, 1, 1) (2, 1, 2)
(2, 2, 1) (2, 2, 2)

رومانی صبحی رزق جرجس مطر

Python Lecture 4: Functions and Modules

September 11, 2025

رومانى صبحى رزق جرجس مطر

1 Functions in Python

Functions are reusable blocks of code that perform a specific task. They help organize code, reduce repetition, and improve readability. A function can take input (parameters), process it, and optionally return output. Functions promote modularity, allowing complex programs to be broken into smaller, manageable parts.

1.1 Syntax for Defining and Calling Functions

To define a function, use the `def` keyword followed by the function name, parameters in parentheses, and a colon. The body is indented.

Structure:

```
def function_name(parameters):
    # code to execute
    return value  # optional
```

2025/2026 2025/2026 2025/2026

To call a function, use its name followed by parentheses with arguments.

Example: A simple function to add two numbers.

```
def add(a, b):
    return a + b

num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))
result = add(num1, num2)
print(result)
```

Example Input: 5, 3
Output: 8.0

2 Types of Functions

2.1 Built-in Functions vs User-Defined Functions

Built-in functions are pre-defined in Python or libraries (e.g., print(), len()) and are ready to use without definition. User-defined functions are created by the programmer using `def` to perform custom tasks.

To use built-in functions from libraries, import the library first. For example, to use the math library, write 'import math'. You can then call functions like math.sqrt(16), which

returns 4.0. Other libraries include: - pandas: Import with 'import pandas as pd', and use pd.DataFrame([[1, 2], [3, 4]]) to create a DataFrame. - numpy: Import with 'import numpy as np', and use np.array([1, 2, 3]) to create an array. - sklearn: Import with 'from sklearn.linear$_{m}odelimportLinearRegression'$, $anduseLinearRegression()tocreateamodel.-keras$: $Importwith'fromkeras.modelsimportSequential'$, $anduseSequential()tocreateaneuralnetworkmodel.$

Examples of calling built-in functions from the math library: - math.pow(2, 3) returns 8.0 (2 raised to 3). - math.sin(math.pi / 2) returns 1.0 (sine of 90 degrees). To call these, import math first, then use math.function$_{n}ame(arguments)$.

2.2 Types of User-Defined Functions

User-defined functions can be classified into four types based on parameters and return values. Here are examples using the task of adding three numbers (a, b, c) and optionally returning or printing the sum:

- Takes parameters, returns value: This function accepts inputs and returns a result. Define it as:

```
def add(a, b, c):
    return a + b + c

a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
c = float(input("Enter third number: "))
sum = add(a, b, c)
print(sum)
```

Call it with sum = add(1, 2, 3) and print(sum) to get 6.0.

- Takes parameters, no return: This function accepts inputs but only prints the result without returning it. Define it as:

```
def print_sum(a, b, c):
    print(a + b + c)

a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
c = float(input("Enter third number: "))
print_sum(a, b, c)
```

Call it with print$_{s}um(1, 2, 3)toget6.0printed.$

- No parameters, returns value: This function has no inputs and returns a computed value. Define it as:

```
def get_sum():
    return 1 + 2 + 3

sum = get_sum()
print(sum)
```

Call it with sum = get$_{s}um()andprint(sum)toget6.$

- No parameters, no return: This function has no inputs or return, performing an action. Define it as:

```
def display_sum():
    print(1 + 2 + 3)

display_sum()
```

Call it with display$_sum()toget6printed.$

2.3 Another Example: Rectangle Area Computation

Using the four types for computing rectangle area (length * width).

- Takes parameters, returns value:

```
def area(length, width):
    return length * width

length = float(input("Enter length: "))
width = float(input("Enter width: "))
rect_area = area(length, width)
print(rect_area)
```

Example Input: 5, 3
Output: 15.0

- Takes parameters, no return:

2025/2026 2025/2026 2025/2026

```
def print_area(length, width):
    print(length * width)

length = float(input("Enter length: "))
width = float(input("Enter width: "))
print_area(length, width)
```

Example Input: 5, 3
Output: 15.0

- No parameters, returns value:

```
def get_area():
    length = float(input("Enter length: "))
    width = float(input("Enter width: "))
    return length * width

rect_area = get_area()
print(rect_area)
```

Example Input: 5, 3
Output: 15.0

- No parameters, no return:

رومانی صبحی رزق جرجس مطر

```
  def display_area():
      length = float(input("Enter length: "))
      width = float(input("Enter width: "))
      print(length * width)

  display_area()
```

Example Input: 5, 3
Output: 15.0

2.4 Additional Function Examples

2.4.1 Function to Decide if a Number is Even or Odd

Write a function that takes a number and returns whether it is even or odd.

.

```
def is_even_odd():
    num = float(input("Enter a number: "))
    if num % 2 == 0:
        return "Even"
    else:
        return "Odd"

result = is_even_odd()
print(result)
```


Example Input: 4
Output: Even

2.4.2 Function for Computing the Grade

Write a function that takes a score and returns the grade based on the score range.

.

```
def compute_grade():
    score = float(input("Enter score (0-100): "))
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    else:
        return "Fail"

grade = compute_grade()
print(grade)
```

Example Input: 85
Output: B

رومانی صبحی رزق جرجس مطر

2.4.3 Function to Add Even Numbers

Write a function that takes a list of numbers and returns the sum of even numbers.

.

```
def sum_even_numbers():
    numbers = input("Enter numbers separated by spaces: ").
       split()
    total = 0
    for num in numbers:
        num = float(num)
        if num % 2 == 0:
            total += num
    return total

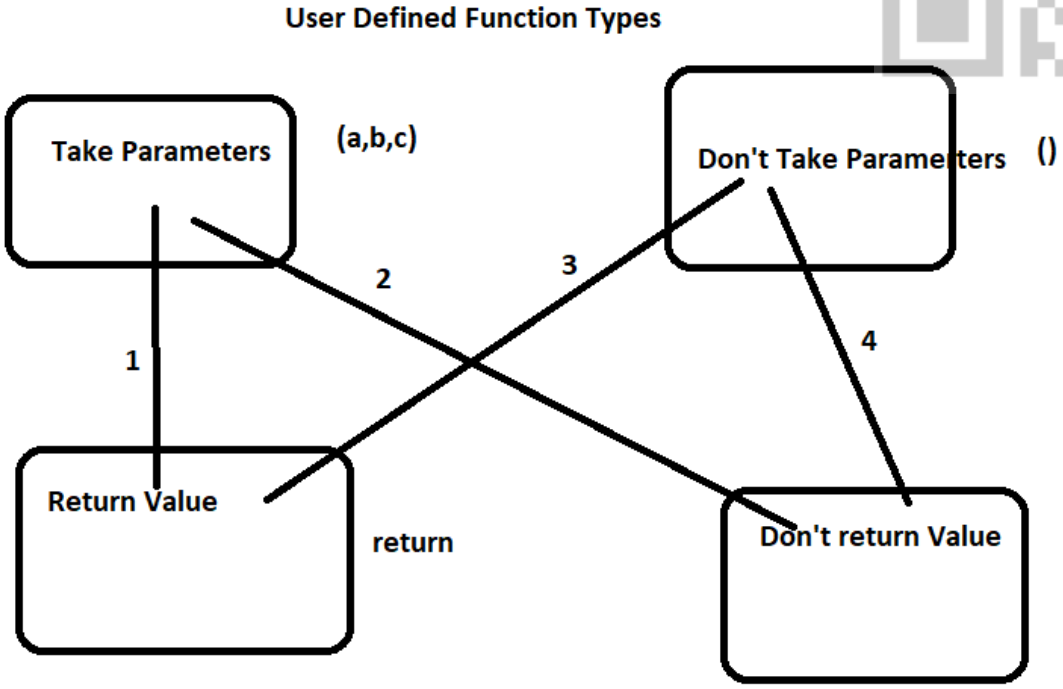
even_sum = sum_even_numbers()
print(even_sum)
```

Example Input: 1 2 3 4 5 6
Output: 12.0

رومانی صبحی رزق جرجس مطر

رومانی صبحی رزق جرجس مطر

User Defined Functions



Example:

```python
# Fuction Rect_Area

# Type 1:  Function take parameters and return value

def Rect_Area_1(length, width):
    area = width * length
    return area

# or Call -->Best

L = float(input("Enter Length: "))
W= float(input("Enter Width: "))
print("Rectangle Area = ",Rect_Area_1(L, W))
```

رومانی صبحی رزق جرجس مطر

```
#-----------------------------------------------------

# Type 2: Function take parameters but don't return value

def Rect_Area_2(length, width):
    area= length * width
    print("Rectangel Area = ", area)

# Calling
L = float(input("Enter Length: "))
W= float(input("Enter Width: "))
Rect_Area_2(L, W)

#-----------------------------------------
```

```
#-----------------------------------------


# Type 3: Function Donot take parameters but return value

def Rect_Area_3():
    length = float(input("Enter Length: "))
    width = float (input("Enter Width"))
    area = length* width

    return area

# call

print("Rectangel Area =", Rect_Area_3())


#-----------------------------------------
```

رومانی صبحی رزق جرجس مطر

```python
# Type 4: Function Donot take parameters but Donot return value

def Rect_Area_4():
    length = float(input("Enter Length: "))
    width = float (input("Enter Width"))
    area = length* width

    print("Rectangle Area = ", area )

# Call

Rect_Area_4()
```

Assignment

```python
# Assignment4: compute cylinder volume using the four types
#cyl_vol= 3.14 * radius ** 2 * Height

#Cylinder_volume_1
#Cylinder_volume_2
#Cylinder_volume_3
#Cylinder_volume_4
```

Python Lecture 4: Functions and Modules

September 11, 2025

رومانى صبحى رزق جرجس مطر

1 Functions in Python

Functions are reusable blocks of code that perform a specific task. They help organize code, reduce repetition, and improve readability. A function can take input (parameters), process it, and optionally return output. Functions promote modularity, allowing complex programs to be broken into smaller, manageable parts.

1.1 Syntax for Defining and Calling Functions

To define a function, use the `def` keyword followed by the function name, parameters in parentheses, and a colon. The body is indented.

Structure:

```
def function_name(parameters):
    # code to execute
    return value  # optional
```

2025/2026 2025/2026 2025/2026

To call a function, use its name followed by parentheses with arguments.

Example: A simple function to add two numbers.

```
def add(a, b):
    return a + b

num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))
result = add(num1, num2)
print(result)
```

Example Input: 5, 3
Output: 8.0

2 Types of Functions

2.1 Built-in Functions vs User-Defined Functions

Built-in functions are pre-defined in Python or libraries (e.g., print(), len()) and are ready to use without definition. User-defined functions are created by the programmer using `def` to perform custom tasks.

To use built-in functions from libraries, import the library first. For example, to use the math library, write 'import math'. You can then call functions like math.sqrt(16), which

returns 4.0. Other libraries include: - pandas: Import with 'import pandas as pd', and use pd.DataFrame([[1, 2], [3, 4]]) to create a DataFrame. - numpy: Import with 'import numpy as np', and use np.array([1, 2, 3]) to create an array. - sklearn: Import with 'from sklearn.linear$_{m}odelimportLinearRegression'$, $anduseLinearRegression()tocreateamodel.-keras$: $Importwith'fromkeras.modelsimportSequential'$, $anduseSequential()tocreateaneuralnetworkmodel.$

Examples of calling built-in functions from the math library: - math.pow(2, 3) returns 8.0 (2 raised to 3). - math.sin(math.pi / 2) returns 1.0 (sine of 90 degrees). To call these, import math first, then use math.function$_{n}ame(arguments)$.

2.2 Types of User-Defined Functions

User-defined functions can be classified into four types based on parameters and return values. Here are examples using the task of adding three numbers (a, b, c) and optionally returning or printing the sum:

- Takes parameters, returns value: This function accepts inputs and returns a result. Define it as:

```
def add(a, b, c):
    return a + b + c

a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
c = float(input("Enter third number: "))
sum = add(a, b, c)
print(sum)
```

Call it with sum = add(1, 2, 3) and print(sum) to get 6.0.

- Takes parameters, no return: This function accepts inputs but only prints the result without returning it. Define it as:

```
def print_sum(a, b, c):
    print(a + b + c)

a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
c = float(input("Enter third number: "))
print_sum(a, b, c)
```

Call it with print$_{s}um(1, 2, 3)toget6.0printed.$

- No parameters, returns value: This function has no inputs and returns a computed value. Define it as:

```
def get_sum():
    return 1 + 2 + 3

sum = get_sum()
print(sum)
```

Call it with sum = get$_{s}um()andprint(sum)toget6.$

- No parameters, no return: This function has no inputs or return, performing an action. Define it as:

```
def display_sum():
    print(1 + 2 + 3)

display_sum()
```

Call it with display$_sum()toget6printed.$

2.3 Another Example: Rectangle Area Computation

Using the four types for computing rectangle area (length * width).

- Takes parameters, returns value:

```
def area(length, width):
    return length * width

length = float(input("Enter length: "))
width = float(input("Enter width: "))
rect_area = area(length, width)
print(rect_area)
```

Example Input: 5, 3
Output: 15.0

- Takes parameters, no return:

2025/2026 2025/2026 2025/2026

```
def print_area(length, width):
    print(length * width)

length = float(input("Enter length: "))
width = float(input("Enter width: "))
print_area(length, width)
```

Example Input: 5, 3
Output: 15.0

- No parameters, returns value:

```
def get_area():
    length = float(input("Enter length: "))
    width = float(input("Enter width: "))
    return length * width

rect_area = get_area()
print(rect_area)
```

Example Input: 5, 3
Output: 15.0

- No parameters, no return:

رومانی صبحی رزق جرجس مطر

```
  def display_area():
      length = float(input("Enter length: "))
      width = float(input("Enter width: "))
      print(length * width)

  display_area()
```

Example Input: 5, 3
Output: 15.0

2.4 Additional Function Examples

2.4.1 Function to Decide if a Number is Even or Odd

Write a function that takes a number and returns whether it is even or odd.

.

```
def is_even_odd():
    num = float(input("Enter a number: "))
    if num % 2 == 0:
        return "Even"
    else:
        return "Odd"

result = is_even_odd()
print(result)
```


Example Input: 4
Output: Even

2.4.2 Function for Computing the Grade

Write a function that takes a score and returns the grade based on the score range.

.

```
def compute_grade():
    score = float(input("Enter score (0-100): "))
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    else:
        return "Fail"

grade = compute_grade()
print(grade)
```

Example Input: 85
Output: B

رومانی صبحی رزق جرجس مطر

2.4.3 Function to Add Even Numbers

Write a function that takes a list of numbers and returns the sum of even numbers.

.

```
def sum_even_numbers():
    numbers = input("Enter numbers separated by spaces: ").
       split()
    total = 0
    for num in numbers:
        num = float(num)
        if num % 2 == 0:
            total += num
    return total

even_sum = sum_even_numbers()
print(even_sum)
```

Example Input: 1 2 3 4 5 6
Output: 12.0

رومانی صبحی رزق جرجس مطر

رومانی صبحی رزق جرجس مطر

Python Lecture 5: Data Structures

September 18, 2025

1 Review of Functions from Lecture 4

To build on the concepts from Lecture 4, we start with a quick refresh of built-in functions, importing packages, and discovering library functions. This connects to our new topic by showing how functions can manipulate data structures.

1.1 Refreshing Built-in Functions

Built-in functions like `print()`, `len()`, and `input()` are always available. Here's a simple program to refresh their use, incorporating user input and output as seen in Lecture 4 examples.

2025/2026 2025/2026 2025/2026

```
def refresh_builtins():
    name = input("Enter your name: ")
    print("Hello, " + name + "! Your name has " + str(len(
    name)) + " characters.")

refresh_builtins()
```

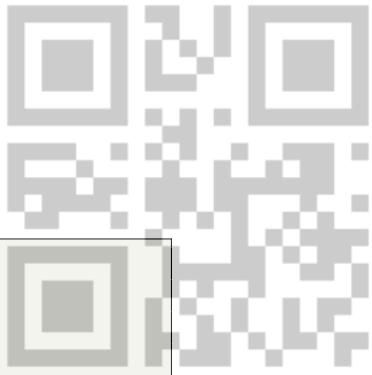
Example Input: Alice
Output: Hello, Alice! Your name has 5 characters.

This function takes no parameters but uses built-ins to interact with the user, similar to the no-parameters examples in Lecture 4.

1.2 How to Import Packages and Use Existing Functions

As discussed in Lecture 4, import packages using 'import library_name' or 'from library import function'. Examples include math, pandas, numpy, sklearn, and keras. After importing, call functions with library_name.function(arguments).

Example: Using math library (from Lecture 4).



```python
import math

def use_math_functions():
    print(math.sqrt(25))  # Returns 5.0
    print(math.pow(2, 3))  # Returns 8.0

use_math_functions()
```

Other examples:
- import pandas as pd; df = pd.DataFrame([[1, 2], [3, 4]])
- import numpy as np; arr = np.array([1, 2, 3])
- from sklearn.linear$_{m}$odelimport$LinearRegression$; $model = LinearRegression()$
- from keras.models import Sequential; nn$_{m}$odel = $Sequential()$

These build on Lecture 4 by reusing the import syntax in functions.

1.3 How to Know the Existing Functions of a Library

To discover functions in a library, use built-in tools like dir(library_name) to list attributes and methods, or help(library_name) for documentation. This is useful after importing.

Example: Discovering math functions.

```python
import math

def discover_math():
    print(dir(math))  # Lists all functions like ['acos', 'asin', 'sqrt', ...]
    help(math.sqrt)   # Shows documentation for sqrt

discover_math()
```

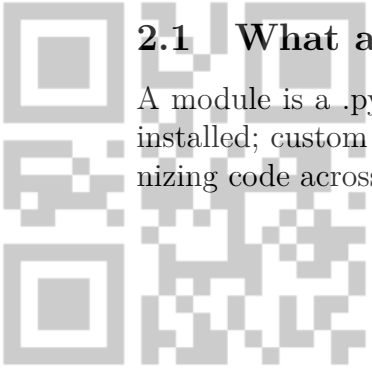
Use this to explore libraries mentioned in Lecture 4, like pandas or numpy.

2 Modules in Python

Modules are files containing Python code (functions, variables, classes) that can be imported and reused, promoting modularity as introduced in Lecture 4.

2.1 What are Modules

A module is a .py file with definitions. Built-in modules like math are pre-installed; custom modules are user-created. They extend functions by organizing code across files.



2025 رومانی صبحی رزق جرجس مطر

رومانی صبحی رزق جرجس مطر

2.2 How to Create a Module

Save functions in a file like my_module.py. Then import it in another script.

Example: Create my_module.py with functions from Lecture 4.

Content of my_module.py:

```python
def add(a, b, c):
    return a + b + c

def is_even_odd(num):
    if num % 2 == 0:
        return "Even"
    else:
        return "Odd"
```

To use it:

```python
import my_module

sum_result = my_module.add(1, 2, 3)  # Returns 6
print(sum_result)

even_check = my_module.is_even_odd(4)  # Returns "Even"
print(even_check)
```

This connects to Lecture 4 by reusing user-defined functions in a module.

3 Data Structures in Python

Data structures store and organize data efficiently. Python's built-in ones include lists, tuples, dictionaries, and sets. Arrays are available via numpy. We'll explain each with simple programs, building on Lecture 4 functions.

3.1 Comparison of Data Structures

Before diving into each data structure, here is a comparison table summarizing their key characteristics and usage.

3.2 Lists

Lists are one of the most versatile data structures in Python. They are ordered collections, meaning elements have a specific position and can be accessed by index (starting from 0). Lists are mutable, allowing addition, removal, or modification of elements. They can contain duplicates and elements of different types (integers, strings, floats, etc.). Lists support slicing

(e.g., my_list[1:3]), iteration with loops, and comprehension for creating new lists.

Common operations include: - Appending elements with `append()` - Inserting at a position with `insert()` - Removing with `remove()` or `pop()` - Sorting with `sort()` or `sorted()` - Reversing with `reverse()`

Example: Simple list operations.

```
def list_demo():
    my_list = [1, 2, 3, "apple", 4.5]
    print(my_list[0])  # Access first element: 1
    my_list.append(6)  # Add element
    print(len(my_list))  # Length: 6
    print(my_list[1:4])  # Slicing: [2, 3, 'apple']

list_demo()
```

Connect to Lecture 4: Modify sum_even_numbers() to take a list parameter and use it.

Modified sum_even_numbers in my_module.py:

```
def sum_even_numbers(numbers):
    total = 0
    for num in numbers:
        num = float(num)
        if num % 2 == 0:
            total += num
    return total
```

Usage:

```
from my_module import sum_even_numbers

numbers = [1, 2, 3, 4, 5, 6]
even_sum = sum_even_numbers(numbers)
print(even_sum)  # 12.0
```

3.2.1 Additional List Examples

Example 1: Iterating over a list with mixed types.

```
fruits = ["apple", "banana", "cherry", "dfsdfsfsd", "mrmrmr",
    2323, 232355.3434]
for fruit in fruits:
    print(fruit)
    print("Hi in For")

print("Hi out For")
```

2025 رومانی صبحی رزق جرجس مطر

رومانی صبحی رزق جرجس مطر

Example 2: Creating a list using range.

```
y = list(range(10, 0, -2))  # countdown by 2
print(y)  # [10, 8, 6, 4, 2]
```

Example 3: Append, insert, remove operations.

```
numbers = [1, 2, 3, 4, 5]
print("Initial list:", numbers)  # Initial list: [1, 2, 3, 4,
    5]

numbers.append(10)
print(numbers)  # [1, 2, 3, 4, 5, 10]

numbers.insert(0, 100)
print(numbers)  # [100, 1, 2, 3, 4, 5, 10]

numbers.insert(3, 200)
print(numbers)  # [100, 1, 2, 200, 3, 4, 5, 10]

numbers.remove(200)
numbers.remove(100)
print(numbers)  # [1, 2, 3, 4, 5, 10]
```

Example 4: Pop operations.

```
list2 = [1, 2, 3, 5, 6, 9, 4, 70]
print(list2)  # [1, 2, 3, 5, 6, 9, 4, 70]

list2.pop()
list2.pop()
list2.pop()
print(list2)  # [1, 2, 3, 5, 6, 9]
```

Example 5: Sorting lists.

```
list3 = [100, 90, 10, 45, 400, 700, 12]
list4 = [100, 90, 10, 45, 400, 700, 12]

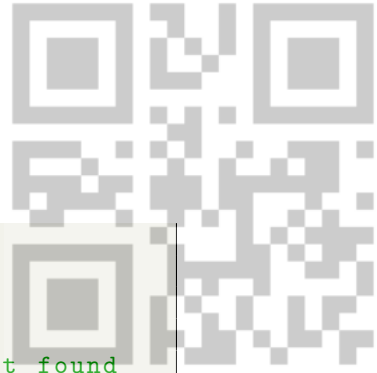
print(list3)  # [100, 90, 10, 45, 400, 700, 12]

list3.sort()  # Asc Order
print(list3)  # [10, 12, 45, 90, 100, 400, 700]

list4.sort(reverse=True)  # Desc order
print(list4)  # [700, 400, 100, 90, 45, 12, 10]
```

Example 6: Index and modifying elements.

```
my_list = [1, 2, 3, 4]
print(my_list)  # [1, 2, 3, 4]
```



```
try:
    index = my_list.index(5)  # 5 is not in the list
    print(index)
except ValueError:
    print("Value not found in the list")  # Value not found
    in the list

my_list[0] = 10
print(my_list)  # [10, 2, 3, 4]

my_list[2] = 30
print(my_list)  # [10, 2, 30, 4]
```

Example 7: Comprehensive list methods.

```
numbers = [1, 2, 3, 4, 5]

# Accessing elements
print("First item (numbers[0]):", numbers[0])  # 1
print("Second item (numbers[1]):", numbers[1])  # 2
print("Last item (numbers[-1]):", numbers[-1])  # 5
print("Second last item (numbers[-2]):", numbers[-2])  # 4
print("-------------------------------------------------")

# Append - adds 6 to the end
numbers.append(6)
print("After append(6):", numbers)  # [1, 2, 3, 4, 5, 6]
print("-------------------------------------------------")

# Insert - adds 6 at index 0
numbers.insert(0, 6)
print("After insert(0, 6):", numbers)  # [6, 1, 2, 3, 4, 5,
    6]
print("-------------------------------------------------")

# Remove - removes the first occurrence of 6
numbers.remove(6)
print("After remove(6):", numbers)  # [1, 2, 3, 4, 5, 6]
print("-------------------------------------------------")

# Pop - removes the last item
popped = numbers.pop()
print("After pop():", numbers)  # [1, 2, 3, 4, 5]
print("Popped value:", popped)  # 6
print("-------------------------------------------------")

# Copy - creates a copy of the list
copied_list = numbers.copy()
print("Copied list:", copied_list)  # [1, 2, 3, 4, 5]
print("-------------------------------------------------")
```

رومانی صبحی رزق جرجس مطر



رومانی صبحی رزق جرجس مطر

```
35
36 # Sort - sorts the list in ascending order
37 numbers.sort()
38 print("After sort():", numbers)  # [1, 2, 3, 4, 5]
39 print("-----------------------------------------------")
40
41 # Reverse - reverses the list
42 numbers.reverse()
43 print("After reverse():", numbers)  # [5, 4, 3, 2, 1]
44 print("-----------------------------------------------")
45
46 # Index - finds the position of the first occurrence of 3
47 if 3 in numbers:
48     print("Index of 3:", numbers.index(3))  # 2
49 else:
50     print("3 is not in the list.")
51 print("-----------------------------------------------")
52
53 # Clear - removes all items
54 numbers.clear()
55 print("After clear():", numbers)  # []
56 print("-----------------------------------------------")
```

3.3 Tuples

Tuples are similar to lists but immutable, meaning once created, their elements cannot be changed, added, or removed. This makes them suitable for fixed collections of items, like coordinates or constants. Tuples are faster and use less memory than lists. They support indexing, slicing, and unpacking (e.g., a, b = my_tuple).

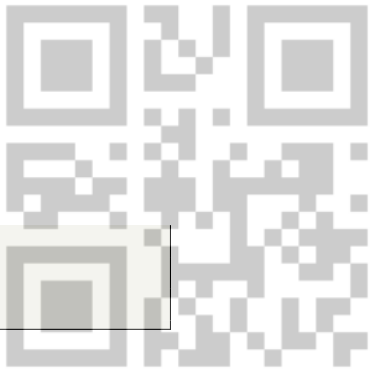
Common uses: As keys in dictionaries (since immutable), returning multiple values from functions.

Example:

```
1 def tuple_demo():
2     my_tuple = (1, 2, "banana", 3.5)
3     print(my_tuple[1])  # 2
4     # my_tuple[0] = 0  # Error: immutable
5     a, b, _, _ = my_tuple  # Unpacking
6     print(a + b)  # 3
7
8 tuple_demo()
```

Connect: Use add() from Lecture 4 on tuple elements.

```
1 from my_module import add
2
```



```
coords = (1, 2, 3)
total = add(*coords)  # Unpack tuple: 6
print(total)
```

3.3.1 Additional Tuple Examples

Example 1: Accessing and unpacking tuples.

```
x = (10, 14, 19)
print(x)  # (10, 14, 19)
print(x[0])  # 10
print(x[1])  # 14
print(x[2])  # 19

item1, item2, item3 = x
print(item1)  # 10
print(item2)  # 14
print(item3)  # 19

item1 = 100
print(item1)  # 100
```

Example 2: Converting tuple to list.

```
# Convert Tuple to list
my_tuple = (1, 2, 3, 4)
my_list = list(my_tuple)
print(my_list)  # [1, 2, 3, 4]

my_list[0] = 1000
print(my_list)  # [1000, 2, 3, 4]
```

Example 3: Comprehensive tuple operations.

```
# Creating a tuple
coordinates = (1, 2, 3)
print("Tuple:", coordinates)  # Tuple: (1, 2, 3)

# Accessing elements
print("First element:", coordinates[0])  # 1
print("Second element:", coordinates[1])  # 2
print("Third element:", coordinates[2])  # 3
print("----------------------------------------")

# Trying to change a value (this will raise an error)
# coordinates[0] = 10  #  TypeError: 'tuple' object does not
    support item assignment

# Length of the tuple
```

2025 رومانی صبحی رزق جرجس مطر



رومانی صبحی رزق جرجس مطر

```
print("Length of tuple:", len(coordinates))  # 3
print("-----------------------------------------")

# Unpacking the tuple into separate variables
x, y, z = coordinates
print("Unpacked values:")
print("x =", x)  # 1
print("y =", y)  # 2
print("z =", z)  # 3
print("-----------------------------------------------------")

# Tuple with one item (note the comma)
single_item_tuple = (5,)
print("Single-item tuple:", single_item_tuple)  # (5,)
print("-----------------------------------------------------")

# Using tuple in a function return: the function return tuple (list of values to be returned)
def get_student_info():
    return ("Alice", 20, "Computer Science")

name, age, major = get_student_info()
print("\nStudent Info:")
print("Name:", name)  # Alice
print("Age:", age)  # 20
print("Major:", major)  # Computer Science
print("-----------------------------------------")
```

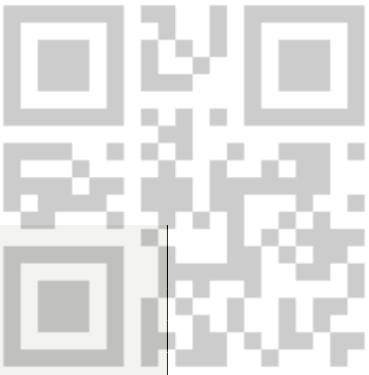
3.4 Dictionaries

Dictionaries store data in key-value pairs, where keys are unique and immutable (e.g., strings, numbers, tuples), and values can be any type, including other data structures. Since Python 3.7, dictionaries preserve insertion order. They provide fast lookups, insertions, and deletions based on keys. Not indexed by numbers, but by keys.

Common methods: - Access with `dict[key]` or `get(key, default)` - Add/update with `dict[key] = value` or `update()` - Iterate with `keys()`, `values()`, `items()`

Example: Use dictionaries to store Kaggle dataset metadata and print in the CLI.

```
def kaggle_metadata():
    dataset = {
        "name": "Titanic",
        "size": "1MB",
```



```
        "rows": 891,
        "columns": 12,
        "description": "Passenger data for survival
    prediction"
    }
    print("Dataset Name: " + dataset["name"])
    print("Size: " + dataset.get("size", "Unknown"))
    print("Rows: " + str(dataset["rows"]))
    for key, value in dataset.items():
        print(key + ": " + str(value))

kaggle_metadata()
```

Connect: Compute grade average from dict of scores using compute_grade(). Assuming compute_grade takes a score parameter (from Lecture 4).

```
from my_module import compute_grade

scores = {"Alice": 85, "Bob": 92}
for name, score in scores.items():
    print(name + ": " + compute_grade(score))
```

3.4.1 Additional Dictionary Examples

Example 1: Accessing dictionary values.

2025/2026 2025/2026 2025/2026

```
# Create a dictionary to store customer information
customer = {
    "name": "John Smith",
    "age": 30,
    "is_verified": True
}

# Access existing keys
print("Customer name:", customer["name"])       # John Smith
print("Customer age:", customer["age"])         # 30
print("Is verified:", customer["is_verified"])  # True
print("----------------------------------------")
```

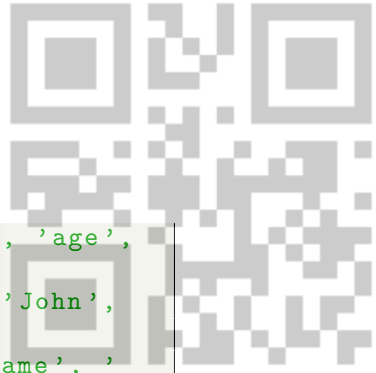
Example 2: Dictionary methods.

```
customer = {
    "name": "John",
    "age": 25,
    "type": "silver"
}

# keys, values, and items
```

رومانی صبحی رزق جرجس مطر





رومانی صبحی رزق جرجس مطر

```
print("Keys:", customer.keys())  # dict_keys(['name', 'age', 'type'])
print("Values:", customer.values())  # dict_values(['John', 25, 'silver'])
print("Items:", customer.items())  # dict_items([('name', 'John'), ('age', 25), ('type', 'silver')])
print("----------------------------------------")

# get and update
print("Membership type:", customer.get("type", "standard"))  # silver
customer.update({"type": "gold", "email": "john@example.com"})
print("Updated customer:", customer)  # {'name': 'John', 'age': 25, 'type': 'gold', 'email': 'john@example.com'}
print("----------------------------------------")

# pop and popitem
customer.pop("age")
print("After pop('age'):", customer)  # {'name': 'John', 'type': 'gold', 'email': 'john@example.com'}
print("----------------------------------------")

last_item = customer.popitem()
print("Last item removed:", last_item)  # ('email', 'john@example.com')
print("----------------------------------------")

# copy and clear
copy_customer = customer.copy()  # Copy dictionary
print("Copied dictionary:", copy_customer)  # {'name': 'John', 'type': 'gold'}
print("----------------------------------------")

customer.clear()  # CLEAR DICTIONARY
print("Cleared dictionary:", customer)  # {}
print("----------------------------------------")
```

3.5 Sets

Sets are collections of unique elements, with no duplicates allowed. They are unordered, so no indexing or slicing. Elements must be immutable. Sets are highly efficient for membership testing (in operator) and mathematical operations like union, intersection, difference, symmetric difference.

Common methods: - Add with `add()` - Remove with `remove()` or `discard()` - Set operations: `union()`, `intersection()`, etc.

رومانی صبحی رزق جرجس مطر

Example:

```
def set_demo():
    my_set = {1, 2, 3, 2}  # Duplicates removed: {1,2,3}
    my_set.add(4)
    print(my_set)  # {1,2,3,4}
    print(2 in my_set)  # True
    other_set = {3, 4, 5}
    print(my_set.union(other_set))  # {1,2,3,4,5}

set_demo()
```

Connect: Check even/odd in a set using is_even_odd().

```
from my_module import is_even_odd

num_set = {1, 2, 3, 4}
for num in num_set:
    print(str(num) + " is " + is_even_odd(num))
```

3.6 Arrays and Other Data Structures

Arrays in Python are provided by the NumPy library, not built-in. They are designed for efficient storage and manipulation of numerical data, especially in scientific computing. Arrays are homogeneous (all elements same type), support multi-dimensional shapes (e.g., matrices), and enable vectorized operations without loops for speed.

Common operations: Arithmetic on entire arrays, slicing, reshaping with `reshape()`, aggregation like `sum()`, `mean()`.

Example:

```
import numpy as np

def array_demo():
    arr = np.array([1, 2, 3])
    print(arr * 2)  # [2 4 6]
    matrix = np.array([[1, 2], [3, 4]])
    print(matrix.shape)  # (2, 2)
    print(np.mean(arr))  # 2.0

array_demo()
```

Other structures: Strings (immutable sequences), Queues/Deques (from collections module for FIFO/LIFO).

رومانی صبحی رزق جرجس مطر

4 Example Programs on Data Structures

Here are 10 programs, each using functions from Lecture 4 to connect concepts.

4.1 Program 1: List Sum Using add()

```
from my_module import add

def sum_list(lst):
    return add(lst[0], lst[1], lst[2])  # For 3 elements

print(sum_list([1, 2, 3]))  # 6
```

4.2 Program 2: Tuple Area Calculation

Assuming area from Lecture 4 rectangle example.

```
from my_module import area

dimensions = (5, 3)
print(area(*dimensions))  # 15.0
```


4.3 Program 3: Dictionary Even Check

```
from my_module import is_even_odd

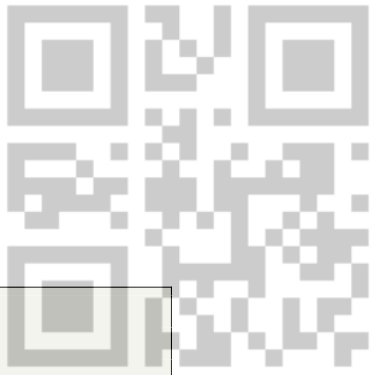
ages = {"Alice": 20, "Bob": 21}
for name, age in ages.items():
    print(name + "'s age is " + is_even_odd(age))
```

4.4 Program 4: Set Grade Computation

Assuming compute$_g$*radetakesscore*.

```
from my_module import compute_grade

scores_set = {85, 92, 65}
for score in scores_set:
    print("Grade for " + str(score) + ": " + compute_grade(
    score))
```



4.5 Program 5: Numpy Array Sum Even

```
import numpy as np
from my_module import sum_even_numbers

arr = np.array([1, 2, 3, 4])
print(sum_even_numbers(arr.tolist()))  # 6.0
```

4.6 Program 6: List Rectangle Areas

```
from my_module import area

rects = [(5,3), (4,2), (6,1)]
for l, w in rects:
    print(area(l, w))
```

4.7 Program 7: Dictionary Kaggle Print with print$_sum()$

```
from my_module import print_sum

metadata = {"rows": 891, "columns": 12, "features": 10}
print_sum(metadata["rows"], metadata["columns"], metadata["
    features"])
```

4.8 Program 8: Tuple Even Sum

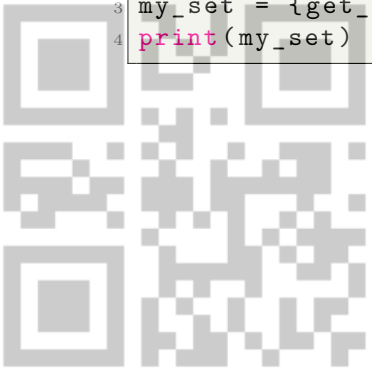
```
from my_module import sum_even_numbers

nums = (1, 2, 3, 4, 5, 6)
print(sum_even_numbers(list(nums)))  # 12.0
```

4.9 Program 9: Set Addition Using get$_sum()$

```
from my_module import get_sum

my_set = {get_sum(), 7, 8}  # get_sum() returns 6
print(my_set)  # {6,7,8}
```



رومانی صبحی رزق جرجس مطر

4.10 Program 10: Array Grade Average

```python
import numpy as np
from my_module import compute_grade

scores_arr = np.array([85, 92, 65])
avg = np.mean(scores_arr)
print("Average grade: " + compute_grade(avg))
```

5 Assignments

5.1 Assignment 1

Create a module with a function that takes a list of numbers and returns a dictionary with even/odd counts. Use $\texttt{is}_{e}ven_{o}dd()fromLecture4$.

5.2 Assignment 2

Write a program using a tuple to store rectangle dimensions, compute areas with area() from Lecture 4, and store results in a set.

5.3 Assignment 3

Use a dictionary for Kaggle dataset metadata (add at least 3 datasets), import numpy, and compute average rows using array operations.

5.4 Assignment 4

Build a function that takes a set of scores, uses $\texttt{compute}_{g}rade()toassigngrades, and$

رومانى صبحى رزق جرجس مطر

رومانی صبحی رزق جرجس مطر

Data Structure	Characteristics	How to Use It
List	Ordered, mutable (can change elements), allows duplicates, can hold different data types (heterogeneous).	Create: `my_list = [1, 'a', 2.5]` Access: `print(my_list[0])` Modify: `my_list.append(3)` Other methods: extend, insert, remove, pop, sort.
Tuple	Ordered, immutable (cannot change elements after creation), allows duplicates, heterogeneous. Faster and more memory-efficient than lists for fixed data.	Create: `my_tuple = (1, 'a', 2.5)` Access: `print(my_tuple[1])` No modification methods; used for unpacking or as keys in dicts.
Dictionary	Unordered (insertion order preserved since Python 3.7), mutable, stores key-value pairs, keys must be unique and immutable, values can be any type. Fast lookups.	Create: `my_dict = {'key': 'value'}` Access: `print(my_dict['key'])` Modify: `my_dict['new'] = 10` Methods: get, keys, values, items, update.
Set	Unordered, mutable, stores unique elements only (no duplicates), elements must be immutable. Fast membership testing and set operations.	Create: `my_set = {1, 2, 3}` Add: `my_set.add(4)` Operations: union (`|`), intersection (`&`), difference (`-`).
Array (NumPy)	Ordered, mutable, homogeneous (all elements same type, usually numbers), supports multidimensional, efficient for numerical computations and vectorized operations.	Import: `import numpy as np` Create: `arr = np.array([1, 2, 3])` Operations: `print(arr + 1)` (broadcasting), slicing, reshaping, mathematical functions.

Table 1: Comparison of Python Data Structures

رومانی صبحی رزق جرجس مطر

Python Lecture 5: Data Structures

September 18, 2025

1 Review of Functions from Lecture 4

To build on the concepts from Lecture 4, we start with a quick refresh of built-in functions, importing packages, and discovering library functions. This connects to our new topic by showing how functions can manipulate data structures.

1.1 Refreshing Built-in Functions

Built-in functions like `print()`, `len()`, and `input()` are always available. Here's a simple program to refresh their use, incorporating user input and output as seen in Lecture 4 examples.

2025/2026 2025/2026 2025/2026

```
def refresh_builtins():
    name = input("Enter your name: ")
    print("Hello, " + name + "! Your name has " + str(len(
    name)) + " characters.")

refresh_builtins()
```

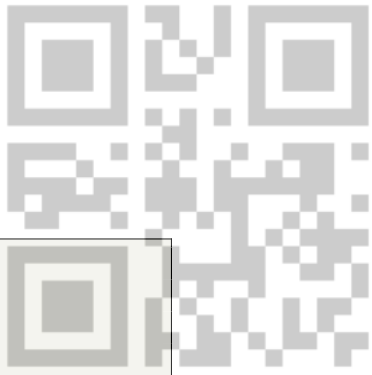
Example Input: Alice
Output: Hello, Alice! Your name has 5 characters.

This function takes no parameters but uses built-ins to interact with the user, similar to the no-parameters examples in Lecture 4.

1.2 How to Import Packages and Use Existing Functions

As discussed in Lecture 4, import packages using 'import library_name' or 'from library import function'. Examples include math, pandas, numpy, sklearn, and keras. After importing, call functions with library_name.function(arguments).

Example: Using math library (from Lecture 4).



```python
import math

def use_math_functions():
    print(math.sqrt(25))  # Returns 5.0
    print(math.pow(2, 3))  # Returns 8.0

use_math_functions()
```

Other examples:
- import pandas as pd; df = pd.DataFrame([[1, 2], [3, 4]])
- import numpy as np; arr = np.array([1, 2, 3])
- from sklearn.linear$_{m}$odelimport$LinearRegression$; $model = LinearRegression()$
- from keras.models import Sequential; nn$_{m}$odel = $Sequential()$

These build on Lecture 4 by reusing the import syntax in functions.

1.3 How to Know the Existing Functions of a Library

To discover functions in a library, use built-in tools like dir(library_name) to list attributes and methods, or help(library_name) for documentation. This is useful after importing.

Example: Discovering math functions.

```python
import math

def discover_math():
    print(dir(math))  # Lists all functions like ['acos', 'asin', 'sqrt', ...]
    help(math.sqrt)   # Shows documentation for sqrt

discover_math()
```

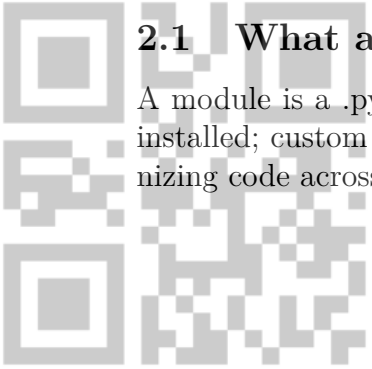
Use this to explore libraries mentioned in Lecture 4, like pandas or numpy.

2 Modules in Python

Modules are files containing Python code (functions, variables, classes) that can be imported and reused, promoting modularity as introduced in Lecture 4.

2.1 What are Modules

A module is a .py file with definitions. Built-in modules like math are pre-installed; custom modules are user-created. They extend functions by organizing code across files.



2025 رومانی صبحی رزق جرجس مطر

رومانی صبحی رزق جرجس مطر

2.2 How to Create a Module

Save functions in a file like my_module.py. Then import it in another script.

Example: Create my_module.py with functions from Lecture 4.

Content of my_module.py:

```python
def add(a, b, c):
    return a + b + c

def is_even_odd(num):
    if num % 2 == 0:
        return "Even"
    else:
        return "Odd"
```

To use it:

```python
import my_module

sum_result = my_module.add(1, 2, 3)  # Returns 6
print(sum_result)

even_check = my_module.is_even_odd(4)  # Returns "Even"
print(even_check)
```

This connects to Lecture 4 by reusing user-defined functions in a module.

3 Data Structures in Python

Data structures store and organize data efficiently. Python's built-in ones include lists, tuples, dictionaries, and sets. Arrays are available via numpy. We'll explain each with simple programs, building on Lecture 4 functions.

3.1 Comparison of Data Structures

Before diving into each data structure, here is a comparison table summarizing their key characteristics and usage.

3.2 Lists

Lists are one of the most versatile data structures in Python. They are ordered collections, meaning elements have a specific position and can be accessed by index (starting from 0). Lists are mutable, allowing addition, removal, or modification of elements. They can contain duplicates and elements of different types (integers, strings, floats, etc.). Lists support slicing

(e.g., my_list[1:3]), iteration with loops, and comprehension for creating new lists.

Common operations include: - Appending elements with `append()` - Inserting at a position with `insert()` - Removing with `remove()` or `pop()` - Sorting with `sort()` or `sorted()` - Reversing with `reverse()`

Example: Simple list operations.

```
def list_demo():
    my_list = [1, 2, 3, "apple", 4.5]
    print(my_list[0])  # Access first element: 1
    my_list.append(6)  # Add element
    print(len(my_list))  # Length: 6
    print(my_list[1:4])  # Slicing: [2, 3, 'apple']

list_demo()
```

Connect to Lecture 4: Modify sum_even_numbers() to take a list parameter and use it.

Modified sum_even_numbers in my_module.py:

```
def sum_even_numbers(numbers):
    total = 0
    for num in numbers:
        num = float(num)
        if num % 2 == 0:
            total += num
    return total
```

Usage:

```
from my_module import sum_even_numbers

numbers = [1, 2, 3, 4, 5, 6]
even_sum = sum_even_numbers(numbers)
print(even_sum)  # 12.0
```

3.2.1 Additional List Examples

Example 1: Iterating over a list with mixed types.

```
fruits = ["apple", "banana", "cherry", "dfsdfsfsd", "mrmrmr",
    2323, 232355.3434]
for fruit in fruits:
    print(fruit)
    print("Hi in For")

print("Hi out For")
```

2025 رومانی صبحی رزق جرجس مطر

رومانی صبحی رزق جرجس مطر

Example 2: Creating a list using range.

```
y = list(range(10, 0, -2))  # countdown by 2
print(y)  # [10, 8, 6, 4, 2]
```

Example 3: Append, insert, remove operations.

```
numbers = [1, 2, 3, 4, 5]
print("Initial list:", numbers)  # Initial list: [1, 2, 3, 4,
    5]

numbers.append(10)
print(numbers)  # [1, 2, 3, 4, 5, 10]

numbers.insert(0, 100)
print(numbers)  # [100, 1, 2, 3, 4, 5, 10]

numbers.insert(3, 200)
print(numbers)  # [100, 1, 2, 200, 3, 4, 5, 10]

numbers.remove(200)
numbers.remove(100)
print(numbers)  # [1, 2, 3, 4, 5, 10]
```

Example 4: Pop operations.

```
list2 = [1, 2, 3, 5, 6, 9, 4, 70]
print(list2)  # [1, 2, 3, 5, 6, 9, 4, 70]

list2.pop()
list2.pop()
list2.pop()
print(list2)  # [1, 2, 3, 5, 6, 9]
```

Example 5: Sorting lists.

```
list3 = [100, 90, 10, 45, 400, 700, 12]
list4 = [100, 90, 10, 45, 400, 700, 12]

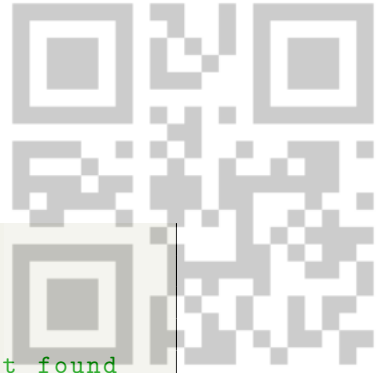
print(list3)  # [100, 90, 10, 45, 400, 700, 12]

list3.sort()  # Asc Order
print(list3)  # [10, 12, 45, 90, 100, 400, 700]

list4.sort(reverse=True)  # Desc order
print(list4)  # [700, 400, 100, 90, 45, 12, 10]
```

Example 6: Index and modifying elements.

```
my_list = [1, 2, 3, 4]
print(my_list)  # [1, 2, 3, 4]
```



```
try:
    index = my_list.index(5)  # 5 is not in the list
    print(index)
except ValueError:
    print("Value not found in the list")  # Value not found
    in the list

my_list[0] = 10
print(my_list)  # [10, 2, 3, 4]

my_list[2] = 30
print(my_list)  # [10, 2, 30, 4]
```

Example 7: Comprehensive list methods.

```
numbers = [1, 2, 3, 4, 5]

# Accessing elements
print("First item (numbers[0]):", numbers[0])  # 1
print("Second item (numbers[1]):", numbers[1])  # 2
print("Last item (numbers[-1]):", numbers[-1])  # 5
print("Second last item (numbers[-2]):", numbers[-2])  # 4
print("-------------------------------------------------")

# Append - adds 6 to the end
numbers.append(6)
print("After append(6):", numbers)  # [1, 2, 3, 4, 5, 6]
print("-------------------------------------------------")

# Insert - adds 6 at index 0
numbers.insert(0, 6)
print("After insert(0, 6):", numbers)  # [6, 1, 2, 3, 4, 5,
    6]
print("-------------------------------------------------")

# Remove - removes the first occurrence of 6
numbers.remove(6)
print("After remove(6):", numbers)  # [1, 2, 3, 4, 5, 6]
print("-------------------------------------------------")

# Pop - removes the last item
popped = numbers.pop()
print("After pop():", numbers)  # [1, 2, 3, 4, 5]
print("Popped value:", popped)  # 6
print("-------------------------------------------------")

# Copy - creates a copy of the list
copied_list = numbers.copy()
print("Copied list:", copied_list)  # [1, 2, 3, 4, 5]
print("-------------------------------------------------")
```

رومانی صبحی رزق جرجس مطر



رومانی صبحی رزق جرجس مطر

```
35
36 # Sort - sorts the list in ascending order
37 numbers.sort()
38 print("After sort():", numbers)  # [1, 2, 3, 4, 5]
39 print("-----------------------------------------------")
40
41 # Reverse - reverses the list
42 numbers.reverse()
43 print("After reverse():", numbers)  # [5, 4, 3, 2, 1]
44 print("-----------------------------------------------")
45
46 # Index - finds the position of the first occurrence of 3
47 if 3 in numbers:
48     print("Index of 3:", numbers.index(3))  # 2
49 else:
50     print("3 is not in the list.")
51 print("-----------------------------------------------")
52
53 # Clear - removes all items
54 numbers.clear()
55 print("After clear():", numbers)  # []
56 print("-----------------------------------------------")
```

3.3 Tuples

Tuples are similar to lists but immutable, meaning once created, their elements cannot be changed, added, or removed. This makes them suitable for fixed collections of items, like coordinates or constants. Tuples are faster and use less memory than lists. They support indexing, slicing, and unpacking (e.g., a, b = my_tuple).

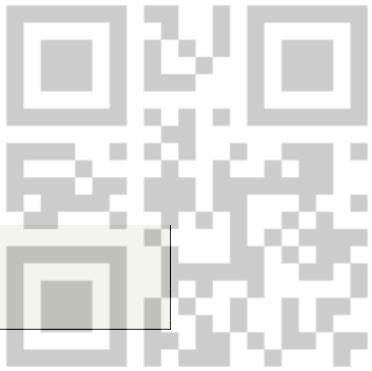
Common uses: As keys in dictionaries (since immutable), returning multiple values from functions.

Example:

```
1 def tuple_demo():
2     my_tuple = (1, 2, "banana", 3.5)
3     print(my_tuple[1])  # 2
4     # my_tuple[0] = 0  # Error: immutable
5     a, b, _, _ = my_tuple  # Unpacking
6     print(a + b)  # 3
7
8 tuple_demo()
```

Connect: Use add() from Lecture 4 on tuple elements.

```
1 from my_module import add
2
```



```
coords = (1, 2, 3)
total = add(*coords)  # Unpack tuple: 6
print(total)
```

3.3.1 Additional Tuple Examples

Example 1: Accessing and unpacking tuples.

```
x = (10, 14, 19)
print(x)  # (10, 14, 19)
print(x[0])  # 10
print(x[1])  # 14
print(x[2])  # 19

item1, item2, item3 = x
print(item1)  # 10
print(item2)  # 14
print(item3)  # 19

item1 = 100
print(item1)  # 100
```

Example 2: Converting tuple to list.

```
# Convert Tuple to list
my_tuple = (1, 2, 3, 4)
my_list = list(my_tuple)
print(my_list)  # [1, 2, 3, 4]

my_list[0] = 1000
print(my_list)  # [1000, 2, 3, 4]
```

Example 3: Comprehensive tuple operations.

```
# Creating a tuple
coordinates = (1, 2, 3)
print("Tuple:", coordinates)  # Tuple: (1, 2, 3)

# Accessing elements
print("First element:", coordinates[0])  # 1
print("Second element:", coordinates[1])  # 2
print("Third element:", coordinates[2])  # 3
print("----------------------------------------")

# Trying to change a value (this will raise an error)
# coordinates[0] = 10  #  TypeError: 'tuple' object does not
    support item assignment

# Length of the tuple
```

2025 رومانی صبحی رزق جرجس مطر



رومانی صبحی رزق جرجس مطر

```
print("Length of tuple:", len(coordinates))  # 3
print("-----------------------------------------")

# Unpacking the tuple into separate variables
x, y, z = coordinates
print("Unpacked values:")
print("x =", x)  # 1
print("y =", y)  # 2
print("z =", z)  # 3
print("-----------------------------------------------------")

# Tuple with one item (note the comma)
single_item_tuple = (5,)
print("Single-item tuple:", single_item_tuple)  # (5,)
print("-----------------------------------------------------")

# Using tuple in a function return: the function return tuple (list of values to be returned)
def get_student_info():
    return ("Alice", 20, "Computer Science")

name, age, major = get_student_info()
print("\nStudent Info:")
print("Name:", name)  # Alice
print("Age:", age)  # 20
print("Major:", major)  # Computer Science
print("-----------------------------------------")
```

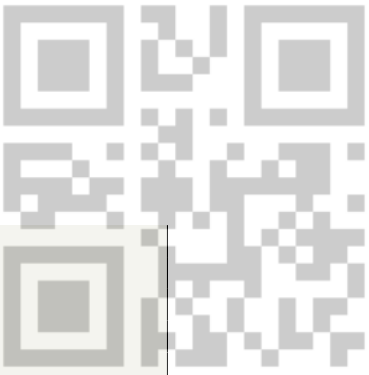
3.4 Dictionaries

Dictionaries store data in key-value pairs, where keys are unique and immutable (e.g., strings, numbers, tuples), and values can be any type, including other data structures. Since Python 3.7, dictionaries preserve insertion order. They provide fast lookups, insertions, and deletions based on keys. Not indexed by numbers, but by keys.

Common methods: - Access with `dict[key]` or `get(key, default)` - Add/update with `dict[key] = value` or `update()` - Iterate with `keys()`, `values()`, `items()`

Example: Use dictionaries to store Kaggle dataset metadata and print in the CLI.

```
def kaggle_metadata():
    dataset = {
        "name": "Titanic",
        "size": "1MB",
```



```
        "rows": 891,
        "columns": 12,
        "description": "Passenger data for survival
    prediction"
    }
    print("Dataset Name: " + dataset["name"])
    print("Size: " + dataset.get("size", "Unknown"))
    print("Rows: " + str(dataset["rows"]))
    for key, value in dataset.items():
        print(key + ": " + str(value))

kaggle_metadata()
```

Connect: Compute grade average from dict of scores using compute_grade(). Assuming compute_grade takes a score parameter (from Lecture 4).

```
from my_module import compute_grade

scores = {"Alice": 85, "Bob": 92}
for name, score in scores.items():
    print(name + ": " + compute_grade(score))
```

3.4.1 Additional Dictionary Examples

Example 1: Accessing dictionary values.

2025/2026 2025/2026 2025/2026

```
# Create a dictionary to store customer information
customer = {
    "name": "John Smith",
    "age": 30,
    "is_verified": True
}

# Access existing keys
print("Customer name:", customer["name"])       # John Smith
print("Customer age:", customer["age"])         # 30
print("Is verified:", customer["is_verified"])  # True
print("----------------------------------------")
```

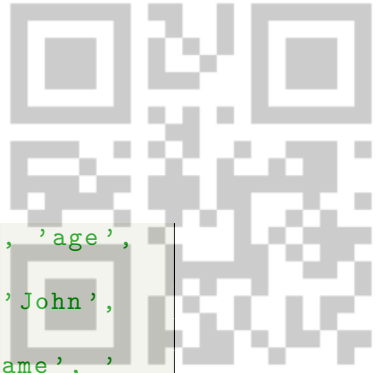
Example 2: Dictionary methods.

```
customer = {
    "name": "John",
    "age": 25,
    "type": "silver"
}

# keys, values, and items
```

رومانی صبحی رزق جرجس مطر





رومانی صبحی رزق جرجس مطر

```
print("Keys:", customer.keys())  # dict_keys(['name', 'age', 'type'])
print("Values:", customer.values())  # dict_values(['John', 25, 'silver'])
print("Items:", customer.items())  # dict_items([('name', 'John'), ('age', 25), ('type', 'silver')])
print("----------------------------------------")

# get and update
print("Membership type:", customer.get("type", "standard"))  # silver
customer.update({"type": "gold", "email": "john@example.com"})
print("Updated customer:", customer)  # {'name': 'John', 'age': 25, 'type': 'gold', 'email': 'john@example.com'}
print("----------------------------------------")

# pop and popitem
customer.pop("age")
print("After pop('age'):", customer)  # {'name': 'John', 'type': 'gold', 'email': 'john@example.com'}
print("----------------------------------------")

last_item = customer.popitem()
print("Last item removed:", last_item)  # ('email', 'john@example.com')
print("----------------------------------------")

# copy and clear
copy_customer = customer.copy()  # Copy dictionary
print("Copied dictionary:", copy_customer)  # {'name': 'John', 'type': 'gold'}
print("----------------------------------------")

customer.clear()  # CLEAR DICTIONARY
print("Cleared dictionary:", customer)  # {}
print("----------------------------------------")
```

3.5 Sets

Sets are collections of unique elements, with no duplicates allowed. They are unordered, so no indexing or slicing. Elements must be immutable. Sets are highly efficient for membership testing (in operator) and mathematical operations like union, intersection, difference, symmetric difference.

Common methods: - Add with `add()` - Remove with `remove()` or `discard()` - Set operations: `union()`, `intersection()`, etc.

رومانی صبحی رزق جرجس مطر

Example:

```
def set_demo():
    my_set = {1, 2, 3, 2}  # Duplicates removed: {1,2,3}
    my_set.add(4)
    print(my_set)  # {1,2,3,4}
    print(2 in my_set)  # True
    other_set = {3, 4, 5}
    print(my_set.union(other_set))  # {1,2,3,4,5}

set_demo()
```

Connect: Check even/odd in a set using is_even_odd().

```
from my_module import is_even_odd

num_set = {1, 2, 3, 4}
for num in num_set:
    print(str(num) + " is " + is_even_odd(num))
```

3.6 Arrays and Other Data Structures

Arrays in Python are provided by the NumPy library, not built-in. They are designed for efficient storage and manipulation of numerical data, especially in scientific computing. Arrays are homogeneous (all elements same type), support multi-dimensional shapes (e.g., matrices), and enable vectorized operations without loops for speed.

Common operations: Arithmetic on entire arrays, slicing, reshaping with `reshape()`, aggregation like `sum()`, `mean()`.

Example:

```
import numpy as np

def array_demo():
    arr = np.array([1, 2, 3])
    print(arr * 2)  # [2 4 6]
    matrix = np.array([[1, 2], [3, 4]])
    print(matrix.shape)  # (2, 2)
    print(np.mean(arr))  # 2.0

array_demo()
```

Other structures: Strings (immutable sequences), Queues/Deques (from collections module for FIFO/LIFO).

رومانی صبحی رزق جرجس مطر

4 Example Programs on Data Structures

Here are 10 programs, each using functions from Lecture 4 to connect concepts.

4.1 Program 1: List Sum Using add()

```
from my_module import add

def sum_list(lst):
    return add(lst[0], lst[1], lst[2])  # For 3 elements

print(sum_list([1, 2, 3]))  # 6
```

4.2 Program 2: Tuple Area Calculation

Assuming area from Lecture 4 rectangle example.

```
from my_module import area

dimensions = (5, 3)
print(area(*dimensions))  # 15.0
```


4.3 Program 3: Dictionary Even Check

```
from my_module import is_even_odd

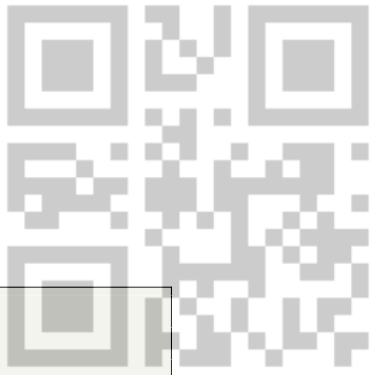
ages = {"Alice": 20, "Bob": 21}
for name, age in ages.items():
    print(name + "'s age is " + is_even_odd(age))
```

4.4 Program 4: Set Grade Computation

Assuming compute$_g$*radetakesscore*.

```
from my_module import compute_grade

scores_set = {85, 92, 65}
for score in scores_set:
    print("Grade for " + str(score) + ": " + compute_grade(
    score))
```



4.5 Program 5: Numpy Array Sum Even

```
import numpy as np
from my_module import sum_even_numbers

arr = np.array([1, 2, 3, 4])
print(sum_even_numbers(arr.tolist()))  # 6.0
```

4.6 Program 6: List Rectangle Areas

```
from my_module import area

rects = [(5,3), (4,2), (6,1)]
for l, w in rects:
    print(area(l, w))
```

4.7 Program 7: Dictionary Kaggle Print with print$_sum()$

```
from my_module import print_sum

metadata = {"rows": 891, "columns": 12, "features": 10}
print_sum(metadata["rows"], metadata["columns"], metadata["
    features"])
```

4.8 Program 8: Tuple Even Sum

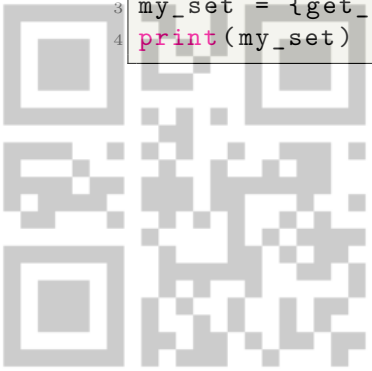
```
from my_module import sum_even_numbers

nums = (1, 2, 3, 4, 5, 6)
print(sum_even_numbers(list(nums)))  # 12.0
```

4.9 Program 9: Set Addition Using get$_sum()$

```
from my_module import get_sum

my_set = {get_sum(), 7, 8}  # get_sum() returns 6
print(my_set)  # {6,7,8}
```



رومانی صبحی رزق جرجس مطر

4.10 Program 10: Array Grade Average

```python
import numpy as np
from my_module import compute_grade

scores_arr = np.array([85, 92, 65])
avg = np.mean(scores_arr)
print("Average grade: " + compute_grade(avg))
```

5 Assignments

5.1 Assignment 1

Create a module with a function that takes a list of numbers and returns a dictionary with even/odd counts. Use $\texttt{is}_{e}ven_{o}dd()fromLecture4$.

5.2 Assignment 2

Write a program using a tuple to store rectangle dimensions, compute areas with area() from Lecture 4, and store results in a set.

5.3 Assignment 3

Use a dictionary for Kaggle dataset metadata (add at least 3 datasets), import numpy, and compute average rows using array operations.

5.4 Assignment 4

Build a function that takes a set of scores, uses $\texttt{compute}_{g}rade()toassigngrades, and$

رومانى صبحى رزق جرجس مطر

رومانی صبحی رزق جرجس مطر

Data Structure	Characteristics	How to Use It
List	Ordered, mutable (can change elements), allows duplicates, can hold different data types (heterogeneous).	Create: `my_list = [1, 'a', 2.5]` Access: `print(my_list[0])` Modify: `my_list.append(3)` Other methods: extend, insert, remove, pop, sort.
Tuple	Ordered, immutable (cannot change elements after creation), allows duplicates, heterogeneous. Faster and more memory-efficient than lists for fixed data.	Create: `my_tuple = (1, 'a', 2.5)` Access: `print(my_tuple[1])` No modification methods; used for unpacking or as keys in dicts.
Dictionary	Unordered (insertion order preserved since Python 3.7), mutable, stores key-value pairs, keys must be unique and immutable, values can be any type. Fast lookups.	Create: `my_dict = {'key': 'value'}` Access: `print(my_dict['key'])` Modify: `my_dict['new'] = 10` Methods: get, keys, values, items, update.
Set	Unordered, mutable, stores unique elements only (no duplicates), elements must be immutable. Fast membership testing and set operations.	Create: `my_set = {1, 2, 3}` Add: `my_set.add(4)` Operations: union (`|`), intersection (`&`), difference (`-`).
Array (NumPy)	Ordered, mutable, homogeneous (all elements same type, usually numbers), supports multidimensional, efficient for numerical computations and vectorized operations.	Import: `import numpy as np` Create: `arr = np.array([1, 2, 3])` Operations: `print(arr + 1)` (broadcasting), slicing, reshaping, mathematical functions.

Table 1: Comparison of Python Data Structures

Python Lecture 6: File Handling and Exception Handling

September 24, 2025

رومانى صبحى رزق جرجس مطر

1 Revision of Lecture 5: Functions Using Data Structures

To revise the data structures from Lecture 5, we will create five functions, each utilizing a different data structure: list, dictionary, tuple, array (from NumPy), and set. These functions will perform simple operations. Finally, a main program will call all five functions.

1.1 Function Using List: Sum of Elements

This function takes a list of numbers and returns their sum.

```
def sum_list(numbers):
    total = 0
    for num in numbers:
        total += num
    return total
```

Example: $\text{sum}_list([1,2,3])returns6$.

1.2 Function Using Dictionary: Get Value by Key

This function takes a dictionary and a key, returning the corresponding value or "Not Found".

```
def get_dict_value(my_dict, key):
    return my_dict.get(key, "Not Found")
```

Example: $\text{get}_dict_value(”name” : ”Alice”, ”age” : 25, ”age”)returns25$.

1.3 Function Using Tuple: Unpack and Add

This function takes a tuple of three numbers, unpacks them, and returns their sum.

```
def sum_tuple(tup):
    a, b, c = tup
    return a + b + c
```

Example: $\text{sum}_tuple((4,5,6))returns15$.

1.4 Function Using Array (NumPy): Compute Mean

This function uses NumPy to compute the mean of an array.

```
import numpy as np

def compute_mean_array(arr):
    return np.mean(arr)
```

Example: $\text{compute}_{m}ean_{a}rray(np.array([10, 20, 30]))returns20.0$.

1.5 Additional NumPy Examples

This subsection provides a detailed explanation and examples of using NumPy arrays, including 1D and 2D arrays, stacking operations, and mathematical functions. These examples demonstrate NumPy's power for numerical computations.

Vertical stacking (np.vstack) combines arrays along the vertical axis (axis 0), effectively adding rows from one array below another. It requires arrays to have the same number of columns. Horizontal stacking (np.hstack) combines arrays along the horizontal axis (axis 1), adding columns from one array to the right of another. It requires arrays to have the same number of rows.

```
# Array (1D or 2D)
import numpy as np

arr = np.array([1, 2, 4, 10])
print(arr)
print(arr.shape)
print("Sum: ", arr.sum())
print("Mean = Average = ", arr.mean())

arr2D = np.array([[1, 2, 10], [2, 5, 7], [4, 6, 12], [5, 8, 9]])
print(arr2D)
print(arr2D.shape)
print("Sum: ", arr2D.sum())
print("Mean = Average = ", arr2D.mean())

arr1 = np.array([[1, 2, 3], [4, 5, 6]])
print("Array:\n", arr1)

arr2 = np.array([[10, 20, 30], [40, 50, 60]])
print("\nZeros:\n", arr2)
print(arr2.shape)

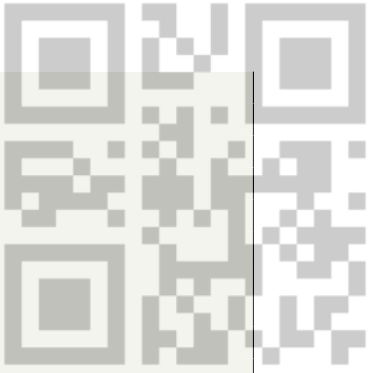
arr3 = np.array([[10, 20, 30, 40], [40, 50, 60, 40]])
print("\nZeros:\n", arr3)
print(arr3.shape)

stack_v = np.vstack((arr1, arr2))
print("\nVertical Stack:\n", stack_v)
print(stack_v.shape)

stack_v2 = np.vstack((arr1, arr3))
print("\nVertical Stack:\n", stack_v2)
print(stack_v2.shape)

stack_h = np.hstack((arr1, arr2))
```

رومانی صبحی رزق جرجس مطر



```
print("\nHorizontal Stack:\n", stack_h)
print(stack_h.shape)

stack_h2 = np.hstack((arr1, arr3))
print("\nHorizontal Stack:\n", stack_h2)
print(stack_h2.shape)

# Mathematical Operations
arr1 = np.array([[1, 2, 3], [4, 5, 6]])
print("Array:\n", arr1)

arr2 = np.array([[10, 20, 30], [40, 50, 60]])
print("\nZeros:\n", arr2)
print(arr2.shape)

print("\nAdd:\n", np.add(arr1, arr2))
print("Subtract:\n", np.subtract(arr1, arr2))
print("Multiply:\n", np.multiply(arr1, arr2))
print("Divide:\n", np.divide(arr1, arr2))
print("Dot product:\n", np.dot(arr1, arr2.T))

arr4 = np.array([1, 2, 3, 10, 4, 5, 6])
print("Array:\n", arr4)
print("\nSum:", np.sum(arr4))
print("Mean:", np.mean(arr4))
print("Median:", np.median(arr4))
print("Variance:", np.var(arr4))
print("Std Dev:", np.std(arr4))
print("Min:", np.min(arr4))
print("Max:", np.max(arr4))
print("Argmin (index of min):", np.argmin(arr4))
print("Argmax (index of max):", np.argmax(arr4))
```

Expected Output (partial, varies by environment): “‘ [1 2 4 10] (4,) Sum: 17 Mean = Average = 4.25 [[1 2 10] [2 5 7] [4 6 12] [5 8 9]] (4, 3) Sum: 71 Mean = Average = 5.916666666666667 ... “‘

The following table summarizes common NumPy array functions, their meanings, simple examples, and results. It provides a quick reference for array operations.

1.6 Function Using Set: Check Membership

This function takes a set and a value, checking if the value is in the set.

```
def check_in_set(my_set, value):
    if value in my_set:
        return "Present"
    else:
        return "Absent"
```

Example: $\text{check}_in_set(1, 2, 3, 2) returns “Present”$.

1.7 Main Program Calling All Functions

The main program demonstrates calling all five functions.

```
import numpy as np

```

رومانى صبحى رزق جرجس مطر

```
def main():
    # List example
    my_list = [1, 2, 3]
    print("Sum of list:", sum_list(my_list))

    # Dictionary example
    my_dict = {"name": "Alice", "age": 25}
    print("Value for 'name':", get_dict_value(my_dict, "name"))

    # Tuple example
    my_tuple = (4, 5, 6)
    print("Sum of tuple:", sum_tuple(my_tuple))

    # Array example
    my_array = np.array([10, 20, 30])
    print("Mean of array:", compute_mean_array(my_array))

    # Set example
    my_set = {1, 2, 3}
    print("Check 3 in set:", check_in_set(my_set, 3))

main()
```

Output: Sum of list: 6 Value for 'name': Alice Sum of tuple: 15 Mean of array: 20.0 Check 3 in set: Present

2 File Handling in Python

File handling allows Python programs to read from and write to files. Python provides built-in functions to create, read, update, and delete files.

2.1 Opening a File

Use the `open()` function to open a file. It takes the file path and mode as arguments. The following table summarizes all available modes for opening text and binary files, providing a comprehensive reference for file handling.

Modes: - 'r': Read (default) - 'w': Write (creates file if not exists, truncates if exists) - 'a': Append (creates if not exists) - 'b': Binary mode (e.g., 'rb' for reading binary) - 'x': Exclusive creation (fails if file exists) - '+': Read and write (e.g., 'r+')

Example: Opening a file for reading.

```
file = open("example.txt", "r")
print(file.read())
file.close()
```

2.2 Closing a File

Always close files using `close()` to free resources. Better to use `with` statement for automatic closing.

Example with `with`:

```
with open("example.txt", "r") as file:
    content = file.read()
```

```
    print(content)
# File is automatically closed here
```

2.3 Reading from a File

- `read()`: Reads the entire file. - `readline()`: Reads one line. - `readlines()`: Reads all lines into a list.

Example: Reading entire file.

```
with open("example.txt", "r") as file:
    content = file.read()
    print(content)
```

Example: Reading line by line.

```
with open("example.txt", "r") as file:
    line = file.readline()
    while line:
        print(line.strip())
        line = file.readline()
```

Example: Reading all lines.

```
with open("example.txt", "r") as file:
    lines = file.readlines()
    for line in lines:
        print(line.strip())
```

2.4 Writing to a File

- `write()`: Writes a string. - `writelines()`: Writes a list of strings.

Example: Writing to a file (overwrites).

```
with open("output.txt", "w") as file:
    file.write("Hello, World!\n")
    file.write("This is a test.")
```

Example: Appending to a file.

```
with open("output.txt", "a") as file:
    file.write("\nAppended line.")
```

Example: Writelines.

```
lines = ["Line 1\n", "Line 2\n", "Line 3\n"]
with open("output.txt", "w") as file:
    file.writelines(lines)
```

2.5 Binary Files

For images, videos, etc., use binary modes ('rb', 'wb').

Example: Reading a binary file (e.g., image).

```
with open("image.jpg", "rb") as file:
    data = file.read()
    # Process binary data
```

رومانی صبحی رزق جرجس مطر

Example: Writing binary.

```
data = b'\x00\x01\x02'  # Example binary data
with open("binary.bin", "wb") as file:
    file.write(data)
```

2.6 File Methods and Attributes

- `tell()`: Current position. - `seek(offset, whence)`: Move position. - `name`: File name. - `mode`: File mode. - `closed`: If closed.

Example:

```
with open("example.txt", "r") as file:
    print("File name:", file.name)
    print("Mode:", file.mode)
    file.seek(5)  # Move to position 5
    print("Position:", file.tell())
    content = file.read()
    print(content)
```

3 Exception Handling in Python

Exception handling manages errors gracefully using try-except blocks.

3.1 Try-Except Block

Try code that may raise an exception, catch with except.

Example:

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero!")
```

Output: Cannot divide by zero!

3.2 Multiple Except Blocks

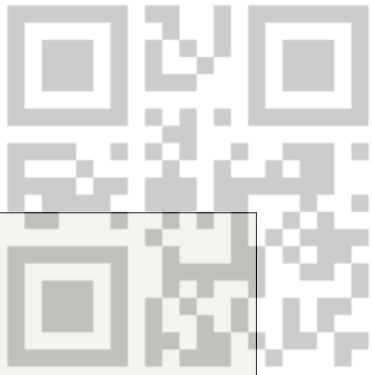
Handle different exceptions.

Example:

```
try:
    num = int("abc")
except ValueError:
    print("Invalid number!")
except TypeError:
    print("Type error!")
```

Output: Invalid number!

رومانی صبحی رزق جرجس مطر



3.3 Else and Finally

- Else: Runs if no exception. - Finally: Always runs.
Example:

```
try:
    result = 10 / 2
except ZeroDivisionError:
    print("Division error!")
else:
    print("Result:", result)
finally:
    print("Execution complete.")
```

Output: Result: 5.0 Execution complete.

3.4 Raising Exceptions

Use `raise` to throw exceptions.
Example:

```
def check_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative!")
    return age

try:
    check_age(-1)
except ValueError as e:
    print(e)
```

Output: Age cannot be negative!

3.5 File Handling with Exceptions

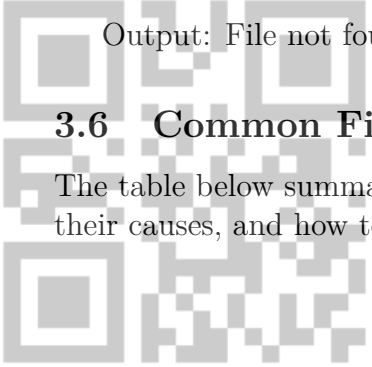
Common in files: FileNotFoundError, IOError.
Example: Handling file not found.

```
try:
    with open("nonexistent.txt", "r") as file:
        print(file.read())
except FileNotFoundError:
    print("File not found!")
except IOError:
    print("IO error occurred!")
finally:
    print("File operation attempted.")
```

Output: File not found! File operation attempted.

3.6 Common File-Related Exceptions

The table below summarizes common exceptions thrown during file handling in Python, their causes, and how to handle them with simple examples.



4 Summary of File Types

4.1 Comparison of Text and Binary Files

The following table compares text and binary files in terms of data representation, file modes, use cases, advantages, and disadvantages. It highlights how text files are suited for readable content while binary files handle raw data efficiently, providing a clear distinction to help choose the appropriate type based on the applications needs.

4.2 When to Use Text vs. Binary Files

This table outlines scenarios for using text versus binary files, including specific examples of file extensions. It serves as a guide for selecting file types based on the nature of the data and the required operations, emphasizing practicality in real-world applications.

4.3 How to Use Text and Binary Files

The table below explains the methods for handling text and binary files in Python, with concise code examples. It demonstrates the differences in opening modes and data handling, making it easier to implement file operations correctly.

5 Assignments

5.1 Assignment 1

Write a program that reads the content of a text file specified by the user. Include exception handling for FileNotFoundError and print an appropriate message if the file does not exist.

2025/2026 2025/2026 2025/2026

5.2 Assignment 2

Create a program that writes a list of strings to a binary file using 'wb' mode and then reads the binary data back, printing it as bytes. Handle any IOError that may occur during the process.

5.3 Assignment 3

Develop a function that appends user input to an existing text file. Use exception handling to catch and manage any permission-related errors or other IOExceptions, ensuring the program doesn't crash.

5.4 Assignment 4

Implement a script that opens a file, uses seek to move to a specific position (e.g., the middle of the file), reads from there, and handles exceptions like ValueError for invalid seek positions or FileNotFoundError if the file is missing.

رومانی صبحی رزق جرجس مطر

Function	Meaning	Example	Result
np.array()	Creates an array from a list.	np.array([1, 2, 3])	[1 2 3]
np.zeros()	Creates an array of zeros.	np.zeros(3)	[0. 0. 0.]
np.ones()	Creates an array of ones.	np.ones(3)	[1. 1. 1.]
np.empty()	Creates an uninitialized array.	np.empty(3)	Random values
shape	Returns the shape (dimensions).	arr.shape for [1,2,3]	(3,)
ndim	Returns the number of dimensions.	arr.ndim for 2D array	2
size	Returns the total number of elements.	arr.size for 2x3 array	6
sum()	Computes the sum of elements.	np.sum([1,2,3])	6
mean()	Computes the mean.	np.mean([1,2,3])	2.0
median()	Computes the median.	np.median([1,2,3])	2.0
var()	Computes the variance.	np.var([1,2,3])	0.666...
std()	Computes the standard deviation.	np.std([1,2,3])	0.816...
min()	Finds the minimum value.	np.min([1,2,3])	1
max()	Finds the maximum value.	np.max([1,2,3])	3
argmin()	Index of minimum value.	np.argmin([1,2,3])	0
argmax()	Index of maximum value.	np.argmax([1,2,3])	2
add()	Element-wise addition.	np.add([1,2], [3,4])	[4 6]
subtract()	Element-wise subtraction.	np.subtract([1,2], [3,4])	[-2 -2]
multiply()	Element-wise multiplication.	np.multiply([1,2], [3,4])	[3 8]
divide()	Element-wise division.	np.divide([1,2], [3,4])	[0.333 0.5]
dot()	Dot product.	np.dot([1,2], [3,4])	11
reshape()	Reshapes the array.	np.reshape([1,2,3,4], (2,2))	[[1 2] [3 4]]
transpose()	Transposes the array.	np.transpose([[1,2],[3,4]])	[[1 3] [2 4]]
vstack()	Stacks arrays vertically.	np.vstack([[1,2],[3,4]])	[[1 2] [3 4]] (same if same shape)
hstack()	Stacks arrays horizontally.	np.hstack([[1,2],[3,4]])	[1 2 3 4] (for 1D)

Table 1: Common NumPy Array Functions with Examples and Results

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

رومانی صبحی رزق جرجس مطر

Mode	Description	Example Use	File Existence Behavior
'r'	Read (default, text mode)	Reading a text file (e.g., "example.txt")	Must exist, else FileNotFoundError
'w'	Write (creates file, truncates if exists, text mode)	Writing a new text file	Creates if not exists, overwrites if exists
'a'	Append (creates if not exists, text mode)	Adding to a log file	Creates if not exists, appends if exists
'r+'	Read and write (text mode)	Modifying an existing text file	Must exist, else FileNotFoundError
'w+'	Write and read (creates, truncates, text mode)	Creating and reading a text file	Creates if not exists, overwrites if exists
'a+'	Append and read (creates if not exists, text mode)	Appending and reading a log	Creates if not exists, appends if exists
'rb'	Read (binary mode)	Reading an image (e.g., "image.jpg")	Must exist, else FileNotFoundError
'wb'	Write (creates, truncates, binary mode)	Writing a binary file (e.g., "data.bin")	Creates if not exists, overwrites if exists
'ab'	Append (creates if not exists, binary mode)	Appending binary data	Creates if not exists, appends if exists
'rb+'	Read and write (binary mode)	Modifying a binary file	Must exist, else FileNotFoundError
'wb+'	Write and read (creates, truncates, binary mode)	Creating and reading binary data	Creates if not exists, overwrites if exists
'ab+'	Append and read (creates if not exists, binary mode)	Appending and reading binary data	Creates if not exists, appends if exists
'x'	Exclusive creation (fails if file exists, text mode)	Creating a new text file only if it doesnt exist	Throws FileExistsError if exists, creates if not
'xb'	Exclusive creation (fails if file exists, binary mode)	Creating a new binary file only if it doesnt exist	Throws FileExistsError if exists, creates if not

2025/2026 2025/2026 2025/2026

Table 2: File Opening Modes in Python

رومانی صبحی رزق جرجس مطر

Exception Type	When Thrown	How to Handle	Example Code
FileNotFoundError	When opening a non-existent file in 'r' or 'r+' mode.	Use try-except to catch and print a message or create the file.	`try: open('nonexistent.txt', 'r') except FileNotFoundError: print("File not found!")` Output: File not found!
PermissionError	When no permission to read/write (e.g., read-only file in 'w' mode).	Catch and inform user, or change permissions if possible.	`try: open('protected.txt', 'w') except PermissionError: print("Permission denied!")` Output: Permission denied!
IsADirectoryError	When treating a directory as a file (e.g., open('dir/')).	Ensure path is a file, not directory.	`try: open('directory/', 'r') except IsADirectoryError: print("Path is a directory!")` Output: Path is a directory!
NotADirectoryError	When a non-directory path is expected as a directory.	Verify path structure.	`import os try: os.listdir('file.txt') except NotADirectoryError: print("Not a directory!")` Output: Not a directory!
IOError/OSError	General IO errors (e.g., disk full, invalid path).	Catch broadly and log error.	`try: open('/invalid/path', 'r') except IOError as e: print(f"IO error: {e}")` Output: IO error: [Errno 2] No such file...
ValueError	Invalid mode or seek position.	Validate inputs before operations.	`try: open('file.txt', 'invalid') except ValueError: print("Invalid mode!")` Output: Invalid mode!

Table 3: Common File Handling Exceptions in Python

رومانی صبحی رزق جرجس مطر

Aspect	Text Files	Binary Files
Data Representation	Human-readable characters (e.g., ASCII, UTF-8). Stored as strings.	Raw binary data (bytes). Not human-readable.
File Modes	'r', 'w', 'a', 'r+', etc.	'rb', 'wb', 'ab', 'rb+', etc.
Use Cases	Configuration files, logs, scripts, CSV/JSON data. When data needs to be edited manually.	Images (JPEG, PNG), videos (MP4), executables, serialized objects. When preserving exact byte structure is crucial.
Advantages	Easy to read/edit with text editors. Platform-independent for plain text.	Efficient for non-text data. No encoding issues.
Disadvantages	Inefficient for large non-text data. Encoding/decoding required.	Not readable without specialized tools. Platform-dependent sometimes (e.g., endianness).

Table 4: Differences Between Text and Binary Files

File Type	When to Use	Examples
Text Files	When data is string-based, needs human readability, or for data exchange (e.g., APIs, configs). Suitable for logging or simple data storage.	.txt (plain text notes), .csv (data tables), .json (structured data), .py (Python scripts).
Binary Files	When dealing with non-text media, serialized objects, or performance-critical data where byte-level precision is needed.	.jpg (images), .mp3 (audio), .exe (executables), .pickle (serialized Python objects).

Table 5: When to Use Text and Binary Files with Examples

File Type	How to Use in Python	Example Code

Text Files	Open with text modes, use read/write for strings. Handle encoding if needed (default UTF-8).	`with open('data.txt', 'w') as f:`
		`  f.write('Hello')`
		`with open('data.txt', 'r') as f:`
		`  print(f.read())  # Hello`

Binary Files	Open with binary modes, use bytes for read/write. No automatic encoding.	`with open('data.bin', 'wb') as f:`
		`  f.write(b'\x48\x65\x6C\x6C\x6F')`
		`with open('data.bin', 'rb') as f:`
		`  print(f.read())  # b'Hello'`

Table 6: How to Use Text and Binary Files with Examples

Python Lecture 6: File Handling and Exception Handling

September 24, 2025

رومانى صبحى رزق جرجس مطر

1 Revision of Lecture 5: Functions Using Data Structures

To revise the data structures from Lecture 5, we will create five functions, each utilizing a different data structure: list, dictionary, tuple, array (from NumPy), and set. These functions will perform simple operations. Finally, a main program will call all five functions.

1.1 Function Using List: Sum of Elements

This function takes a list of numbers and returns their sum.

```
def sum_list(numbers):
    total = 0
    for num in numbers:
        total += num
    return total
```

Example: $\text{sum}_list([1,2,3])returns6$.

1.2 Function Using Dictionary: Get Value by Key

This function takes a dictionary and a key, returning the corresponding value or "Not Found".

```
def get_dict_value(my_dict, key):
    return my_dict.get(key, "Not Found")
```

Example: $\text{get}_dict_value(”name” : ”Alice”, ”age” : 25, ”age”)returns25$.

1.3 Function Using Tuple: Unpack and Add

This function takes a tuple of three numbers, unpacks them, and returns their sum.

```
def sum_tuple(tup):
    a, b, c = tup
    return a + b + c
```

Example: $\text{sum}_tuple((4,5,6))returns15$.

1.4 Function Using Array (NumPy): Compute Mean

This function uses NumPy to compute the mean of an array.

```
import numpy as np

def compute_mean_array(arr):
    return np.mean(arr)
```

Example: $\text{compute}_{m}ean_{a}rray(np.array([10, 20, 30]))returns20.0$.

1.5 Additional NumPy Examples

This subsection provides a detailed explanation and examples of using NumPy arrays, including 1D and 2D arrays, stacking operations, and mathematical functions. These examples demonstrate NumPy's power for numerical computations.

Vertical stacking (np.vstack) combines arrays along the vertical axis (axis 0), effectively adding rows from one array below another. It requires arrays to have the same number of columns. Horizontal stacking (np.hstack) combines arrays along the horizontal axis (axis 1), adding columns from one array to the right of another. It requires arrays to have the same number of rows.

```
# Array (1D or 2D)
import numpy as np

arr = np.array([1, 2, 4, 10])
print(arr)
print(arr.shape)
print("Sum: ", arr.sum())
print("Mean = Average = ", arr.mean())

arr2D = np.array([[1, 2, 10], [2, 5, 7], [4, 6, 12], [5, 8, 9]])
print(arr2D)
print(arr2D.shape)
print("Sum: ", arr2D.sum())
print("Mean = Average = ", arr2D.mean())

arr1 = np.array([[1, 2, 3], [4, 5, 6]])
print("Array:\n", arr1)

arr2 = np.array([[10, 20, 30], [40, 50, 60]])
print("\nZeros:\n", arr2)
print(arr2.shape)

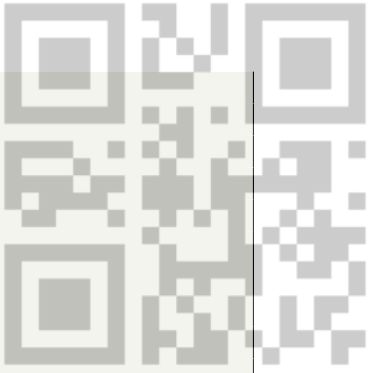
arr3 = np.array([[10, 20, 30, 40], [40, 50, 60, 40]])
print("\nZeros:\n", arr3)
print(arr3.shape)

stack_v = np.vstack((arr1, arr2))
print("\nVertical Stack:\n", stack_v)
print(stack_v.shape)

stack_v2 = np.vstack((arr1, arr3))
print("\nVertical Stack:\n", stack_v2)
print(stack_v2.shape)

stack_h = np.hstack((arr1, arr2))
```

رومانی صبحی رزق جرجس مطر



```
print("\nHorizontal Stack:\n", stack_h)
print(stack_h.shape)

stack_h2 = np.hstack((arr1, arr3))
print("\nHorizontal Stack:\n", stack_h2)
print(stack_h2.shape)

# Mathematical Operations
arr1 = np.array([[1, 2, 3], [4, 5, 6]])
print("Array:\n", arr1)

arr2 = np.array([[10, 20, 30], [40, 50, 60]])
print("\nZeros:\n", arr2)
print(arr2.shape)

print("\nAdd:\n", np.add(arr1, arr2))
print("Subtract:\n", np.subtract(arr1, arr2))
print("Multiply:\n", np.multiply(arr1, arr2))
print("Divide:\n", np.divide(arr1, arr2))
print("Dot product:\n", np.dot(arr1, arr2.T))

arr4 = np.array([1, 2, 3, 10, 4, 5, 6])
print("Array:\n", arr4)
print("\nSum:", np.sum(arr4))
print("Mean:", np.mean(arr4))
print("Median:", np.median(arr4))
print("Variance:", np.var(arr4))
print("Std Dev:", np.std(arr4))
print("Min:", np.min(arr4))
print("Max:", np.max(arr4))
print("Argmin (index of min):", np.argmin(arr4))
print("Argmax (index of max):", np.argmax(arr4))
```

Expected Output (partial, varies by environment): “‘ [1 2 4 10] (4,) Sum: 17 Mean = Average = 4.25 [[1 2 10] [2 5 7] [4 6 12] [5 8 9]] (4, 3) Sum: 71 Mean = Average = 5.916666666666667 ... “‘

The following table summarizes common NumPy array functions, their meanings, simple examples, and results. It provides a quick reference for array operations.

1.6 Function Using Set: Check Membership

This function takes a set and a value, checking if the value is in the set.

```
def check_in_set(my_set, value):
    if value in my_set:
        return "Present"
    else:
        return "Absent"
```

Example: $\text{check}_in_set(1, 2, 3, 2) returns “Present”$.

1.7 Main Program Calling All Functions

The main program demonstrates calling all five functions.

```
import numpy as np

```

رومانى صبحى رزق جرجس مطر

```
def main():
    # List example
    my_list = [1, 2, 3]
    print("Sum of list:", sum_list(my_list))

    # Dictionary example
    my_dict = {"name": "Alice", "age": 25}
    print("Value for 'name':", get_dict_value(my_dict, "name"))

    # Tuple example
    my_tuple = (4, 5, 6)
    print("Sum of tuple:", sum_tuple(my_tuple))

    # Array example
    my_array = np.array([10, 20, 30])
    print("Mean of array:", compute_mean_array(my_array))

    # Set example
    my_set = {1, 2, 3}
    print("Check 3 in set:", check_in_set(my_set, 3))

main()
```

Output: Sum of list: 6 Value for 'name': Alice Sum of tuple: 15 Mean of array: 20.0 Check 3 in set: Present

2 File Handling in Python

File handling allows Python programs to read from and write to files. Python provides built-in functions to create, read, update, and delete files.

2.1 Opening a File

Use the `open()` function to open a file. It takes the file path and mode as arguments. The following table summarizes all available modes for opening text and binary files, providing a comprehensive reference for file handling.

Modes: - 'r': Read (default) - 'w': Write (creates file if not exists, truncates if exists) - 'a': Append (creates if not exists) - 'b': Binary mode (e.g., 'rb' for reading binary) - 'x': Exclusive creation (fails if file exists) - '+': Read and write (e.g., 'r+')

Example: Opening a file for reading.

```
file = open("example.txt", "r")
print(file.read())
file.close()
```

2.2 Closing a File

Always close files using `close()` to free resources. Better to use `with` statement for automatic closing.

Example with `with`:

```
with open("example.txt", "r") as file:
    content = file.read()
```

```
    print(content)
# File is automatically closed here
```

2.3 Reading from a File

- `read()`: Reads the entire file. - `readline()`: Reads one line. - `readlines()`: Reads all lines into a list.

Example: Reading entire file.

```
with open("example.txt", "r") as file:
    content = file.read()
    print(content)
```

Example: Reading line by line.

```
with open("example.txt", "r") as file:
    line = file.readline()
    while line:
        print(line.strip())
        line = file.readline()
```

Example: Reading all lines.

```
with open("example.txt", "r") as file:
    lines = file.readlines()
    for line in lines:
        print(line.strip())
```

2.4 Writing to a File

- `write()`: Writes a string. - `writelines()`: Writes a list of strings.

Example: Writing to a file (overwrites).

```
with open("output.txt", "w") as file:
    file.write("Hello, World!\n")
    file.write("This is a test.")
```

Example: Appending to a file.

```
with open("output.txt", "a") as file:
    file.write("\nAppended line.")
```

Example: Writelines.

```
lines = ["Line 1\n", "Line 2\n", "Line 3\n"]
with open("output.txt", "w") as file:
    file.writelines(lines)
```

2.5 Binary Files

For images, videos, etc., use binary modes ('rb', 'wb').

Example: Reading a binary file (e.g., image).

```
with open("image.jpg", "rb") as file:
    data = file.read()
    # Process binary data
```

رومانی صبحی رزق جرجس مطر

Example: Writing binary.

```
data = b'\x00\x01\x02'  # Example binary data
with open("binary.bin", "wb") as file:
    file.write(data)
```

2.6 File Methods and Attributes

- `tell()`: Current position. - `seek(offset, whence)`: Move position. - `name`: File name. - `mode`: File mode. - `closed`: If closed.

Example:

```
with open("example.txt", "r") as file:
    print("File name:", file.name)
    print("Mode:", file.mode)
    file.seek(5)  # Move to position 5
    print("Position:", file.tell())
    content = file.read()
    print(content)
```

3 Exception Handling in Python

Exception handling manages errors gracefully using try-except blocks.

3.1 Try-Except Block

Try code that may raise an exception, catch with except.

Example:

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero!")
```

Output: Cannot divide by zero!

3.2 Multiple Except Blocks

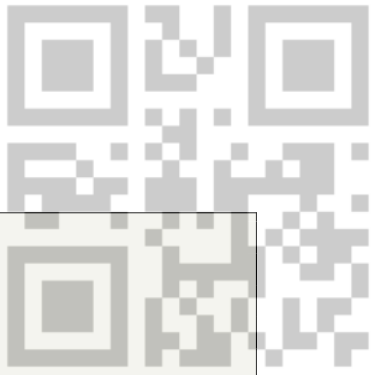
Handle different exceptions.

Example:

```
try:
    num = int("abc")
except ValueError:
    print("Invalid number!")
except TypeError:
    print("Type error!")
```

Output: Invalid number!

رومانی صبحی رزق جرجس مطر



3.3 Else and Finally

- Else: Runs if no exception. - Finally: Always runs.
Example:

```
try:
    result = 10 / 2
except ZeroDivisionError:
    print("Division error!")
else:
    print("Result:", result)
finally:
    print("Execution complete.")
```

Output: Result: 5.0 Execution complete.

3.4 Raising Exceptions

Use `raise` to throw exceptions.
Example:

```
def check_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative!")
    return age

try:
    check_age(-1)
except ValueError as e:
    print(e)
```

Output: Age cannot be negative!

3.5 File Handling with Exceptions

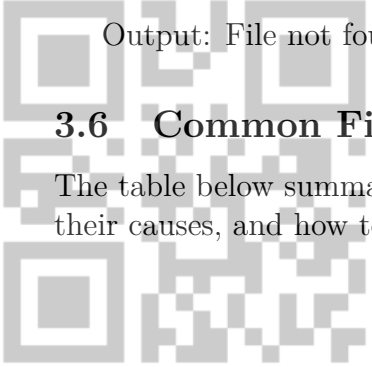
Common in files: FileNotFoundError, IOError.
Example: Handling file not found.

```
try:
    with open("nonexistent.txt", "r") as file:
        print(file.read())
except FileNotFoundError:
    print("File not found!")
except IOError:
    print("IO error occurred!")
finally:
    print("File operation attempted.")
```

Output: File not found! File operation attempted.

3.6 Common File-Related Exceptions

The table below summarizes common exceptions thrown during file handling in Python, their causes, and how to handle them with simple examples.



4 Summary of File Types

4.1 Comparison of Text and Binary Files

The following table compares text and binary files in terms of data representation, file modes, use cases, advantages, and disadvantages. It highlights how text files are suited for readable content while binary files handle raw data efficiently, providing a clear distinction to help choose the appropriate type based on the applications needs.

4.2 When to Use Text vs. Binary Files

This table outlines scenarios for using text versus binary files, including specific examples of file extensions. It serves as a guide for selecting file types based on the nature of the data and the required operations, emphasizing practicality in real-world applications.

4.3 How to Use Text and Binary Files

The table below explains the methods for handling text and binary files in Python, with concise code examples. It demonstrates the differences in opening modes and data handling, making it easier to implement file operations correctly.

5 Assignments

5.1 Assignment 1

Write a program that reads the content of a text file specified by the user. Include exception handling for FileNotFoundError and print an appropriate message if the file does not exist.

2025/2026 2025/2026 2025/2026

5.2 Assignment 2

Create a program that writes a list of strings to a binary file using 'wb' mode and then reads the binary data back, printing it as bytes. Handle any IOError that may occur during the process.

5.3 Assignment 3

Develop a function that appends user input to an existing text file. Use exception handling to catch and manage any permission-related errors or other IOExceptions, ensuring the program doesn't crash.

5.4 Assignment 4

Implement a script that opens a file, uses seek to move to a specific position (e.g., the middle of the file), reads from there, and handles exceptions like ValueError for invalid seek positions or FileNotFoundError if the file is missing.

رومانی صبحی رزق جرجس مطر

Function	Meaning	Example	Result
np.array()	Creates an array from a list.	np.array([1, 2, 3])	[1 2 3]
np.zeros()	Creates an array of zeros.	np.zeros(3)	[0. 0. 0.]
np.ones()	Creates an array of ones.	np.ones(3)	[1. 1. 1.]
np.empty()	Creates an uninitialized array.	np.empty(3)	Random values
shape	Returns the shape (dimensions).	arr.shape for [1,2,3]	(3,)
ndim	Returns the number of dimensions.	arr.ndim for 2D array	2
size	Returns the total number of elements.	arr.size for 2x3 array	6
sum()	Computes the sum of elements.	np.sum([1,2,3])	6
mean()	Computes the mean.	np.mean([1,2,3])	2.0
median()	Computes the median.	np.median([1,2,3])	2.0
var()	Computes the variance.	np.var([1,2,3])	0.666...
std()	Computes the standard deviation.	np.std([1,2,3])	0.816...
min()	Finds the minimum value.	np.min([1,2,3])	1
max()	Finds the maximum value.	np.max([1,2,3])	3
argmin()	Index of minimum value.	np.argmin([1,2,3])	0
argmax()	Index of maximum value.	np.argmax([1,2,3])	2
add()	Element-wise addition.	np.add([1,2], [3,4])	[4 6]
subtract()	Element-wise subtraction.	np.subtract([1,2], [3,4])	[-2 -2]
multiply()	Element-wise multiplication.	np.multiply([1,2], [3,4])	[3 8]
divide()	Element-wise division.	np.divide([1,2], [3,4])	[0.333 0.5]
dot()	Dot product.	np.dot([1,2], [3,4])	11
reshape()	Reshapes the array.	np.reshape([1,2,3,4], (2,2))	[[1 2] [3 4]]
transpose()	Transposes the array.	np.transpose([[1,2],[3,4]])	[[1 3] [2 4]]
vstack()	Stacks arrays vertically.	np.vstack([[1,2],[3,4]])	[[1 2] [3 4]] (same if same shape)
hstack()	Stacks arrays horizontally.	np.hstack([[1,2],[3,4]])	[1 2 3 4] (for 1D)

Table 1: Common NumPy Array Functions with Examples and Results

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

رومانی صبحی رزق جرجس مطر

Mode	Description	Example Use	File Existence Behavior
'r'	Read (default, text mode)	Reading a text file (e.g., "example.txt")	Must exist, else FileNotFoundError
'w'	Write (creates file, truncates if exists, text mode)	Writing a new text file	Creates if not exists, overwrites if exists
'a'	Append (creates if not exists, text mode)	Adding to a log file	Creates if not exists, appends if exists
'r+'	Read and write (text mode)	Modifying an existing text file	Must exist, else FileNotFoundError
'w+'	Write and read (creates, truncates, text mode)	Creating and reading a text file	Creates if not exists, overwrites if exists
'a+'	Append and read (creates if not exists, text mode)	Appending and reading a log	Creates if not exists, appends if exists
'rb'	Read (binary mode)	Reading an image (e.g., "image.jpg")	Must exist, else FileNotFoundError
'wb'	Write (creates, truncates, binary mode)	Writing a binary file (e.g., "data.bin")	Creates if not exists, overwrites if exists
'ab'	Append (creates if not exists, binary mode)	Appending binary data	Creates if not exists, appends if exists
'rb+'	Read and write (binary mode)	Modifying a binary file	Must exist, else FileNotFoundError
'wb+'	Write and read (creates, truncates, binary mode)	Creating and reading binary data	Creates if not exists, overwrites if exists
'ab+'	Append and read (creates if not exists, binary mode)	Appending and reading binary data	Creates if not exists, appends if exists
'x'	Exclusive creation (fails if file exists, text mode)	Creating a new text file only if it doesnt exist	Throws FileExistsError if exists, creates if not
'xb'	Exclusive creation (fails if file exists, binary mode)	Creating a new binary file only if it doesnt exist	Throws FileExistsError if exists, creates if not

2025/2026 2025/2026 2025/2026

Table 2: File Opening Modes in Python

رومانی صبحی رزق جرجس مطر

Exception Type	When Thrown	How to Handle	Example Code
FileNotFoundError	When opening a non-existent file in 'r' or 'r+' mode.	Use try-except to catch and print a message or create the file.	`try: open('nonexistent.txt', 'r') except FileNotFoundError: print("File not found!")` Output: File not found!
PermissionError	When no permission to read/write (e.g., read-only file in 'w' mode).	Catch and inform user, or change permissions if possible.	`try: open('protected.txt', 'w') except PermissionError: print("Permission denied!")` Output: Permission denied!
IsADirectoryError	When treating a directory as a file (e.g., open('dir/')).	Ensure path is a file, not directory.	`try: open('directory/', 'r') except IsADirectoryError: print("Path is a directory!")` Output: Path is a directory!
NotADirectoryError	When a non-directory path is expected as a directory.	Verify path structure.	`import os try: os.listdir('file.txt') except NotADirectoryError: print("Not a directory!")` Output: Not a directory!
IOError/OSError	General IO errors (e.g., disk full, invalid path).	Catch broadly and log error.	`try: open('/invalid/path', 'r') except IOError as e: print(f"IO error: {e}")` Output: IO error: [Errno 2] No such file...
ValueError	Invalid mode or seek position.	Validate inputs before operations.	`try: open('file.txt', 'invalid') except ValueError: print("Invalid mode!")` Output: Invalid mode!

Table 3: Common File Handling Exceptions in Python

رومانی صبحی رزق جرجس مطر

Aspect	Text Files	Binary Files
Data Representation	Human-readable characters (e.g., ASCII, UTF-8). Stored as strings.	Raw binary data (bytes). Not human-readable.
File Modes	'r', 'w', 'a', 'r+', etc.	'rb', 'wb', 'ab', 'rb+', etc.
Use Cases	Configuration files, logs, scripts, CSV/JSON data. When data needs to be edited manually.	Images (JPEG, PNG), videos (MP4), executables, serialized objects. When preserving exact byte structure is crucial.
Advantages	Easy to read/edit with text editors. Platform-independent for plain text.	Efficient for non-text data. No encoding issues.
Disadvantages	Inefficient for large non-text data. Encoding/decoding required.	Not readable without specialized tools. Platform-dependent sometimes (e.g., endianness).

Table 4: Differences Between Text and Binary Files

File Type	When to Use	Examples
Text Files	When data is string-based, needs human readability, or for data exchange (e.g., APIs, configs). Suitable for logging or simple data storage.	.txt (plain text notes), .csv (data tables), .json (structured data), .py (Python scripts).
Binary Files	When dealing with non-text media, serialized objects, or performance-critical data where byte-level precision is needed.	.jpg (images), .mp3 (audio), .exe (executables), .pickle (serialized Python objects).

Table 5: When to Use Text and Binary Files with Examples

File Type	How to Use in Python	Example Code

Text Files	Open with text modes, use read/write for strings. Handle encoding if needed (default UTF-8).	`with open('data.txt', 'w') as f:`
		`  f.write('Hello')`
		`with open('data.txt', 'r') as f:`
		`  print(f.read())  # Hello`

Binary Files	Open with binary modes, use bytes for read/write. No automatic encoding.	`with open('data.bin', 'wb') as f:`
		`  f.write(b'\x48\x65\x6C\x6C\x6F')`
		`with open('data.bin', 'rb') as f:`
		`  print(f.read())  # b'Hello'`

Table 6: How to Use Text and Binary Files with Examples

Python Lecture 7: Advanced Exception Handling and Introduction to Object-Oriented Programming

September 30, 2025

رومانی صبحی رزق جرجس مطر

1 Summary of File Types

This section provides a summary table of text and binary file types, highlighting their differences and including simple Python code examples for usage.

2 Try-Except-Else-Finally and Usage with Files

The try-except-else-finally block is used for exception handling in Python. - **try**: Contains code that might raise an exception. - **except**: Catches and handles the exception. - **else**: Executes if no exception occurs in the try block. - **finally**: Always executes, regardless of whether an exception occurred or not. Useful for cleanup tasks like closing files.

When working with files, these blocks ensure graceful handling of errors such as file not found or I/O issues.

2.1 Example 1: Basic File Reading with Try-Except-Else-Finally

```
try:
    with open('example.txt', 'r') as file:
        content = file.read()
except FileNotFoundError:
    print("File not found!")
except IOError:
    print("An I/O error occurred!")
else:
    print("File content:", content)
finally:
    print("File operation complete.")
```

If the file exists, it prints the content and "File operation complete." If not, it handles the error and still runs finally.

2.2 Example 2: Writing to File with Error Handling

```
try:
    with open('output.txt', 'w') as file:
```

```
        file.write("Hello, World!")
except IOError:
    print("Error writing to file!")
else:
    print("Write successful.")
finally:
    print("Cleanup done.")
```

This ensures the write operation is attempted, handles errors, confirms success in else, and always runs finally.

3 Starting in Object-Oriented Programming in Python

3.1 Definition of OOP and Its Characteristics

Object-Oriented Programming (OOP) is a programming paradigm that uses "objects" to design applications and computer programs. Objects are instances of classes and can contain data (attributes) and code (methods). OOP focuses on modeling real-world entities as software objects that interact with each other.

Key characteristics of OOP: - **Encapsulation**: Bundling data and methods that operate on the data within a single unit (class), restricting direct access to some components. - **Abstraction**: Hiding complex implementation details and showing only essential features. - **Inheritance**: Allowing a new class to inherit properties and methods from an existing class, promoting code reuse. - **Polymorphism**: Enabling objects of different classes to be treated as objects of a common superclass, typically through method overriding.

3.2 Difference Between Class and Object

- **Class**: A blueprint or template that defines the structure and behavior (attributes and methods) for creating objects. It does not occupy memory until an object is created. - **Object**: An instance of a class. It is a real entity that occupies memory and has specific values for the attributes defined in the class.

How to create an object from a class:

```
class MyClass:
    pass  # Class definition

obj = MyClass()  # Creating an object
```

3.3 Difference Between Procedural Programming and Object-Oriented Programming

- **Procedural Programming**: Focuses on functions and procedures to perform tasks. It follows a top-down approach, where the program is divided into functions that operate on data. Data and functions are separate. Examples: C, Pascal. - **Object-Oriented Programming**: Focuses on objects that combine data and functions. It follows a bottom-up approach, emphasizing data hiding, reusability, and modeling real-world scenarios. Examples: Python, Java.

رومانی صبحی رزق جرجس مطر

Procedural is suitable for simple, linear tasks, while OOP is better for complex, modular systems.

3.4 Content of the Class in Python

A class in Python typically contains: - **Attributes** (data members): Variables that store data. - **Methods**: Functions that define behaviors. - **Constructor** (`__init__`): A special method to initialize objects. - Optional: Class variables, static methods, etc.

Example:

```
class Example:
    class_var = "Shared"  # Class variable

    def __init__(self, value):
        self.instance_var = value  # Instance attribute

    def method(self):
        print(self.instance_var)
```

3.5 Data Members and Methods of the Class

- **Data Members (Attributes)**: Variables associated with the class or its instances. - Instance attributes: Unique to each object (defined in `__init__`). - Class attributes: Shared among all instances (defined directly in the class). - **Methods**: Functions defined inside the class that operate on data members. The first parameter is usually `self` (referring to the instance).

3.6 Difference Between Private, Public, Protected Data Members and Methods

In Python, access modifiers are conventions rather than strict enforcements: - **Public**: No underscore prefix. Accessible from anywhere. Example: `self.var`. - **Protected**: Single underscore prefix (`_var`). Intended for use within the class and subclasses. Accessible outside but by convention, treated as non-public. - **Private**: Double underscore prefix (`__var`). Name mangling makes it harder to access outside the class (becomes `_ClassName__var`). Not truly private but discourages external access.

Example:

```
class AccessExample:
    def __init__(self):
        self.public_var = "Public"
        self._protected_var = "Protected"
        self.__private_var = "Private"
```

3.7 Complete Examples of Classes

3.7.1 Class Rectangle

Here is a complete example of a Rectangle class with constructor, accessors (get methods), modifiers (set methods), area calculation, perimeter calculation, and a print method.

رومانی صبحی رزق جرجس مطر

```python
class Rectangle:
    def __init__(self, length, width):
        self._length = length  # Protected attribute
        self._width = width    # Protected attribute

    # Accessors (Getters)
    def get_length(self):
        return self._length

    def get_width(self):
        return self._width

    # Modifiers (Setters)
    def set_length(self, length):
        if length > 0:
            self._length = length
        else:
            print("Length must be positive.")

    def set_width(self, width):
        if width > 0:
            self._width = width
        else:
            print("Width must be positive.")

    # Area method
    def rectangle_area(self):
        return self._length * self._width

    # Perimeter method
    def rect_perimeter(self):
        return 2 * (self._length + self._width)

    # Print details
    def print_rectangle(self):
        print(f"Length: {self._length}, Width: {self._width}")
        print(f"Area: {self.rectangle_area()}")
        print(f"Perimeter: {self.rect_perimeter()}")

# Usage example
rect = Rectangle(5, 3)  # Create object
rect.print_rectangle()  # Print details

# Use setters
rect.set_length(10)
rect.set_width(4)

# Use getters and methods
print("New Length:", rect.get_length())
print("New Area:", rect.rectangle_area())
```

2025/2026 2025/2026 2025/2026

```
rect.print_rectangle()  # Print updated details
```

Output:

```
Length: 5, Width: 3
Area: 15
Perimeter: 16
New Length: 10
New Area: 40
Length: 10, Width: 4
Area: 40
Perimeter: 28
```

3.7.2 Class Square

A Square class, similar to Rectangle but with a single side length.

```
class Square:
    def __init__(self, side):
        self._side = side  # Protected attribute

    # Accessors (Getters)
    def get_side(self):
        return self._side

    # Modifiers (Setters)
    def set_side(self, side):
        if side > 0:
            self._side = side
        else:
            print("Side must be positive.")

    # Area method
    def square_area(self):
        return self._side ** 2

    # Perimeter method
    def square_perimeter(self):
        return 4 * self._side

    # Print details
    def print_square(self):
        print(f"Side: {self._side}")
        print(f"Area: {self.square_area()}")
        print(f"Perimeter: {self.square_perimeter()}")

# Usage example
sq = Square(4)  # Create object
sq.print_square()  # Print details

# Use setter
sq.set_side(6)
```

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

```
# Use getter and methods
print("New Side:", sq.get_side())
print("New Area:", sq.square_area())
sq.print_square()  # Print updated details
```

Output:

```
Side: 4
Area: 16
Perimeter: 16
New Side: 6
New Area: 36
Side: 6
Area: 36
Perimeter: 24
```

3.7.3 Class Cylinder

A Cylinder class with radius and height, calculating volume and surface area.

```
import math

class Cylinder:
    def __init__(self, radius, height):
        self._radius = radius  # Protected attribute
        self._height = height  # Protected attribute
2025/2026 2025/2026 2025/2026
    # Accessors (Getters)
    def get_radius(self):
        return self._radius

    def get_height(self):
        return self._height

    # Modifiers (Setters)
    def set_radius(self, radius):
        if radius > 0:
            self._radius = radius
        else:
            print("Radius must be positive.")

    def set_height(self, height):
        if height > 0:
            self._height = height
        else:
            print("Height must be positive.")

    # Volume method
    def cylinder_volume(self):
        return math.pi * (self._radius ** 2) * self._height

```

رومانی صبحی رزق جرجس مطر

```
    # Surface area method
    def cylinder_surface_area(self):
        return 2 * math.pi * self._radius * (self._radius +
            self._height)

    # Print details
    def print_cylinder(self):
        print(f"Radius: {self._radius}, Height: {self._height}")
        print(f"Volume: {self.cylinder_volume():.2f}")
        print(f"Surface Area:
            {self.cylinder_surface_area():.2f}")

# Usage example
cyl = Cylinder(3, 5)  # Create object
cyl.print_cylinder()  # Print details

# Use setters
cyl.set_radius(4)
cyl.set_height(6)

# Use getters and methods
print("New Radius:", cyl.get_radius())
print("New Volume:", f"{cyl.cylinder_volume():.2f}")
cyl.print_cylinder()  # Print updated details
```

Output:

```
Radius: 3, Height: 5
Volume: 141.37
Surface Area: 150.80
New Radius: 4
New Volume: 301.59
Radius: 4, Height: 6
Volume: 301.59
Surface Area: 251.33
```

3.7.4 Class Complex

A Complex number class with real and imaginary parts, supporting basic operations.

```
class Complex:
    def __init__(self, real, imag):
        self._real = real  # Protected attribute
        self._imag = imag  # Protected attribute

    # Accessors (Getters)
    def get_real(self):
        return self._real

    def get_imag(self):
        return self._imag

```

رومانی صبحی رزق جرجس مطر

```
    # Modifiers (Setters)
    def set_real(self, real):
        self._real = real

    def set_imag(self, imag):
        self._imag = imag

    # Add method
    def add(self, other):
        return Complex(self._real + other._real, self._imag +
            other._imag)

    # Subtract method
    def subtract(self, other):
        return Complex(self._real - other._real, self._imag -
            other._imag)

    # Multiply method
    def multiply(self, other):
        real = self._real * other._real - self._imag *
            other._imag
        imag = self._real * other._imag + self._imag *
            other._real
        return Complex(real, imag)

    # Modulus method
    def modulus(self):
        return (self._real ** 2 + self._imag ** 2) ** 0.5

    # Print details
    def print_complex(self):
        print(f"{self._real} + {self._imag}i")
        print(f"Modulus: {self.modulus():.2f}")

# Usage example
c1 = Complex(2, 3)  # Create object
c1.print_complex()  # Print details

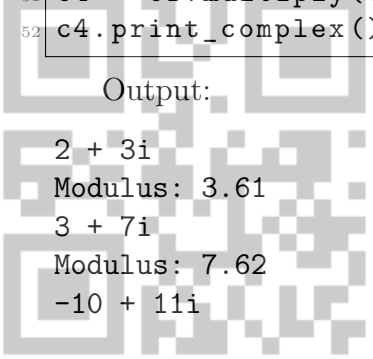
c2 = Complex(1, 4)
c3 = c1.add(c2)
c3.print_complex()

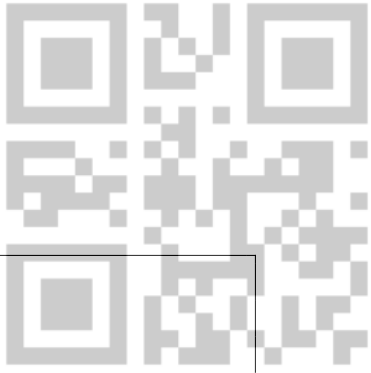
c4 = c1.multiply(c2)
c4.print_complex()
```

Output:

```
2 + 3i
Modulus: 3.61
3 + 7i
Modulus: 7.62
-10 + 11i
```

2025/2026 2025/2026 2025/2026
رومانى صبحى رزق جرجس مطر





```
Modulus: 14.87
```

3.7.5 Class Student

A Student class with name, age, and grades list.

```python
class Student:
    def __init__(self, name, age):
        self._name = name  # Protected attribute
        self._age = age    # Protected attribute
        self._grades = []  # Protected list

    # Accessors (Getters)
    def get_name(self):
        return self._name

    def get_age(self):
        return self._age

    def get_grades(self):
        return self._grades

    # Modifiers (Setters)
    def set_name(self, name):
        self._name = name

    def set_age(self, age):
        if age > 0:
            self._age = age
        else:
            print("Age must be positive.")

    # Add grade method
    def add_grade(self, grade):
        if 0 <= grade <= 100:
            self._grades.append(grade)
        else:
            print("Grade must be between 0 and 100.")

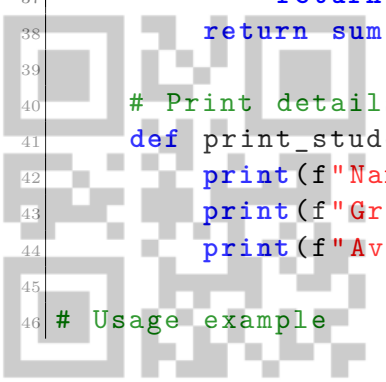
    # Average grade method
    def average_grade(self):
        if not self._grades:
            return 0
        return sum(self._grades) / len(self._grades)

    # Print details
    def print_student(self):
        print(f"Name: {self._name}, Age: {self._age}")
        print(f"Grades: {self._grades}")
        print(f"Average Grade: {self.average_grade():.2f}")

# Usage example
```

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر



```python
stu = Student("Alice", 20)  # Create object
stu.add_grade(85)
stu.add_grade(92)
stu.print_student()  # Print details

# Use setters
stu.set_age(21)
stu.add_grade(78)

# Use getters and methods
print("New Age:", stu.get_age())
print("New Average:", f"{stu.average_grade():.2f}")
stu.print_student()  # Print updated details
```

Output:

```
Name: Alice, Age: 20
Grades: [85, 92]
Average Grade: 88.50
New Age: 21
New Average: 85.00
Name: Alice, Age: 21
Grades: [85, 92, 78]
Average Grade: 85.00
```

3.7.6 Class Circle

A Circle class with radius, calculating area and circumference.

```python
import math

class Circle:
    def __init__(self, radius):
        self._radius = radius  # Protected attribute

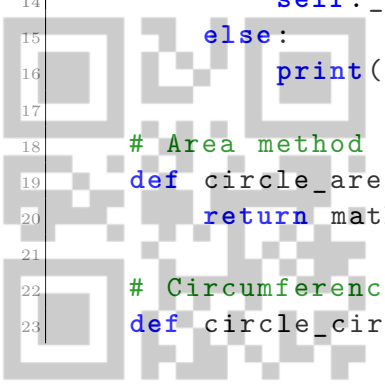
    # Accessors (Getters)
    def get_radius(self):
        return self._radius

    # Modifiers (Setters)
    def set_radius(self, radius):
        if radius > 0:
            self._radius = radius
        else:
            print("Radius must be positive.")

    # Area method
    def circle_area(self):
        return math.pi * (self._radius ** 2)

    # Circumference method
    def circle_circumference(self):
```

رومانی صبحی رزق جرجس مطر



```
        return 2 * math.pi * self._radius

    # Print details
    def print_circle(self):
        print(f"Radius: {self._radius}")
        print(f"Area: {self.circle_area():.2f}")
        print(f"Circumference:
           {self.circle_circumference():.2f}")

# Usage example
circ = Circle(5)  # Create object
circ.print_circle()  # Print details

# Use setter
circ.set_radius(7)

# Use getter and methods
print("New Radius:", circ.get_radius())
print("New Area:", f"{circ.circle_area():.2f}")
circ.print_circle()  # Print updated details
```

Output:

```
Radius: 5
Area: 78.54
Circumference: 31.42
New Radius: 7
New Area: 153.94
Radius: 7
Area: 153.94
Circumference: 43.98
```


3.7.7 Class BankAccount

A BankAccount class with account number, balance, and methods for deposit/withdraw.

```
class BankAccount:
    def __init__(self, account_number, balance=0):
        self._account_number = account_number  # Protected
           attribute
        self._balance = balance                # Protected
           attribute

    # Accessors (Getters)
    def get_account_number(self):
        return self._account_number

    def get_balance(self):
        return self._balance

    # Deposit method
    def deposit(self, amount):
```

رومانی صبحی رزق جرجس مطر

```
        if amount > 0:
            self._balance += amount
        else:
            print("Amount must be positive.")

    # Withdraw method
    def withdraw(self, amount):
        if 0 < amount <= self._balance:
            self._balance -= amount
        else:
            print("Invalid withdrawal amount.")

    # Print details
    def print_account(self):
        print(f"Account Number: {self._account_number}")
        print(f"Balance: {self._balance:.2f}")

# Usage example
acc = BankAccount("123456", 100)  # Create object
acc.print_account()  # Print details

# Deposit and withdraw
acc.deposit(50)
acc.withdraw(30)

# Use getters
print("New Balance:", acc.get_balance())
acc.print_account()  # Print updated details
```

2025/2026 2025/2026 2025/2026

Output:

```
Account Number: 123456
Balance: 100.00
New Balance: 120.0
Account Number: 123456
Balance: 120.00
```

رومانی صبحی رزق جرجس مطر

رومانی صبحی رزق جرجس مطر

Aspect	Text Files	Binary Files	Simple Python Code
Data Representation	Human-readable characters (e.g., ASCII, UTF-8). Stored as strings.	Raw binary data (bytes). Not human-readable.	-
File Modes	'r', 'w', 'a', 'r+', etc.	'rb', 'wb', 'ab', 'rb+', etc.	-
Use Cases	Configuration files, logs, scripts, CSV/JSON data. When data needs to be edited manually.	Images (JPEG, PNG), videos (MP4), executables, serialized objects. When preserving exact byte structure is crucial.	-
Advantages	Easy to read/edit with text editors. Platform-independent for plain text.	Efficient for non-text data. No encoding issues.	-
Disadvantages	Inefficient for large non-text data. Encoding/decoding required.	Not readable without specialized tools. Platform-dependent sometimes (e.g., endianness).	-

Code Example	Writing and Reading Text File
	`with open('text.txt', 'w')as f: f.write('Hello World')0open('text.txt', 'r')as f: print(f.read())#Output: Hello World`
	Writing and Reading Binary File
	`with open('binary.bin', 'wb')as f: f.write(b'Hello World')0open('binary.bin', 'rb')as f: print(f.read())#Output: b'Hello World'`		

Table 1: Summary of Text and Binary Files with Differences and Code Examples

Python Programming

1- Traditional Programming

2- Machine Learning

Environment:

1- Anaconda

- Spyder
- Jypter

2- Using Kaggle

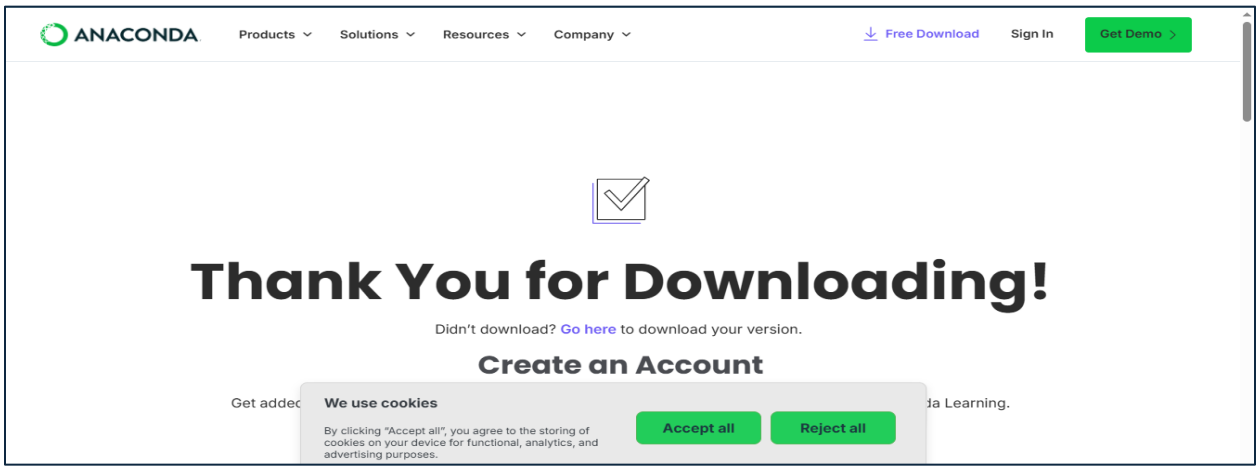
3- Using Colab

رومانی صبحی رزق جرجس مطر

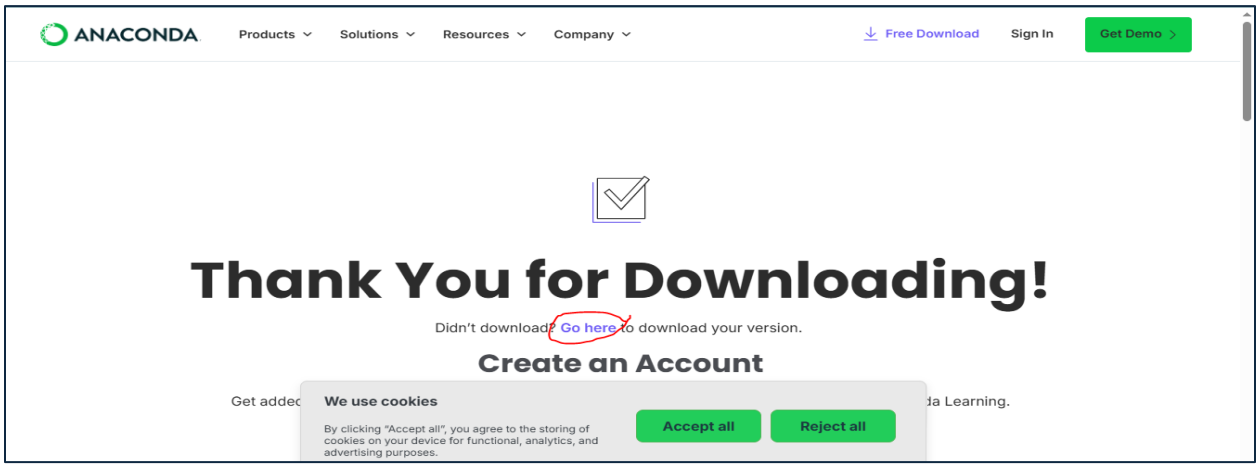
1-Using Anaconda (Spyder)

Steps to install and download Anaconda

https://www.anaconda.com/download-success

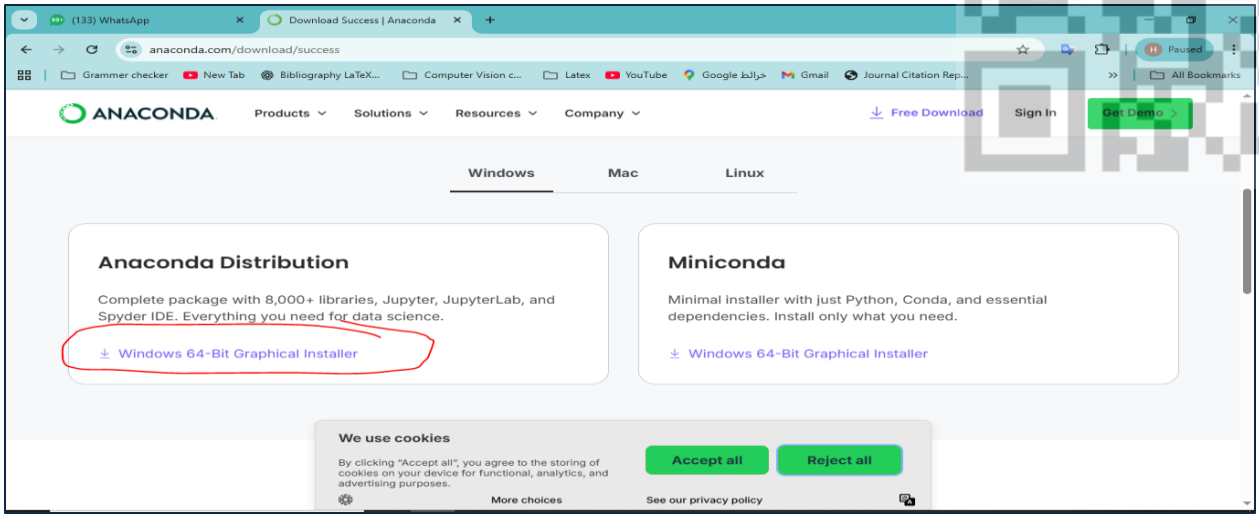


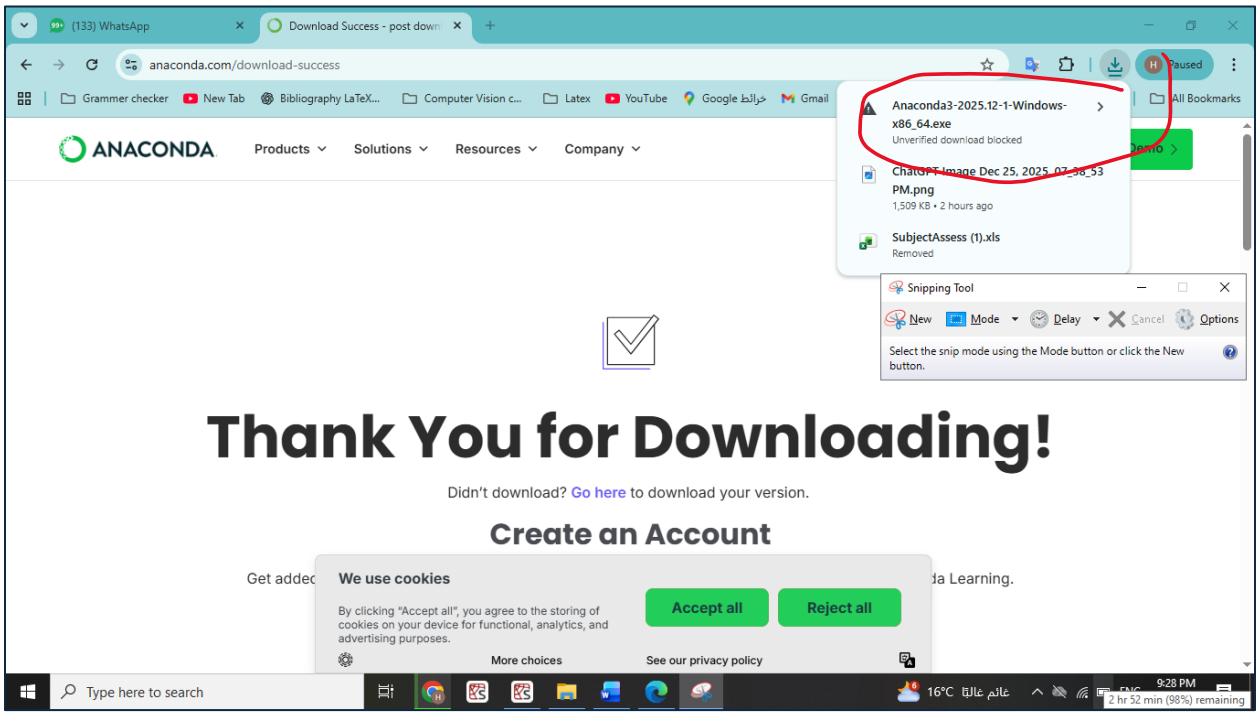
2025/2026 2025/2026 2025/2026



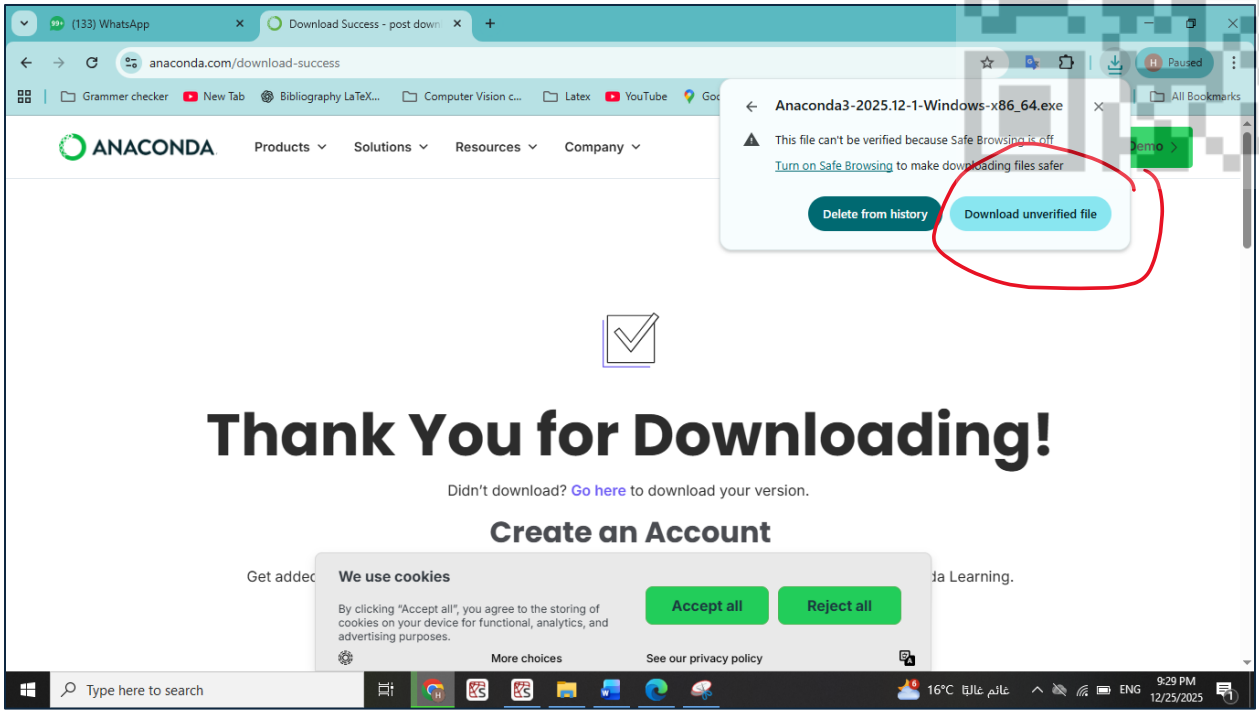
رومانی صبحی رزق جرجس مطر

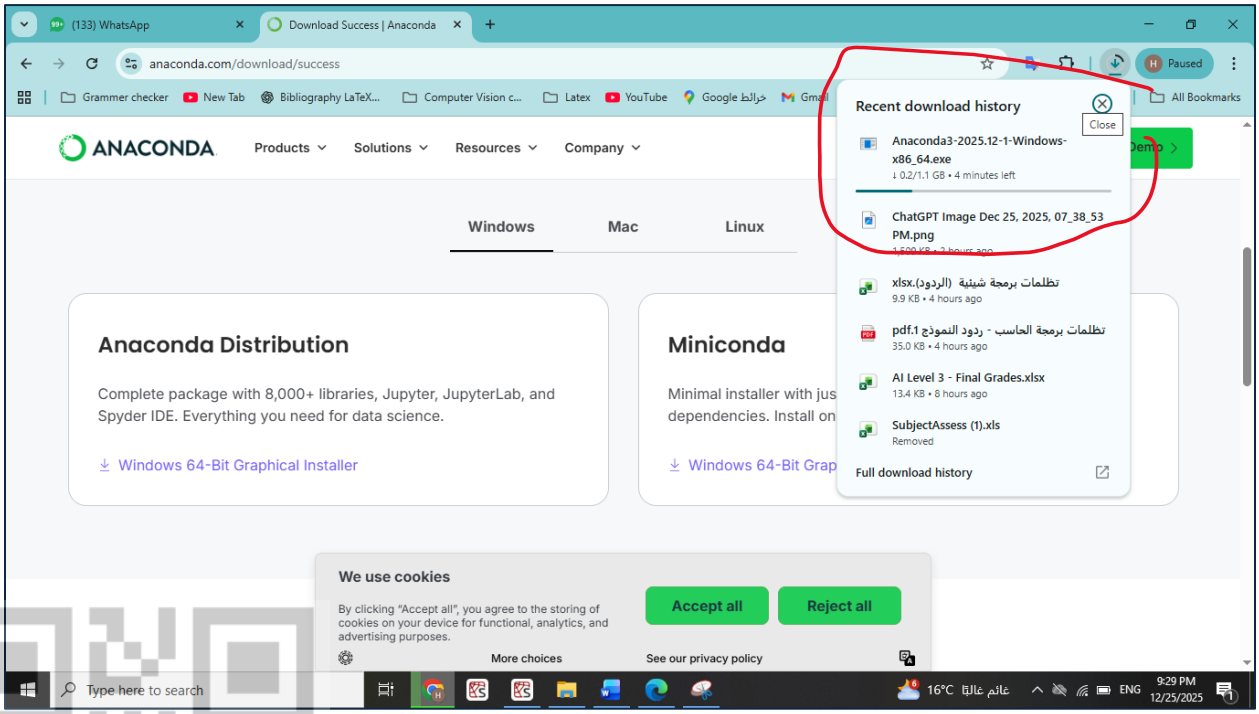
رومانی صبحی رزق جرجس مطر



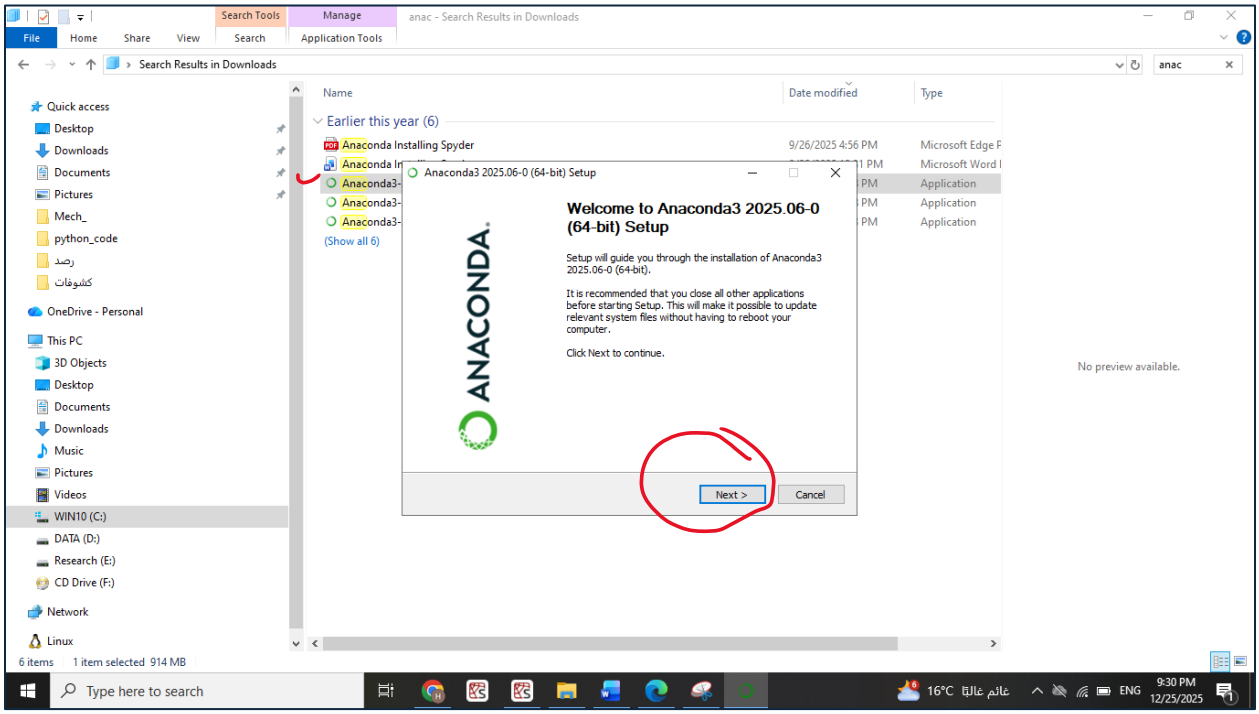


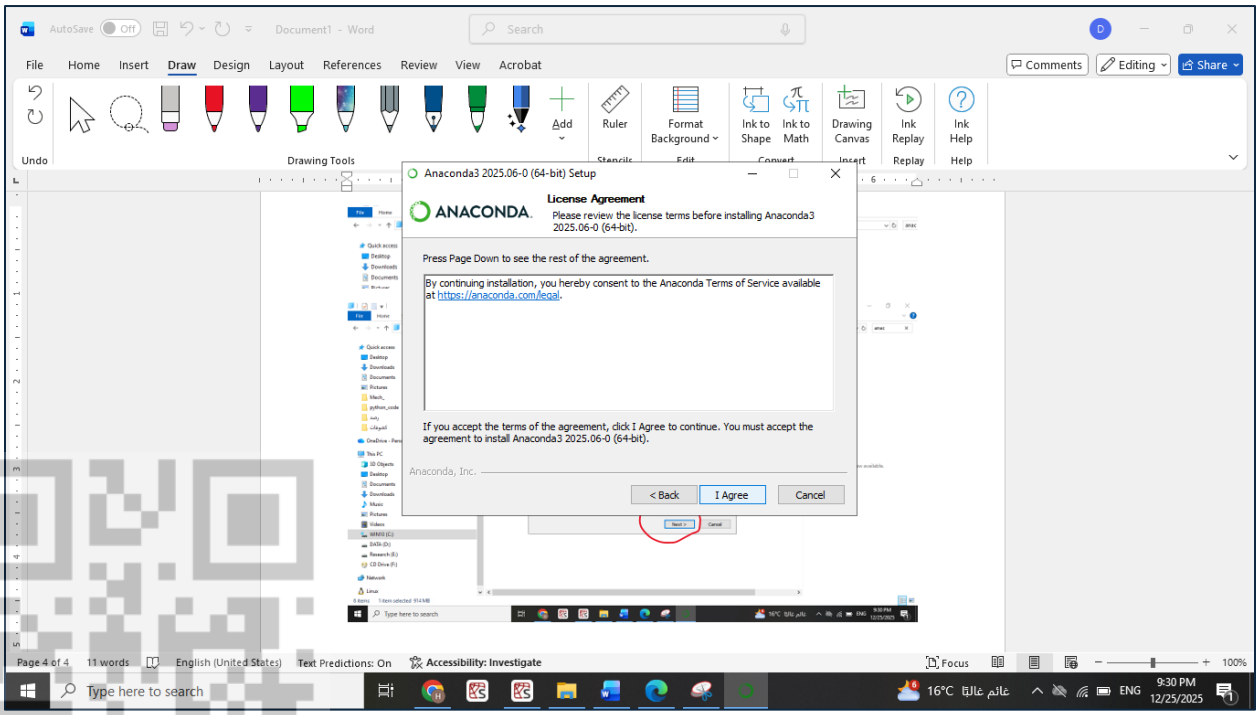
رومانی صبحی رزق جرجس مطر



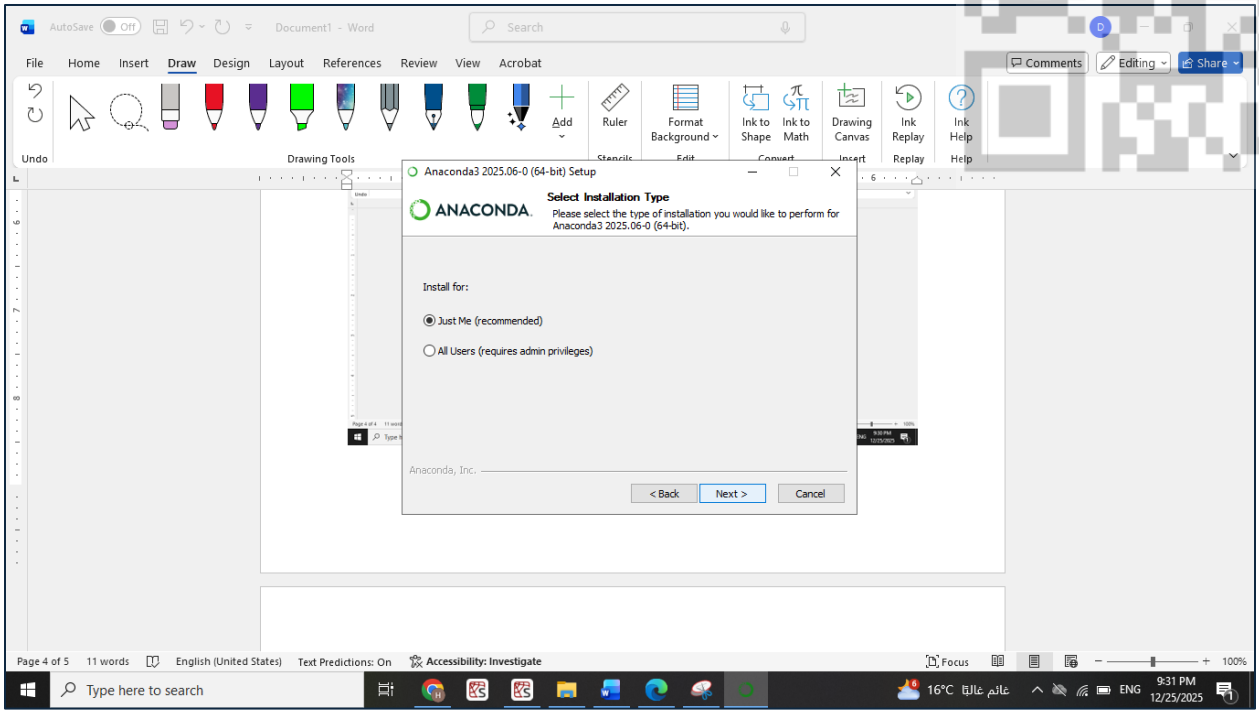


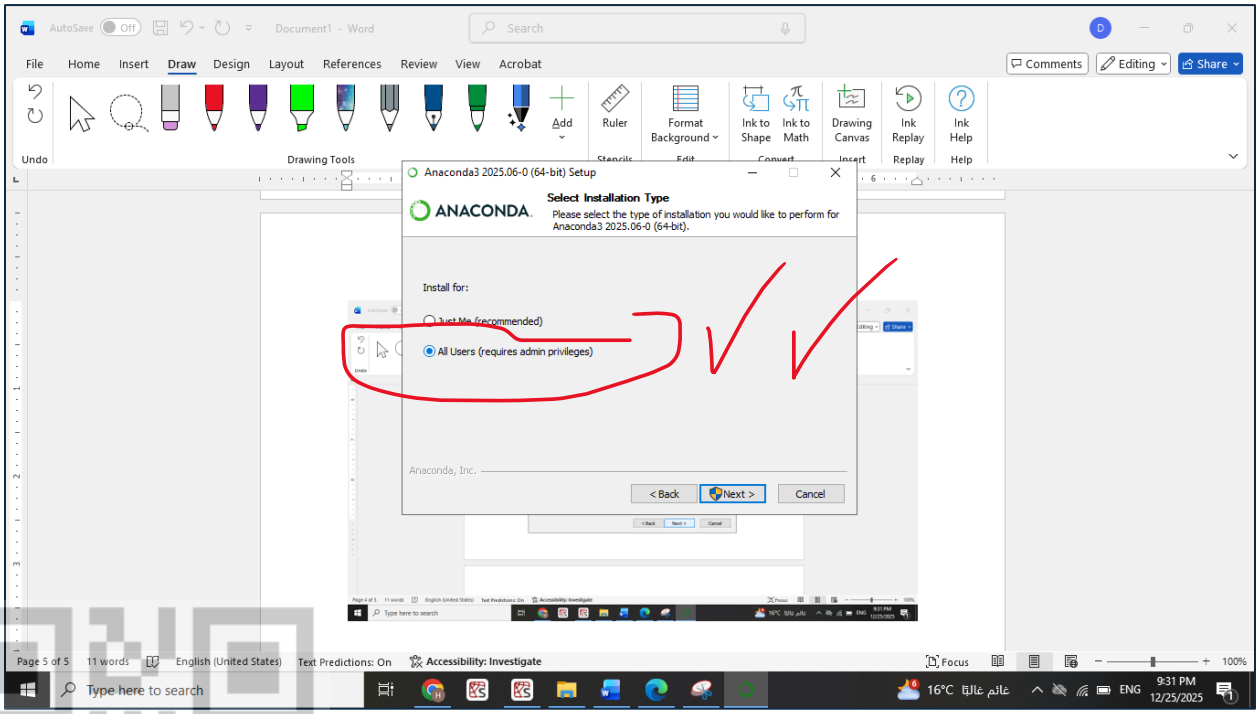






رومانی صبحی رزق برجس مطر

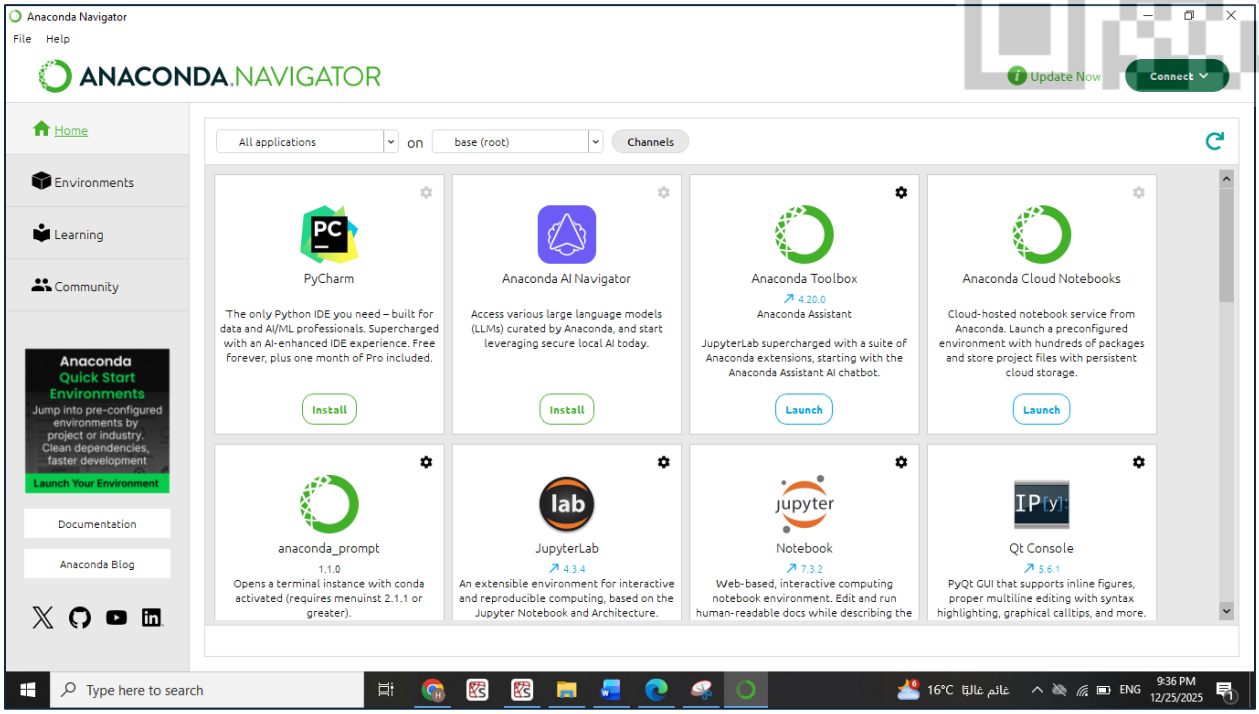


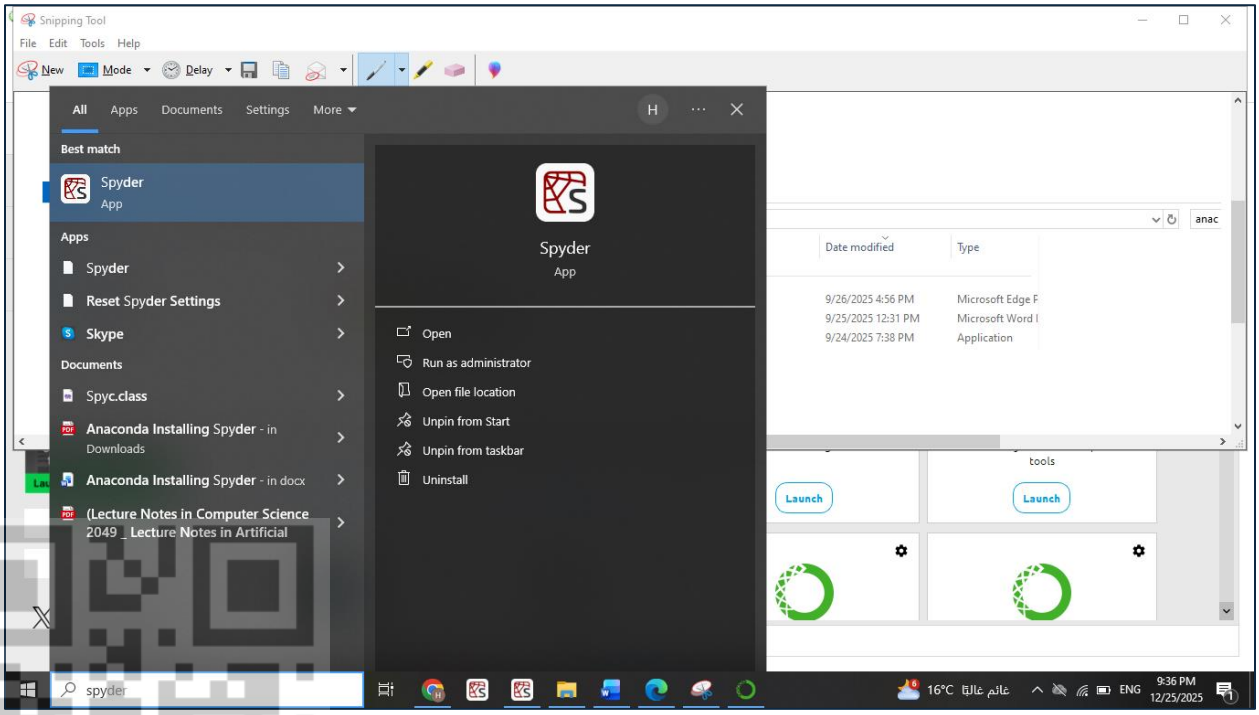


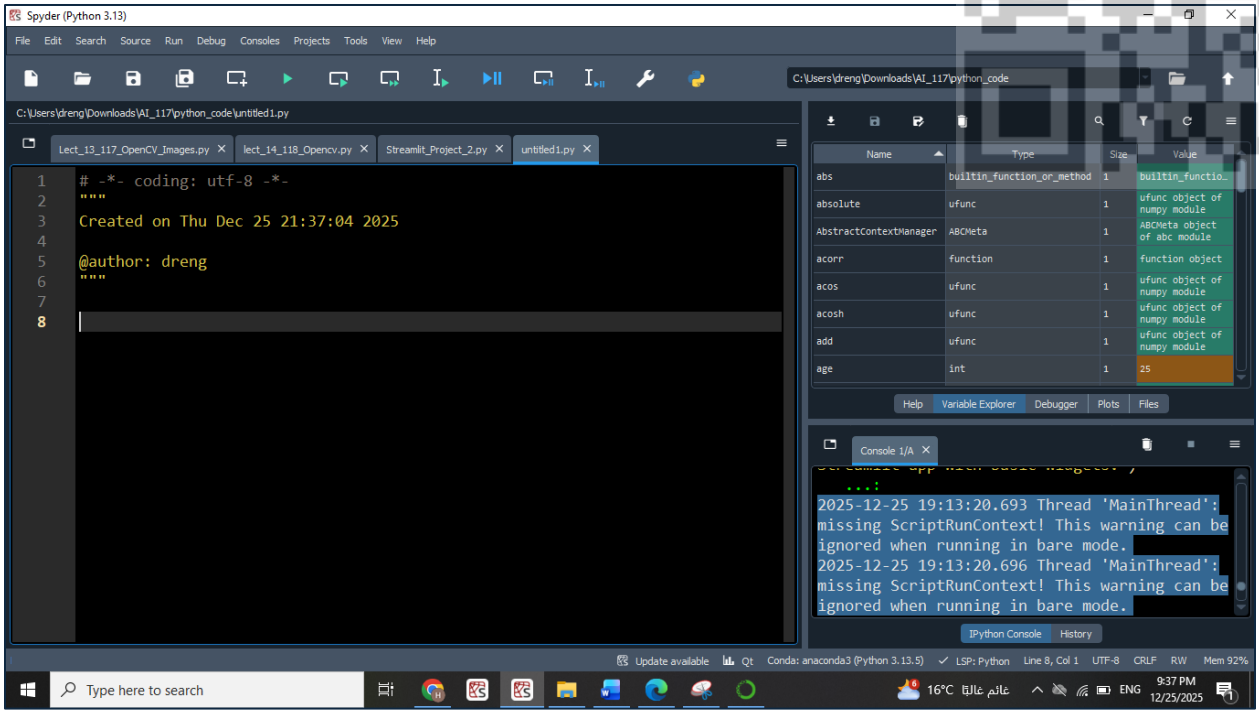
رومانی صبحی رزق جرجس مطر

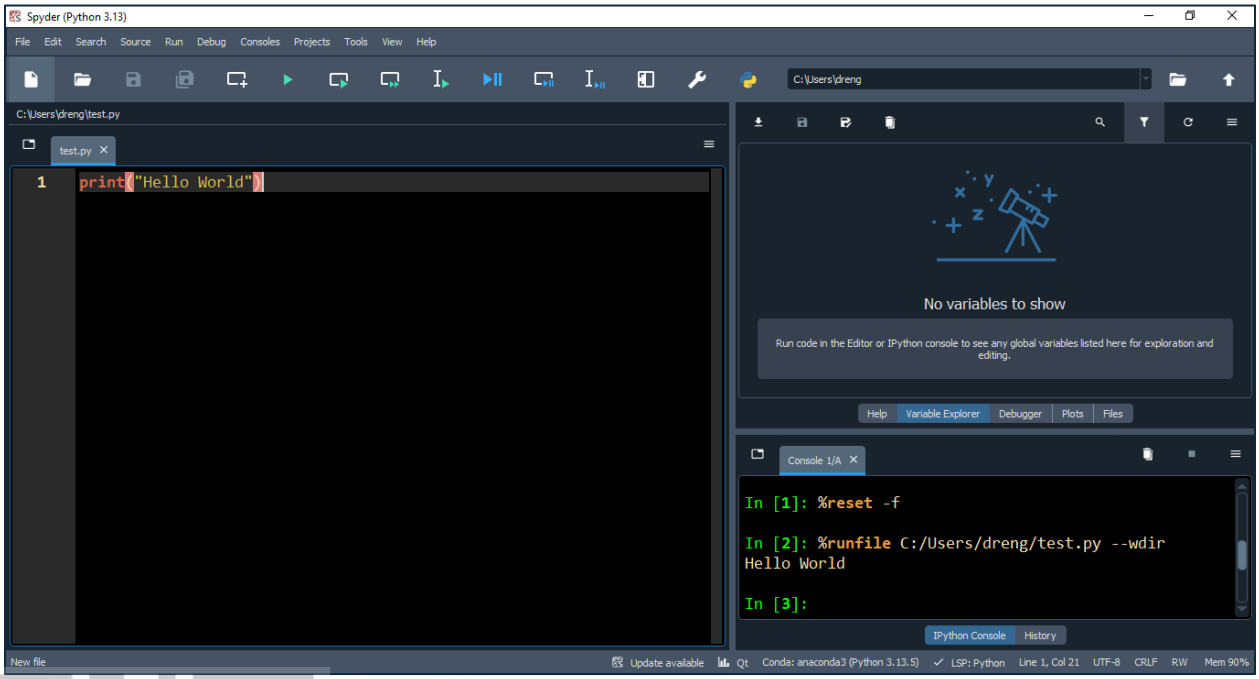
رومانی صبحی رزق برجس مطر

Opening Spyder



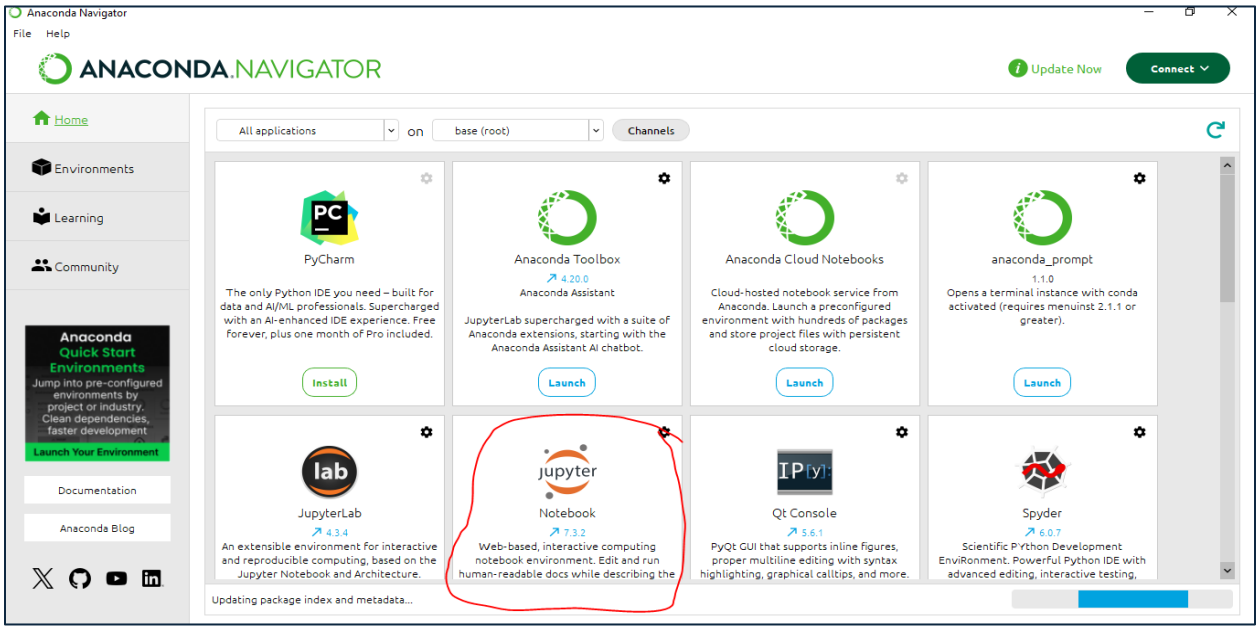




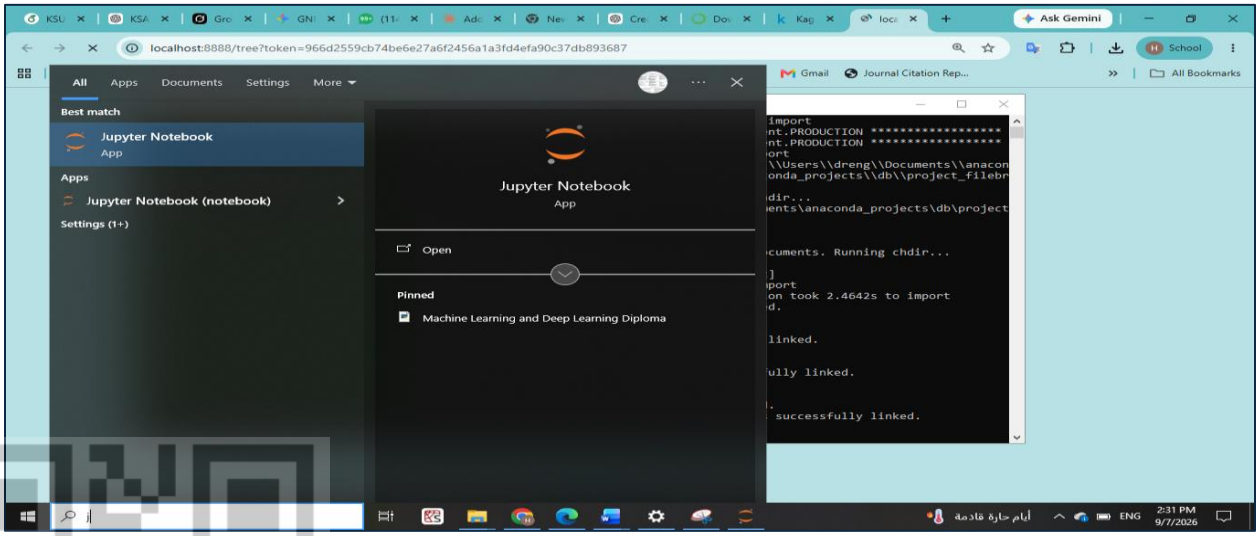


رومانی صبحی رزق جرجس مطر

2-Using Anaconda (Jupter)

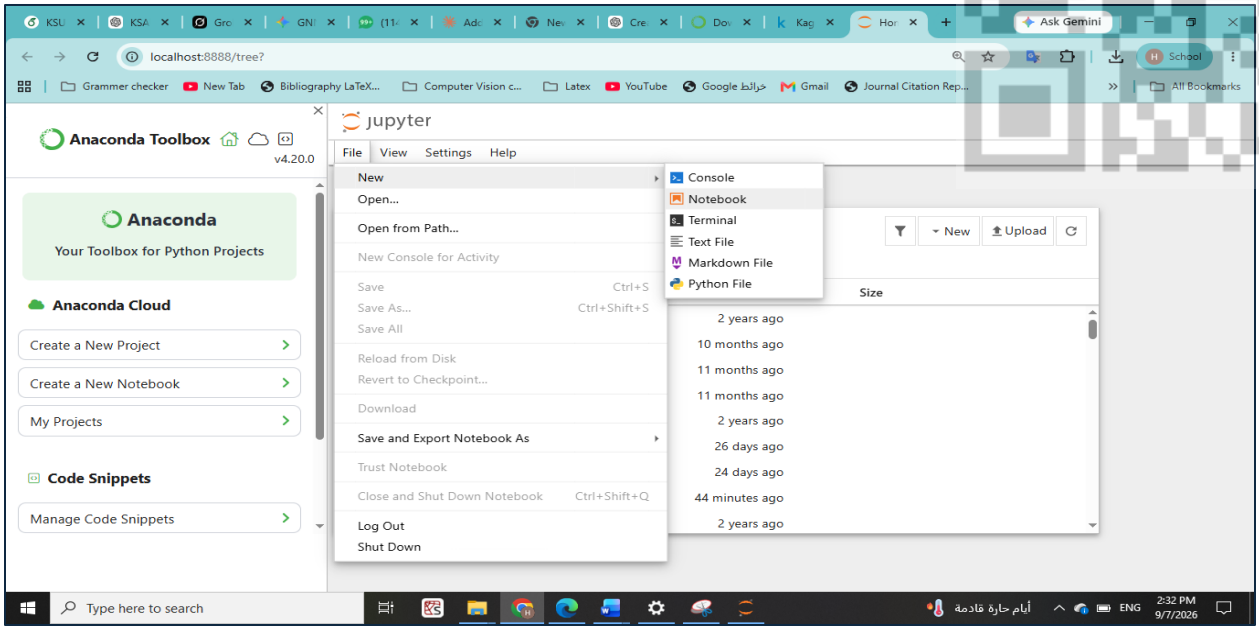


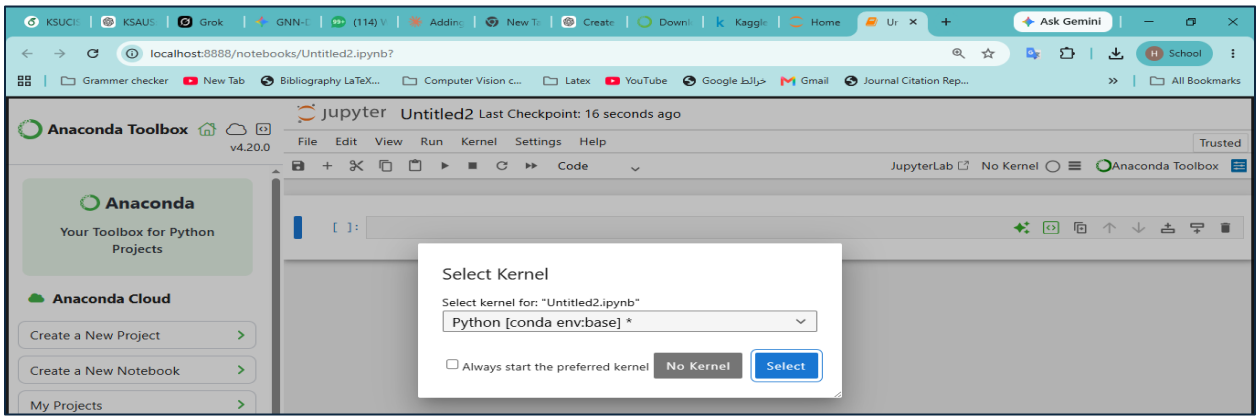
2025/2026 2025/2026 2025/2026

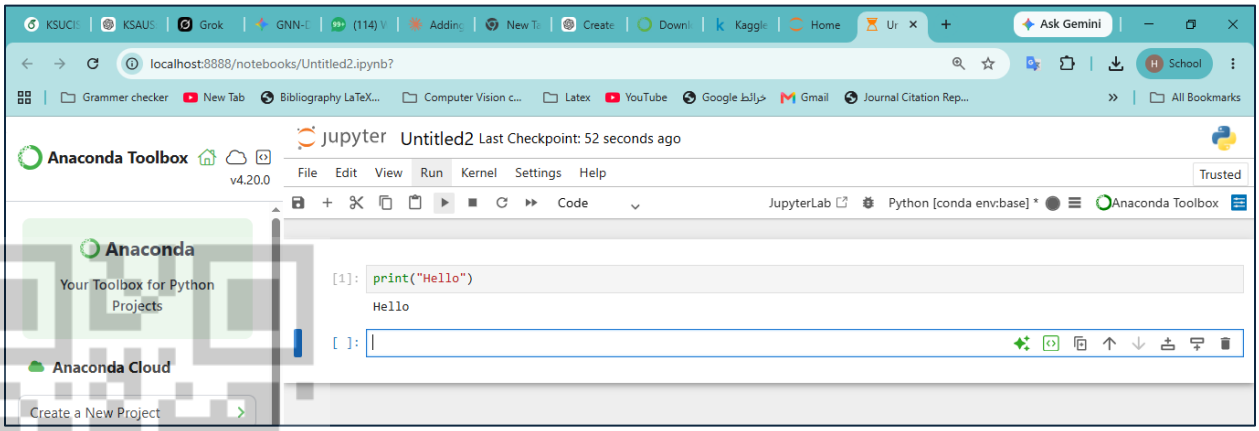


رومانی صبحی رزق جرجس مطر

رومانی صبحی رزق جرجس مطر





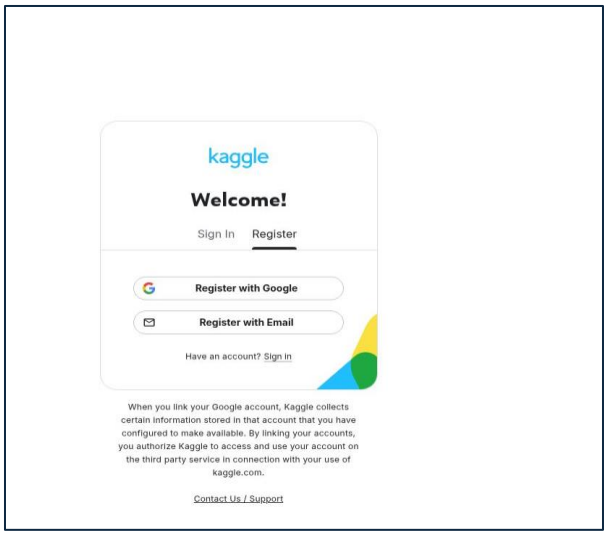


2-Using Kaggle

Create an Account and Start Using Kaggle

https://www.kaggle.com

رومانی صبحی رزق جرجس مطر



2025/2026

Kaggle registration page (captured 7 September 2026).

Practical self-study handout • Updated 7 September 2026

ممكن الدخول بحساب Gmaill

1 **Open the registration page**

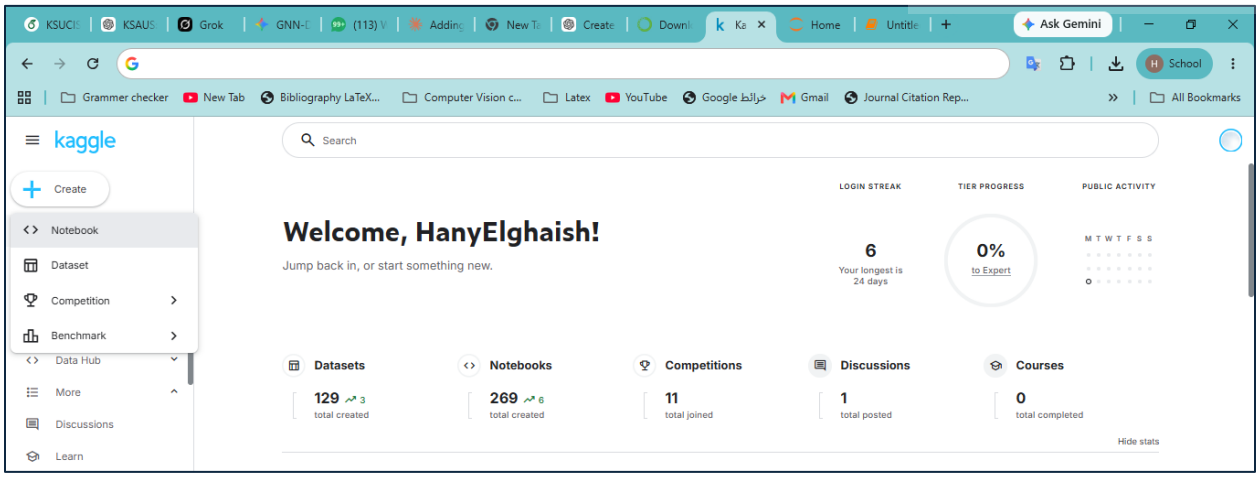
2 **Choose a registration method**

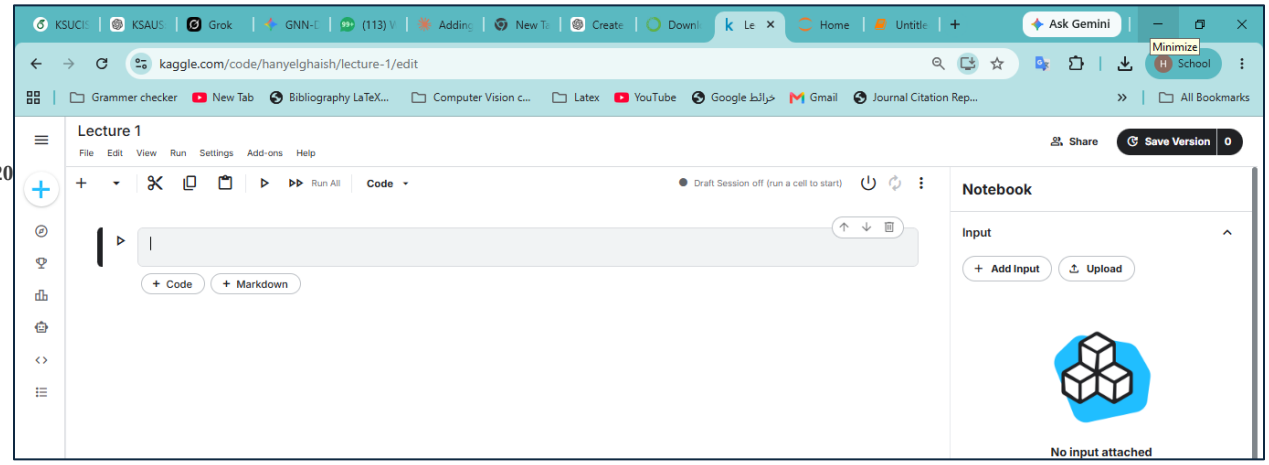
3 **Complete the verification**

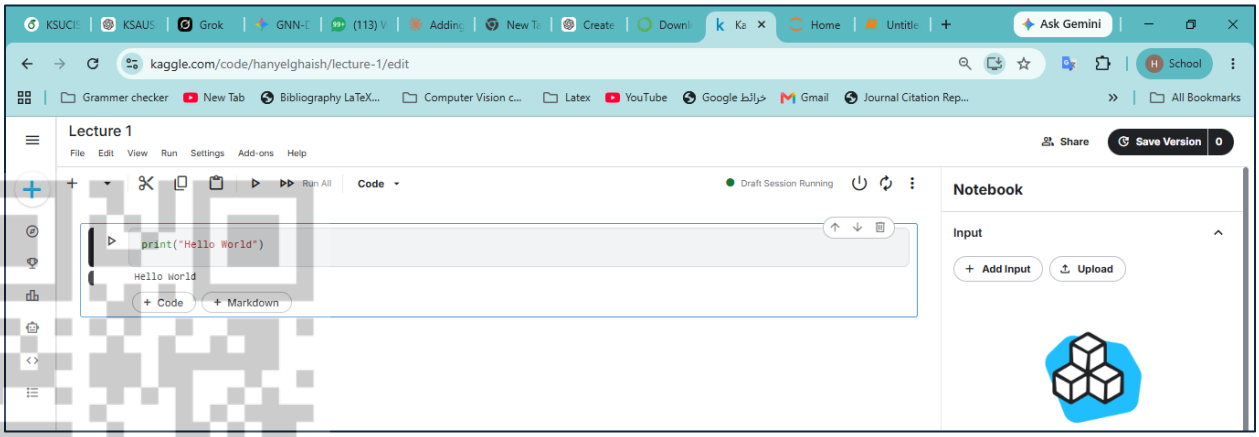
4 **Finish your profile**

Create Your First Kaggle Notebook

1 Open the Code area



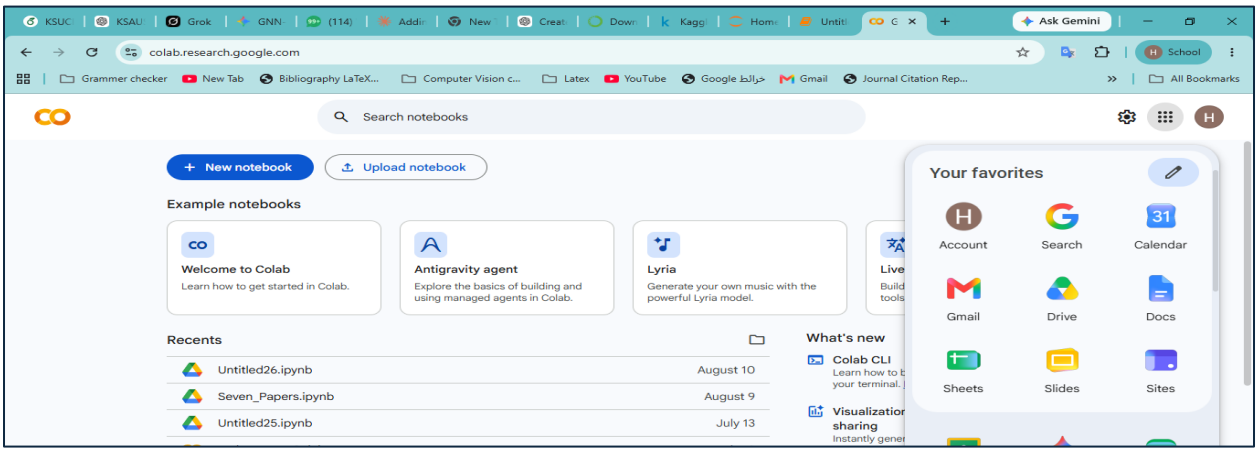


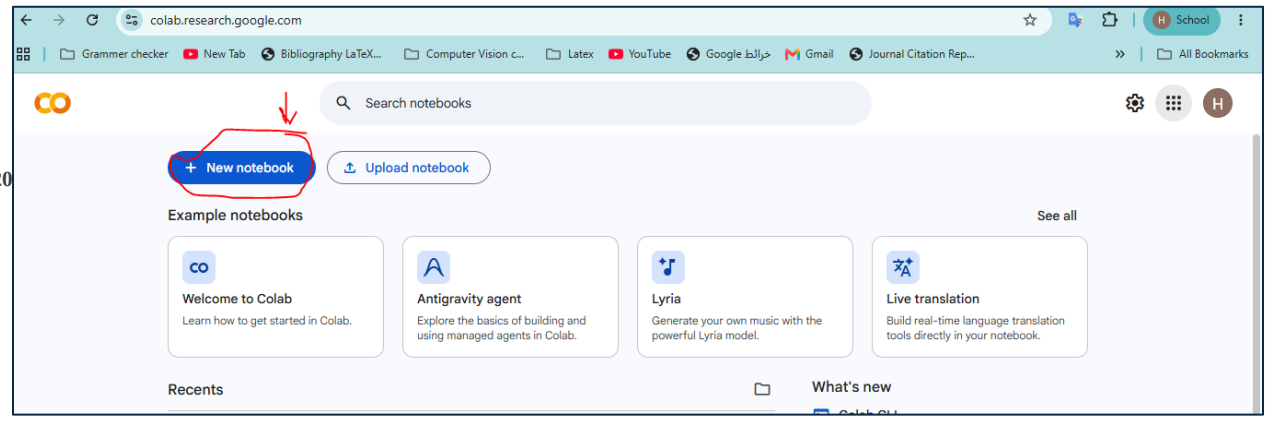


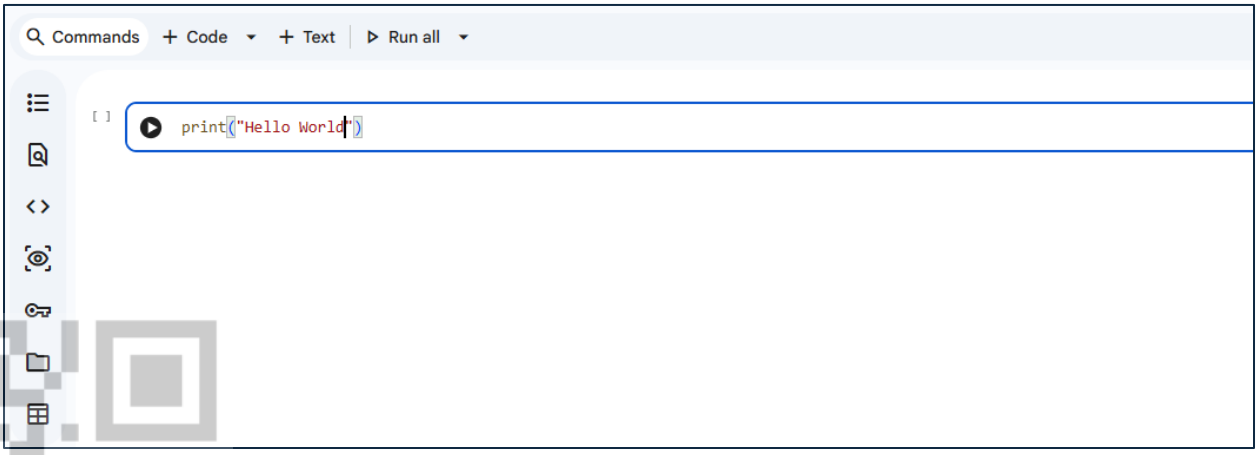
رومانی صبحی رزق جرجس مطر

4- Using Colab

https://colab.research.google.com/







2025/20

رومانی صبحی رزق جرجس مطر



رومانی صبحی رزق جرجس مطر

Assignment 1

Write a python program that asks the user to enter the cylinder radius and Height then compute the cylinder volume and print this volume on the screen.

Please execute this program on Spyder, Jupter, Kaggle, and Colab

رومانی صبحی رزق جرجس مطر

Python Lecture 2: Practice Assignments on Arithmetic, Relational, Logical Operators, and Branching

رومانی صبحی رزق جرجس مطر

Instructions

Complete the following assignments to practice the concepts covered in Lecture 2, including arithmetic operators, operator precedence, relational operators, logical operators, and branching structures (if, if-else, if-elif-else). Write, run, and test your code in a Python environment (e.g., IDLE, VS Code, or an online interpreter). Ensure proper indentation (4 spaces), handle input errors (e.g., division by zero, invalid input types), and use descriptive print statements. Submit your code files and outputs for review. If you encounter errors, note them and attempt to fix them to deepen your understanding.

1 Assignments

1. **Assignment Arithmetic Operations Calculator: Write a Python program that takes two numbers (integers or floats) as input using `input()`. Perform all arithmetic operations (+, -, *, /, //, %, **), and print each result with a clear label (e.g., "Sum: 8"). Check for division by zero before performing division, floor division, or modulus, and print an error message if the second number is zero.:** Arithmetic Operations Calculator: Write a Python program that takes two numbers (integers or floats) as input using `input()`. Perform all arithmetic operations (+, -, *, /, //, %, **), and print each result with a clear label (e.g., "Sum: 8"). Check for division by zero before performing division, floor division, or modulus, and print an error message if the second number is zero.

2025/2026 2025/2026 2025/2026

2. **Assignment Arithmetic Operator Precedence: Write a program that evaluates the following expressions with different operator precedence and prints the results with explanations in comments. Use variables and `input()` for numbers where appropriate. Test each:**
   - **5 + 4 * 2**
   - **(5 + 4) * 2**
   - **3 ** 2 + 1**
   - **10 / 5 * 2**
   - **10 % 3 // 2**

   Explain in comments why the results differ based on precedence.: Arithmetic Operator Precedence: Write a program that evaluates the following expressions with different operator precedence and prints the results with explanations in comments. Use variables and `input()` for numbers where appropriate. Test each:
   - 5 + 4 * 2

- (5 + 4) * 2
- 3 ** 2 + 1
- 10 / 5 * 2
- 10 % 3 // 2

Explain in comments why the results differ based on precedence.

3. **Assignment Data Type Compatibility: Write a program to test the following statements about arithmetic operators and data types using try-except blocks. For each, attempt the operation, catch any errors, and print whether the statement is True or False with a brief explanation:**
   - **Addition (+) works on strings (e.g., "5" + "3").**
   - **Multiplication (*) works on strings and integers (e.g., "hello" * 3).**
   - **Division (/) works on strings (e.g., "10" / "2").**
   - **Addition (+) works on integers and floats (e.g., 5 + 3.2).**
   - **Exponentiation (**) works on floats (e.g., 2.5 ** 2).**

   : Data Type Compatibility: Write a program to test the following statements about arithmetic operators and data types using try-except blocks. For each, attempt the operation, catch any errors, and print whether the statement is True or False with a brief explanation:
   - Addition (+) works on strings (e.g., "5" + "3").
   - Multiplication (*) works on strings and integers (e.g., "hello" * 3).
   - Division (/) works on strings (e.g., "10" / "2").

2025/2026 2025/2026 2025/2026

   - Addition (+) works on integers and floats (e.g., 5 + 3.2).
   - Exponentiation (**) works on floats (e.g., 2.5 ** 2).
4. **Assignment Relational and Logical Operators: Write a program that takes a number as input and uses relational and logical operators to check the following conditions. Print True or False for each with a descriptive message:**
   - **Is the number between 1 and 10 (inclusive)?**
   - **Is the number positive and even?**
   - **Is the number not equal to zero or greater than 100?**

   Use try-except to handle invalid inputs.: Relational and Logical Operators: Write a program that takes a number as input and uses relational and logical operators to check the following conditions. Print True or False for each with a descriptive message:
   - Is the number between 1 and 10 (inclusive)?
   - Is the number positive and even?
   - Is the number not equal to zero or greater than 100?

   Use try-except to handle invalid inputs.
5. **Assignment Simple Branching with if: Write a program that takes a number as input and uses an `if` statement to check if it is positive. If true, print "The number is positive." Ensure proper indentation (4 spaces). Test with positive,**

رومانی صبحی رزق جرجس مطر

negative, and zero inputs.: Simple Branching with if: Write a program that takes a number as input and uses an `if` statement to check if it is positive. If true, print "The number is positive." Ensure proper indentation (4 spaces). Test with positive, negative, and zero inputs.

6. **Assignment if-else Voting Eligibility: Write a program that takes a persons age as input and uses an `if-else` statement to check if they are eligible to vote (age $\geq$ 18). Print "You are eligible to vote." or "You are not eligible to vote." Use try-except for invalid inputs and ensure correct indentation.:** if-else Voting Eligibility: Write a program that takes a persons age as input and uses an `if-else` statement to check if they are eligible to vote (age $\geq$ 18). Print "You are eligible to vote." or "You are not eligible to vote." Use try-except for invalid inputs and ensure correct indentation.

7. **Assignment if-elif-else Grading System: Write a program that takes a score (0100) as input and uses an `if-elif-else` structure to assign a grade: A (90+), B (8089), C (7079), or Fail (below 70). Print the grade with a message (e.g., "Grade: A"). Use try-except for invalid inputs and ensure proper indentation.:** if-elif-else Grading System: Write a program that takes a score (0100) as input and uses an `if-elif-else` structure to assign a grade: A (90+), B (8089), C (7079), or Fail (below 70). Print the grade with a message (e.g., "Grade: A"). Use try-except for invalid inputs and ensure proper indentation.

8. **Assignment Rectangle Calculations: Write a sequential program that takes the length and width of a rectangle as input. Calculate and print the area (length * width) and perimeter (2 * (length + width)). Use try-except to handle invalid inputs (e.g., non-numeric values).:** Rectangle Calculations: Write a sequential program that takes the length and width of a rectangle as input. Calculate and print the area (length * width) and perimeter (2 * (length + width)). Use try-except to handle invalid inputs (e.g., non-numeric values).

2025/2026 2025/2026 2025/2026

9. **Assignment Simple Interest Calculator: Write a sequential program that takes the principal amount, annual interest rate (as a decimal), and time (in years) as input. Calculate the simple interest using the formula (principal * rate * time) / 100, and print the result. Ensure proper input validation with try-except.:** Simple Interest Calculator: Write a sequential program that takes the principal amount, annual interest rate (as a decimal), and time (in years) as input. Calculate the simple interest using the formula (principal * rate * time) / 100, and print the result. Ensure proper input validation with try-except.

10. **Assignment Tax with Dependents: Write a program that combines sequential and branching structures. Take the users annual income and number of dependents as input. Calculate a tax amount (income * 0.15). Use an `if-elif-else` structure to reduce the tax: 20% reduction for 3 or more dependents, 10% reduction for 1 or 2 dependents, no reduction otherwise. Print the income, initial tax, reduction, and final tax. Use try-except and proper indentation.:** Tax with Dependents: Write a program that combines sequential and branching structures. Take the users annual income and number of dependents as input. Calculate a tax amount (income * 0.15). Use an `if-elif-else` structure to reduce the tax: 20% reduction for 3 or more dependents, 10% reduction for 1 or 2 dependents, no reduction otherwise. Print the income, initial tax, reduction, and final tax. Use try-except and proper indentation.

رومانی صبحی رزق جرجس مطر

Submission Guidelines

- Save each assignment as a separate Python file (e.g., `assignment1.py`, `assignment2.py`).
- Include comments in your code to explain your logic, especially for operator precedence, try-except blocks, and branching conditions.
- Run each program with at least two different sets of inputs and note the outputs or any errors.
- Submit your code files and a document with the outputs or error messages for each assignment.

رومانی صبحی رزق جرجس مطر

Assignment 4

Task 1 — While Loop: Factorial

Write a program that:

1. Prompts the user for a single integer n.
2. If n is negative or not an integer, asks again until a valid non-negative integer is entered.
3. Uses a while loop to compute n! (factorial).
4. Prints the result in the form: n! = value.

Sample run

Enter a non-negative integer: -3

Invalid input. Try again.

Enter a non-negative integer: 5

5! = 120 = 5 * 4 * 3 *2 * 1

Task 2 — For Loop with continue

Write a program that prints the numbers from 1 to 20 inclusive, **skipping** numbers divisible by 5.
Use a for loop and continue.

Expected output

1 2 3 4 6 7 8 9 11 12 13 14 16 17 18 19

رومانی صبحی رزق جرجس مطر

Task 3 — Nested Loops: Triangle Pattern

File: triangle_pattern.py

Write a program that prints the following numeric triangle using **nested** for loops.
For a user input rows, print:

Example for rows = 3:

1

22

333

Input rules:

- rows must be an integer in the range [1, 20]. Keep asking until valid.

Try This code

```
rows = int(input("Enter Number of rows"))
for i in range(1,rows):
    for j in range (1,i+1):
        print(i, end="")

    print()
```

2025/20

رومانی صبحی رزق جرجس مطر

Task 4:

Write then run this Program to compute the grade of students

```
Answer = input("Enter Y to Compute Grade or other letter to exist: ")
while Answer == "Y":


    print("Computing Grade for Student: ")
    d1 = float(input("Enter First degree out of 10: "))
    d2 = float(input("Enter Second degree out of 10 : "))
    d3 = float(input("Enter Third degree out of 10: "))

    sum = d1 + d2 + d3
    print("total degree = " , sum)
    percent = sum / 30  * 100
    print("Percentage %= " , percent)

    if (percent >= 0 and percent<=100):

        if(percent>=85 and percent <= 100):
            print("Excellent")
        elif(percent>=75 and percent <85):
            print("Very Good")
        elif(percent>=65 and percent <75):
            print(" Good")
        elif(percent>=50 and percent <65):
            print("Pass")
        else:
            print("Fail")

    else:
        print("You Must Enter all your degrees between 0 and 10")


    Answer = input("Enter Y to continue or N to exist: ")


print("--- END --- ")
```

رومانی صبحی رزق جرجس مطر

Assignment 4: User Defined Functions

Instructions
- Write Python programs as required in each task.
- Ensure that your functions are properly named, indented, and tested with sample inputs.
- Each function should be written in a separate code block.
- Document your code with comments for clarity.

Question 1
Define a function that receives a student's score (integer) as input and prints the corresponding grade based on the following scale:

- A: 90–100
- B: 80–89
- C: 70–79
- D: 60–69
- F: Below 60

After defining the function, call it with at least two test values to verify that it works correctly.

Question 2
Write a function that receives any positive integer number and determines whether the number is **even** or **odd**. The function should return the result as a string.

Question 3
Create a function that generates and prints the **multiplication table** for a given number up to 10. For example, if the input is 5, the function should print the multiplication table of 5.

2025/2026 2025/2026 2025/2026

Question 4
Develop a program that contains four separate functions, each performing one arithmetic operation:

1. A function to **add** two numbers and return the result.
2. A function to **subtract** two numbers and return the result.
3. A function to **multiply** two numbers and return the result.
4. A function to **divide** two numbers and return the result.

Note: The division function should check that the denominator is not zero before performing the operation.

رومانی صبحی رزق جرجس مطر

Assignment # 5

Task 1: write this code in your machines

```
#%%
# Write a program that do the following:
    # 1- Function: receives two numbers and return their sum
    # 2- Function: to print the even numbers from 0 : 30
    # 3- Function  to add number from specific start till secific end and return this sum
    # 4- Function to check and print if a given number is Even or odd
    # 5- Function that receives the student percentage and print his/her grade
    # Write a main program that call all of these functions
```

```python
# fn1: add two numbers
def Add(num1, num2):
    sum = num1+num2
    return sum

# fn2: print even numbers: 0-30
def printEven():
    for i in range(0,31,2):
        print(f"{i} is Even")

#fn3: add range(start : end)
def AddRange(start, end):
    sum = 0
    for i in range(start,end+1):
        sum = sum + i

    return sum
```

رومانی صبحی رزق جرجس مطر

رومانی صبحی رزق جرجس مطر

```python
#fn4: check Even/Odd
def CheckEvenOdd(num):
    if num % 2 == 0:
        print(f"{num} is Even")
    else:
        print(f"{num} is Odd")

#fn5: print student grade on the received percentage

def PrintStudentGrade(percent):

    if percent >=0 and percent <= 100:


        if percent >=85 and percent <= 100:
            print("Excellent ")
        elif percent >=75 and percent <85:
            print("Very Good ")
        elif percent >=65 and percent <75:
            print("Good ")
        elif percent >=50 and percent <65:
            print("Pass ")
        else:
            print("Fail")
    else:
        print("You must enter your percent between 0 and 100")
```

رومانی صبحی رزق جرجس مطر

```
#-----------------------------------------------------
# Calling
# calling fn1
num1= float(input("Enter num 1: "))
num2= float(input("Enter num 2: "))
add = Add(num1, num2)
print(f"{num1}+ {num2} = {add}")

# ----------------------
# calling fn2
printEven()

#---------------------
# calling fn3
start = int(input("Enter your int start: "))
end = int(input("Enter your int end: "))

summation = AddRange(start, end)

print(f"sum from {start}: {end} = {summation}")

#---------------------
# calling fn4
num = int(input("Enter a number to check if it is Even or ODD: "))
CheckEvenOdd(num)
```

```
#-----------------------------
# calling fn5
# compute grade for 5 students

for i in range(0,5):

    percent= float(input("Enter your percent between 0 and 100: "))
    PrintStudentGrade(percent)
```

Task 2: using the four types of creating the functions, write functions to compute the cylinder volumes:

Type 1: function that receives the radius and height and return the cylinder volume

Type 2: function that receives the radius and height and print the cylinder volumes

Type 3: function that does not take parameters and return the cylinder volume

Type 4: function that does not take parameters and print the cylinder volume

رومانی صبحی رزق جرجس مطر

Assignment 7: Create a Cylinder Class in Python

Objective

Design and implement a Python class named `Cylinder` to represent a cylinder with properties like radius and height. The class should include methods to manage its attributes, perform calculations, and display information. This exercise will help you practice creating user-defined data types and applying object-oriented programming (OOP) principles.

Instructions

1. Create a class called `Cylinder`.
2. Define a constructor (`__init__`) that initializes the cylinder with a `radius` and `height`. Ensure both values are positive numbers (use error handling to validate this).
3. Implement the following methods:
   - Getter methods to retrieve the `radius` and `height`.
   - Setter methods to update the `radius` and `height` with validation for positive values.
   - A method to calculate the `volume` of the cylinder (use the formula: $V = \pi \cdot r^2 \cdot h$).
   - A method to calculate the `surface_area` of the cylinder (use the formula: $SA = 2 \cdot \pi \cdot r \cdot (r + h)$).
   - A method to `print_details` that displays the cylinder's radius, height, volume, and surface area.
4. Add appropriate comments to explain your code.
5. Test your class by creating at least two cylinder objects with different dimensions, using the setters to modify values, and calling the `print_details` method to verify the results.
6. Submit your Python script with the class definition and test code.

Requirements

- Use the `math` module for the value of π.
- Implement basic error handling (e.g., raise a `ValueError` if radius or height is negative or zero).
- Use protected attributes (e.g., `_radius`, `_height`) to follow encapsulation principles.

2025/2026 2025/2026 2025/2026
رومانی صبحی رزق جرجس مطر

Example Output

For a cylinder with an initial radius of 3 and height of 5, and after modifying to radius 4 and height 6, the output might look like:

```
Radius: 3, Height: 5
Volume: 141.37
Surface Area: 150.80
Radius: 4, Height: 6
Volume: 301.59
Surface Area: 251.33
```

Submission

- Save your code in a file named `cylinder_assignment.py`.
- Include your name and date in a comment at the top of the file.
- Submit the file by the due date: October 15, 2025.

Evaluation Criteria

- **Correctness (40%)**: All methods work as specified, with accurate calculations and proper error handling.
- **OOP Principles (30%)**: Use of encapsulation (protected attributes), clear method structure, and appropriate use of `self`.

2025/2026 2025/2026 2025/2026

- **Code Quality (20%)**: Includes comments, follows Python naming conventions, and is well-organized.
- **Testing (10%)**: Includes a test section with at least two cylinder objects and demonstrates all methods.

رومانی صبحی رزق جرجس مطر

رومانی صبحی رزق جرجس مطر

Assignments for Python Lecture 6: File Handling and Exception Handling

September 17, 2025

1 Assignments

1.1 Assignment 1

Write a program that reads the content of a text file specified by the user. Include exception handling for FileNotFoundError and print an appropriate message if the file does not exist.

1.2 Assignment 2

Create a program that writes a list of strings to a binary file using 'wb' mode and then reads the binary data back, printing it as bytes. Handle any IOError that may occur during the process.

1.3 Assignment 3

Develop a function that appends user input to an existing text file. Use exception handling to catch and manage any permission-related errors or other IOExceptions, ensuring the program doesn't crash.

1.4 Assignment 4

Implement a script that opens a file, uses seek to move to a specific position (e.g., the middle of the file), reads from there, and handles exceptions like ValueError for invalid seek positions or FileNotFoundError if the file is missing.

رومانی صبحی رزق جرجس مطر

2 Answers

2.1 Answer to Assignment 1

This program prompts the user for a file name, attempts to read its content, and handles the FileNotFoundError with a custom message. The with statement ensures the file is properly closed.

```
file_name = input("Enter the file name: ")
try:
    with open(file_name, "r") as file:
        content = file.read()
        print("File content:", content)
except FileNotFoundError:
    print(f"The file '{file_name}' was not found.")
```

Expected Output (if file exists, e.g., "example.txt" with "Hello"): File content: Hello Expected Output (if file does not exist): The file 'example.txt' was not found.

2.2 Answer to Assignment 2

This program writes a list of strings converted to bytes into a binary file and reads it back, handling potential IOError. The binary data is printed as bytes.


```
data = ["Hello", "World"]
try:
    with open("binary_data.bin", "wb") as file:
        for item in data:
            file.write(item.encode() + b"\n")
    with open("binary_data.bin", "rb") as file:
        binary_content = file.read()
        print("Binary content:", binary_content)
except IOError as e:
    print(f"An IO error occurred: {e}")
```

Expected Output: Binary content: b'Hello' (may vary slightly based on encoding and newline handling).

2.3 Answer to Assignment 3

This function takes user input and appends it to a text file, handling permission issues or other IOExceptions with a try-except block.

```
def append_to_file(file_name):
    try:
```

```
        with open(file_name, "a") as file:
            user_input = input("Enter text to append: ")
            file.write(user_input + "\n")
            print("Text appended successfully.")
    except IOError as e:
        print(f"Error appending to file: {e}")

# Example usage
append_to_file("output.txt")
```

Expected Output (if successful): Enter text to append: Test Text appended successfully. Expected Output (if permission denied): Error appending to file: [Errno 13] Permission denied: 'output.txt'.

2.4 Answer to Assignment 4

This script opens a file, moves to the middle using seek, reads from there, and handles exceptions for invalid seek positions or missing files.

```
file_name = input("Enter the file name: ")
try:
    with open(file_name, "r") as file:
        file.seek(0, 2)  # Move to end
        size = file.tell()
        middle = size // 2
        file.seek(middle)
        content = file.read()
        print("Content from middle:", content)
except FileNotFoundError:
    print(f"The file '{file_name}' was not found.")
except ValueError as e:
    print(f"Invalid seek position: {e}")
except IOError as e:
    print(f"IO error: {e}")
```

Expected Output (if file exists, e.g., "example.txt" with "Hello World"): Content from middle: World Expected Output (if file does not exist): The file 'example.txt' was not found.

رومانی صبحی رزق جرجس مطر

Python File Handling — Assignment

Important Note for Students: For this assignment, do not consider exception handling. Focus only on implementing the required file handling logic. Exception handling will be introduced in the next assignment.

Assignment 1

Write a program that reads the content of a text file specified by the user and prints its content on the screen. Do not add exception handling at this stage.

Assignment 2

Create a program that writes a list of strings to a binary file using 'wb' mode and then reads the binary data back, printing it as bytes. Do not use exception handling.

Assignment 3

Develop a function that appends user input to an existing text file. After appending, read and display the updated content. Ignore exceptions for now.

Assignment 4

Implement a script that opens a file, uses seek to move to a specific position (e.g., the middle of the file), reads from there, and prints the result. Skip exception handling in this task.

رومانی صبحی رزق جرجس مطر

Helpful Reference (No Exception Handling)

Use these minimal examples to guide your solutions. They show core file operations without try/except blocks.

Common Text & Binary Modes (Cheat Sheet)

Mode	Meaning	Notes
r	Read text	File must exist
w	Write text	Create or overwrite
a	Append text	Create if missing
r+	Read & write text	File must exist
w+	Write & read text	Create or overwrite
a+	Append & read text	Create if missing
rb	Read binary	File must exist
wb	Write binary	Create or overwrite
ab	Append binary	Create if missing

1) Read a Text File (Assignment 1)

```
# Read a user-specified text file and print its contents
path = input("Enter text file path: ")
f = open(path, "r")
content = f.read()
print(content)
f.close()
```


2) Write List of Strings to Binary and Read Back (Assignment 2)

```
# Write a list of strings as bytes, then read it back
lines = ["Line 1\n", "Line 2\n", "Line 3\n"]
# Encode to bytes and write
with open("data.bin", "wb") as f:
    for s in lines:
        f.write(s.encode("utf-8"))

# Read back as bytes
with open("data.bin", "rb") as f:
    data = f.read()
print(data)              # e.g., b'Line 1\nLine 2\nLine 3\n'
```

3) Append User Input to a Text File (Assignment 3)

```
# Append new text to an existing file and then show new content
path = input("Enter existing text file path: ")
user_text = input("Enter a line to append: ")

# Append
with open(path, "a") as f:
    f.write(user_text + "\n")

# Read and display updated content
with open(path, "r") as f:
    print(f.read())
```

4) Use seek() to Read from a Specific Position (Assignment 4)

```
# Open a file, seek to the middle, then read from there
path = input("Enter text file path: ")
with open(path, "r") as f:
    # Find file size (in bytes for text files this is the encoded size)
    f.seek(0, 2)          # move to end
    end_pos = f.tell()
    middle = end_pos // 2
    f.seek(middle, 0)     # move to 'middle' bytes from start
    chunk = f.read(100)   # read next 100 characters (or fewer if near EOF)
    print("Started at byte:", middle)
    print("Chunk:", chunk)
```

رومانی صبحی رزق جرجس مطر

Assignment_7_118

Data Structure and File Handling

Program 1:

- **Prompt the user to enter 4 course scores, each out of 10.**
- **Append each score to a list.**
- **Pass the list to a function that:**
  - **Computes the total (out of 40) and percentage.**
  - **Returns a textual grade: Excellent, Very Good, Good, Pass, or Fail.**
- **Print the list, total, percentage, and grade.**

---

program 2

- **Create a program that:**

1. **Opens file.txt in write mode and writes any content.**
2. **Opens the same file in append mode and adds more content.**
3. **Opens the file in read mode and prints its full contents to the screen.**

رومانی صبحی رزق جرجس مطر

Assignment_8_119

Data Structure, File Handling, and OOP

رومانی صبحی رزق جرجس مطر

Program 1:

- **Prompt the user to enter 4 course scores, each out of 10.**
- **Append each score to a list.**
- **Pass the list to a function that:**
  - **Computes the total (out of 40) and percentage.**
  - **Returns a textual grade: Excellent, Very Good, Good, Pass, or Fail.**
- **Print the list, total, percentage, and grade.**

---

program 2

- **Create a program that:**

1. **Opens file.txt in write mode and writes any content.**
2. **Opens the same file in append mode and adds more content.**
3. **Opens the file in read mode and prints its full contents to the screen.**

Program 3: **Write this Rectangle Class then create two objects from this class**

```
#%%
# User Defined data type
# self.__width  --> private data member (Accessed only inside the class)
# self._width --> protected data member (Accessed inside the class and on the protected)
# self.width    --> public   data member accessed inside and outside the class
```

رومانی صبحی رزق جرجس مطر

```python
class Rectangle:
   # Constructor
    def __init__(self, w, l):
        if w> 0 and l>0:
            self.__width =w
            self.__length =l
        else:
            print("You must postive number")
            self.__width =w
            self.__length =0

    # Getters or Accessors
    def getWidth(self):
        return self.__width

    def getLength(self):
        return self.__length
```

```python
# Setters or Modifiers

def setWidth(self, w):
    if w > 0:
        self.__width =w
    else:
        print("You must postive number")
        self.__width =0

def setLength(self, l):
    if l > 0:
        self.__length =l
    else:
        print("You must postive number")
        self.__length =0

def setWidthLength(self, w,l):
    if w> 0 and l>0:
        self.__width =w
        self.__length =l
    else:
        print("You must postive number")
        self.__width =w
        self.__length =0
```

```
def Area(self):
    return self.__width * self.__length

def Perimeter(self):
    return 2 * (self.__width + self.__length)

def Display(self):
    print("--- Rectangle Details ----")
    print("Width = ", self.__width)
    print("Length = ", self.__length)
    print("Area = ", self.Area())
    print("Perimeter = ", self. Perimeter())
    print("========================")
```

```
# Calling
rect1= Rectangle(3,2)

rect1.Display()


w= float(input("Enter width: "))
l= float(input("Enter length: "))
rect2 = Rectangle(w,l)
rect2.Display()
```


رومانی صبحی رزق جرجس مطر

Assignment 8: Bank Account Class in Python

رومانی صبحی رزق جرجس مطر

1 Objective

This assignment requires you to design and implement a Python `BankAccount` class. This exercise will help you practice object-oriented programming concepts such as encapsulation, method implementation, and error handling in Python.

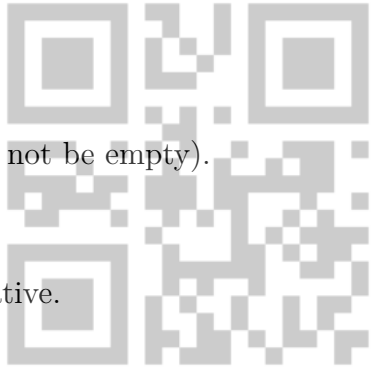
2 Instructions

Create a Python `BankAccount` class as described below. Ensure your code includes proper error handling and follows good programming practices (e.g., encapsulation with private attributes, clear comments, and input validation using PEP 8 guidelines). Test the class with a main script to demonstrate its functionality.

2.1 Part 1: BankAccount Class

Create a `BankAccount` class with the following requirements:

1. **Private Data Members**:
   - `_user_name` (str): The account holder's name (use underscore prefix for private attributes).
   - `_balance` (float): The current balance in the account.
   - `_account_number` (str): A unique identifier for the account.
2. **Constructor**:
   - An `__init__` method that initializes `_user_name`, `_balance`, and `_account_number`.
   - Ensure the initial balance is non-negative (set to 0 if negative).
   - Ensure the account number and user name are not empty strings.
3. **Methods**:
   - **Getters and Setters** (use `@property` and `@<attribute>.setter` decorators):



  - `user_name`: Returns the account holder's name.
  - `user_name.setter`: Sets the account holder's name (must not be empty).
  - `balance`: Returns the current balance.
  - `balance.setter`: Sets the balance, ensuring it is non-negative.
  - `account_number`: Returns the account number.
  - `account_number.setter`: Sets the account number (must not be empty).

- `deposit(amount)`:
  - Adds the specified amount to the balance if the amount is positive.
  - Prints a confirmation message with the new balance or an error message if the amount is invalid.
- `withdraw(amount)`:
  - Subtracts the specified amount from the balance if the amount is positive and sufficient funds are available.
  - Prints a confirmation message with the new balance or an error message if the amount is invalid or funds are insufficient.
- `transfer(amount, other_account)`:
  - Transfers the specified amount to another `BankAccount` if the amount is positive and sufficient funds are available.
  - Calls the `withdraw` method on the current account and the `deposit` method on the other account.
  - Prints a confirmation message or an error message if the transfer is invalid.
- `print_account_details()`:
  - Prints the account holder's name, account number, and current balance in a formatted way (e.g., "Account Holder: [name], Account Number: [number], Balance: $[balance]").

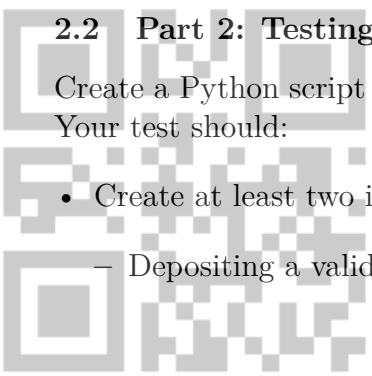
2025/2026 2025/2026 2025/2026

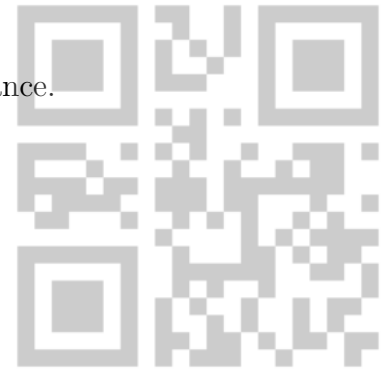
رومانی صبحی رزق جرجس مطر

2.2 Part 2: Testing

Create a Python script (e.g., `main.py`) with a main block to test the `BankAccount` class. Your test should:

- Create at least two instances of `BankAccount` and demonstrate:
  - Depositing a valid and invalid amount.





- Withdrawing a valid amount and an amount exceeding the balance.
- Transferring funds between two `BankAccount` instances.
- Printing account details for each account.

2.3 Example Output

Your programs output should look similar to this:

```
Bank Account Details:
Account Holder: John Doe, Account Number: ACC12345, Balance: $1000.0
Deposited: $500.0, New Balance: $1500.0
Withdrawn: $200.0, New Balance: $1300.0
Error: Insufficient funds.
Transferred: $300.0 to Jane Smith, New Balance: $1000.0
Bank Account Details:
Account Holder: John Doe, Account Number: ACC12345, Balance: $1000.0

Bank Account Details:
Account Holder: Jane Smith, Account Number: ACC67890, Balance: $2000.0
Deposited: $300.0, New Balance: $2300.0
Bank Account Details:
Account Holder: Jane Smith, Account Number: ACC67890, Balance: $2300.0
```

2025/2026 2025/2026 2025/2026

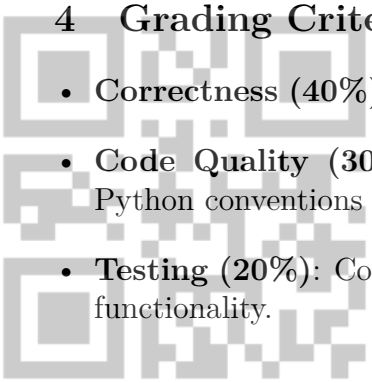
رومانی صبحی رزق جرجس مطر

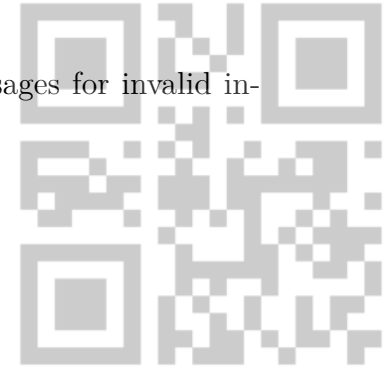
3 Submission Guidelines

- Submit your Python files (`bank_account.py` and `main.py`).
- Ensure your code is well-commented and follows proper indentation (use PEP 8 guidelines).
- Include error handling for invalid inputs (e.g., negative amounts, insufficient funds, empty strings for names or account numbers).
- Test your code thoroughly to ensure all methods work as expected.

4 Grading Criteria

- **Correctness (40%)**: Code meets all functional requirements and handles edge cases.
- **Code Quality (30%)**: Proper encapsulation, clear comments, and adherence to Python conventions (e.g., PEP 8).
- **Testing (20%)**: Comprehensive testing in the main script demonstrating all required functionality.





- **Error Handling (10%)**: Appropriate validation and error messages for invalid inputs.

Due Date: [Instructor to specify]

Good luck, and happy coding!

رومانی صبحی رزق جرجس مطر

Assignment # 9 _ 118

Task 1: Create class Complex with Real and imaginary as private data members. Also, create two complex numbers from this class. Then Add, subtract and compute the magnitude of both. Please write the below class Complex and then use it as in the calling section.

```
class Complex:

    def __init__(self, R,I):
        self.__Real = R
        self.__Img = I
    # Setters
    def SetReal(self, R):
        self.__Real = R
    def SetImg(self, I):
        self.__Img = I
    def SetComplex(self, R, I):
        self.__Real = R
        self.__Img = I

    # Getter
    def GetReal(self):
        return self.__Real

    def GetImg(self):
        return self.__Img
```

```
# AddComplex
def AddComplex(self, Other):
    return Complex(self.__Real + Other.GetReal(), self.__Img + Other.GetImg())

# Subtract
def SubtractComplex(self, Other):
    return Complex(self.__Real - Other.GetReal(), self.__Img - Other.GetImg())

# Magitude

def Magnitude(self):
    return math.sqrt(pow(self.__Real,2) + pow(self.__Img,2))

def PrintComplex(self):
    print("Real Part = ", self.__Real  )
    print("Imaginary Part = ", self.__Img  )
```

رومانی صبحی رزق جرجس مطر

Complex class calling

```
# Calling

C1 = Complex(10,10)
C1.PrintComplex()
print("----------------")
C2 = Complex (20,20)
C2.PrintComplex()
print("----------------")

print("-----ADD Complex-----------")
C3 = C1.AddComplex(C2)
C3.PrintComplex()

print("----Subtract Complex------------")
C4 = C1.SubtractComplex(C2)
C4.PrintComplex()
```


رومانی صبحی رزق جرجس مطر

Task 2: Create a package that contains all the classes that we have done in the last two lectures about OOP. Then create another python file that call and use the methods in this package.

Hany_OOP_Classes_9_118.py | Main_Calling.py

```
1   import math
2
3   class Rectangle:
41
42
43  class Student:
109
110 class Cylinder:
170
171 class BankAccount:
238
239 class Complex:
276
```

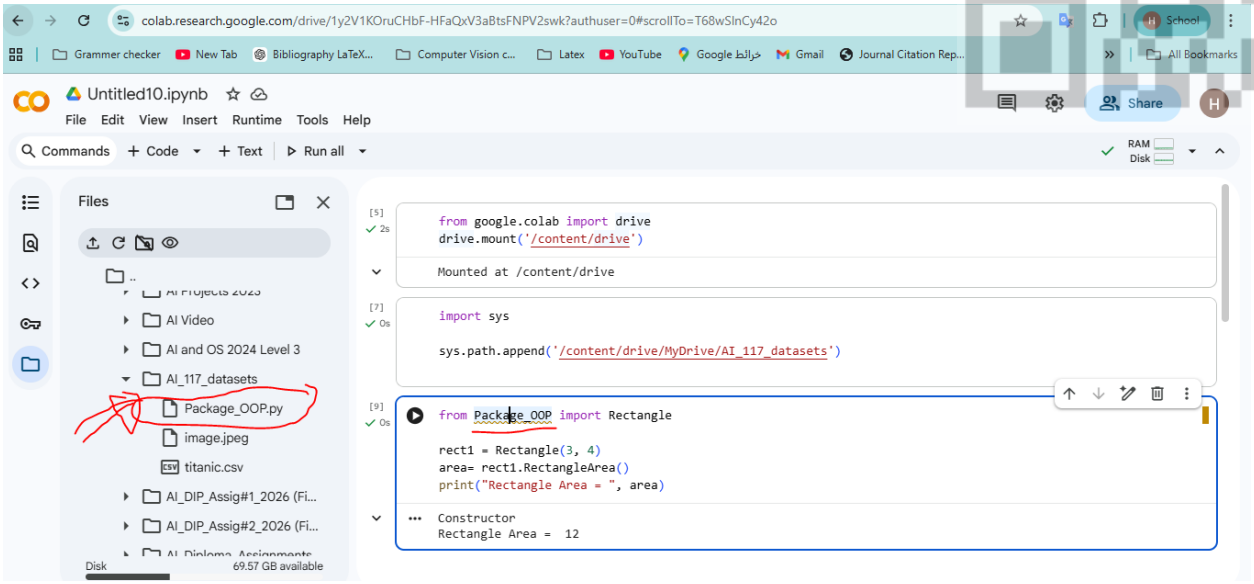
Calling of the package: Complete the calling of the Student class methods

Hany_OOP_Classes_9_118.py | Main_Calling.py

```
1
2
3   from Hany_OOP_Classes_9_118 import  Complex
4
5
6
7   # Calling
8
9   C1 = Complex(10, 20)
10  C2 = Complex (20, 10)
11  C3 = C1.AddComplex(C2)
12
13
14  C1.PrintComplex()
15  C2.PrintComplex()
16  C3.PrintComplex()
17  #%%
18  # Assignment
19  from Hany_OOP_Classes_9_118 import  Student
20
```

رومانی صبحی رزق جرجس مطر

Task 3: Upload your package in your google derive then use it as in the previous Task



رومانی صبحی رزق جرجس مطر